

Companion to Bird & de Moor, Algebra of Programming

§0. Laws from Relation Algebra

§0.1. Adjunctions

(0.1a)

	$F \dashv G$	F's type	monad FG	$1 \sqsubseteq FG$	$GF \sqsubseteq 1$	F preserves $\cup$	G preserves $\cap$	$FGF=F$	$GFG=G$
1	$\triangleleft \dashv \triangleright$	$A \rightarrow A \otimes A$	$\triangleleft \triangleright = 1$	$1 \sqsubseteq \triangleleft \triangleright$	$\triangleright \triangleleft \sqsubseteq 1$	$(R \cup S) \triangleleft = R \triangleleft \cup S \triangleleft$	$(R \cap S) \triangleright = R \triangleright \cap S \triangleright$	$\triangleleft \triangleright \triangleleft = \triangleleft$	$\triangleright \triangleleft \triangleright = \triangleright$
2	$\multimap \dashv \multimap$	$A \rightarrow I$	$\multimap \multimap = T$	$1 \sqsubseteq \multimap \multimap$	$\multimap \multimap \sqsubseteq 1$	$(R \cup S) \multimap = R \multimap \cup S \multimap$	$(R \cap S) \multimap = R \multimap \cap S \multimap$	$\multimap \multimap \multimap = \multimap$	$\multimap \multimap \multimap = \multimap$
3	$\circ \dashv \circ$	$(A \rightarrow B) \rightarrow (B \rightarrow A)$	$R^{\circ \circ} = R$	$R^{\circ \circ} = R$	$R^{\circ \circ} = R$	$(R \cup S)^{\circ} = S^{\circ} \cup R^{\circ}$	$(R \cap S)^{\circ} = R^{\circ} \cap S^{\circ}$	$R^{\circ \circ} = R$	$R^{\circ \circ} = R$
4	$\multimap \triangleleft \triangleright \multimap$	$(X \otimes A \rightarrow Y) \rightarrow (X \rightarrow Y \otimes A)$	1	$(\multimap \triangleleft \otimes 1) (1 \otimes \triangleright \multimap) = 1$	$(1 \otimes \multimap \triangleleft) (\triangleright \multimap \otimes 1) = 1$	$=$	$=$	$=$	$=$
5	$\triangleright \multimap \triangleleft \triangleright$	$(X \rightarrow Y \otimes A) \rightarrow (X \otimes A \rightarrow Y)$	1	$(1 \otimes \multimap \triangleleft) (\triangleright \multimap \otimes 1) = 1$	$(\multimap \triangleleft \otimes 1) (1 \otimes \triangleright \multimap) = 1$	$=$	$=$	$=$	$=$
6	$\Delta \dashv \cap$	$(A \rightarrow B) \rightarrow (A \rightarrow B)^2$	$R \cap R = R$	$R \cap R = R$	$R \cap S \sqsubseteq R$	$\Delta(R \cup S) = \Delta R \cup \Delta S$	$(R \cap T) \cap (S \cap U) = (R \cap S) \cap (T \cap U)$	$R \cap R = R$	$R \cap R = R$
7	$\cup \dashv \Delta$	$(A \rightarrow B)^2 \rightarrow (A \rightarrow B)$	$(R, S) \mapsto (R \cup S, R \cup S)$	$R \sqsubseteq R \cup S$	$R \cup R = R$	$(R \cup T) \cup (S \cup U) = (R \cup S) \cup (T \cup U)$	$\Delta(R \cap S) = \Delta R \cap \Delta S$	$R \cup R = R$	$R \cup R = R$
8	$\emptyset \dashv !$	$\{*\} \rightarrow (A \rightarrow B)$			$\emptyset \sqsubseteq R$				
9	$\cdot S \dashv / S$	$(A \rightarrow B) \rightarrow (A \rightarrow C)$	S/S	$R \sqsubseteq (RS)/S$	$(T/S)S \sqsubseteq T$	$(S \cup T)R = (SR) \cup (TR)$	$(R \cap R') / S = (R / S) \cap (R' / S)$	$((RS)/S)S = RS$	$((T/S)S)/S = T/S$
10	$S \cdot \dashv S \setminus$	$(B \rightarrow C) \rightarrow (A \rightarrow C)$	$S \setminus S$	$R \sqsubseteq S \setminus (SR)$	$S(S \setminus T) \sqsubseteq T$	$R(S \cup T) = (RS) \cup (RT)$	$S \setminus (R \cap R') = (S \setminus R) \cap (S \setminus R')$	$S(S \setminus (SR)) = SR$	$S \setminus (S(S \setminus T)) = S \setminus T$
11	$R \cap \dashv R \Rightarrow$	$(A \rightarrow B) \rightarrow (A \rightarrow B)$	$X \mapsto R \Rightarrow (X \cap R)$	$X \sqsubseteq R \Rightarrow (X \cap R)$	$R \cap (R \Rightarrow Y) \sqsubseteq Y$	$R \cap (X \cup Y) = (R \cap X) \cup (R \cap Y)$	$R \Rightarrow S \cap T = (R \Rightarrow S) \cap (R \Rightarrow T)$	$R \cap (R \Rightarrow (X \cap R)) = X \cap R$	$R \Rightarrow (R \cap (R \Rightarrow Y)) = R \Rightarrow Y$
12	$\mathcal{D} \dashv \cdot T$	$(A \rightarrow B) \rightarrow \text{Cor } A$	$R \mapsto (\mathcal{D}R)T$	$R \sqsubseteq (\mathcal{D}R)T$	$\mathcal{D}(AT) \sqsubseteq A$	$\mathcal{D}(R \cup S) = \mathcal{D}R \cup \mathcal{D}S$	$(A \cap B)T = AT \cap BT$	$\mathcal{D}((\mathcal{D}R)T) = \mathcal{D}R$	$(\mathcal{D}(AT))T = AT$
13	$\mathcal{R} \dashv T \cdot$	$(A \rightarrow B) \rightarrow \text{Cor } B$	$R \mapsto T(\mathcal{R}R)$	$R \sqsubseteq T(\mathcal{R}R)$	$\mathcal{R}(TA) \sqsubseteq A$	$\mathcal{R}(R \cup S) = \mathcal{R}R \cup \mathcal{R}S$	$T(A \cap B) = TA \cap TB$	$\mathcal{R}(T(\mathcal{R}R)) = \mathcal{R}R$	$T(\mathcal{R}(TA)) = TA$
14	$\cdot f \dashv \cdot f^{\circ}$	$(A \rightarrow B) \rightarrow (A \rightarrow C)$	$f f^{\circ}$	$\text{Map}(f) \Rightarrow 1 \sqsubseteq f f^{\circ}$	$f^{\circ} f \sqsubseteq 1$	$(S \cup T)R = (SR) \cup (TR)$	$\text{Simple}(g) \Rightarrow (X \cap Y) g^{\circ} = (X g^{\circ}) \cap (Y g^{\circ})$	$f f^{\circ} f = f$	$f^{\circ} f f^{\circ} = f^{\circ}$
15	$f^{\circ} \cdot \dashv f \cdot$	$(A \rightarrow C) \rightarrow (B \rightarrow C)$	$f f^{\circ}$	$\text{Map}(f) \Rightarrow 1 \sqsubseteq f f^{\circ}$	$f^{\circ} f \sqsubseteq 1$	$R(S \cup T) = (RS) \cup (RT)$	$f(X \cap Y) = fX \cap fY$	$f f^{\circ} f = f$	$f^{\circ} f f^{\circ} = f^{\circ}$
16	$i \dashv E$	$\text{Map} \multimap \text{Rel}$	E	$\frac{1}{\exists} : A \rightarrow PA$	$\exists : PB \rightarrow B$	—	—	$\frac{\exists}{\exists} = 1$	$\frac{R}{\exists} \exists = R$
17	$\cdot \exists \dashv \frac{\exists}{\exists}$	$\text{Map}(A, PB) \rightarrow (A \rightarrow B)$	1	$\frac{f \exists}{\exists} = f$	$\frac{R}{\exists} \exists = R$	—	—	$\frac{\exists}{\exists} \frac{f \exists}{\exists} = f \exists$	$\frac{R}{\exists} \frac{f \exists}{\exists} \exists = f \exists$
18	$\frac{\exists}{\exists} \dashv \cdot \exists$	$(A \rightarrow B) \rightarrow \text{Map}(A, PB)$	1	$\frac{R}{\exists} \exists = R$	$\frac{f \exists}{\exists} = f$	—	—	$\frac{\exists}{\exists} \frac{R}{\exists} = \frac{R}{\exists}$	$\frac{f \exists}{\exists} \frac{R}{\exists} \exists = f \exists$

§0.2. Composing adjunctions

(0.2a)

	$\bullet T$	$T \bullet$	$\bullet g$	$g^\circ \bullet$	Tn	$^\circ$	$\bullet \triangleleft$	Δ
1	$\bullet S$	$R / (S_1 \ S_2)$ $= R / S_2 / S_1$	$S \setminus (R / W)$ $= (S \setminus R) / W$	$R / (Sg) = (Rg^\circ) / S$	$\text{Map}(f) \Rightarrow f$ $(R / S) = (f R) / S$	$(Y/S)^\circ = S^\circ \setminus (Y^\circ)$	$(RS)^\wedge = R^\wedge (S \otimes 1)$	$(R \cap R') / S = (R / S) \cap (R' / S)$
2	$S \bullet$	$S \setminus (R / W)$ $= (S \setminus R) / W$	$(S_1 \ S_2) \setminus R$ $= S_2 \setminus (S_1 \setminus R)$	$S \setminus (Rg^\circ) = (S \setminus R)g^\circ$	$(g^\circ S) \setminus R = S \setminus (gR)$	$(S \setminus Y)^\circ = Y^\circ / S^\circ$	$((S \otimes 1)R)^\wedge = SR^\wedge$	$S \setminus (R \cap R') = (S \setminus R) \cap (S \setminus R')$
3	$\bullet f$	$\text{Map}(f) \Rightarrow R / (f S) = (R / S) f^\circ$	$T \setminus (Rf^\circ) = (T \setminus R) f^\circ$	$(R S)^\circ = S^\circ R^\circ$		$(R S)^\circ = S^\circ R^\circ$		$\text{Simple}(g) \Rightarrow (X \cap Y) g^\circ = (X g^\circ) \cap (Y g^\circ)$
4	$f^\circ \bullet$	$\text{Map}(f) \Rightarrow f (R / S) = (f R) / S$	$(Tf^\circ) \setminus R = f(T \setminus R)$	$(R S)^\circ = S^\circ R^\circ$		$(R S)^\circ = S^\circ R^\circ$		$f(T_1 \cap T_2) = fT_1 \cap fT_2$
5	Rn	—	—	—	$R \Rightarrow S \Rightarrow T = R \cap S \Rightarrow T$	$(R \Rightarrow Y)^\circ = R^\circ \Rightarrow (Y^\circ)$	—	$R \Rightarrow S \cap T = (R \Rightarrow S) \cap (R \Rightarrow T)$
6	$^\circ$	$(Y/T)^\circ = T^\circ \setminus (Y^\circ)$	$(T \setminus Y)^\circ = Y^\circ / T^\circ$	$(R S)^\circ = S^\circ R^\circ$	$(T \Rightarrow Y)^\circ = T^\circ \Rightarrow (Y^\circ)$	$R^\circ{}^\circ = R$	$(R^\wedge)^\circ = (R^\circ \otimes 1)$ $(1 \otimes \triangleright \bullet)$ both bends: the $^\circ$ section's display	$(T_1 \cap T_2)^\circ = T_1^\circ \cap T_2^\circ$
7	$\bullet \triangleleft$	$(RT)^\wedge = R^\wedge (T \otimes 1)$	$((T \otimes 1)R)^\wedge = TR^\wedge$	—	—	$(R^\wedge)^\circ = (R^\circ \otimes 1)$ $(1 \otimes \triangleright \bullet)$ both bends: the $^\circ$ section's display	—	—
8	u	$(S \cup T) R = (S R) \cup (T R)$	$R (S \cup T) = (R S) \cup (R T)$	$(S \cup T) R = (S R) \cup (T R)$	$R (S \cup T) = (R S) \cup (R T)$	$R \cap (S \cup T) = R \cap S \cup R \cap T$	$(R \cup S)^\circ = S^\circ \cup R^\circ$	
9	$\emptyset$	$\emptyset R = \emptyset$	$R \emptyset = \emptyset$	$\emptyset R = \emptyset$	$R \emptyset = \emptyset$	$R \cap \emptyset = \emptyset$	$\emptyset^\circ = \emptyset$	

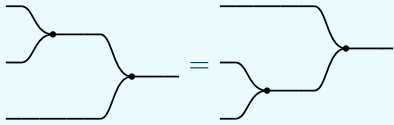
§0.3. Adjoint triples

(0.3a)

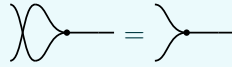
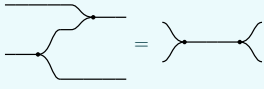
$$\begin{aligned} & \cdot f \dashv \cdot f^\circ \dashv / f^\circ \\ & f^\circ \cdot \dashv f \cdot \dashv f \backslash \end{aligned}$$

§0.4. Frobenius

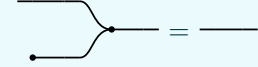
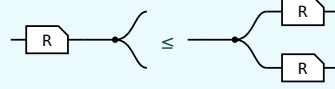
Frobenius.



▷ associative



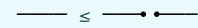
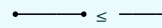
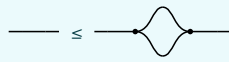
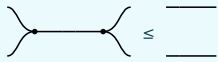
▷ commutative



◻ is its unit



(0.4a)

Frobenius, one half — the other is its $^{\circ}$ $R \triangleleft \leq \triangleleft (R \otimes R)$ $R \dashv \leq \dashv$ 

(0.4b)

▷ ◁ ≤ 1

1 ≤ ◁ ▷

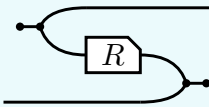
◻ ◻ ≤ 1

1 ≤ ◻ ◻

§0.5. $\circ$ is a contravariant 2-functor

(0.5a)

$\circ$ is primitive, part of the data the first section lists. It turns both of R 's wires round, and the Frobenius structure DRAWS that — the picture and the formula below are R° , not its definition:



$$R^\circ = (\circ \text{--} \triangleleft \otimes 1) (1 \otimes R \otimes 1) (1 \otimes \triangleright \text{--} \circ)$$

where $\circ \text{--} \triangleleft : \mathbb{I} \rightarrow a \otimes a$ opens a pair of wires out of nothing and $\triangleright \text{--} \circ : b \otimes b \rightarrow \mathbb{I}$ closes one, so the input of R° is where the output of R was. Taking that formula as the definition would now be circular: $\triangleleft$ and $\text{--} \circ$ are $\triangleright^\circ$ and $\circ \text{--}$. The laws of $\circ$ are that it is a contravariant 2-functor $\circ : \mathcal{C}^\circ \rightarrow \mathcal{C}$:

$$(i) \ 1^\circ = 1 \qquad (ii) \ (RS)^\circ = S^\circ R^\circ \qquad (iii) \ (R \otimes S)^\circ = R^\circ \otimes S^\circ \qquad (iv) \ R \leq S \text{ implies } R^\circ \leq S^\circ$$

§0.6. Meet and composition

(0.6a)

semi-distributivity, and what supplies it	picture
$R \ (S \cap T) \sqsubseteq RS \cap RT$ — the lax copy law. Equality exactly when R is single valued: the Maps section's $F(R \cap S) = FR \cap FS$.	

§0.7. Domain and range

(0.7a)

1	$\text{dom}(R) \triangleq 1 \cap (R \circ R^\circ)$
2	$\text{Coreflexive}(X) \implies \text{dom}(R) \subseteq X \iff R \subseteq X \times R$
3	$\text{dom}(R \circ S) \subseteq \text{dom}(R)$
4	$\text{dom}(R \cap S) = 1 \cap (S \circ R^\circ)$
5	$R \text{ entire} \iff \text{Entire}(R) \triangleq \text{dom}(R) = 1$
6	$R \text{ simple} \iff \text{Simple}(R) \triangleq R^\circ \circ R \subseteq 1$
7	$R \text{ a map} \iff \text{Map}(R) \triangleq \text{Entire}(R) \wedge \text{Simple}(R)$
8	$\text{Entire}(R) \wedge \text{Entire}(S) \implies \text{Entire}(R \circ S)$ $\text{Simple}(f) \wedge \text{Simple}(g) \implies \text{Simple}(f \circ g)$ $\text{Map}(f) \wedge \text{Map}(g) \implies \text{Map}(f \circ g)$
9	$\text{Entire}(R \circ S) \implies \text{Entire}(R)$

§0.8. Sliding the discard

(0.8a)



§0.9. Division

1	$X \in R/S \iff XS \in R$ X is any x -to- y pairing; one that only pairs x with a y such that x admires everyone y hates lies inside R/S , and R/S is the largest such.	
2	$X \in S \setminus R \iff SX \in R$ The mirror — divide on the left when x comes first.	
3	$(R/S)S \in R$ There is a y such that x admires everyone y hates, and p is one of the people y hates — then x admires p too. Strict at $S=\emptyset$: R/S is everyone, $(R/S)S=\emptyset$.	
4	$S(S \setminus R) \in R$ The mirror.	
5	associate: $R/(S_1 S_2) = (R/S_2)/S_1$ A friend's enemy is two hops: divide by the far end first.	
6	$(S_1 S_2) \setminus R = S_2 \setminus (S_1 \setminus R)$ The mirror.	
7	maps: $f(R/S) = (fR)/S$ Rename x before or after dividing — the licence to write fR/S .	
8	$R/(fS) = (R/S)f^\circ$ Rename y : a map leaves a denominator as f° outside the box.	
9	$(R/S)(S/W) \in R/W$ Someone who admires all of a hate-set that already covers everyone z works for admires those people too.	
10	$1 \in R/R$ R/R runs admirer to admirer: each admires everyone they admire. Strict: two people who each admire only a and b admire each other's idols too, and still stay two people.	
11	$(R/R)(R/R) = R/R$ R/R is the preorder admires at least as much as , and a preorder is idempotent. Freyd writes $\sqsubseteq$; with $1 \in R/R$ above it is an equality.	
12	$(R/R)R = R$ Reaching p through someone whose idols x fully admires is reaching p directly, since x admires their own idols.	
13	$R/1 = R$ Dividing by 1 : p 's set is just $\{p\}$, so admiring all of it is admiring p .	
14	$R/(S_1 \cup S_2) = R/S_1 \cap R/S_2$ Admiring a combined hate-set is admiring each set in full.	
15	$S \setminus (R/W) = (S \setminus R)/W$ Which is why $S \setminus R/W$ needs no bracket.	

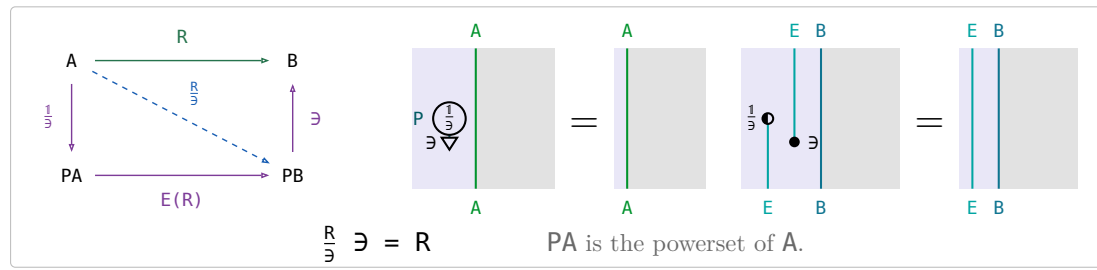
§0.10. Power allegories

(0.10a)

	law	the reading
1	$\exists_R \square = R\square, \exists_R = \exists_R \square$	One $\exists$ per object, not per arrow.
2	$\exists$ is thick	Comprehension: every x has a set of exactly the people x admires.
3	$\frac{R}{\exists} : A \rightarrow PB$, for $R : A \rightarrow B$	convert a relation to a function. $a \frac{R}{\exists} = \{b \mid a R b\}$
4	$\text{Map}(\frac{R}{\exists})$	
5	$\frac{R}{\exists} \exists = R$	reading the chosen set back through $\exists$ returns the relation.
6	$\text{Simple}(F) \implies F \sqsubseteq \frac{F \exists}{\exists}$	A partial choice of sets is inside the total one.
7	the singleton map is monic: $\frac{1}{\exists} (\frac{1}{\exists})^\circ \sqsubseteq 1$ two points with the same one-person set are the same point.	The one-person set.
8	$\frac{\exists}{\exists} = 1$	Make the set of a set, then read it back one level down.
9	fusion: $\text{Map}(f) \implies \frac{f R}{\exists} = f \frac{R}{\exists}$	Naturality of the unit, $\text{Map}(f) \implies f \frac{1}{\exists} = \frac{1}{\exists} E(f)$. renaming a point and then taking its one-person set is taking the set and renaming inside it.
10	$\text{Map}(f) \wedge 1\square = \exists\square \implies \frac{f}{\exists} = f \frac{1}{\exists}$	Rename first or take singletons first — the fusion row above at $R=1$.
11	$R\square = \exists\square \implies R /_s S = \frac{R}{\exists} (\frac{S}{\exists})^\circ$	x and y match when R sends x and S sends y to the same set.
12	$E(R) \triangleq \frac{\exists R}{\exists}$	$E(R) : PA \rightarrow PB$, image of a set of A
13	$\frac{R}{\exists} = \frac{1}{\exists} E(R)$	$\frac{1}{\exists} : x \mapsto \{x\}$
14	$\frac{S}{\exists} E(R) = \frac{S R}{\exists}$	absorption — the monad's composition law, $\frac{S}{\exists} \diamond \frac{R}{\exists} = \frac{S R}{\exists}$, $\mathcal{A} \cong \text{Kleisli}(E)$ in Relation Algebra
15	$E(R) = E(\frac{R}{\exists})$ union	the image of a set under R is the union of the R -sets of its points.
16	$\supseteq \triangleq \exists / \exists$	$xs \supseteq ys \iff \forall a. ys \exists a \rightarrow xs \exists a$
17	$\subseteq = \supseteq^\circ$	$xs \subseteq ys \iff ys \supseteq xs$
18	$T^\circ \frac{T}{\exists} \sqsubseteq E$	every T -value of a point is a member of its T -set: $\Lambda(T)$ is a map and $\Lambda(T)\exists = T$.

§0.11. $i \dashv E$: power transpose through singleton

(0.11a)



§1.2. Natural numbers

§1.3. Lists

§1.4. Trees

§1.5. Inverses

§1.6. Polymorphic functions

§1.7. Pointwise and point-free

$\langle 1, \text{list}(p) \rangle \text{ zip filter}(\pi_2) \text{ list}(\pi_1) = \text{filter}(p)$ (1.7a)		
pairing each element with its answer under p ($\langle 1, \text{list}(p) \rangle \text{ zip}$), keeping the pairs answered true ($\text{filter}(\pi_2)$) and dropping the answers ($\text{list}(\pi_1)$) is $\text{filter}(p)$		
the starting composite		
(1) (2) (3) (4) (5)		
1	$\langle 1, \text{list}(p) \rangle \text{ zip filter}(\pi_2) \text{ list}(\pi_1)$	definition of filter: $\text{filter}(p) \triangleq \text{list}((p \rightarrow \text{wrap}, \text{nil}))$ concat
2	$\triangleq \langle 1, \text{list}(p) \rangle \text{ zip list}((\pi_2 \rightarrow \text{wrap}, \text{nil})) \text{ concat list}(\pi_1)$	(1.6): $\text{concat list}(f) = \text{list}(\text{list}(f)) \text{ concat}$
3	$= \langle 1, \text{list}(p) \rangle \text{ zip list}((\pi_2 \rightarrow \text{wrap}, \text{nil})) \text{ list}(\text{list}(\pi_1)) \text{ concat}$	(1.8) backwards: $\text{list}(f) \text{ list}(g) = \text{list}(fg)$
4	$= \langle 1, \text{list}(p) \rangle \text{ zip list}((\pi_2 \rightarrow \text{wrap}, \text{nil}) \text{ list}(\pi_1)) \text{ concat}$	(1.11): $(p \rightarrow f, g)h = (p \rightarrow fh, gh)$; (1.5): $\text{wrap list}(f) = f \text{ wrap}$; (1.4): $\text{nil list}(f) = \text{nil}$
5	$= \langle 1, \text{list}(p) \rangle \text{ zip list}((\pi_2 \rightarrow \pi_1 \text{ wrap}, \text{nil})) \text{ concat}$	(1.9) backwards: $1 = \text{list}(1)$
(6) (7) (8) (9)		
6	$= \langle \text{list}(1), \text{list}(p) \rangle \text{ zip list}((\pi_2 \rightarrow \pi_1 \text{ wrap}, \text{nil})) \text{ concat}$	(1.7): $\langle \text{list}(f), \text{list}(g) \rangle \text{ zip} = \text{list}(\langle f, g \rangle)$ (1.8) backwards: $\text{list}(f) \text{ list}(g) = \text{list}(fg)$
7	$= \text{list}(\langle 1, p \rangle (\pi_2 \rightarrow \pi_1 \text{ wrap}, \text{nil})) \text{ concat}$	(1.10): $h(p \rightarrow f, g) = (hp \rightarrow hf, hg)$; (1.1): $\langle f, g \rangle \pi_1 = f$; (1.3): $f \text{ nil} = \text{nil}$
8	$= \text{list}((p \rightarrow \text{wrap}, \text{nil})) \text{ concat}$	(1.12): $1 f = f$
9	$= \text{filter}(p)$	definition of filter

§2. Functions and Categories

§2.1. Categories

§2.2. Functors

§2.3. Natural transformations

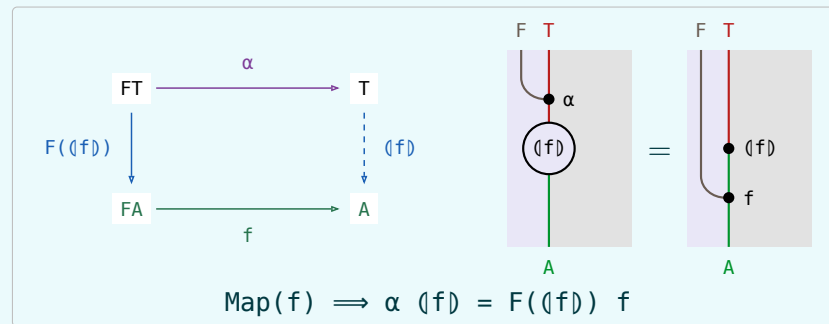
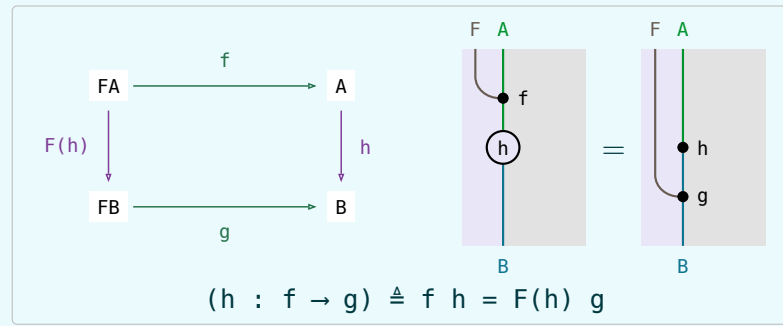
§2.4. Constructing datatypes

§2.5. Products and coproducts

§2.6. Initial algebras

(2.6a)

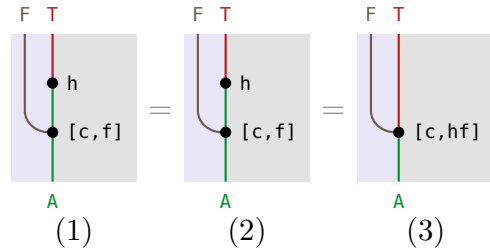
The **initial algebra** $\alpha : F(T) \rightarrow T$ has exactly one F -homomorphism $\langle f \rangle : T \rightarrow A$ to every F -algebra $f : F(A) \rightarrow A$.



$$\alpha h = F(h) [c, f] \iff \text{zero } h = c \wedge \text{succ } h = h f \quad (2.6b)$$

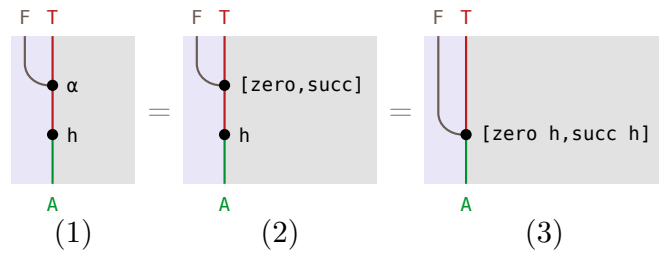
h is a homomorphism from $\alpha = [\text{zero}, \text{succ}]$ to $[c, f]$ exactly when h sends **zero** to **c** and **succ** then h equals h then f

the right side

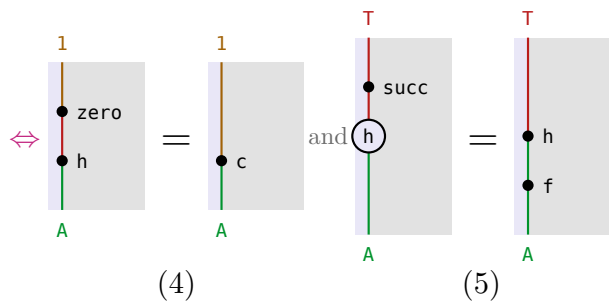


1	$F(h) [c, f]$	definition of F
2	$= (1 + h) [c, f]$	coproduct
3	$= [c, h f]$	

the left side



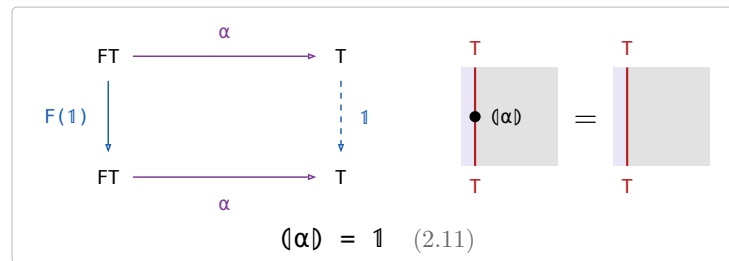
1	αh	since $\alpha = [\text{zero}, \text{succ}]$
2	$= [\text{zero}, \text{succ}] h$	coproduct
3	$= [\text{zero } h, \text{succ } h]$	cancellation, zero arm



4	$\iff [\text{zero } h, \text{succ } h] = [c, h f] \implies \text{zero } h = c$	cancellation, succ arm
5	and $[\text{zero } h, \text{succ } h] = [c, h f] \implies \text{succ } h = h f$	

§2.6.1. Reflection

(2.6.1a)

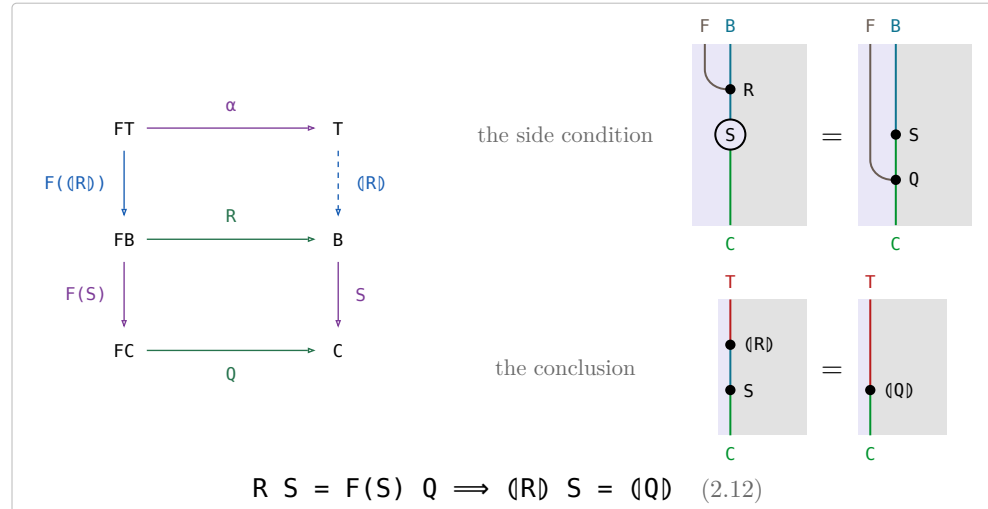


Taking a value apart with α and putting it straight back is doing nothing.

§2.6.2. $R S = F(S) Q \implies (R) S = (Q)$ — folding with R , then applying S , is folding with Q

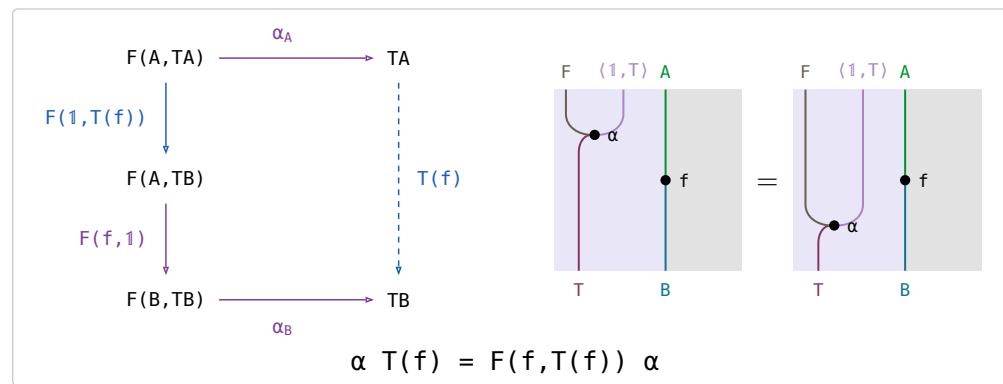
When $S : B \rightarrow C$ is an F -homomorphism from $R : F(B) \rightarrow B$ to $Q : F(C) \rightarrow C$, folding with R and then applying S is folding with Q .

(2.6.2a)

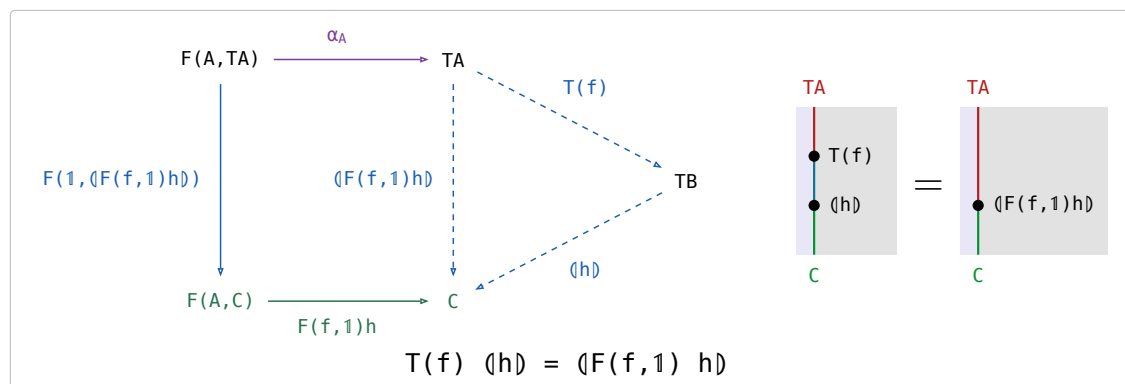


§2.7. Type functors

$T(R) \triangleq \langle F(R, 1) \ \alpha \rangle$ — the type functor T acts on a map $f : A \rightarrow B$ through the initial algebras $\alpha_A : F(A, TA) \rightarrow TA$.



- $F : \mathcal{A} \times \mathcal{A} \rightarrow \mathcal{A}$ is a bifunctor and a wire is a unary functor, so the two arguments are packed first: $\langle 1, T \rangle : \mathcal{A} \rightarrow \mathcal{A} \times \mathcal{A}$ sends A to (A, TA) , and $F(\langle 1, T \rangle(A))$ is $F(A, TA)$.
- The picture is three wires — F , $\langle 1, T \rangle$, and the object — and the region between the first two is $\mathcal{A} \times \mathcal{A}$.
- α is then an ordinary natural transformation $F \circ \langle 1, T \rangle \Rightarrow T$: its bead eats the F and $\langle 1, T \rangle$ wires, and the T wire is born under it.
- $F(f, T(f))$ costs no notation. It is the bead f on the object wire with $\langle 1, T \rangle$ and F running past: $\langle 1, T \rangle$ is what turns f into the pair $(f, T(f))$, and F is what applies it.
- The law is the naturality of α , which is exactly the freedom to slide that f bead past it.



name	law	what it says
1 functor	$T(1) = 1$ and $T(R) \ T(S) = T(R \ S)$	Acting by the identity changes nothing, and two actions in a row are one action.
2 type functor fusion	$T(f) \ (h) = (F(f, 1) \ h)$	A relator action followed by a fold is a single fold — the intermediate structure is never built. The side condition holds because F is a bifunctor — $F(R, 1)F(1, (Q)) = F(R, (Q)) = F(1, (Q))F(R, 1)$.
3 naturality of α	$\alpha \ T(f) = F(f, T(f)) \ \alpha$	Building and then mapping is the same as mapping the parts and then building, so α is natural from $G(R) = F(R, T(R))$ to T .

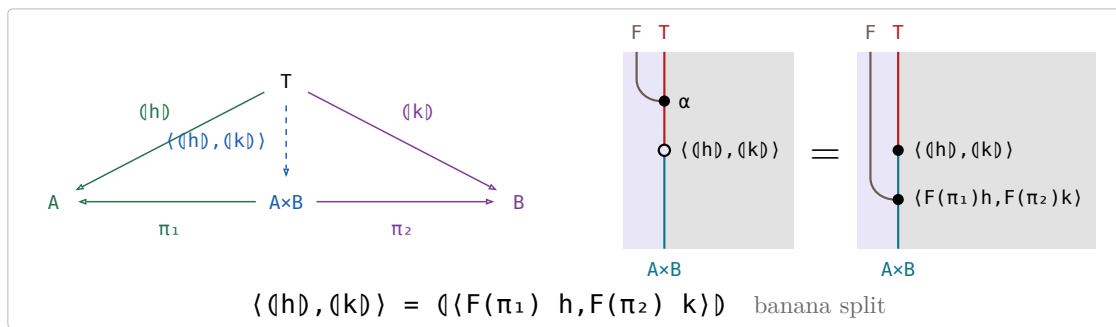
§3. Applications

§3.1. Banana-split

(3.1a)

	definition	type	note
1	$\text{listr } A ::= \text{nil} \mid \text{cons}(A, \text{listr } A) \quad \text{list}(A) = [A]$	$\mathcal{A} \rightarrow \mathcal{A}$	The cons-lists over A , the datatype every row below folds.
2	$\text{sum} = \langle [\text{zero}, \text{plus}] \rangle$	$[\mathbb{N}] \rightarrow \mathbb{N}$	$\text{plus}(a, b) = a + b$.
3	$\text{length} = \langle [\text{zero}, \pi_2 \text{ succ}] \rangle$	$[A] \rightarrow \mathbb{N}$	π_2 drops the head and keeps the count of the tail, succ adds one for the head.
4	$\text{average} \triangleq \langle \text{sum}, \text{length} \rangle \text{ div}$	$[\mathbb{N}] \rightarrow \mathbb{N}$	$\text{div}(m, n) = m/n$, with $\text{div}(0, 0) = 0$ so average is total. Traverses the list twice.
5	banana-split law $\langle \langle h \rangle, \langle k \rangle \rangle = \langle \langle F(\pi_1) h, F(\pi_2) k \rangle \rangle$	$T \rightarrow A \times B$	Any fork of folds is a single fold, hence one traversal — F the base functor.
6	what it reduces to $\alpha \langle \langle h \rangle, \langle k \rangle \rangle = F(\langle \langle h \rangle, \langle k \rangle \rangle) \langle F(\pi_1) h, F(\pi_2) k \rangle$	$FT \rightarrow A \times B$	All that $\triangleq$ reduce leaves to check: the fork satisfies the defining equation.
7	the instance $\langle \text{sum}, \text{length} \rangle = \langle [\text{zeros}, \text{pluss}] \rangle$	$[\mathbb{N}] \rightarrow \mathbb{N} \times \mathbb{N}$	$\text{pluss}(a, (b, n)) = (a + b, n + 1)$, so average runs in one pass.
8	$\text{preds } n = [n, n-1, \dots, 1]$	$\mathbb{N} \rightarrow [\mathbb{N}]$	apply the earlier special case to write preds as $\langle k \rangle \pi_1$.

(3.1b)

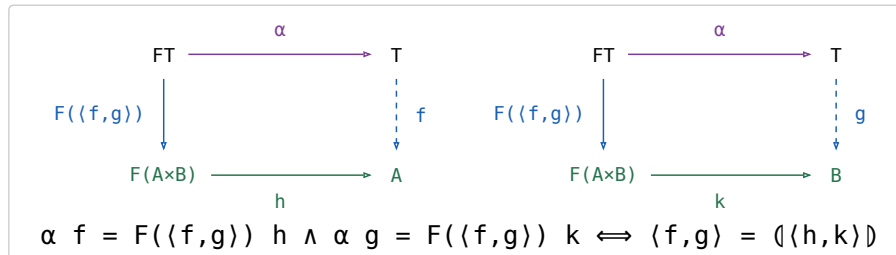


(3.1a)

§3.1.1. Fokkinga's mutual recursion theorem

- $F\text{-Alg}(\mathcal{A})$ — the algebras and their homomorphisms — has binary products whenever $\mathcal{A}$ does, and the forgetful $U : F\text{-Alg}(\mathcal{A}) \rightarrow \mathcal{A}$ **creates** them: the product of $h : FA \rightarrow A$ and $k : FB \rightarrow B$ is carried by $A \times B$, with structure $\langle F(\pi_1)h, F(\pi_2)k \rangle : F(A \times B) \rightarrow A \times B$.
- π_1 and π_2 are homomorphisms out of it, and $\langle p, q \rangle : X \rightarrow A \times B$ is a homomorphism **iff** p and q both are — substitute the structure and compare the two forks component by component.
- The banana-split law of $\langle \langle h \rangle, \langle k \rangle \rangle = \langle \langle F(\pi_1)h, F(\pi_2)k \rangle \rangle$ IS that product's universal property read at the initial algebra: $\langle h \rangle$ and $\langle k \rangle$ are the unique homomorphisms to (A, h) and (B, k) , so their fork is the unique homomorphism into the product, which is $\langle \langle F(\pi_1)h, F(\pi_2)k \rangle \rangle$.
- Fokkinga's theorem is **strictly more general** and is not that product: in mutual recursion the two arrows leave $F(A \times B)$, not FA and FB , so each sees BOTH components. The product is the case where they factor as $F(\pi_1)h$ and $F(\pi_2)k$. What it says: **any algebra on a product carrier is folded by a fork**.

(3.1.1a)



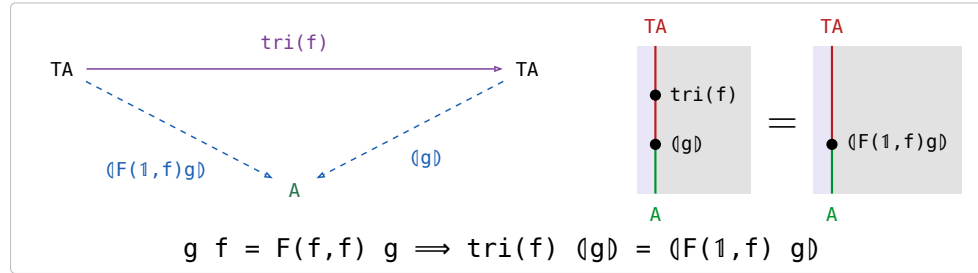
§3.2. Ruby triangles and Horner's rule

(3.2a)

stage	definition
1 informally	$\text{tri}(f)[a_0, a_1, \dots, a_i, \dots, a_n] = [a_0, f(a_1), \dots, f^i(a_i), \dots, f^n(a_n)]$
2 for cons-lists	$\text{tri}(f) = ([\text{nil}, (1 \times \text{listr}(f)) \text{ cons}])$
3 with the base functor named	$F(A, B) = 1 + A \times B$, $\alpha = [\text{nil}, \text{cons}]$, so $\text{tri}(f) = (F(1, \text{listr}(f)) \alpha)$
4 abstractly	F a bifunctor with initial type (α, T) : $\text{tri}(f) = (F(1, T(f)) \alpha)$

For the definition to make sense $f : A \rightarrow A$ is required, and then $\text{tri}(f) : TA \rightarrow TA$.

(3.2b)



Horner's rule is Horner's rule, generalised from cons-lists to any initial type. Fusion reduces it to $F(1, T(f)) \alpha (g) = F(1, (g)) F(1, f) g$.

§3.2.1. Depth of a tree

(3.2.1a)

1	the datatype	$\text{tree } A ::= \text{tip}(A) \mid \text{bin}(\text{tree } A, \text{tree } A)$, base functor $F(A,B)=A+B \times B$, initial type $([\text{tip}, \text{bin}], \text{tree})$
2	the fold	$\langle [g, h] \rangle$ is the unique f with $f(\text{tip}(a))=g(a)$ and $f(\text{bin}(x, y))=h(f(x), f(y))$
3	$\text{tree}(f)$	$\text{tree}(f) = \langle F(f, 1) \mid [\text{tip}, \text{bin}] \rangle$; pointwise $\text{tree}(f)(\text{tip}(a))=\text{tip}(f(a))$ and $\text{tree}(f)(\text{bin}(x, y))=\text{bin}(\text{tree}(f)(x), \text{tree}(f)(y))$
4	max	$\text{max} = \langle [1, \text{bmax}] \rangle$, where $\text{bmax}(a, b)$ is the larger of a and b
5	depths	$\text{depths} = \text{tree}(\text{zero}) \text{ tri}(\text{succ})$ — replaces every tip by its depth in the tree, zero the constant function returning 0 and succ the successor
6	depth	$\text{depth} = \text{depths } \text{max}$

§3.3. The TeX problem

§3.4. Conditions and conditionals

§3.5. Concatenation and currying

If φ is natural, i.e. $G(h \times \text{id}) \cdot \varphi = \varphi \cdot (F(h) \times \text{id})$, then $h \cdot G(f) \cdot \varphi = f \iff f = \text{apply} \cdot ((h \cdot G(\text{apply}) \cdot \varphi) \times \text{id})$ Theorem 3.1
 The unique solution of a two-argument recursion $f(a, x) = h(G(f)(a, x))$ built from a natural φ is **apply** composed with a one-argument fold, which is how **cat** — not itself a fold — is derived from one.

§4. Relations and Allegories

§4.1. Allegories

§4.2. Special properties of arrows

$R:A \rightarrow B$, $S:B \rightarrow A$, $1_A \in RS$, $SR \in 1_B \implies S=R^\circ$ and R is a map

Proposition 4.1

A relation with a two-sided partial inverse under composition — identity below one order, identity above the other — is that inverse's converse, and R is itself a map.

§4.3. Tabular allegories

(f,g) a tabulation of R $\implies k^\circ h \in R \iff \exists! \text{ map } m \text{ with } h=mf, k=mg$ Every pair below R's tabulating pair factors, uniquely, through a single map into the tabulation's apex.

Proposition 4.2

§4.4. Locally complete allegories

§4.5. Boolean allegories

§4.6. Power allegories

§5. Datatypes in Allegories

§5.1. Relators

(5.1a)

A **relator** $F : \mathcal{C} \rightarrow \mathcal{D}$ is a 2-functor between allegories: it preserves $\mathbf{1}$, composition and $\sqsubseteq$.

(5.1b)

the statement

- 1 $\text{Map}(f) \Rightarrow \text{Map}(F(f))$ and $\text{Map}(f) \Rightarrow F(f^\circ) = F(f)^\circ$. Lemma 5.1
- 2 Over a **tabular** allegory a functor is a relator $\iff$ it preserves $^\circ$. Theorem 5.1 the book's reverse direction (preserving $^\circ \implies$ relator) needs the source to be **Rel**: the counterexample over a tabular allegory that is not **Rel** shows it can fail
- 3 $F(R^\circ) = F(R)^\circ$ for every R , so $F(R)^\circ$ needs no bracket.
- 4 Two relators agreeing on maps are equal. Corollary 5.1
- 5 $\text{Coreflexive}(X) \wedge \text{Coreflexive}(Y) \implies F(X \cap Y) = F(X) \cap F(Y)$ Ex 5.2
- 6 $F(R \cap S) \sqsubseteq F(R) \cap F(S)$, and strictly. Ex 5.2
- 7 $F(\text{dom}(R)) = \text{dom}(F(R))$ Ex 5.5

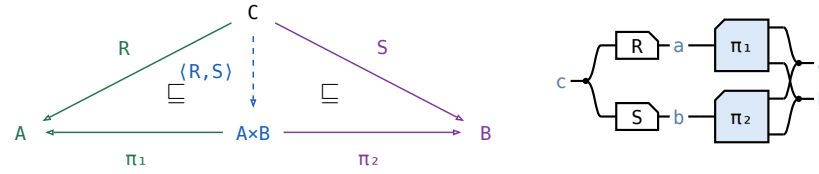
The **power relator** $P \multimap x s \ P(R) \ y s \iff (\forall a \in x s. \exists b \in y s. a \ R \ b) \wedge (\forall b \in y s. \exists a \in x s. a \ R \ b)$ — is where the fourth is strict: for $R = \{(a_1, b_1), (a_2, b_2)\}$ and $S = \{(a_1, b_2), (a_2, b_1)\}$ the pair $(\{a_1, a_2\}, \{b_1, b_2\})$ is in $P(R) \cap P(S)$, while $R \cap S = \emptyset$.

§5.2. Relational products

(5.1) $\langle R, S \rangle \triangleq (R \pi_1^\circ) \cap (S \pi_2^\circ)$ — the **fork** of $R : C \rightarrow A$ and $S : C \rightarrow B$, with (π_1, π_2) the tabulation of τ .
 $\langle R, S \rangle \triangleq R \pi_1^\circ \cap S \pi_2^\circ \iff c \langle R, S \rangle (a, b) \iff c R a \wedge c S b$.

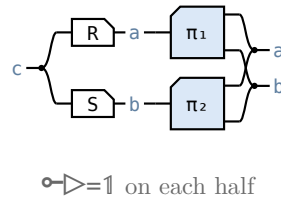
$$\langle R, S \rangle \pi_1 = \text{dom}(S) R \quad (5.6)$$

$$\langle R, S \rangle \pi_2 = \text{dom}(R) S \quad (5.7)$$



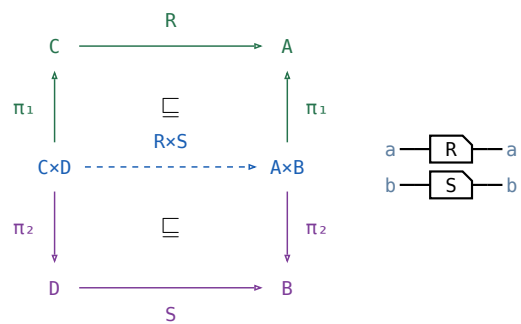
A domain is coreflexive, so $\langle R, S \rangle \pi_1 \sqsubseteq R$, with equality exactly when S is entire; for maps both triangles commute and $\langle f, g \rangle$ is unique. In $\mathbf{Rel}$, $c \langle R, S \rangle (a, b)$ iff $c R a$ and $c S b$ — copy c , then R on one strand and S on the other, which is $\triangleleft(R \otimes S)$ on the right.

No $^\circ$ survives the translation. $\pi_1 = 1 \otimes \bullet$ discards the second component, so $\pi_1^\circ = 1 \otimes \bullet$ **creates** one out of nothing, and $\cap$ is copy, run both, merge. Draw that and the created strands — the two dots with no left end, and the crossing they force — are merged against real ones, which is the monoid's unit law:



§5.2.1. Relational product $R \times S$

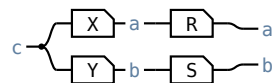
(5.2) $R \times S \triangleq \langle \pi_1 R, \pi_2 S \rangle$ — the relational product $R \times S \triangleq \langle \pi_1 R, \pi_2 S \rangle$.



Right-then-up is $(R \times S) \pi_1$, up-then-right is $\pi_1 R$, and $(R \times S) \pi_1 \sqsubseteq \pi_1 R$, equality when S is entire. In $\mathbf{Rel}$, $(c, d) (R \times S) (a, b)$ iff $c R a$ and $d S b$ — two strands side by side, no copy dot: $R \times S = R \otimes S$.

§5.2.2. Absorption

For $X : E \rightarrow C$ and $Y : E \rightarrow D$, $\langle X, Y \rangle (R \times S) = \langle XR, YS \rangle$. Both sides are this picture:



(5.2.2a)

	the law	picture
1	$\langle X, Y \rangle \langle R, S \rangle^\circ = (X \ R^\circ) \cap (Y \ S^\circ)$	
2	$\langle R, S \rangle^\circ \langle U, V \rangle \subseteq (R^\circ U) \times (S^\circ V)$	
3	$\text{Map}(f) \Rightarrow f \langle R, S \rangle = \langle f R, f S \rangle$ f a map; it fails for an arbitrary arrow;	
4	$X \langle R, S \rangle \subseteq \langle X R, X S \rangle$ the half of the row above that holds for every X	
5	$F(R \times S) \text{ unzip}(F) \subseteq \text{unzip}(F) (F(R) \times F(S))$ $\text{unzip}(F) \triangleq \langle F(\pi_1), F(\pi_2) \rangle$; only $\subseteq$ for an arbitrary R, S	
6	$g = \text{curry}(f) \iff (g \times 1) \text{ eval} = f$ reading $\times$ as the relational product, does $\mathbf{Rel}$ have exponentials?	

the statement	picture
$R : a_1 \rightarrow c, S : a_2 \rightarrow c$	
$[R, S] \triangleq (l^\circ R) \cup (r^\circ S)$ (5.9) The tape is the union — a particle entering at $A+B$ takes exactly one branch — and the two mirrored boxes are what makes the branches disjoint.	
$[R, S] = [\frac{R}{S}, \frac{S}{S}] \ni$	
$l [R, S] = R, r [R, S] = S$ $l T = R \wedge r T = S \implies T = [R, S],$	
$U : a_1 \rightarrow D, V : a_2 \rightarrow D$ $[U, V]^\circ [R, S] = (U^\circ R) \cup (V^\circ S) \quad (5.11)$	
$Z : c \rightarrow D$ $[R, S] Z = [R Z, S Z]$	

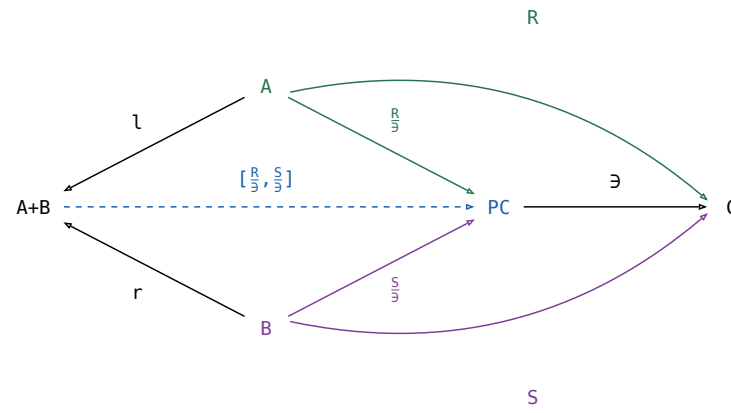
the statement	picture
$R : a_1 \rightarrow b_1, S : a_2 \rightarrow b_2$	
$R + S \triangleq [R \text{ l}, S \text{ r}]$ (5.10)	
$\text{l l}^\circ = 1, \text{r r}^\circ = 1$ Ex 5.12	
$\text{l r}^\circ = 0, \text{r l}^\circ = 0$ Ex 5.12	
$(\text{l}^\circ \text{l}) \cup (\text{r}^\circ \text{r}) = 1$ Ex 5.12	

The universal-property row is not free: $\mathbf{l}, \mathbf{r}$ were only ever asked to be a coproduct of **maps**. They stay one once every arrow is allowed because Ξ sends an arrow $\mathbf{A} \rightarrow \mathbf{C}$ to a map $\mathbf{A} \rightarrow \mathbf{P}\mathbf{C}$ reversibly, so the map coproduct can be applied underneath it. For any $\mathbf{T} : \mathbf{A} + \mathbf{B} \rightarrow \mathbf{C}$,

The diagram illustrates the fusion of two maps into a coproduct of maps. It shows a sequence of transformations between boxes representing maps and coproducts of maps.

- Leftmost box:** A box containing two maps stacked vertically: $\frac{lT}{R}$ (top) and $\frac{rT}{S}$ (bottom).
- First transformation:** A double-headed arrow labeled $\cdot \exists \vdash_{\exists}$ points to the next box.
- Second box:** A box containing two maps stacked vertically: $\frac{lT}{\exists}$ (top) and $\frac{rT}{\exists}$ (bottom).
- Second transformation:** A double-headed arrow labeled "fusion" points to the next box.
- Third box:** A box containing two maps stacked vertically: $\frac{l \exists}{R}$ (top) and $\frac{r \exists}{S}$ (bottom).
- Third transformation:** A double-headed arrow labeled $\cdot \exists \vdash_{\exists}$ points to the next box.
- Fourth box:** A box containing two maps stacked vertically: $\frac{l \exists}{\exists}$ (top) and $\frac{r \exists}{\exists}$ (bottom).
- Bottom transformation:** A double-headed arrow labeled "coproduct of maps" points from the leftmost box to a box below the main sequence.
- Bottom box:** A box containing a single map $\frac{T}{\exists}$ above a bracketed pair of maps $[\frac{R}{\exists}, \frac{S}{\exists}]$.
- Bottom transformation:** A double-headed arrow labeled $\cdot \exists \vdash_{\exists}$ points from the bottom box to the rightmost box.
- Rightmost box:** A box containing a single map $\frac{T}{\exists}$ above a bracketed pair of maps $[\frac{R}{\exists}, \frac{S}{\exists}] \exists$.

(5.3.1b)



Nothing here holds only up to $\cong$: every triangle commutes on the nose, which is the difference from the fork above. The border spells $[R, S] = [\begin{smallmatrix} R & S \\ \tilde{l} & \tilde{r} \end{smallmatrix}] \exists$, and pushing $\exists$ into the union that $[\cdot, \cdot]$ on maps already is turns that back into the definition, $(\tilde{l} \circ \begin{smallmatrix} R \\ \tilde{l} \end{smallmatrix} \cup \tilde{r} \circ \begin{smallmatrix} S \\ \tilde{r} \end{smallmatrix}) \exists = \tilde{l} \circ R \cup \tilde{r} \circ S$.

(5.3.1c)

the law	picture
1 prove (5.11), and say why duality does not carry it over from the product law	
2 $(R + S) \cap ([P, Q] [U, V]^\circ)$ $= (R \cap (P \cup^\circ)) + (S \cap (Q \cup^\circ))$ $[P, Q] [U, V]^\circ$ is a full 2×2 of composites; $R+S$ is diagonal, so the meet cuts the two off-diagonal branches	

§5.4. The power relator $P(R)$

(5.4a)

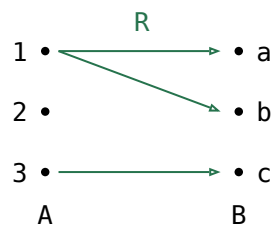
$P(R) \triangleq (\epsilon \setminus (R \epsilon)) \cap ((\exists R) / \exists)$ — the power relator $P(R) : PA \rightarrow PB$, R lifted to sets.

$xs \ P(R) \ ys \iff (\forall a \in xs. \exists b \in ys. a \ R \ b) \wedge (\forall b \in ys. \exists a \in xs. a \ R \ b)$

(5.4b)

	in Rel	in words	im(xs)	partner
1	$(\exists R) / \exists$	$\forall y \in ys. \exists x \in xs. x \ R \ y$	$ys \subseteq \text{im}(xs)$	every y has a partner
2	$(\exists / (\exists R))^\circ$	$\forall y. (\exists x \in xs. x \ R \ y) \rightarrow y \in ys$	$ys \supseteq \text{im}(xs)$	ys leaves out no partner
3	$E(R)$	$ys = \{y \mid \exists x \in xs. x \ R \ y\}$	$ys = \text{im}(xs)$	every y has a partner, and none is left out
4	$((\exists R^\circ) / \exists)^\circ$	$\forall x \in xs. \exists y \in ys. x \ R \ y$	—	every x has a partner
5	$P(R)$	$P(R)(xs, ys) \iff (\forall y. y \in ys \implies (\exists x. x \in xs \wedge R(x, y))) \wedge (\forall x. x \in xs \implies (\exists y. y \in ys \wedge R(x, y)))$	—	every x and every y has a partner

(5.4c)



	xs	$(\exists R) / \exists$	$(\exists / (\exists R))^\circ$	$E(R)$	$((\exists R^\circ) / \exists)^\circ$	$P(R)$
1	$\emptyset$	$\emptyset$	all 8 subsets of abc	$\emptyset$	all 8 subsets of abc	$\emptyset$
2	$\{1\}$	$\emptyset, a, b, ab$	ab, abc	ab	a, b, ab, ac, bc, abc	a, b, ab
3	$\{2\}$	$\emptyset$	all 8 subsets of abc	$\emptyset$	none	none
4	$\{3\}$	$\emptyset, c$	c, ac, bc, abc	c	c, ac, bc, abc	c
5	$\{1, 2\}$	$\emptyset, a, b, ab$	ab, abc	ab	none	none
6	$\{1, 3\}$	all 8 subsets of abc	abc	abc	ac, bc, abc	ac, bc, abc
7	$\{2, 3\}$	$\emptyset, c$	c, ac, bc, abc	c	none	none
8	$\{1, 2, 3\}$	all 8 subsets of abc	abc	abc	none	none

(5.4d)

	law	the reading
1	$X \sqsubseteq P(R) \iff X \exists \sqsubseteq \exists R \wedge X^\circ \exists \sqsubseteq \exists R^\circ$	One containment, and the same one at R° — which is the definition read off the two divisions. Hence $P(R^\circ) = P(R)^\circ$, and $R \sqsubseteq S \implies P(R) \sqsubseteq P(S)$.
2	$P(1) = 1$	The straightness axiom verbatim: extensionality is P 's unit law.
3	$\text{Map}(f) \implies P(f) = E(f)$	In Rel, $xs \ P(f) \ ys \iff ys = \{f(a) \mid a \in xs\}$. The half at f° says every $a \in xs$ has its $f(a)$ on ys ; f has just the one image per a , so that already says ys contains everything xs reaches, which is the fraction's second half. For a map the two definitions coincide.
4	$P(R \ S) = P(R) \ P(S)$	$\exists$ is the division cancellation laws. $\sqsubseteq$ is the one law in this section that is not a calculation: it needs a tabulation of $P(RS)$.

§5.4.1. Type relators

(5.4.1a)

$T(R) = \langle F(R, 1) \ \alpha \rangle$ — the type functor T acts on an arrow $R : A \rightarrow B$ by rebuilding with α , applying R to the parameter.

(5.4.1b)

	name	law	what it says
1	the defining equation	$T(R) = \langle F(R, 1) \ \alpha \rangle$	Rebuild the structure with α , applying R to the parameter on the way.
2	type relator	$T(R)^\circ = T(R^\circ)$	A datatype acts on relations, not only on maps — the map of the converse is the converse of the map.

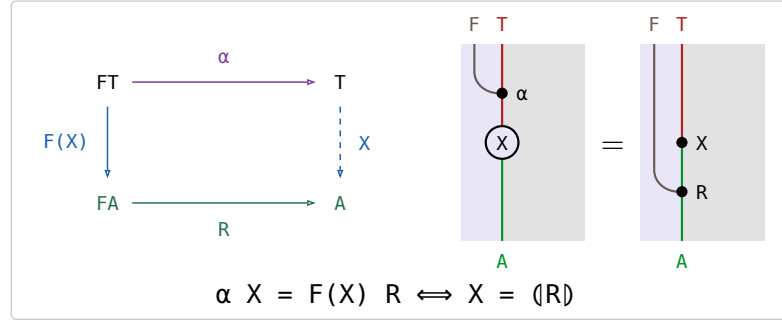
§5.5. Relational catamorphisms

(5.5a)

The initial algebra $\alpha : F(T) \rightarrow T$ among the maps is also initial in the allegory:

§5.5.1. The defining equation

(5.5.1a)



An F-algebra is a **weakened** natural transformation. A transformation $F \Rightarrow 1$ on $\mathcal{A}$ would need a component $FX \rightarrow X$ at every object and a commuting square at every arrow, but F-Algebra only need it works on T and A .

(5.5.1b)

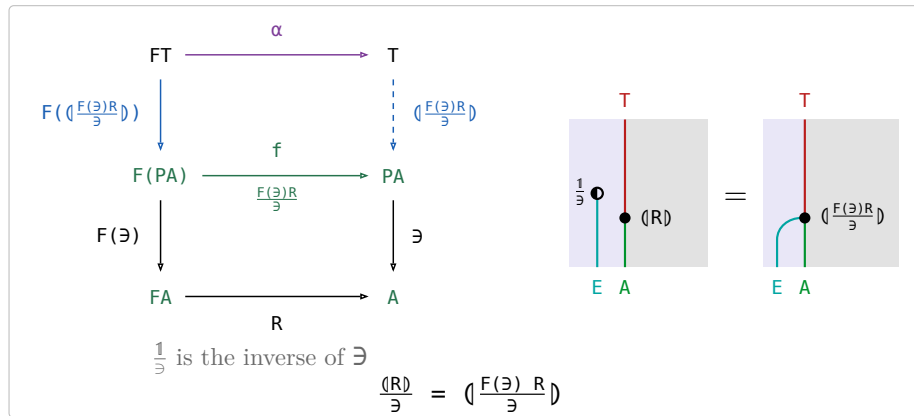
$$\llbracket [c, f] \rrbracket = \text{reduce}(c, f)$$

catamorphism: arrows of the form $\llbracket f \rrbracket$ are called catamorphisms, and these two examples reveal the notion to be a familiar idea in abstract clothing

1	part	Nat	[A]
2	datatype	$\text{Nat} ::= \text{zero} \mid \text{succ Nat}$	$[A] ::= \text{nil} \mid \text{cons}(A, [A])$
3	base functor F	$F(X) = 1 + X$	$F(X) = 1 + A \times X$
4	initial algebra α	$\alpha = [\text{zero}, \text{succ}]$ $: 1 + \text{Nat} \rightarrow \text{Nat}$	$\alpha = [\text{nil}, \text{cons}]$ $: 1 + A \times [A] \rightarrow [A]$
5	the fold, pointwise	$\llbracket [c, f] \rrbracket (\text{zero}) = c$ $\llbracket [c, f] \rrbracket (\text{succ}(n)) = f(\llbracket [c, f] \rrbracket (n))$	$\llbracket [c, f] \rrbracket (\text{nil}) = c$ $\llbracket [c, f] \rrbracket (\text{cons}(a, x)) = f(a, \llbracket [c, f] \rrbracket (x))$

§5.5.2. $\llbracket R \rrbracket = \llbracket \frac{F(\exists) R}{\exists} \rrbracket \exists$ — folding with R is a fold of maps into sets, then $\exists$

(5.5.2a)



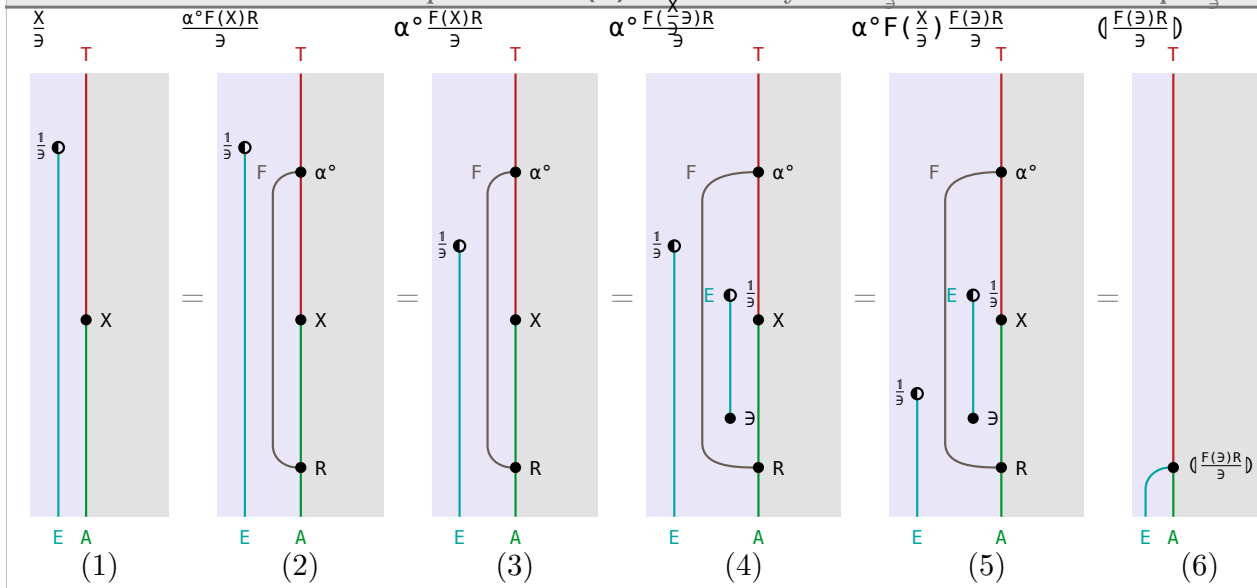
(5.12)

$$\begin{array}{c}
 \frac{\alpha X}{F(X)R} \xleftrightarrow{\cdot \exists^{-1} \frac{1}{\exists}} \frac{\frac{\alpha X}{\exists}}{\frac{F(X)R}{\exists}} \xleftrightarrow{\cdot \exists^{-1} \frac{1}{\exists}} \frac{\frac{\alpha X}{\exists}}{\frac{F(\frac{X}{\exists})R}{\exists}} \\
 \xleftrightarrow{\text{relator, fusion twice}} \frac{\frac{\alpha \frac{X}{\exists}}{F(\frac{X}{\exists}) \frac{F(\exists)R}{\exists}}}{\frac{F(\frac{X}{\exists})R}{\exists}} \xleftrightarrow{\text{reduce of maps}} \frac{\frac{X}{\exists}}{\frac{\llbracket \frac{F(\exists)R}{\exists} \rrbracket}{\exists}} \xleftrightarrow{\cdot \exists^{-1} \frac{1}{\exists}} \frac{X}{\llbracket \frac{F(\exists)R}{\exists} \rrbracket \exists}
 \end{array}$$

the last two rows at $X := \llbracket R \rrbracket$:

$$\alpha X = F(X) R \iff \frac{X}{\exists} = \langle \frac{F(\exists) R}{\exists} \rangle \quad (5.5.2b)$$

in a tabular allegory, with F a relator and $\alpha : F(T) \rightarrow T$ its initial algebra: a relation X out of T satisfies the fold equation $\alpha X = F(X) R$ of R exactly when $\frac{X}{\exists}$ is the fold of the map $\frac{F(\exists) R}{\exists}$



1	$\frac{X}{\exists}$	α iso; $\alpha X = F(X) R$
2	$= \frac{\alpha^o F(X) R}{\exists}$	α^o is a map because α is an iso, so it leaves Λ : $\frac{\alpha^o F(X) R}{\exists} = \alpha^o \frac{F(X) R}{\exists}$ — $\text{Map}(f)$ $\implies \frac{f R}{\exists} = f \frac{R}{\exists}$
3	$= \alpha^o \frac{F(X) R}{\exists}$	$X = \frac{X}{\exists} \frac{R}{\exists} \implies \frac{R}{\exists} \exists = R$
4	$= \alpha^o \frac{F(\frac{X}{\exists}) R}{\exists}$	$F(\frac{X}{\exists}) = F(\frac{X}{\exists}) F(\exists)$ as F is a functor, and $F(\frac{X}{\exists})$ is a map (a relator sends maps to maps), so it leaves Λ — $\triangleq$ relator, $\text{Map}(f)$ $\implies \frac{f R}{\exists} = f \frac{R}{\exists}$
5	$= \alpha^o F(\frac{X}{\exists}) \frac{F(\exists) R}{\exists}$	fold uniqueness, $\frac{F(\exists) R}{\exists}$ being a map — $(h : f \rightarrow g) \triangleq f h = F(h) g$, $\text{Map}(f)$ $\implies \alpha \langle f \rangle = F(\langle f \rangle) f$
6	$= \langle \frac{F(\exists) R}{\exists} \rangle$	

§5.6. Combinatorial functions

(5.6a)

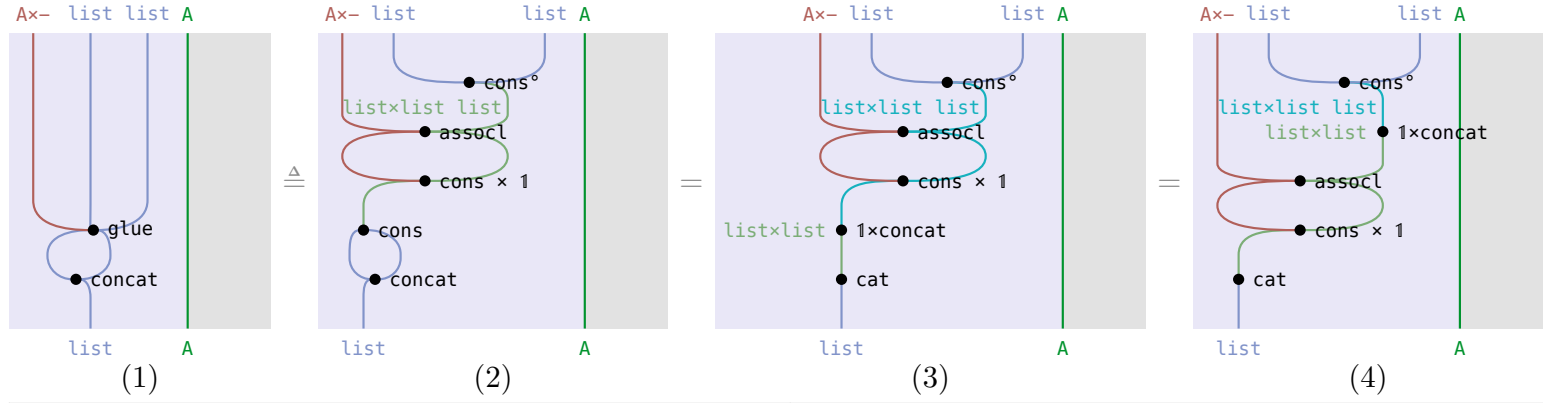
	definition	type	note
1	$[A] ::= \text{nil} \mid \text{cons}(A, [A]) \quad \text{list}(A) = [A]$	$\mathcal{A} \rightarrow \mathcal{A}$	The list type, under the short name it keeps.
2	$\text{list}(R) = \langle [\text{nil}, (R \times 1) \text{ cons}] \rangle$	$[A] \rightarrow [B]$	The relator's action on $R : A \rightarrow B$: one R per element, the shape untouched.
3	$\text{subseq} = \langle [\text{nil}, \text{cons} \cup \pi_2] \rangle$	$[A] \rightarrow [A]$	$xs \text{ subseq } ys$: ys is xs with elements dropped — cons keeps the head, π_2 drops it.
4	$\text{prefix} = \langle [\text{nil}, \text{cons} \cup \rightarrow \text{nil}] \rangle$ $\text{prefix} = \text{cat}^\circ \pi_1 = \text{init}^*$	$[A] \rightarrow [A]$	ys is an initial segment of xs ; the first nil is where it stops early. $\text{init} \triangleq \text{snoc}^\circ \pi_1$.
5	$\text{suffix} = \text{cat}^\circ \pi_2 = \text{tail}^*$	$[A] \rightarrow [A]$	The dual, $\text{tail} \triangleq \text{cons}^\circ \pi_2$; as a reduce it needs snoc -lists.
6	$\text{partition} = \text{concat}^\circ$	$[A] \rightarrow [[A]]$	This cat is restricted to $[A]^+ \times [A] \rightarrow [A]$, so ys is a list of non-empty segments of xs .
7	$\text{concat} = \langle [\text{nil}, \text{cat}] \rangle$	$[[A]] \rightarrow [A]$	Joins the segments back up, which is why its converse splits a list.
8	inits	$[A] \rightarrow [[A]]$	Implements $\frac{\text{prefix}}{\exists}$, listing the prefixes by increasing length.
9	tails	$[A] \rightarrow [[A]]$	Implements $\frac{\text{suffix}}{\exists}$ by decreasing length — the opposite order.
10	$\text{cpp}, \text{cpl}, \text{cpr}$	$[A] \times [B] \rightarrow [A \times B]$ $[A] \times B \rightarrow [A \times B]$ $A \times [B] \rightarrow [A \times B]$	Every pair (a, b) with a from the first list and b from the second; cpl , cpr keep one side a single element.
11	$\text{cpp setify} = (\text{setify} \times \text{setify}) \frac{\exists \times \exists}{\exists}$ $\text{cpl setify} = (\text{setify} \times 1) \frac{\exists \times 1}{\exists}$ $\text{cpr setify} = (1 \times \text{setify}) \frac{1 \times \exists}{\exists}$	$[A] \times [B] \rightarrow P(A \times B)$	Forgetting the order with setify , cpp is the transpose of $\exists \times \exists$.
12	$\text{cp}(F) \triangleq \frac{F(\exists)}{\exists}$	$F(PA) \rightarrow PFA$	$\text{cp}(F)$, the transpose of a relator's action on $\exists$; cpp implements it at $F(A) = A \times A$.
13	$\text{cp}(\text{list}) = \langle \frac{[\text{nil}, (\exists \times \exists) \text{ cons}]}{\exists} \rangle$ $\frac{[\text{nil}, (\exists \times \exists) \text{ cons}]}{\exists} = [\text{nil} \frac{1}{\exists}, \frac{\exists \times \exists}{\exists} \text{ cons}]$ $E(\text{cons})]$	$\text{list}(PA) \rightarrow P(\text{list}(A))$	$\text{cp}(\text{list})$: (5.12) $\langle R \rangle = \langle \frac{F(\exists)}{\exists} R \rangle \exists$ at $\text{list}(\exists) = \langle [\text{nil}, (\exists \times 1) \text{ cons}] \rangle$, then the algebra expanded.
14	$\text{cplist} = \langle [\text{nil} \text{ wrap}, \text{cpp list}(\text{cons})] \rangle$	$[[A]] \rightarrow [[A]]$	$\text{cp}(\text{list})$ with every set a list: cpp in place of $\frac{\exists \times \exists}{\exists}$.

glue concat $\sqsubseteq (1 \times \text{concat}) \text{ cons}$

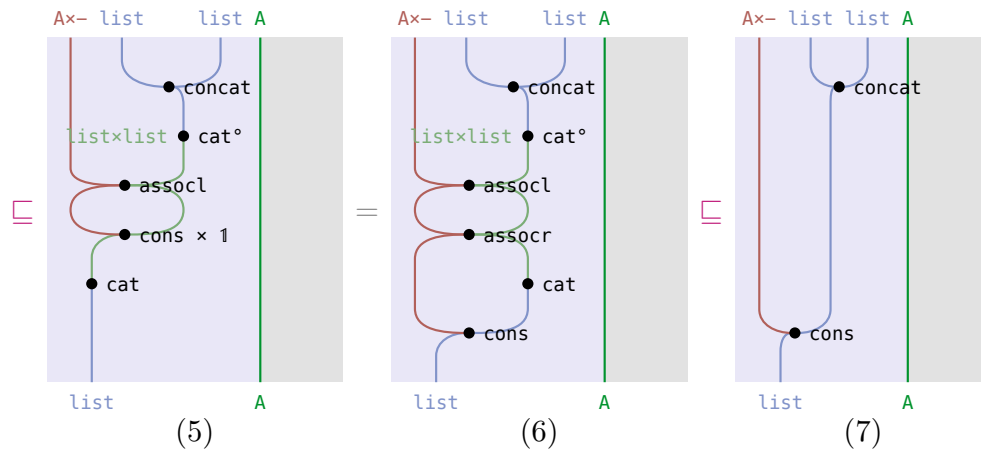
(5.6b)

putting **a** on the front of the first segment and flattening (**glue concat**) gives at most flattening the segments and putting **a** on the front of the result (**(1×concat) cons**)

the starting composite



1	glue concat	definition of glue
2	$\triangleq (1 \times \text{cons}^\circ) \text{ assocl } \text{cons} \times 1 \text{ cons } \text{concat}$	since $\text{concat} = ([\text{nil}, \text{cat}])$
3	$= (1 \times \text{cons}^\circ) \text{ assocl } \text{cons} \times 1 (1 \times \text{concat}) \text{ cat}$	naturality of assocl
4	$= (1 \times (\text{cons}^\circ (1 \times \text{concat}))) \text{ assocl } \text{cons} \times 1 \text{ cat}$	since $\text{concat} = ([\text{nil}, \text{cat}])$



5	$\sqsubseteq (1 \times \text{concat}) (1 \times \text{cat}^\circ) \text{ assocl } \text{cons} \times 1 \text{ cat}$	since $(\text{cons} \times 1) \text{ cat} = \text{assocr } (1 \times \text{cat}) \text{ cons}$
6	$= (1 \times \text{concat}) (1 \times \text{cat}^\circ) \text{ assocl } \text{assocr } (1 \times \text{cat}) \text{ cons}$	since $\text{assocl } \text{assocr} = 1$ and $\text{cat}^\circ \text{ cat} \sqsubseteq 1$
7	$\sqsubseteq (1 \times \text{concat}) \text{ cons}$	

§5.6.1. $\frac{\text{subseq}}{\exists} = ([\text{nil } \frac{1}{\exists}, (\frac{1 \times \exists}{\exists} \text{ E}(\text{cons}), \pi_2) \text{ cup}])$ — the set of subsequences is a fold of maps into sets

(5.6.1a)

$\text{cup} \triangleq \frac{(\pi_1 \exists) \cup (\pi_2 \exists)}{\exists}$ — cup unions two sets.

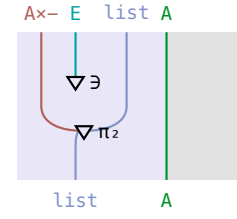
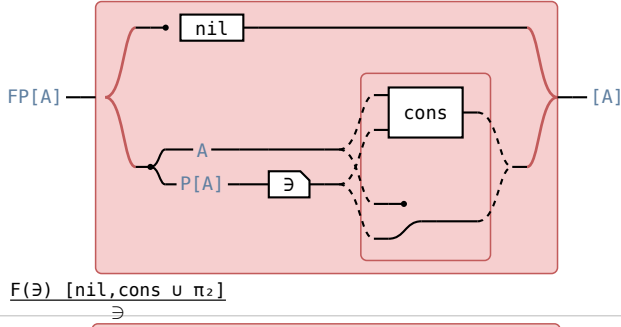
$\text{cup} \triangleq \frac{\pi_1 \exists \cup \pi_2 \exists}{\exists} : \text{PA} \times \text{PA} \rightarrow \text{PA}$

$$\frac{F(\exists) [\text{nil}, \text{cons} \cup \pi_2]}{\exists} = [\text{nil} \frac{1}{\exists}, \frac{(1 \times \exists) (\text{cons} \cup \pi_2)}{\exists}] \quad (5.6.1b)$$

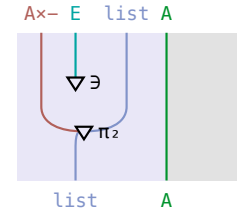
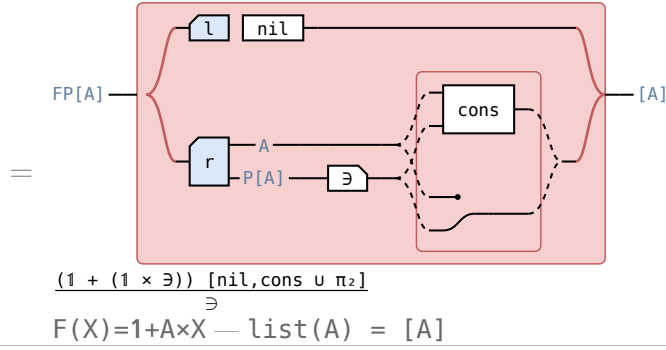
the set of lists the algebra builds is, from nothing, just **nil**, and from a head and a set of tails, every tail in the set with the head put on or left off — (5.12) $\langle R \rangle = \langle \frac{F(\exists)}{\exists} R \rangle \exists$ at **subseq** = $\langle [\text{nil}, \text{cons} \cup \pi_2] \rangle$, **subseq** = $\langle [\text{nil}, \text{cons} \cup \pi_2] \rangle$.

circuit — the fork is $F([A])=1+A \times [A]$: **nil** above, the pair below

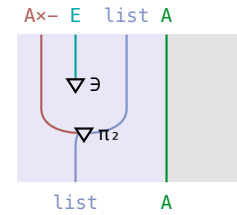
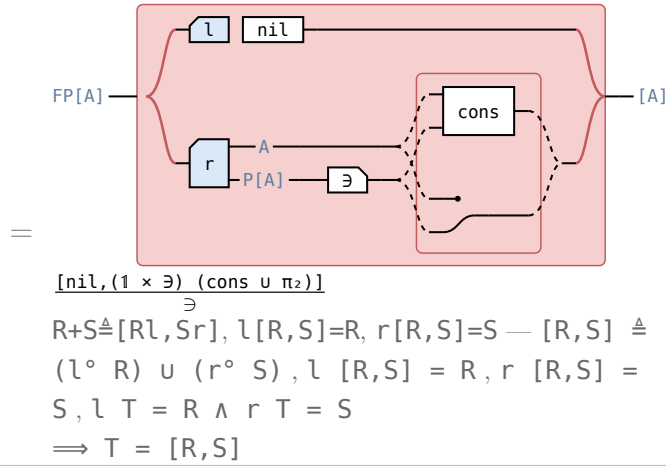
Hinze–Marsden



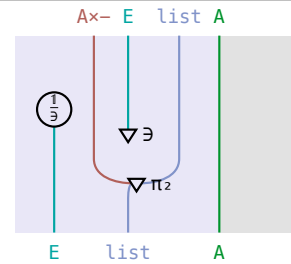
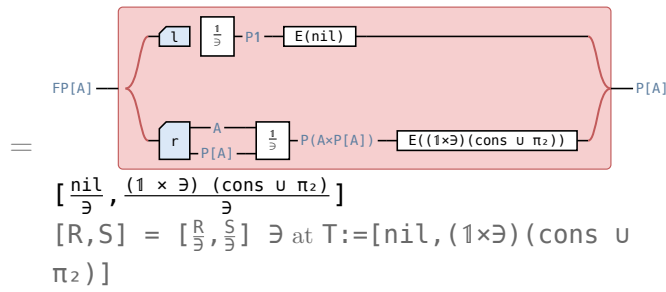
the π_2 operand of $\text{cons} \cup \pi_2$ under the $1 \times \exists$ summand of $F(\exists)$, i.e. $(1 \times \exists) \pi_2$



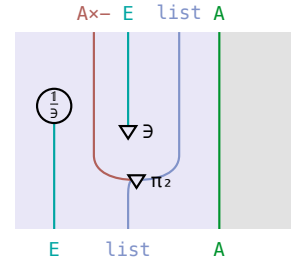
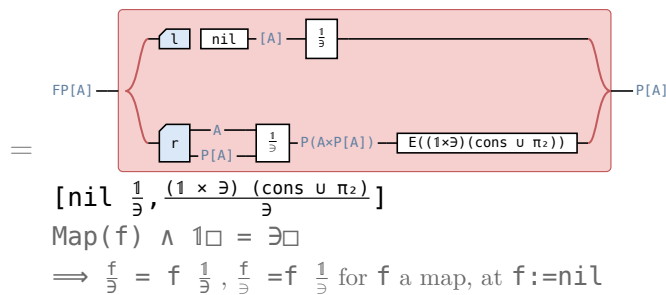
the same operand under $1+1 \times \exists$, whose $1 \times \exists$ summand it sits in



the π_2 operand of the second arm $(1 \times \exists) (\text{cons} \cup \pi_2)$

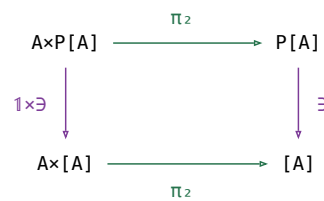


the π_2 operand under its $\frac{1}{\exists}$, the arm $(1 \times \exists) \pi_2 = \pi_2 \exists$ rewrites



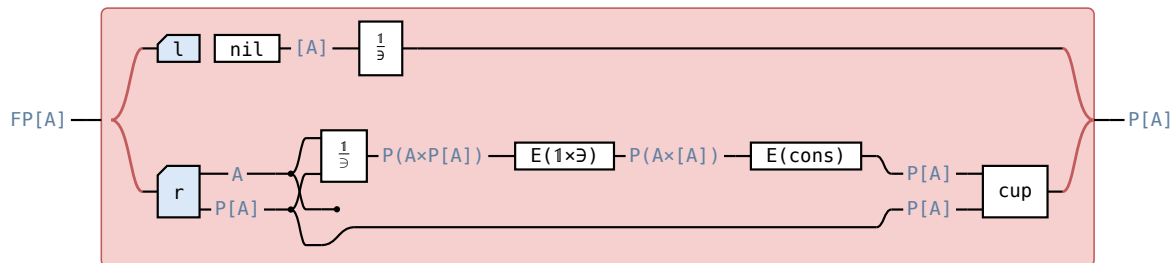
the same operand; the two rows differ only in the **nil** arm

(5.6.1c)



$\frac{(1 \times \exists) (\text{cons} \cup \pi_2)}{\exists} = \langle \frac{1 \times \exists}{\exists} E(\text{cons}), \pi_2 \rangle \text{ cup}$ power transpose of join: the power transpose of the join of two relations is $\langle \frac{R}{\exists}, \frac{S}{\exists} \rangle \text{ cup}$, where cup is the function that returns the union of two sets	(5.6.1d)
circuit	Hinze–Marsden
$\frac{(1 \times \exists) (\text{cons} \cup \pi_2)}{\exists}$ $F(\exists) [\text{nil}, \text{cons} \cup \pi_2] = [\text{nil} \frac{1}{\exists}, \frac{(1 \times \exists) (\text{cons} \cup \pi_2)}{\exists}]$'s second branch	the cons operand of cons $\cup$ π_2
$\frac{((1 \times \exists) \text{cons}) \cup ((1 \times \exists) \pi_2)}{\exists}$ $T(X_1 \cup X_2) = TX_1 \cup TX_2 \text{ — } (S \cup T) R = (S R) \cup (T R), R (S \cup T) = (R S) \cup (R T), R \cap (S \cup T) = R \cap S \cup R \cap T, (R \cup S)^\circ = S^\circ \cup R^\circ$	the π_2 operand of $(1 \times \exists) \text{cons} \cup (1 \times \exists) \pi_2$
$\frac{((1 \times \exists) \text{cons}) \cup (\pi_2 \exists)}{\exists}$ $(1 \times \exists) \pi_2 = \pi_2 \exists \text{ — } (R \times S) \pi_1 \sqsubseteq \pi_1 R \wedge (R \times S) \pi_2 \sqsubseteq \pi_2 S \text{ at } \pi_2, \text{ an equality because } 1 \text{ is entire}$	the π_2 operand of $(1 \times \exists) \text{cons} \cup \pi_2 \exists$
$\langle \frac{(1 \times \exists) \text{cons}}{\exists}, \frac{\pi_2 \exists}{\exists} \rangle \text{ cup}$ $\frac{R \cup S}{\exists} = \langle \frac{R}{\exists}, \frac{S}{\exists} \rangle \text{ cup} \text{ — } \triangleq \text{ cup}$	the cons operand under its $\frac{1}{\exists}$
$\langle \frac{1 \times \exists}{\exists} E(\text{cons}), \pi_2 \rangle \text{ cup}$ $\frac{S}{\exists} E(R) = \frac{S}{\exists} R, \text{ absorption } \frac{S}{\exists} E(R) = \frac{SR}{\exists} \text{ at } S := 1 \times \exists, R := \text{cons}; \text{ fusion and } \frac{\exists}{\exists} = 1 \text{ on the } \pi_2 \text{ operand}$	the π_2 operand, bare π_2

(5.6.1e)

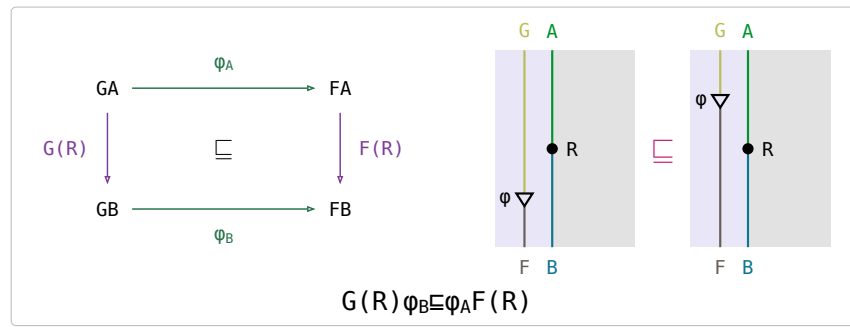


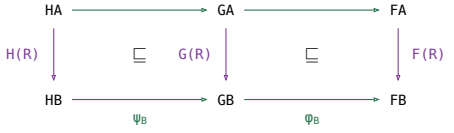
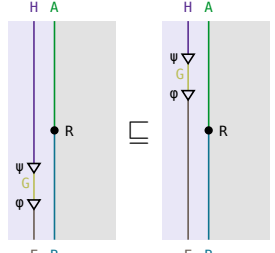
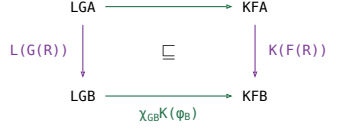
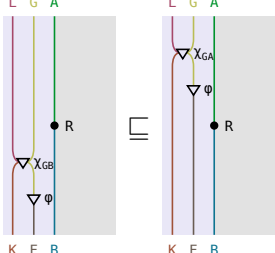
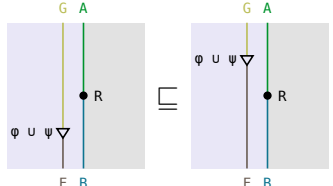
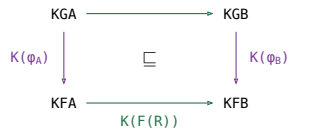
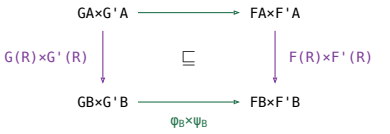
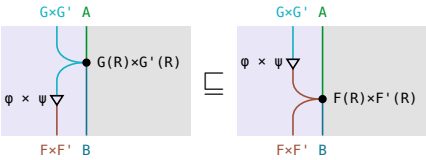
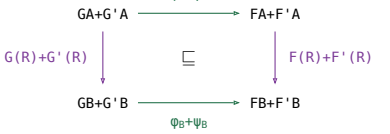
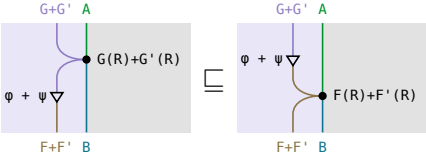
$$[\frac{\text{nil}}{\exists}, \langle \frac{1 \times \exists}{\exists} E(\text{cons}), \pi_2 \rangle \text{ cup}]$$

which writes $P\text{cons}$; **cons** is a map, and there $P(\text{cons}) = E(\text{cons}) \text{ — Map}(f)$
 $\Rightarrow P(f) = E(f)$.

§5.7. Lax natural transformations (LaT)

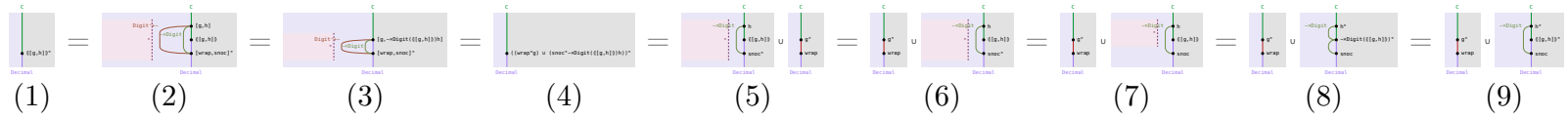
$\varphi : G \Rightarrow F$ is **lax** at $R : A \rightarrow B$ when $G(R)\varphi_B \sqsubseteq \varphi_A F(R)$; it is a **lax natural transformation** (LaT) when lax at every R . (5.7a)



closed under	commutative diagram	string diagram
1 composition $\psi\phi$	 $H(R)\psi_B \sqsubseteq \psi_A G(R)$ and $G(R)\phi_B \sqsubseteq \phi_A F(R)$ give $H(R)(\psi_B\phi_B) \sqsubseteq (\psi_A\phi_A)F(R)$	
2 horizontal composition $\chi\circ\phi$	 $\phi : G \Rightarrow F$ with $G, F : \mathcal{C} \rightarrow \mathcal{D}$ and $\chi : L \Rightarrow K$ with $L, K : \mathcal{D} \rightarrow \mathcal{E}$ give $\chi\circ\phi : L\circ G \Rightarrow K\circ F$, the family $A \mapsto \chi_{GA}K(\phi_A)$ $L(G(R))\chi_{GB} \sqsubseteq \chi_{GA}K(G(R))$ is χ lax at $G(R)$; then $K(G(R)\phi_B) \sqsubseteq K(\phi_A F(R))$ is K applied to ϕ's own inequation; $\chi := \mathbb{1}_K$ gives $K(\phi) : K\circ G \Rightarrow K\circ F$, K composed on the outside; $\phi := \mathbb{1}_G$ gives $\chi G : L\circ G \Rightarrow K\circ G$, G composed on the inside two candidates, $\chi_{GA}K(\phi_A)$ and $L(\phi_A)\chi_{FA}$, ordered by $\sqsubseteq$; this row is the first	
3 union $\phi \cup \psi$	 $G(R)\phi_B \sqsubseteq \phi_A F(R)$ and $G(R)\psi_B \sqsubseteq \psi_A F(R)$ give $G(R)(\phi_B \cup \psi_B) \sqsubseteq (\phi_A \cup \psi_A)F(R)$	
4 a relator K $K(\phi)$	 $G(R)\phi_B \sqsubseteq \phi_A F(R)$ gives $K(G(R))K(\phi_B) \sqsubseteq K(\phi_A)K(F(R))$	
5 product $\phi \times \psi$	 $\phi : G \Rightarrow F$ and $\psi : G' \Rightarrow F'$ give $\phi \times \psi : G \times G' \Rightarrow F \times F'$ $(R \times S)(U \times V) = (RU) \times (SV)$ and monotonicity in both slots — the row above's K applied to the inequation, run on a bifunctor; the fork is the derived case $\langle \phi, \psi \rangle = \triangleleft(\phi \times \psi)$, and its ONE cost is the lax copy law $R \triangleleft \sqsubseteq \triangleleft(R \times R)$ — $R \triangleleft \sqsubseteq \triangleleft(R \otimes R)$, $R \multimap \sqsubseteq \multimap$	
6 coproduct $\phi + \psi$	 $\phi : G \Rightarrow F$ and $\psi : G' \Rightarrow F'$ give $\phi + \psi : G + G' \Rightarrow F + F'$ $(R + S)(U + V) = (RU) + (SV)$ and monotonicity in both slots; the co-fork is the derived case $[\phi, \psi] = (\phi + \psi) \nabla$, and ∇ costs nothing the step it would need is $(\phi_A F(R)) \cap (\psi_A F(R)) \sqsubseteq (\phi \cap \psi)_A F(R)$, the wrong direction of R ($S \cap T \sqsubseteq (R S) \cap (R T)$) a counterexample: $aa = (0, 1) \wedge bb = (0, 0) \wedge (R \times R)(aa, bb) \wedge b_1 = \{0\} \wedge b_1 = E(\pi_1 \cap \pi_2)\{bb\} \wedge a_1 = \emptyset \wedge a_1 = E(\pi_1 \cap \pi_2)\{aa\} \wedge b_2 = \emptyset \wedge b_2 = E(R)(a_1) \Rightarrow \neg((R \times R)(\pi_1 \cap \pi_2) \sqsubseteq (\pi_1 \cap \pi_2) R)$	
7 meet — fails $\phi \cap \psi$		
8 Each row is the general slide $R_A X \sqsubseteq X' R_B$ and $R_B Y \sqsubseteq Y' R_C$ give $R_A(XY) \sqsubseteq (X'Y')R_C$ at $R_A, R_B := G(R), F(R)$ and $X, X' := \phi_B, \phi_A$, quantified over R ; at $X' = X$ with R_A, R_B endorelations it is the monotonicity reading instead — the $\forall R$ sits outside the law, and is the only difference		

$$\llbracket [g,h] \rrbracket^\circ = (g^\circ \text{ wrap}) \cup (h^\circ \text{ --xDigit}(\llbracket [g,h] \rrbracket^\circ) \text{ snoc}) \quad (6.1a)$$

the converse of a fold over decimals undoes the algebra $[g,h]$ and then either **wrap**, or the fold's converse on the front and **snoc**



1	$\llbracket [g,h] \rrbracket^\circ$	$\llbracket \emptyset \rrbracket = [\text{wrap}, \text{snoc}]^\circ \text{ Digit}^{+-}(-\text{xDigit}(\llbracket \emptyset \rrbracket)) \emptyset$
2	$= ([\text{wrap}, \text{snoc}]^\circ \text{ Digit}^{+-}(-\text{xDigit}(\llbracket [g,h] \rrbracket))) [g,h]^\circ$	$(R + S) [P,Q] = [R P, S Q]$
3	$= ([\text{wrap}, \text{snoc}]^\circ [g, -\text{xDigit}(\llbracket [g,h] \rrbracket) h]^\circ$	$[U,V]^\circ [R,S] = (U^\circ R) \cup (V^\circ S)$
4	$= ((\text{wrap}^\circ g) \cup (\text{snoc}^\circ -\text{xDigit}(\llbracket [g,h] \rrbracket) h))^\circ$	$(R \cup S)^\circ = S^\circ \cup R^\circ$
5	$= (\text{snoc}^\circ -\text{xDigit}(\llbracket [g,h] \rrbracket) h)^\circ \cup (\text{wrap}^\circ g)^\circ$	$R \cup S = S \cup R$
6	$= (\text{wrap}^\circ g)^\circ \cup (\text{snoc}^\circ -\text{xDigit}(\llbracket [g,h] \rrbracket) h)^\circ$	$(R S)^\circ = S^\circ R^\circ$
7	$= (g^\circ \text{ wrap}) \cup ((-\text{xDigit}(\llbracket [g,h] \rrbracket) h)^\circ \text{ snoc})$	$(R S)^\circ = S^\circ R^\circ$
8	$= (g^\circ \text{ wrap}) \cup (h^\circ -\text{xDigit}(\llbracket [g,h] \rrbracket)^\circ \text{ snoc})$	$-\text{xDigit}(R)^\circ = -\text{xDigit}(R^\circ)$
9	$= (g^\circ \text{ wrap}) \cup (h^\circ -\text{xDigit}(\llbracket [g,h] \rrbracket^\circ) \text{ snoc})$	

$$\text{val}^\circ = (\text{embed}^\circ \text{ wrap}) \cup (\text{op}^\circ -\text{xDigit}(\text{val}^\circ) \text{ snoc})$$

a number is read back into digits either as one nonzero digit, or by splitting off its last digit and reading back the rest

(6.1b)

- 1 $\text{op}^\circ(m,p) \iff \pi_1(p) = m/10 \wedge \pi_2(p) = m \% 10$
 op° splits a number into its quotient and remainder by 10
- 2 $(\exists p. 0 < \pi_1 p \wedge \text{op}^\circ(m,p)) \iff 10 \leq m$
 op° gives a pair with a nonzero first component exactly at the numbers with two or more digits
- 3 $(\exists d. \text{embed}^\circ(m,d)) \iff 0 < m \wedge m < 10$
 embed° gives a digit exactly at the one-digit numbers

§6.2. Least fixed points

(6.2a)

$\text{Monotonic}(\varphi) \triangleq \forall X. \forall Y. X \sqsubseteq Y \implies \varphi(X) \sqsubseteq \varphi(Y)$ — φ maps the hom-set $A \rightarrow B$ into itself with $X \sqsubseteq Y \implies \varphi(X) \sqsubseteq \varphi(Y)$.

$(\mu X : \varphi(X))$ is the least $X : A \rightarrow B$ with $\varphi(X) \sqsubseteq X$.

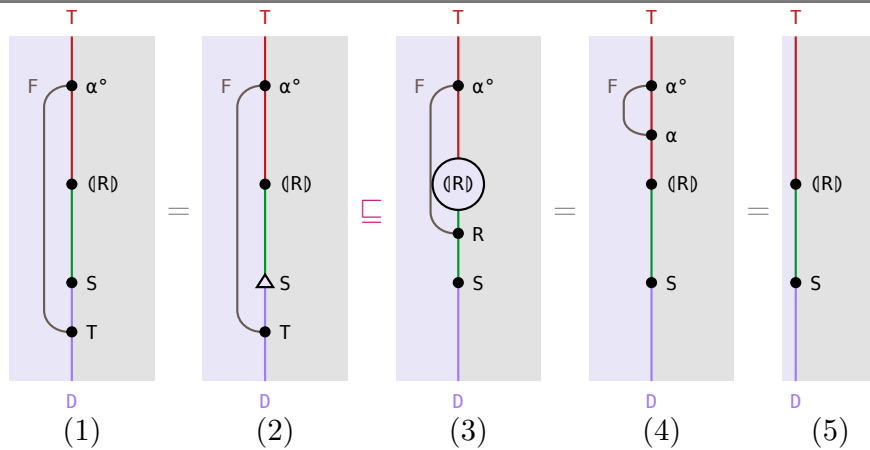
(6.2b)

the law	what it says
1 $\varphi(Y) \sqsubseteq Y \implies (\mu X : \varphi(X)) \sqsubseteq Y$	to bound $(\mu X : \varphi(X))$ above, exhibit one Y the body does not grow past — the half §6.3 and every chapter after it uses
2 $\text{Monotonic}(\varphi) \implies \varphi(\mu X : \varphi(X)) = (\mu X : \varphi(X))$	Knaster–Tarski : the least solution of $\varphi(X) \sqsubseteq X$ already solves $\varphi(X) = X$, so the least prefix point and the least fixed point are one relation
3 $\alpha^\circ F(X) R \sqsubseteq X \implies \langle R \rangle \sqsubseteq X$ (6.2)	a relation X that contains one unfolding of the fold's recursion at X contains the fold
4 $X \sqsubseteq \alpha^\circ F(X) R \implies X \sqsubseteq \langle R \rangle$ (6.3)	a relation X contained in one unfolding of the fold's recursion at X is contained in the fold

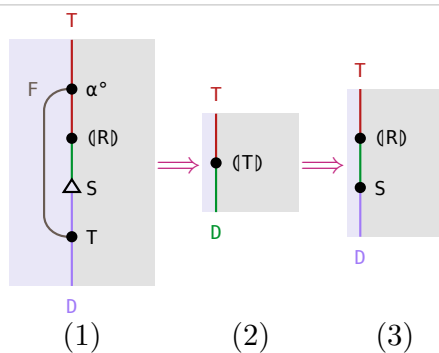
$$F(S) \ T \sqsubseteq R \ S \implies \langle T \rangle \sqsubseteq \langle R \rangle \ S$$

(6.2c)

if S followed by R absorbs T after $F(S)$, the fold of T is below the fold of R followed by S



1 $\alpha^\circ F(\langle R \rangle S) T$	
2 $= \alpha^\circ F(\langle R \rangle) F(S) T$	$F(S) \ T \sqsubseteq R \ S$
3 $\sqsubseteq \alpha^\circ F(\langle R \rangle) R \ S$	$\alpha \langle R \rangle = F(\langle R \rangle) R$
4 $= \alpha^\circ \alpha \langle R \rangle S$	$\alpha^\circ \alpha = \mathbb{1}$
5 $= \langle R \rangle S$	

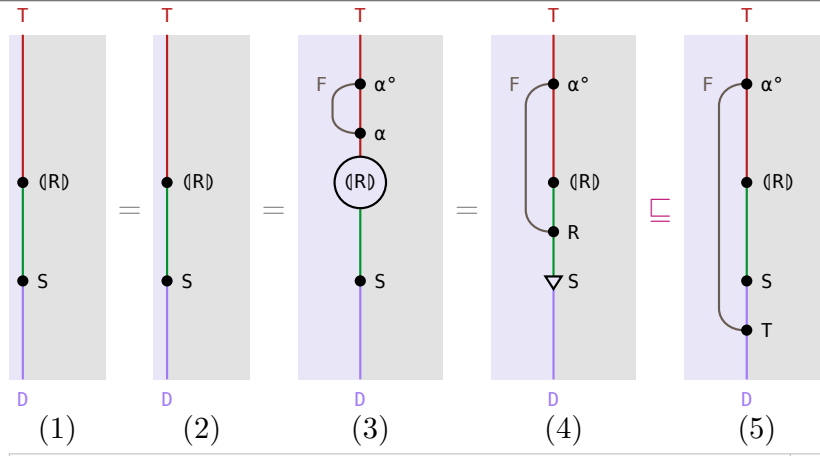


1 $F(S) \ T \sqsubseteq R \ S$	$F(S) \ T \sqsubseteq R \ S$ $\implies \alpha^\circ F(\langle R \rangle S) T \sqsubseteq \langle R \rangle S$
2 $\implies \alpha^\circ F(\langle R \rangle S) T \sqsubseteq \langle R \rangle S$	$\alpha^\circ F(X) R \sqsubseteq X$ $\implies \langle R \rangle \sqsubseteq X$
3 $\implies \langle T \rangle \sqsubseteq \langle R \rangle S$	

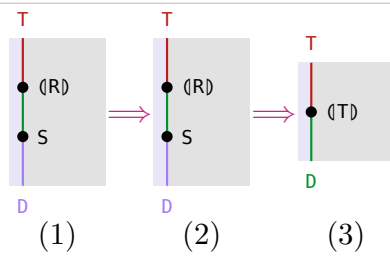
$$R \ S \sqsubseteq F(S) \ T \Rightarrow \langle R \rangle \ S \sqsubseteq \langle T \rangle$$

(6.2d)

if R followed by S is absorbed by $F(S)$ followed by T , the fold of R followed by S is below the fold of T



1	$\langle R \rangle \ S$	
2	$= \langle R \rangle \ S$	$\alpha^0 \ \alpha = 1$
3	$= \alpha^0 \ \alpha \ \langle R \rangle \ S$	$\alpha \ \langle R \rangle = F(\langle R \rangle) \ R$
4	$= \alpha^0 \ F(\langle R \rangle) \ R \ S$	$R \ S \sqsubseteq F(S) \ T$
5	$\sqsubseteq \alpha^0 \ F(\langle R \rangle \ S) \ T$	



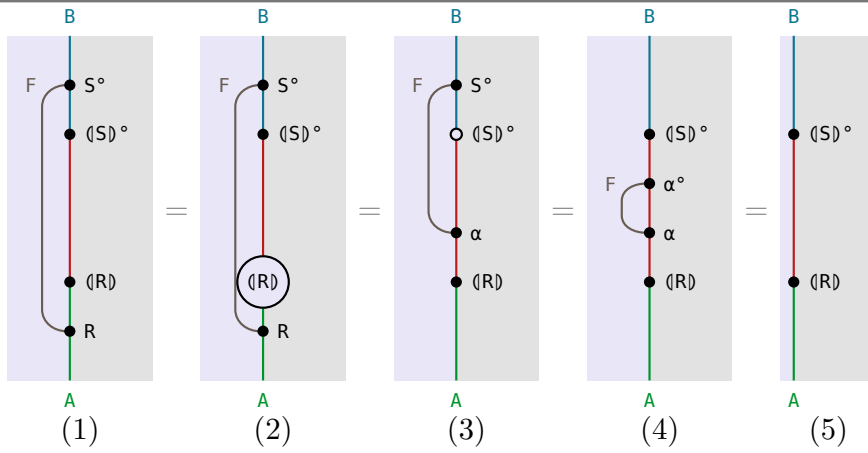
1	$R \ S \sqsubseteq F(S) \ T$	$R \ S \sqsubseteq F(S) \ T$ $\Rightarrow \langle R \rangle \ S \sqsubseteq \alpha^0 \ F(\langle R \rangle \ S) \ T$
2	$\Rightarrow \langle R \rangle \ S \sqsubseteq \alpha^0 \ F(\langle R \rangle \ S) \ T$	$X \sqsubseteq \alpha^0 \ F(X) \ R$ $\Rightarrow X \sqsubseteq \langle R \rangle$
3	$\Rightarrow \langle R \rangle \ S \sqsubseteq \langle T \rangle$	

§6.3. Hylomorphisms

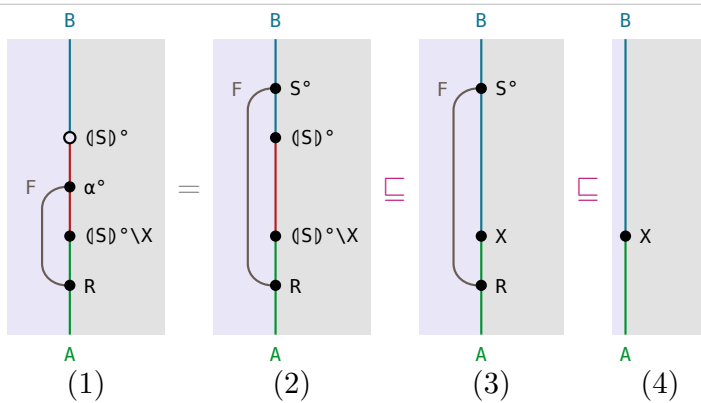
$$(S)^\circ (R) = (\mu X : S^\circ F(X) R)$$

(6.3a)

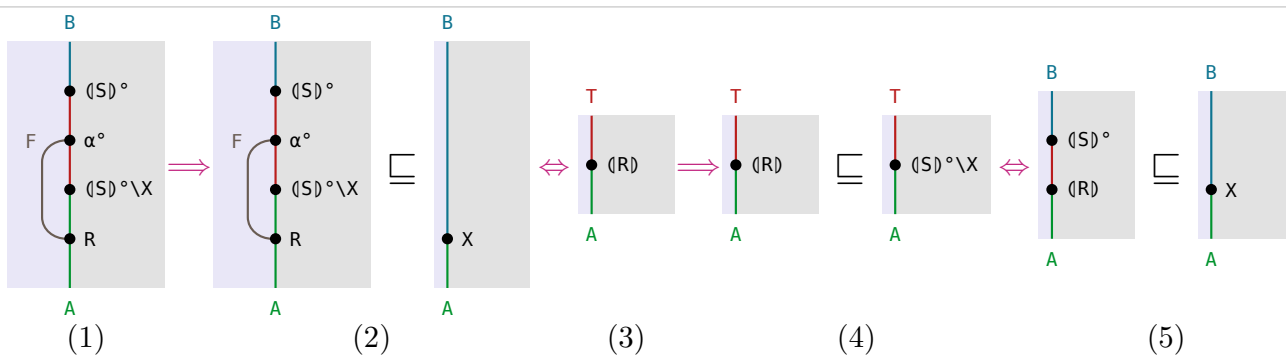
hylomorphism theorem: a hylomorphism is the least fixed point of a certain recursion equation



1	$S^\circ F((S)^\circ (R)) R$	
2	$= S^\circ F((S)^\circ) F((R)) R$	$\alpha (R) = F((R)) R$
3	$= S^\circ F((S)^\circ) \alpha (R)$	$(R)^\circ \alpha^\circ = R^\circ F((R)^\circ)$
4	$= (S)^\circ \alpha^\circ \alpha (R)$	$\alpha^\circ \alpha = 1$
5	$= (S)^\circ (R)$	



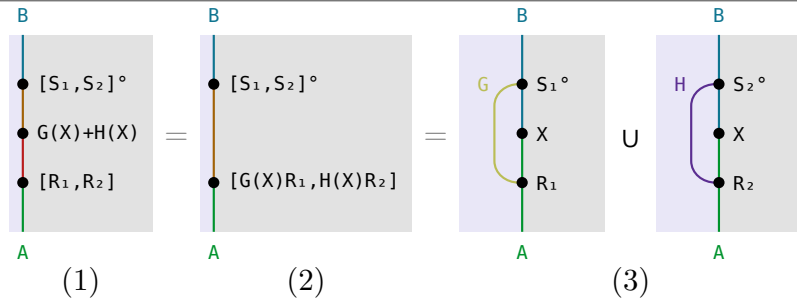
1	$(S)^\circ \alpha^\circ F((S)^\circ \setminus X) R$	$(R)^\circ \alpha^\circ = R^\circ F((R)^\circ)$
2	$= S^\circ F((S)^\circ ((S)^\circ \setminus X)) R$	$S(S \setminus R) \subseteq R$
3	$\sqsubseteq S^\circ F(X) R$	$S^\circ F(X) R \subseteq X$
4	$\sqsubseteq X$	



1	$S^\circ F(X) R \subseteq X$	$S^\circ F(X) R \subseteq X$
2	$\Rightarrow (S)^\circ \alpha^\circ F((S)^\circ \setminus X) R \subseteq X$	$\Rightarrow (S)^\circ \alpha^\circ F((S)^\circ \setminus X) R \subseteq X$
3	$\Leftrightarrow \alpha^\circ F((S)^\circ \setminus X) R \subseteq (S)^\circ \setminus X$	$X \subseteq S \setminus R \Leftrightarrow S X \subseteq R$
4	$\Rightarrow (R) \subseteq (S)^\circ \setminus X$	$\alpha^\circ F(X) R \subseteq X$
5	$\Leftrightarrow (S)^\circ (R) \subseteq X$	$\Rightarrow (R) \subseteq X$
		$X \subseteq S \setminus R \Leftrightarrow S X \subseteq R$

$$([S_1, S_2]^\circ)^\circ ([R_1, R_2])^\circ = (\mu X : (S_1^\circ G(X) R_1) \cup (S_2^\circ H(X) R_2)) \quad (6.3b)$$

when both algebras are case splits over the same sum, the recursion runs each case on its own and unites the results



1	$[S_1, S_2]^\circ (G(X) + H(X)) [R_1, R_2]$	$(R + S) [P, Q] = [R P, S Q]$
2	$= [S_1, S_2]^\circ [G(X) R_1, H(X) R_2]$	$[U, V]^\circ [R, S] = (U^\circ R) \cup (V^\circ S)$
3	$= (S_1^\circ G(X) R_1) \cup (S_2^\circ H(X) R_2)$	

§6.4. Fast exponentiation and modulus computation

$\text{shift } f = (f \times 1) \ h \Rightarrow (\mu X : (\text{zero}^\circ \text{ zero } f) \cup (\text{shift}^\circ (X \times 1) h)) \sqsubseteq f$ once f turns shift into h, the recursion that halves the argument at each step computes f		(6.4a)
(1) (2) (3) (4)		
1	$(\mu X : (\text{zero}^\circ \text{ zero } f) \cup (\text{shift}^\circ (X \times 1) h))$	$\text{convert}^\circ ([g, h]) = (\mu X : (\text{zero}^\circ g) \cup (\text{shift}^\circ (X \times 1) h))$
2	$= \text{convert}^\circ ([\text{zero } f, h])$	$\text{zero } f = g \wedge \text{shift } f = (f \times 1) \ h$
3	$= \text{convert}^\circ \text{ convert } f$	$\Rightarrow \text{convert } f = ([g, h])$
4	$\sqsubseteq f$	$\text{Simple}(\text{convert})$

$(\mu X : (\text{zero}^\circ \text{ one}) \cup (\text{shift}^\circ (X \times 1) \text{ op}(a))) \sqsubseteq \text{exp}(a)$ the recursion that halves the exponent at each step computes a to the power b		(6.4b)
$\text{zero exp}(a) = \text{one}$ a to the power 0 is 1		
$\text{shift exp}(a) = (\text{exp}(a) \times 1) \text{ op}(a)$ a to the power $2n+d$ is $\text{op}(a)$ of a to the power n and d		

$(\mu X : (\text{zero}^\circ \text{ zero}) \cup (\text{shift}^\circ (X \times 1) \text{ op}(b))) \sqsubseteq \text{mod}(b)$ the recursion that halves a at each step computes the remainder of a by b		(6.4c)
$\text{zero mod}(b) = \text{zero}$ $0 \text{ mod } b$ is 0		
$\text{shift mod}(b) = (\text{mod}(b) \times 1) \text{ op}(b)$ $(2a+d) \text{ mod } b$ is $\text{op}(b)$ of $a \text{ mod } b$ and d		

§6.5. Unique fixed points

(6.5a)

$$R R \sqsubseteq S R \Rightarrow (X / R / S) R R \sqsubseteq X$$

((X/R)/S) followed by RR stays below X when $RR \sqsubseteq SR$

$$\text{inductive}(S) \wedge R R \sqsubseteq S R \Rightarrow \text{inductive}(R)$$

R is inductive when $RR \sqsubseteq SR$ for an inductive S

1	$(X / R / S) R R$	$R R \sqsubseteq S R$
2	$\sqsubseteq (X / R / S) S R$	$(R / S) S \sqsubseteq R$
3	$\sqsubseteq (X / R) R$	$(R / S) S \sqsubseteq R$
4	$\sqsubseteq X$	

§6.5.1. Membership

(6.5.1a)

member = $\emptyset$

a constant relator records no elements, so its membership is empty

member = [member, member]

a member of $F+G$ is a member of whichever summand the value is in

member = $(\pi_1 \text{ member}) \cup (\pi_2 \text{ member})$

a member of $F \times G$ is a member of either component

§6.5.2. Consequences

(6.5.2a)

$$\text{inductive}(S \text{ member}) \wedge X = S F(X) R \wedge Y = S F(Y) R \Rightarrow X = Y$$

when $S \text{ member}$ is inductive, two solutions X, Y of $X = SF(X)R$ are equal

(6.5.2b)

$$\text{inductive}(S \text{ member}) \wedge \text{Entire}(S) \wedge \text{Entire}(R) \wedge S F(X) R \sqsubseteq X \Rightarrow \text{Entire}(X)$$

when $S \text{ member}$ is inductive and S, R are entire, a pre-fixed point of $X \mapsto SF(X)R$ is entire

(6.5.2c)

$$\text{inductive}(g \text{ member}) \wedge \text{Map}(g) \wedge \text{Map}(f) \wedge X = g F(X) f \Rightarrow \text{Map}(X)$$

when $g \text{ member}$ is inductive and f, g are maps, the solution of $X = gF(X)f$ is a map

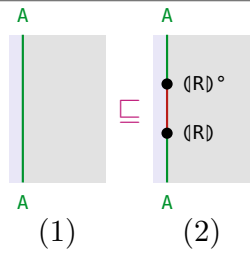
$$X = g F(X) f \wedge \text{Simple}(g) \wedge \text{Simple}(f) \wedge Y^\circ X \sqsubseteq 1 \Rightarrow (g F(Y) f)^\circ X \sqsubseteq 1$$

$(gF(Y)f)^\circ$ followed by X stays below 1 when g, f are simple and $Y^\circ X \sqsubseteq 1$

1	$(g F(Y) f)^\circ X$	$X = g F(X) f$
2	$= (g F(Y) f)^\circ g F(X) f$	$(R S)^\circ = S^\circ R^\circ$
3	$= f^\circ F(Y)^\circ g^\circ g F(X) f$	PreservesRecip(F)
4	$= f^\circ F(Y)^\circ g^\circ g F(X) f$	Simple(g)
5	$\sqsubseteq f^\circ F(Y)^\circ X f$	$Y^\circ X \sqsubseteq 1$
6	$\sqsubseteq f^\circ f$	Simple(f)
7	$\sqsubseteq 1$	

(6.5.2d)

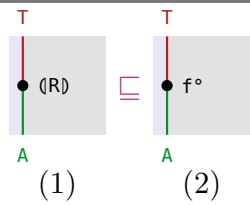
$\text{inductive}(R^\circ \text{ member}) \wedge 1 \sqsubseteq R^\circ R \implies 1 \sqsubseteq (R)^\circ (R)$
 when $R^\circ \text{member}$ is inductive and R is surjective, the fold (R) is surjective



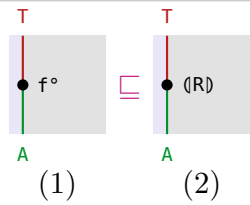
1	1	$\text{inductive}(S \text{ member}) \wedge \text{Entire}(S) \wedge \text{Entire}(R) \wedge S$ $F(X) R \sqsubseteq X$ $\implies \text{Entire}(X) \text{ at } S=R^\circ$
2	$\sqsubseteq (R)^\circ (R)$	

(6.5.2e)

$\text{Map}(f) \wedge R f \sqsubseteq F(f) \alpha \wedge 1 \sqsubseteq R^\circ R \implies f^\circ = (R)$
 a map f with $Rf \sqsubseteq F(f) \alpha$, for a surjective R , has the fold (R) as its converse



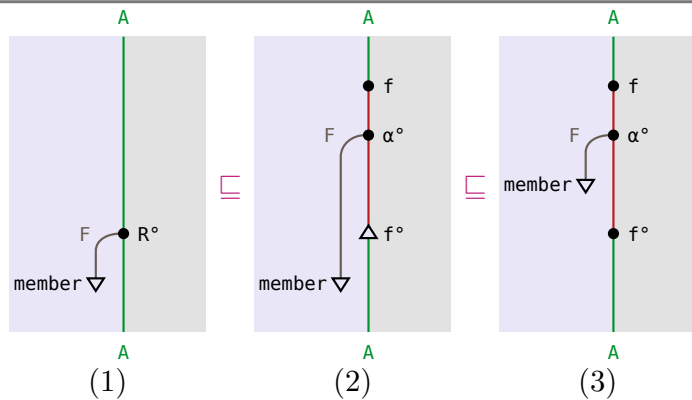
1	(R)	shunting f , fusion, $Rf \sqsubseteq F(f) \alpha$
2	$\sqsubseteq f^\circ$	



1	f°	$\text{inductive}(R^\circ \text{ member}) \wedge 1 \sqsubseteq R^\circ R$ $\implies 1 \sqsubseteq (R)^\circ (R) \text{ and } (R)f \sqsubseteq 1$
2	$\sqsubseteq (R)$	

(6.5.2f)

$\text{Map}(f) \wedge R f \sqsubseteq F(f) \alpha \implies R^\circ \text{ member} \sqsubseteq f \alpha^\circ \text{ member } f^\circ$
 $R^\circ \text{member}$ is below $\alpha^\circ \text{member}$ conjugated by f , so it is inductive when $\alpha^\circ \text{member}$ is



1	$R^\circ \text{ member}$	$\text{Map}(f) \wedge R f \sqsubseteq F(f) \alpha$ $\implies R^\circ \sqsubseteq f \alpha^\circ F(f^\circ)$
2	$\sqsubseteq f \alpha^\circ F(f^\circ) \text{ member}$	$F(R) \text{ member} \sqsubseteq \text{member } R$
3	$\sqsubseteq f \alpha^\circ \text{ member } f^\circ$	

§6.6. Sorting by selection

§6.6.1. The function sort

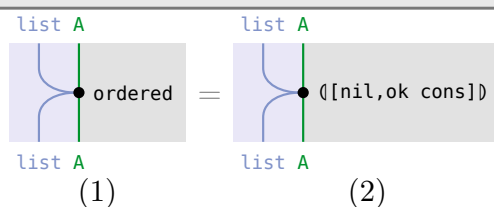
(6.6.1a)

	name	type	definition	meaning
1	orderedP(R)	$P[A]$	$\text{orderedP}[] = \text{true}, \text{orderedP}([a] \# xs) = (\forall x. x \in xs \Rightarrow R(a, x)) \wedge \text{orderedP}(xs)$	every element is R-below each element after it
2	ordered	$[A] \rightarrow [A]$	$xs \text{ ordered } ys \iff xs = ys \wedge \text{orderedP}(xs)$	the coreflexive passing exactly the lists orderedP holds of
3	ok	$A \times [A] \rightarrow A \times [A]$	$(a, xs) \text{ ok } q \iff (a, xs) = q \wedge (\forall x. x \in xs \Rightarrow R(a, x))$	the coreflexive passing (a, x) when a is R-below every element of x

ordered = $\llbracket [\text{nil}, \text{ok cons}] \rrbracket$

(6.6.1b)

a list is sorted exactly when the fold that rebuilds it passes **ok** at every **cons**, i.e. each head is **R**-below every element after it



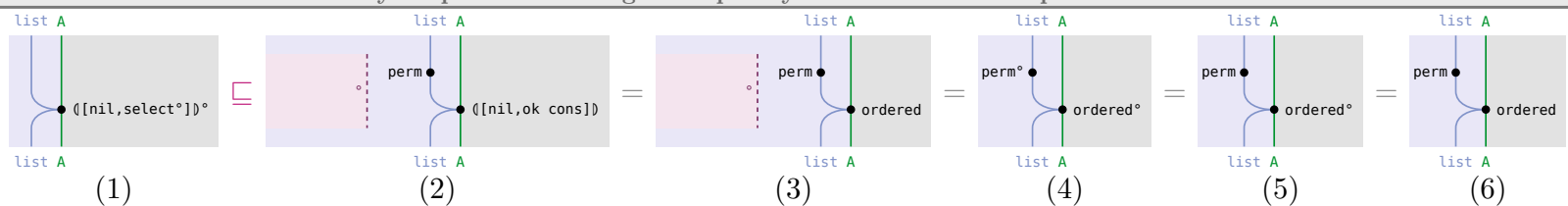
1	ordered	fold uniqueness
2	= $\llbracket [\text{nil}, \text{ok cons}] \rrbracket$	

§6.6.2. Selection sort

select[°] $\sqsubseteq$ ok cons perm $\implies \llbracket [\text{nil}, \text{select}^\circ] \rrbracket^\circ \sqsubseteq$ perm ordered

(6.6.2a)

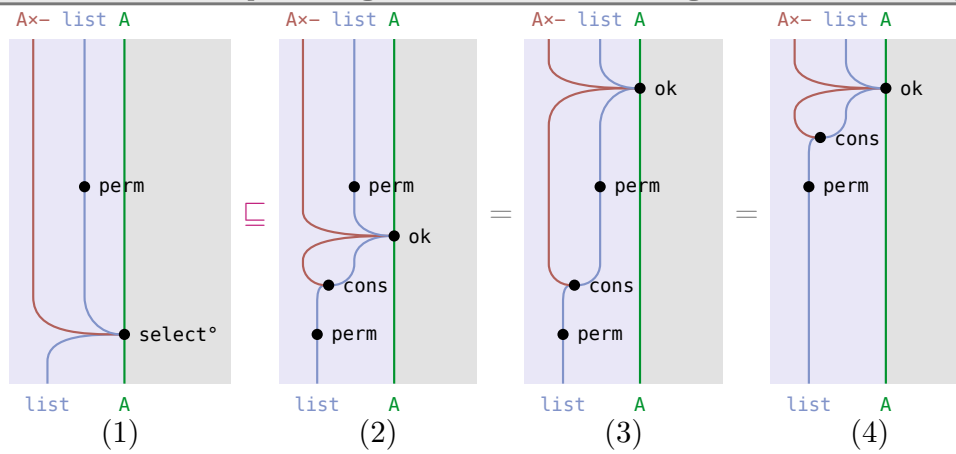
every output of unfolding the input by **select** is a sorted permutation of it



1	$\llbracket [\text{nil}, \text{select}^\circ] \rrbracket^\circ$	$\text{select}^\circ \sqsubseteq \text{ok cons perm}$
2	$\sqsubseteq (\llbracket [\text{nil}, \text{ok cons}] \rrbracket \text{ perm})^\circ$	$\implies \llbracket [\text{nil}, \text{select}^\circ] \rrbracket^\circ \sqsubseteq (\llbracket [\text{nil}, \text{ok cons}] \rrbracket \text{ perm})^\circ$
3	$= (\text{ordered perm})^\circ$	(6.6.1b) $\text{ordered} = \llbracket [\text{nil}, \text{ok cons}] \rrbracket$
4	$= \text{perm}^\circ \text{ ordered}^\circ$	$(R S)^\circ = S^\circ R^\circ$
5	$= \text{perm ordered}^\circ$	$\text{perm}^\circ = \text{perm}$
6	$= \text{perm ordered}$	$\text{ordered}^\circ = \text{ordered}$

$$\text{select}^\circ \sqsubseteq \text{ok cons perm} \Rightarrow (1 \times \text{perm}) \text{select}^\circ \sqsubseteq \text{ok cons perm} \quad (6.6.2b)$$

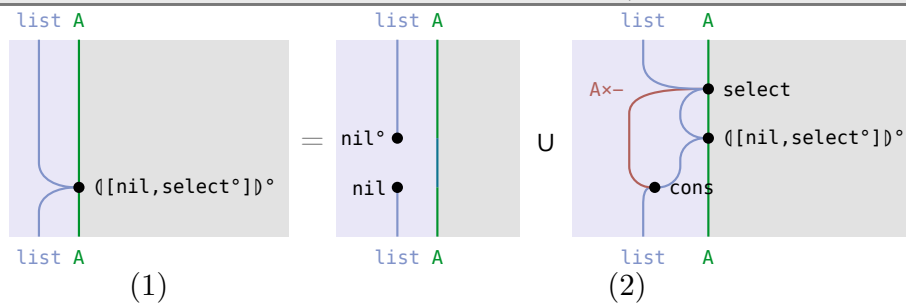
permuting the tail and then undoing **select** lands among the **ok** conses of a permutation



1	$(1 \times \text{perm}) \text{select}^\circ$	$\text{select}^\circ \sqsubseteq \text{ok cons perm}$
2	$\sqsubseteq (1 \times \text{perm}) \text{ok cons perm}$	$\text{ok} (1 \times \text{perm}) = (1 \times \text{perm}) \text{ok}$
3	$= \text{ok} (1 \times \text{perm}) \text{cons perm}$	$\text{cons perm} = (1 \times \text{perm}) \text{cons perm}$
4	$= \text{ok cons perm}$	

$$([[\text{nil}, \text{select}^\circ]]^\circ)^\circ = (\text{nil}^\circ \text{nil}) \cup (\text{select} (1 \times ([[\text{nil}, \text{select}^\circ]]^\circ)^\circ) \text{cons}) \quad (6.6.2c)$$

selection sort returns **[]** on **[]**, and otherwise selects **(a,y)**, sorts **y** and conses **a** back on



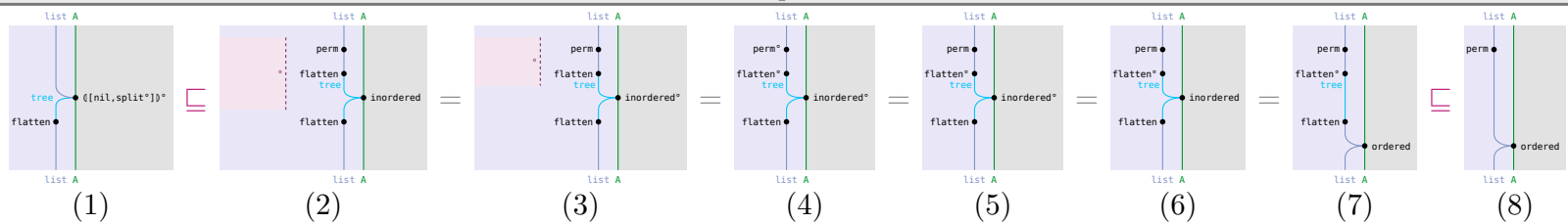
1	$[[\text{nil}, \text{select}^\circ]]^\circ$	unfolding the converse of a fold
2	$= (\text{nil}^\circ \text{nil}) \cup (\text{select} (1 \times ([[\text{nil}, \text{select}^\circ]]^\circ)^\circ) \text{cons})$	

§6.6.3. Quicksort

$$(\forall a. \forall b. \forall c. R(a,b) \Rightarrow R(b,c) \Rightarrow R(a,c)) \wedge \text{split}^\circ \sqsubseteq \text{check}' \text{join perm} \Rightarrow ([[\text{nil}, \text{split}^\circ]]^\circ)^\circ \text{flatten} \sqsubseteq \text{perm ordered} \quad (6.6.3a)$$

flatten $\sqsubseteq$ **perm ordered**

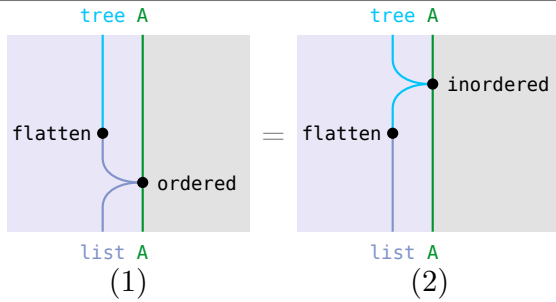
every output of unfolding the input by **split** into a tree and flattening that tree is a sorted permutation of the input



1	$[[\text{nil}, \text{split}^\circ]]^\circ \text{flatten}$	$\text{split}^\circ \sqsubseteq \text{check}' \text{join perm}$ $\Rightarrow ([[\text{nil}, \text{split}^\circ]]^\circ)^\circ \text{flatten} \sqsubseteq (\text{inordered flatten perm})^\circ \text{flatten}$
2	$\sqsubseteq (\text{inordered flatten perm})^\circ \text{flatten}$	$(R S)^\circ = S^\circ R^\circ$
3	$= (\text{flatten perm})^\circ \text{inordered}^\circ \text{flatten}$	$(R S)^\circ = S^\circ R^\circ$
4	$= \text{perm}^\circ \text{flatten}^\circ \text{inordered}^\circ \text{flatten}$	$\text{perm}^\circ = \text{perm}$
5	$= \text{perm flatten}^\circ \text{inordered}^\circ \text{flatten}$	$\text{inordered}^\circ = \text{inordered}$
6	$= \text{perm flatten}^\circ \text{inordered flatten}$	$(6.6.3b) (\forall a. \forall b. \forall c. R(a,b) \Rightarrow R(b,c) \Rightarrow R(a,c))$ $\Rightarrow \text{flatten ordered} = \text{inordered flatten}$
7	$= \text{perm flatten}^\circ \text{flatten ordered}$	$\text{flatten}^\circ \text{flatten} \sqsubseteq 1$
8	$\sqsubseteq \text{perm ordered}$	

($\forall a. \forall b. \forall c. R(a,b) \Rightarrow R(b,c) \Rightarrow R(a,c)$) $\Rightarrow$ flatten ordered = inordered flatten (6.6.3b)

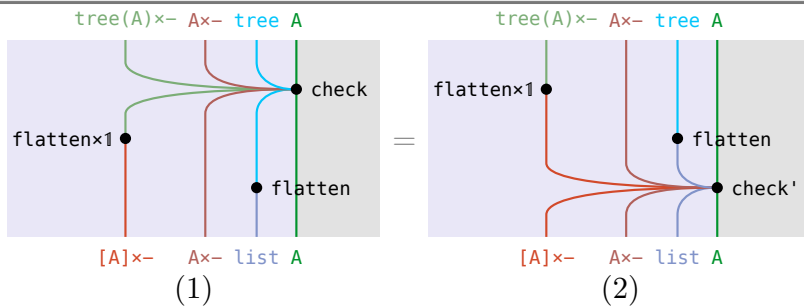
flattening a tree and then testing the list for order is the same as testing the tree with **inordered** and then flattening



1	flatten ordered	R transitive: a flattened tree is ordered iff every node passes check
2	= inordered flatten	

check (flatten $\times$ (1 $\times$ flatten)) = (flatten $\times$ (1 $\times$ flatten)) check' (6.6.3c)

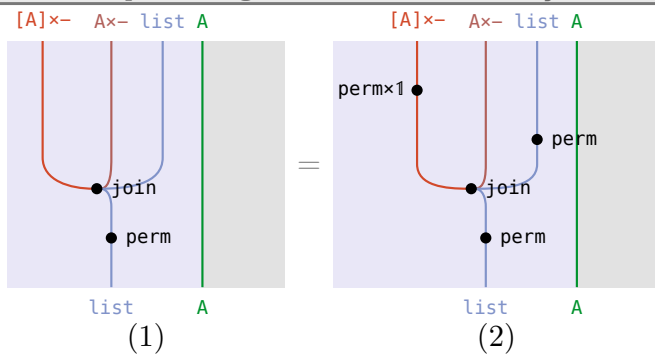
testing a fork with **check** and then flattening both subtrees equals flattening both subtrees and then testing with **check'**, since **intree** of a tree is **inlist** of its flattening



1	check (flatten $\times$ (1 $\times$ flatten))	b intree x $\Leftrightarrow$ b inlist flatten(x)
2	= (flatten $\times$ (1 $\times$ flatten)) check'	

join perm = (perm $\times$ (1 $\times$ perm)) join perm (6.6.3d)

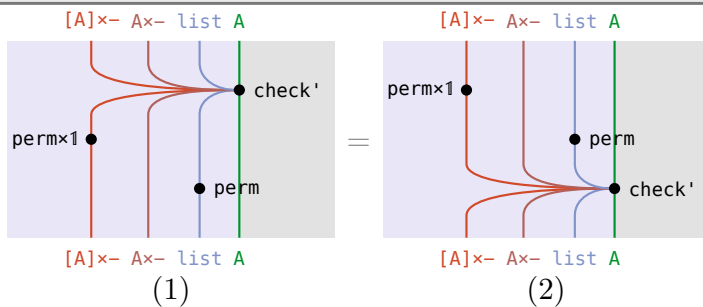
permuting the list **x ++ [a] ++ y** is the same as first permuting **x** and **y** and then permuting the joined list



1	join perm	perm is transitive and closed under ++
2	= (perm $\times$ (1 $\times$ perm)) join perm	

check' (perm $\times$ (1 $\times$ perm)) = (perm $\times$ (1 $\times$ perm)) check' (6.6.3e)

testing a fork of lists with **check'** and then permuting both parts equals permuting both parts and then testing, since permuting a list leaves its members unchanged

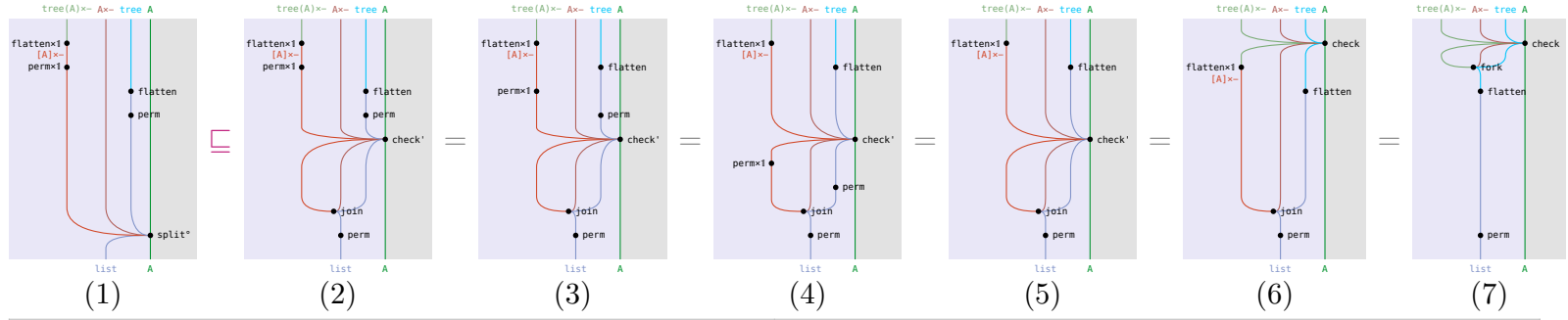


1	check' (perm $\times$ (1 $\times$ perm))	b inlist x $\Leftrightarrow$ b inlist perm(x)
2	= (perm $\times$ (1 $\times$ perm)) check'	

$\text{split}^\circ \sqsubseteq \text{check}' \text{ join perm} \Rightarrow ((\text{flatten perm}) \times (1 \times (\text{flatten perm}))) \text{ split}^\circ \sqsubseteq \text{check fork}$ (6.6.3f)

flatten perm

undoing **split** and then flattening and permuting both parts lands among the **checked** forks whose flattening is permuted

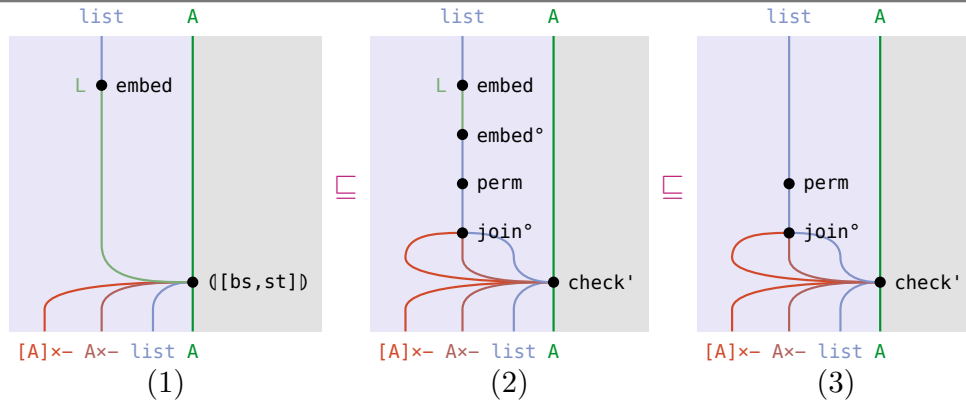


1	$((\text{flatten perm}) \times (1 \times (\text{flatten perm}))) \text{ split}^\circ$	$\text{split}^\circ \sqsubseteq \text{check}' \text{ join perm}$
2	$\sqsubseteq ((\text{flatten perm}) \times (1 \times (\text{flatten perm}))) \text{ check}' \text{ join perm}$	
3	$= (\text{flatten} \times (1 \times \text{flatten})) (\text{perm} \times (1 \times \text{perm})) \text{ check}' \text{ join perm}$	(6.6.3e) $\text{check}' (\text{perm} \times (1 \times \text{perm})) = (\text{perm} \times (1 \times \text{perm})) \text{ check}'$
4	$= (\text{flatten} \times (1 \times \text{flatten})) \text{ check}' (\text{perm} \times (1 \times \text{perm})) \text{ join perm}$	(6.6.3d) $\text{join perm} = (\text{perm} \times (1 \times \text{perm})) \text{ join perm}$
5	$= (\text{flatten} \times (1 \times \text{flatten})) \text{ check}' \text{ join perm}$	(6.6.3c) $\text{check} (\text{flatten} \times (1 \times \text{flatten})) = (\text{flatten} \times (1 \times \text{flatten})) \text{ check}'$
6	$= \text{check} (\text{flatten} \times (1 \times \text{flatten})) \text{ join perm}$	$\text{fork flatten} = (\text{flatten} \times (1 \times \text{flatten})) \text{ join}$
7	$= \text{check fork flatten perm}$	

$\text{bs} \sqsubseteq \text{wrap perm join}^\circ \text{ check}' \wedge (1 \times (\text{perm join}^\circ \text{ check}')) \text{ st} \sqsubseteq \text{cons perm join}^\circ \text{ check}' \Rightarrow$ (6.6.3g)

embed $([\text{bs}, \text{st}]) \sqsubseteq \text{perm join}^\circ \text{ check}'$

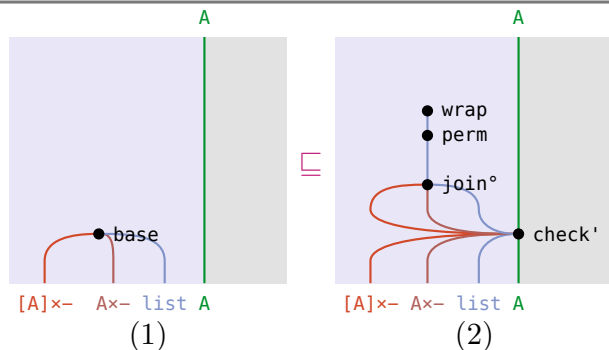
turning a list into a non-empty list and folding it with any $[\text{bs}, \text{st}]$ meeting the two conditions below gives only **checked** triples (x, a, y) whose $\text{join } x \# [a] \# y$ is a permutation of the list



1	$\text{embed } ([\text{bs}, \text{st}])$	$\text{bs} \sqsubseteq \text{wrap perm join}^\circ \text{ check}' \wedge (1 \times (\text{perm join}^\circ \text{ check}')) \text{ st} \sqsubseteq \text{cons perm join}^\circ \text{ check}'$ $\Rightarrow \text{embed } ([\text{bs}, \text{st}]) \sqsubseteq \text{embed embed}^\circ \text{ perm join}^\circ \text{ check}'$
2	$\sqsubseteq \text{embed embed}^\circ \text{ perm join}^\circ \text{ check}'$	$\text{embed embed}^\circ \text{ perm join}^\circ \text{ check}' \sqsubseteq \text{perm join}^\circ \text{ check}'$
3	$\sqsubseteq \text{perm join}^\circ \text{ check}'$	

base $\sqsubseteq \text{wrap perm join}^\circ \text{ check}'$ (6.6.3h)

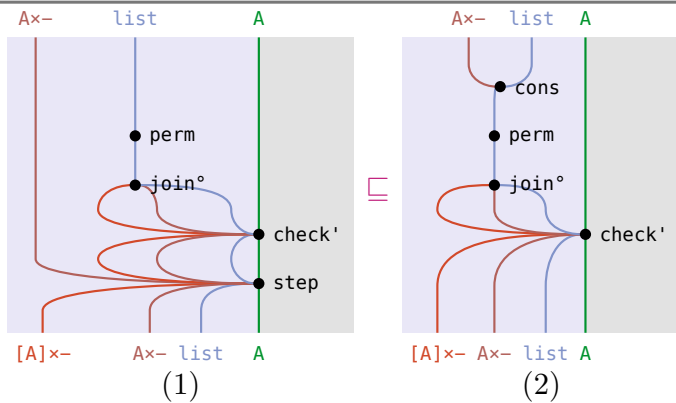
base(a) is a **checked** triple whose join is a permutation of the one-element list $[a]$



1	base	$[\] \# [a] \# [\] = [a]$, and check' holds on empty lists
2	$\sqsubseteq \text{wrap perm join}^\circ \text{ check}'$	

connected(R) $\Rightarrow$ (1 $\times$ (perm join $^\circ$ check')) step $\sqsubseteq$ cons perm join $^\circ$ check' (6.6.3i)

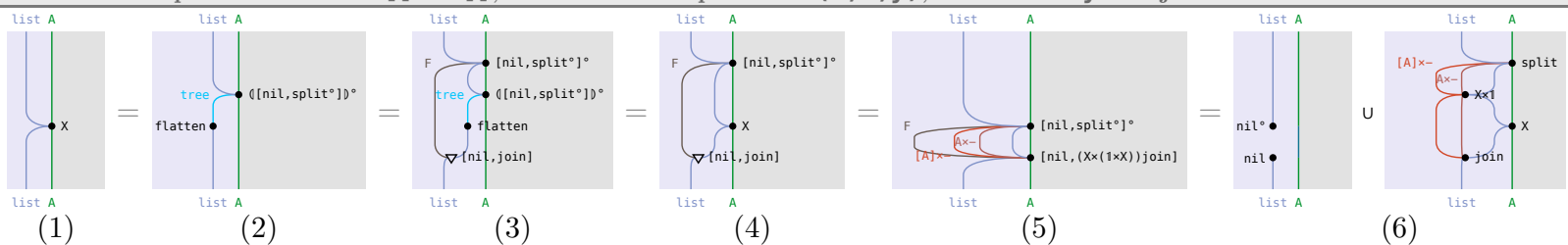
if (x,b,y) is a **checked** triple for a permutation of **l**, then **step(a, (x,b,y))** is a **checked** triple for a permutation of **[a] # l**



1	(1 $\times$ (perm join $^\circ$ check')) step	aRb puts a in front of x; otherwise bRa and a goes in front of y
2	$\sqsubseteq$ cons perm join $^\circ$ check'	

X = ([nil,split $^\circ$]) $^\circ$ flatten $\Rightarrow$ X = (nil $^\circ$ nil) u (split (X $\times$ (1 $\times$ X)) join) (6.6.3j)

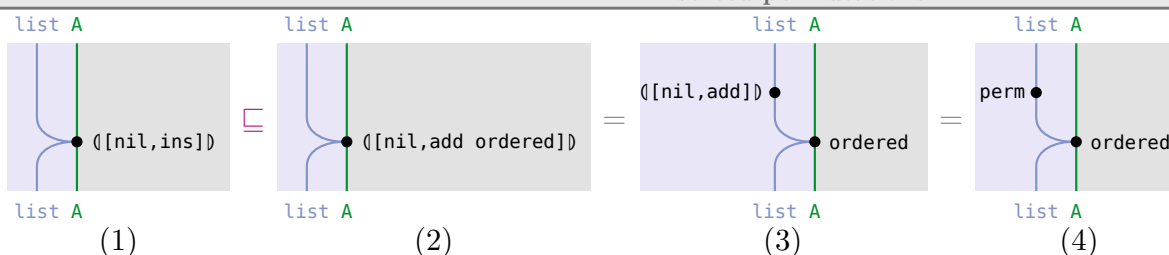
quicksort returns [] on [], and otherwise splits into (x,a,y), sorts x and y and joins them around a



1	X	X = ([nil,split $^\circ$]) $^\circ$ flatten
2	= ([nil,split $^\circ$]) $^\circ$ flatten	S $^\circ$ F((S) $^\circ$ (R)) R = (S) $^\circ$ (R)
3	= [nil,split $^\circ$] $^\circ$ F([nil,split $^\circ$] $^\circ$ flatten) [nil,join]	X = ([nil,split $^\circ$]) $^\circ$ flatten
4	= [nil,split $^\circ$] $^\circ$ F(X) [nil,join]	F(S) [T,U] = [T,(S $\times$ (1 $\times$ S)) U]
5	= [nil,split $^\circ$] $^\circ$ [nil,(X $\times$ (1 $\times$ X)) join]	[U,V] $^\circ$ [R,S] = (U $^\circ$ R) u (V $^\circ$ S)
6	= (nil $^\circ$ nil) u (split (X $\times$ (1 $\times$ X)) join)	

(1 $\times$ ordered) ins $\sqsubseteq$ add ordered $\Rightarrow$ ([nil,ins]) $\sqsubseteq$ perm ordered (6.6.3k)

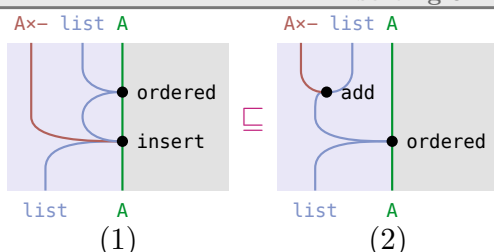
for any **ins** that, on an ordered list, returns only ordered results of **add**, folding with **[nil, ins]** gives only sorted permutations



1	[nil,ins]	(1 $\times$ ordered) ins $\sqsubseteq$ add ordered
2	$\sqsubseteq$ [nil,add ordered]	$\Rightarrow$ ([nil,ins]) $\sqsubseteq$ ([nil,add ordered])
3	= [nil,add] ordered	[nil,add] ordered = [nil,add ordered]
4	= perm ordered	perm = [nil,add]

connected(R) $\wedge$ ($\forall a. \forall b. \forall c. R(a,b) \Rightarrow R(b,c) \Rightarrow R(a,c)$) $\Rightarrow$ (1 $\times$ ordered) insert $\sqsubseteq$ add ordered (6.6.3l)

inserting **a** into an ordered list puts **a** somewhere in it and keeps it ordered



1	(1 $\times$ ordered) insert	insert splices a in, and R transitive and connected
2	$\sqsubseteq$ add ordered	

§6.7. Closure

$$R^* = (\mu X : 1 \cup (X R)) \quad (6.7a)$$

closing R by composing it on the left and closing it by composing on the right give the same relation R^*

$$1 \in X \wedge X X \subseteq X \implies R \subseteq X \iff R^* \subseteq X \quad (6.7b)$$

X contains R exactly when it contains R^*

$$1 \in R^* \quad (6.7c)$$

R^* is reflexive

1	1	$R \subseteq R \cup S$
2	$\sqsubseteq 1 \cup (R R^*)$	$1 \cup (R R^*) = R^*$
3	$= R^*$	

$$R \subseteq R^* \quad (6.7d)$$

R^* contains R

1	R	
2	$= R$	$(6.7c) \ 1 \in R^*$
3	$\sqsubseteq R R^*$	$R R^* \subseteq R^*$
4	$\sqsubseteq R^*$	

$$(1 \cup (R (R^* / R^*))) R^* \subseteq R^* \quad (6.7e)$$

R^*/R^* is a prefixed point of $X \mapsto 1 \cup RX$, stated through division

$$R^* R^* \subseteq R^*$$

R^* is transitive, because R^*/R^* is a prefixed point of $X \mapsto 1 \cup RX$

1	$(1 \cup (R (R^* / R^*))) R^*$	$(S \cup T) R = (S R) \cup (T R)$
2	$= (R^*) \cup (R (R^* / R^*) R^*)$	
3	$= R^* \cup (R (R^* / R^*) R^*)$	$(R / R) R \subseteq R$
4	$\sqsubseteq R^* \cup (R R^*)$	$R R^* \subseteq R^*$
5	$\sqsubseteq R^*$	

$$1 \in X \wedge X X \subseteq X \wedge R \subseteq X \implies 1 \cup (R X) \subseteq X \quad (6.7f)$$

a reflexive transitive X containing R is a prefixed point of $Y \mapsto 1 \cup RY$

$$1 \in X \wedge X X \subseteq X \wedge R \subseteq X \implies R^* \subseteq X$$

a reflexive transitive X containing R is a prefixed point of $X \mapsto 1 \cup RX$, so it contains R^*

1	$1 \cup (R X)$	$1 \in X$
2	$\sqsubseteq X \cup (R X)$	$R \subseteq X$
3	$\sqsubseteq X \cup (X X)$	
4	$\sqsubseteq X$	

$$\frac{R^*}{\exists} = \left(\frac{1}{\exists}, \frac{R R^*}{\exists} \right) \text{ cup} \quad (6.7g)$$

the set of R^* -successors of a is a itself joined with the R^* -successors of its R -successors

1	$\frac{R^*}{\exists}$	$1 \cup (R R^*) = R^*$
2	$= \frac{1 \cup (R R^*)}{\exists}$	$\frac{R \cup S}{\exists} = \left(\frac{R}{\exists}, \frac{S}{\exists} \right) \text{ cup}$
3	$= \left(\frac{1}{\exists}, \frac{R R^*}{\exists} \right) \text{ cup}$	

§6.7.1. Computing closure

$R - \emptyset = R$	(6.7.1a)
taking nothing away leaves R	
$R \cup (S - R) = R \cup S$	
beside R , only the part of S outside R adds anything	
$R - (S \cup T) = R - S - T$	
taking away S and then T takes away $S \cup T$	
$(R \cup S) - T = (R - T) \cup (S - T)$	
taking T away from a union takes it away from each part	

$\text{monotonic}(\varphi) \wedge \text{monotonic}(\psi) \implies (\mu X : \varphi(\psi(X))) = \varphi(\mu Y : \psi(\varphi(Y)))$	(6.7.1b)
the least fixed point of φ after ψ is φ applied to the least fixed point of ψ after φ	

$S R^* = (\mu X : S \cup (X R))$	(6.7.1c)
S followed by any number of R steps is the least X containing S and closed under a further R step	
$R^* S = (\mu X : S \cup (R X))$	
any number of R steps followed by S is the least X containing S and closed under an R step in front	

$\theta(\emptyset, S) = S R^*$	(6.7.1d)
started with nothing found and S to explore, θ computes $S R^*$	
1 $\theta(\emptyset, S)$	$\theta(P, Q) \triangleq P \cup (\mu X : Q \cup ((X R) - P))$
2 $\triangleq \emptyset \cup (\mu X : S \cup ((X R) - \emptyset))$	(6.7.1a) $R - \emptyset = R$
3 $= \emptyset \cup (\mu X : S \cup (X R))$	$\emptyset \cup R = R$
4 $= (\mu X : S \cup (X R))$	(6.7.1c) $S R^* = (\mu X : S \cup (X R))$
5 $= S R^*$	

$\theta(P, \emptyset) = P$	(6.7.1e)
with nothing left to explore, θ returns what it has found	
1 $\theta(P, \emptyset)$	$\theta(P, Q) \triangleq P \cup (\mu X : Q \cup ((X R) - P))$
2 $\triangleq P \cup (\mu X : \emptyset \cup ((X R) - P))$	$(\mu X : \emptyset \cup ((X R) - P)) = \emptyset$
3 $= P \cup \emptyset$	$R \cup \emptyset = R$
4 $= P$	

§7. Optimisation Problems

§7.1. Minimum and maximum

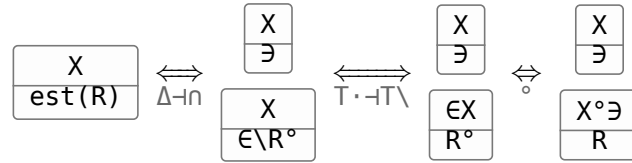
$\text{est}(R) \triangleq \exists n \ (\in \setminus R^\circ) \text{ — } x \text{ is the } \text{est}(R) \text{ of } xs \text{ when } x \in xs \text{ and } x R y \text{ for every } y \in xs; \text{ B\&dM call it } \min R.$	(7.1a)
$\text{est}(\leq) \ xs \ (\text{est}(\leq)) \ x \iff x \in xs \wedge (\forall y \in xs. \ x \leq y).$	

(7.1b)

	the law	what it says
1	$X \sqsubseteq \text{est}(R) \iff X \sqsubseteq \exists \text{ and } X^\circ \exists \sqsubseteq R$	in the set, and below every element of it
2	$\frac{1}{\exists} (\epsilon \setminus R) = R$	bounding a singleton is bounding its element
3	$\frac{S}{\exists} (\epsilon \setminus R) = S^\circ \setminus R$	bound S's image without building the set
4	$\text{union} \triangleq \frac{\exists \exists}{\exists}$	flattens a set of sets
5	$\text{union} (\epsilon \setminus R) = \epsilon \setminus (\epsilon \setminus R)$	bound a union by bounding each member set
6	$\frac{1}{\exists} \text{est}(R) = 1 \sqcap R^\circ$ (7.4)	a singleton's minimum is its element, where R is reflexive $\frac{S}{\exists} \text{est}(R)$ at $S := 1$
7	$\frac{S}{\exists} \text{est}(R) = S \sqcap (S^\circ \setminus R^\circ)$	an S-value that points to every S-value
8	$\frac{S}{\exists} \text{est}(R \sqcap (S^\circ S)) = \frac{S}{\exists} \text{est}(R)$	only R between values S gives one argument counts — context
9	$E(S) \text{est}(R) = (\exists S) \sqcap ((\exists S)^\circ \setminus R^\circ)$ (7.7)	the same for the image of a set $\frac{S}{\exists} \text{est}(R)$ at $S := \exists S$
10	$\text{Map}(f) \implies P(f) \text{est}(R) = \text{est}(f R f^\circ) f$	shunt a function through a minimum
11	$P(S) \text{est}(R) = (\exists S) \sqcap (\epsilon \setminus (SR^\circ))$ (7.9) R reflexive	fusion with the power relator $\exists$ is the only proof here that tabulates
12	$P(S) \text{est}(R) \sqsubseteq (\exists S) \sqcap (\epsilon \setminus (SR^\circ))$	the half of the row above that costs nothing
13	$P(\text{est}(R)) \text{est}(R) \sqsubseteq \text{union} \text{est}(R)$ R transitive	a minimum in each set, then a minimum of those
14	$R R \sqsubseteq R \implies P(\text{est}(R)) \text{est}(R) = P(\text{dom}(\text{est}(R))) \text{union} \text{est}(R)$ (7.12)	the same as an equality, once empty sets are dropped

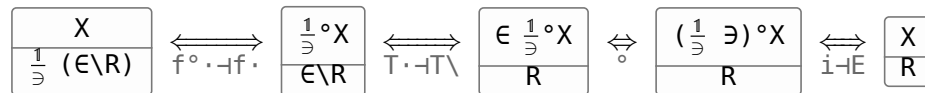
§7.1.1. $X \sqsubseteq \text{est}(R) \iff X \sqsubseteq \exists \wedge \epsilon \in X \sqsubseteq R^\circ$ — X refines est(R) when it picks a member R-related to every member

(7.1.1a)



§7.1.2. $\frac{1}{\exists} (\epsilon \setminus R) = R$ — all of {a} relates by R to c exactly when a does

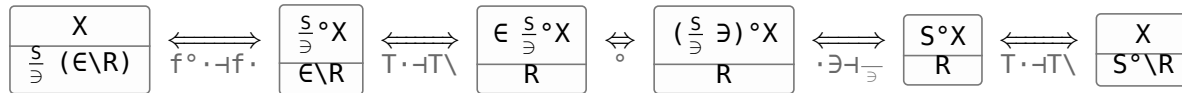
(7.1)



§7.1.3. $\frac{S}{\exists} (\epsilon \setminus R) = S^\circ \setminus R$ — gathering the S-values into one set first changes nothing

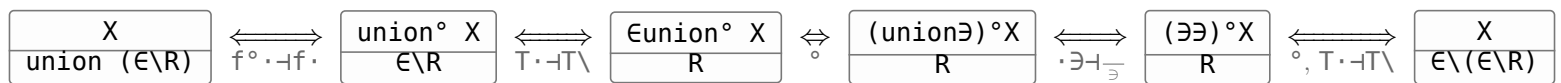
$\frac{S}{\exists}$ gathers the S-image of a point into one set and $\epsilon \setminus R$ asks that every member of that set be R-related to the target, so the set cancels and $S^\circ \setminus R$ asks it of the S-image directly.

(7.2)



§7.1.4. $\text{union} (\epsilon \setminus R) = \epsilon \setminus (\epsilon \setminus R)$ — every member of the union relates by R iff every set's members do

(7.3)



§7.1.5. $\frac{S}{\exists} \text{est}(R) = S \sqcap (S^\circ \setminus R^\circ)$ — the smallest S-value is an S-value R-related to every S-value

$$\begin{array}{c}
 \boxed{\begin{array}{c} X \\ \exists \\ \text{est}(R) \end{array}} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \boxed{\begin{array}{c} S^\circ X \\ \exists \\ \text{est}(R) \end{array}} \xleftrightarrow{\Delta \neg n} \boxed{\begin{array}{c} S^\circ X \\ \exists \\ \exists \\ \epsilon \setminus R^\circ \end{array}} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \boxed{\begin{array}{c} X \\ \exists \\ \exists \\ \epsilon \setminus R^\circ \end{array}} \xleftrightarrow{\cdot \exists \neg \exists, (7.2) \frac{S}{\exists} (\epsilon \setminus R) = S^\circ} \boxed{\begin{array}{c} X \\ S \\ \exists \\ S^\circ \setminus R^\circ \end{array}} \xleftrightarrow{\Delta \neg n} \boxed{\begin{array}{c} X \\ S n (S^\circ \setminus R^\circ) \end{array}}
 \end{array}
 \quad (7.5)$$

§7.1.6. $\frac{S}{\exists} \text{est}(R \cap (S^\circ S)) = \frac{S}{\exists} \text{est}(R)$ — only R between S -values of one argument matters for the smallest

$$\begin{array}{c}
 \boxed{\begin{array}{c} X \\ \exists \\ \text{est}(R n S^\circ S) \end{array}} \xleftrightarrow{(7.5) \frac{S}{\exists} \text{est}(R) = S n (S^\circ \setminus R^\circ), \circ} \boxed{\begin{array}{c} X \\ S n (S^\circ \setminus (R^\circ n S^\circ S)) \end{array}} \xleftrightarrow{\Delta \neg n, T \cdot \neg T \setminus} \boxed{\begin{array}{c} X \\ S \\ S^\circ X \\ R^\circ n S^\circ S \end{array}} \\
 \xleftrightarrow{\Delta \neg n, S^\circ \cdot \text{monotone}} \boxed{\begin{array}{c} X \\ S \\ S^\circ X \\ R^\circ \end{array}} \xleftrightarrow{T \cdot \neg T \setminus, \Delta \neg n} \boxed{\begin{array}{c} X \\ S n (S^\circ \setminus R^\circ) \end{array}} \xleftrightarrow{(7.5) \frac{S}{\exists} \text{est}(R) = S n (S^\circ \setminus R^\circ)} \boxed{\begin{array}{c} X \\ \exists \\ \text{est}(R) \end{array}}
 \end{array}
 \quad (7.6)$$

§7.1.7. $\text{Map}(f) \Rightarrow P(f) \text{est}(R) = \text{est}(f R f^\circ) f$ — the smallest f -image is f of the smallest under $f R f^\circ$

$$\begin{array}{c}
 \boxed{\begin{array}{c} P(f) \text{est}(R) \\ f \text{ a map} \end{array}} \xrightarrow{P=E \text{ on maps, } (7.5) \frac{S}{\exists} \text{est}(R) = S n (S^\circ \setminus R^\circ)} \boxed{(\exists f) n ((\exists f)^\circ \setminus R^\circ)} \xrightarrow{\circ, T \cdot \neg T \setminus, f^\circ \cdot \neg f^\circ} \boxed{(\exists f) n (\epsilon \setminus (f R^\circ))} \\
 \xrightarrow{\text{modular law, } f \text{ simple}} \boxed{(\exists n ((\epsilon \setminus (f R^\circ)) f^\circ)) f} \xrightarrow{\cdot f \neg f^\circ, \circ, \text{est}} \boxed{\text{est}(f R f^\circ) f}
 \end{array}
 \quad (7.8)$$

§7.1.8. $P(S) \text{est}(R) \sqsubseteq (\exists S) n (\epsilon \setminus (S R^\circ))$ — the smallest of the S -image is R -below an S -value of every member

$$\begin{array}{c}
 \boxed{\begin{array}{c} P(S) \text{est}(R) \\ (\exists S) n (\epsilon \setminus (S R^\circ)) \end{array}} \xleftrightarrow{\Delta \neg n, T \cdot \neg T \setminus} \boxed{\begin{array}{c} P(S) \text{est}(R) \\ \exists S \end{array}} \xleftrightarrow{\text{UP of est}} \boxed{\begin{array}{c} P(S) \exists \\ \exists S \end{array}} \\
 \boxed{\begin{array}{c} \epsilon P(S) \text{est}(R) \\ S R^\circ \end{array}} \xleftrightarrow{\text{UP of est}} \boxed{\begin{array}{c} \epsilon P(S) \\ S \epsilon \end{array}}
 \end{array}
 \quad (7.10)$$

§7.1.9. $R R \sqsubseteq R \Rightarrow P(\text{est}(R)) \text{est}(R) \sqsubseteq \text{union est}(R)$ — the smallest of the smallests is a smallest of the union

$$\begin{array}{c}
 \boxed{\begin{array}{c} P(\text{est}(R)) \\ \text{est}(R) \\ \text{union est}(R) \\ R \text{ transitive} \end{array}} \xleftrightarrow{(7.5) \frac{S}{\exists} \text{est}(R) = S n (S^\circ \setminus R^\circ)} \boxed{\begin{array}{c} P(\text{est}(R)) \\ \text{est}(R) \\ (\exists \exists) n ((\exists \exists)^\circ \setminus R^\circ) \end{array}} \xleftrightarrow{\circ, \Delta \neg n, T \cdot \neg T \setminus} \boxed{\begin{array}{c} P(\text{est}(R)) \\ \text{est}(R) \\ \exists \exists \end{array}} \\
 \boxed{\begin{array}{c} \epsilon \epsilon P(\text{est}(R)) \\ \text{est}(R) \\ R^\circ \end{array}} \xleftrightarrow{\text{UP of est, } R \text{ transitive}} \boxed{\begin{array}{c} P(\text{est}(R)) \exists \\ \exists \text{est}(R) \end{array}} \\
 \boxed{\begin{array}{c} \epsilon P(\text{est}(R)) \\ \text{est}(R) \epsilon \end{array}}
 \end{array}
 \quad (7.11)$$

, R transitive

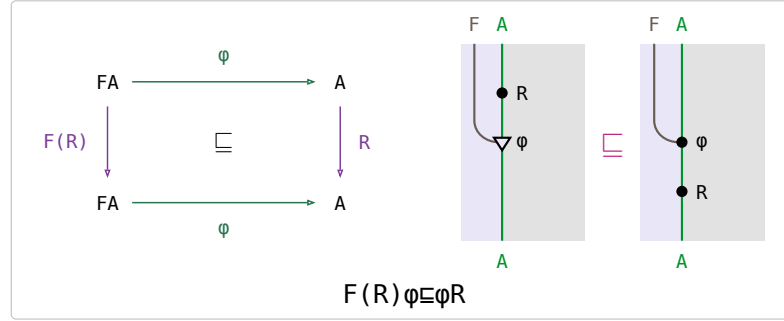
§7.2. Monotonic algebras

(7.2a)

$\varphi \text{ Pres } R \triangleq F(R) \varphi \sqsubseteq \varphi R$ — φ preserves R ; B&dM call φ **monotonic on R** .

$(\leq \times \leq) + \sqsubseteq + \leq$ — addition, $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$, preserves $\leq$: $c = a + b \wedge a \leq a' \wedge b \leq b' \implies c \leq a' + b'$.

(7.2b)

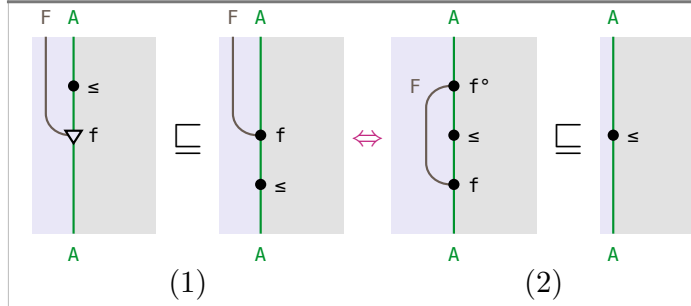


§7.2.1. f monotonic on $\leq$, read at points

(7.2.1a)

$$F(\leq) \ f \sqsubseteq f \leq \iff f^\circ F(\leq) \ f \sqsubseteq \leq$$

$F(\leq)$ then f stays below f then $\leq$ exactly when $f^\circ, F(\leq), f$ in a row stays below $\leq$.



1	$F(\leq) \ f \sqsubseteq f \leq$	Map(f) $\implies f^\circ R \sqsubseteq S \iff R \sqsubseteq f S$
2	$\iff f^\circ F(\leq) \ f \sqsubseteq \leq$	

$$\boxed{f(u)} \xrightarrow{f^\circ} \boxed{u} \xrightarrow{F(\leq)} \boxed{v} \xrightarrow{f} \boxed{f(v)}$$

- $F(\leq)$: from u to any v above it; u, v are in FA , compared component by component. This step is the premise.
- $\sqsubseteq \leq$: the two ends satisfy $f(u) \leq f(v)$, the conclusion. So $u \ F(\leq) \ v \implies f(u) \leq f(v)$: f preserves $\leq$.

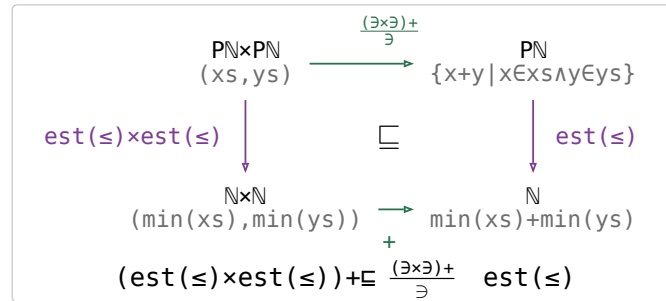
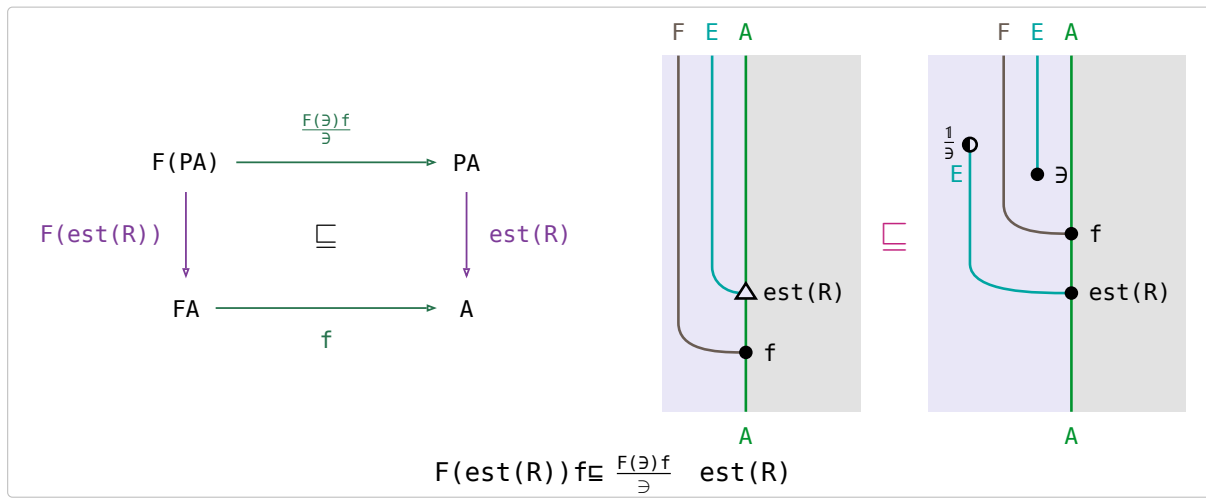
§7.2.2. Function f is monotonic on R iff it distributes over R

(7.2.2a)

$f : FA \rightarrow A$ distributes over R when $F(\text{est}(R)) f \sqsubseteq \frac{F(\exists)f}{\exists} \text{est}(R)$.

$(\text{est}(\leq) \times \text{est}(\leq)) + \sqsubseteq \frac{(\exists \times \exists) +}{\exists} \text{est}(\leq)$ — addition distributes over $\leq$: $\min(xs) + \min(ys)$ is a least element of $\{x+y \mid x \in xs \wedge y \in ys\}$.

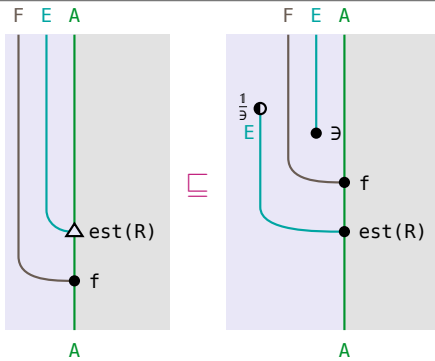
(7.2.2b)



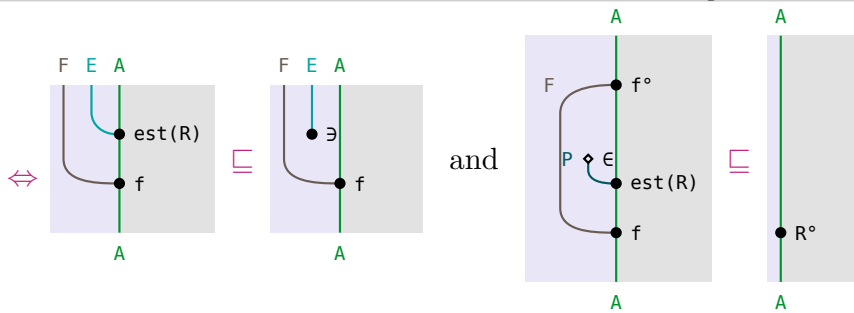
$$\text{Map}(f) \wedge 1 \sqsubseteq R \Rightarrow f^\circ F(R) f \sqsubseteq R \Leftrightarrow F(est(R)) f \sqsubseteq \frac{F(\exists)f}{\exists} est(R)$$

Theorem 7.1

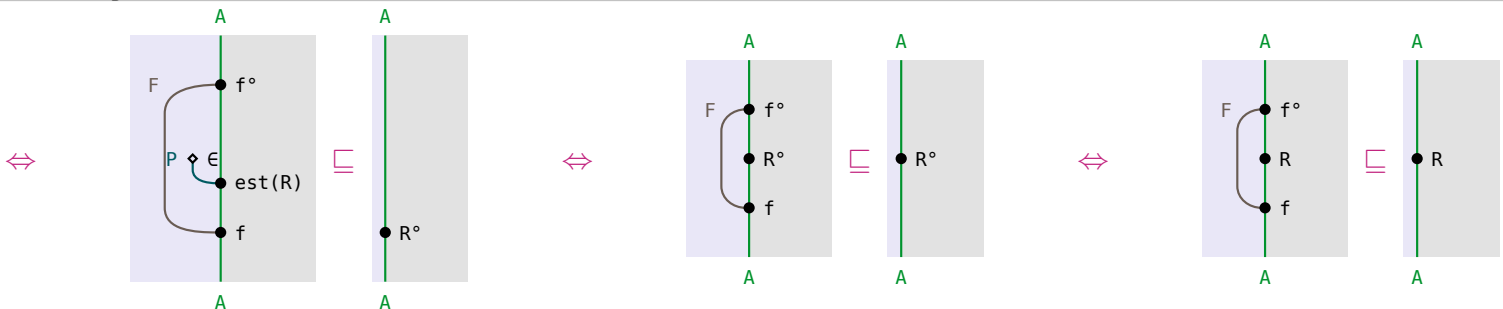
f carries R -related parts to R -related wholes ($f^\circ F(R) f \sqsubseteq R$) if and only if applying f to the R -best parts ($F(est(R))f$) is included in applying f to all parts, then taking an R -best ($F(\exists)f$ then $est(R)$)



f distributes over R — $\triangleq$ distributes — the fraction bent as $\frac{R}{\exists} \exists = R$



(7.5) $\frac{S}{\exists} est(R) = S \cap (S^\circ \setminus R^\circ)$ splits the bound in two, $X \sqsubseteq S \setminus R \Leftrightarrow S X \sqsubseteq R$ moving $(F(\exists)f)^\circ$ across



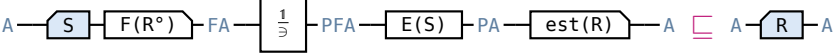
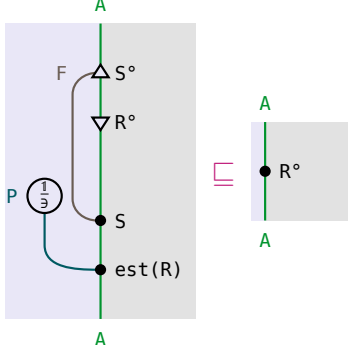
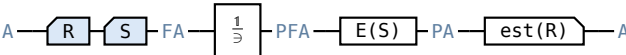
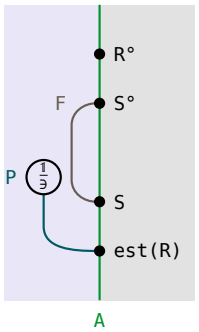
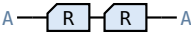
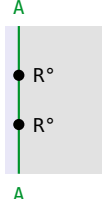
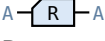
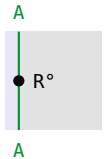
$est(R) \sqsubseteq \exists$ — $\triangleq est$ — so the first conjunct drops

$(F(\exists)f)^\circ = f^\circ F(\epsilon)$ — $\triangleq \circ$ — and $\epsilon est(R) = R^\circ$ — $\triangleq est$, R reflexive

both sides conversed — $F(R^\circ)^\circ = F(R)$, $F(R^\circ) = F(R)^\circ$

§7.2.3. $R \subseteq R \wedge S \text{ Pres } R \Rightarrow (\bigcap_{\mathcal{S}} \text{est}(R)) \subseteq (\bigcap_{\mathcal{S}} \text{est}(R))$ — taking the best at each step gives a best overall result

Theorem 7.2

circuit	Hinze–Marsden
$(\bigcap_{\mathcal{S}} \text{est}(R)) \subseteq (\bigcap_{\mathcal{S}} \text{est}(R))$ the conclusion: one minimum kept at each step is below every result collected and one minimum taken at the end	
$(\bigcap_{\mathcal{S}} \text{est}(R)) \subseteq (S)$ and $\Leftrightarrow (S)^\circ (\bigcap_{\mathcal{S}} \text{est}(R)) \subseteq R^\circ$ (7.5) $\bigcap_{\mathcal{S}} \text{est}(R) = S \cap (S^\circ \setminus R^\circ)$ at (S) ; $(\bigcap_{\mathcal{S}} \text{est}(R)) \subseteq (S)^\circ \setminus R^\circ \Leftrightarrow (S)^\circ (\bigcap_{\mathcal{S}} \text{est}(R)) \subseteq R^\circ$ — $X \subseteq S \setminus R \Leftrightarrow S \setminus X \subseteq R$	
$\Rightarrow \bigcap_{\mathcal{S}} \text{est}(R) \subseteq S$ the left conjunct; (7.5) $\bigcap_{\mathcal{S}} \text{est}(R) = S \cap (S^\circ \setminus R^\circ)$, and $(-)$ is monotone	
 $\Rightarrow$ the right conjunct; $(S)^\circ (\bigcap_{\mathcal{S}} \text{est}(R))$ is the least X with $X = S^\circ F(X) (\bigcap_{\mathcal{S}} \text{est}(R))$ hylomorphism theorem — so Knaster–Tarski leaves this one inequation $\bigcap_{\mathcal{S}} = \frac{1}{3} E(S) \text{ — } \frac{R}{3} \ni = R$	
 $S^\circ F(R^\circ) \subseteq R^\circ S^\circ \text{ — } \triangleq \text{ monotonic at } S, \text{ conversed; } F(R)^\circ = F(R^\circ) \text{ — } F(R^\circ) = F(R)^\circ$	
 $S^\circ (\bigcap_{\mathcal{S}} \text{est}(R)) \subseteq S^\circ (S^\circ \setminus R^\circ) \subseteq R^\circ \text{ — } (7.5) \bigcap_{\mathcal{S}} \text{est}(R) = S \cap (S^\circ \setminus R^\circ), (0.9a.4)$	
 R transitive	

§7.2.4. Coreflexive(p) $\Rightarrow$ $\text{takewhile}(p) = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \rightarrow \text{nil})])$ — $\text{takewhile}(p)$ keeps items while p holds, as one fold

(7.2.4a)

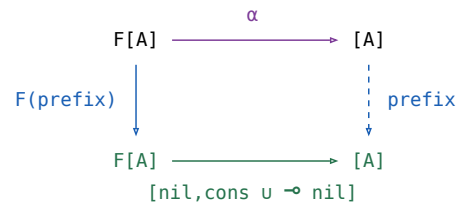
	definition	type	example and in words
1	$FX = L + E \times X$	$\mathcal{A} \rightarrow \mathcal{A}$	nothing, or a head and a tail
2	$\text{nil} \triangleq []$	$1 \rightarrow [A]$	the empty list — the first constructor of $[A]$
3	$\text{cons}(a, [x_1, \dots, x_n]) \triangleq [a, x_1, \dots, x_n]$	$A \times [A] \rightarrow [A]$	$\text{cons}(3, [1, 2]) = [3, 1, 2]$ — a head onto a tail, the second constructor
4	$\alpha \triangleq [\text{nil}, \text{cons}]$	$F[A] \rightarrow [A]$	both constructors as one map
5	$p \sqsubseteq 1$	$A \rightarrow A$	$p \triangleq \text{even}$ — $2 \leq p \leq 2$, and $3 \notin \text{dom}(p)$ — a coreflexive: a passes p when $a \leq p \leq a$
6	$R \triangleq \text{length} \leq \text{length}^\circ$	$[A] \rightarrow [A]$	$[1] R [1, 2]$ — $xs R ys \iff \text{length}(xs) \leq \text{length}(ys)$
7	$(\rightarrow \text{nil})(a, x) \triangleq \text{nil}$	$A \times [A] \rightarrow [A]$	$(\rightarrow \text{nil})(3, [1, 2]) = \text{nil}$ — drop the pair, return nil
8	$\text{prefix} = ([\text{nil}, \text{cons } u \rightarrow \text{nil}])$	$[A] \rightarrow [A]$	$[3, 1, 2] \text{ prefix } [3, 1]$ — $xs \text{ prefix } ys \iff \exists zs. xs = ys \# zs$ — at each cons , keep the head or stop
9	$S \triangleq [\text{nil}, ((p \times 1) \text{ cons}) u \rightarrow \text{nil}]$	$F[A] \rightarrow [A]$	$(4, [2]) S [4, 2]$, and $(3, [2]) S \text{ nil}$ only — prefix 's algebra with one extra p — keep a head that passes p , or stop
10	$(X \rightarrow R, S) \triangleq \text{guard}(C, X) [R, S]$	$A \rightarrow B$	the test X picks the branch: R where X holds, S where it does not
11	$\text{nil } R = \tau, \text{ nil } R^\circ = \text{nil}$		nil is the shortest list — below every list, and above only itself, so it loses every $\text{est}(R^\circ)$

prefix = ([nil, cons u → nil])

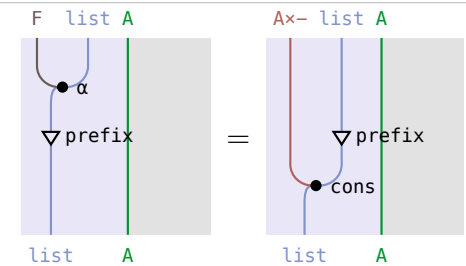
(7.2.4b)

the fold whose algebra, at each **cons**, keeps the head or stops with **nil**: **xs prefix ys $\iff \exists zs. xs = ys \# zs$**

the defining square $\alpha \text{ prefix} = F(\text{prefix})[\text{nil}, \text{cons } u \rightarrow \text{nil}]$ — build the list and then take a prefix, or take a prefix of the tail and then rebuild with the algebra



$F = 1 + A \times -$, so $F(\text{prefix}) = 1 + 1 \times \text{prefix}$: the head passes, the fold recurses on the tail alone

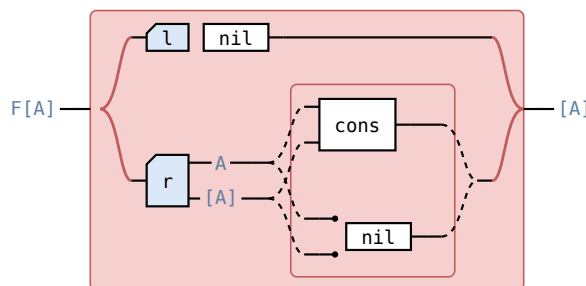
Hinze–Marsden

the **cons** operand of **cons u → nil**; $\rightarrow \text{nil}$ makes a constant and draws nothing

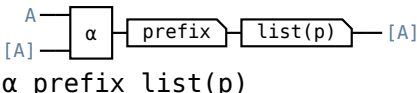
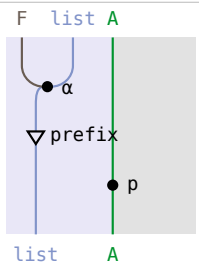
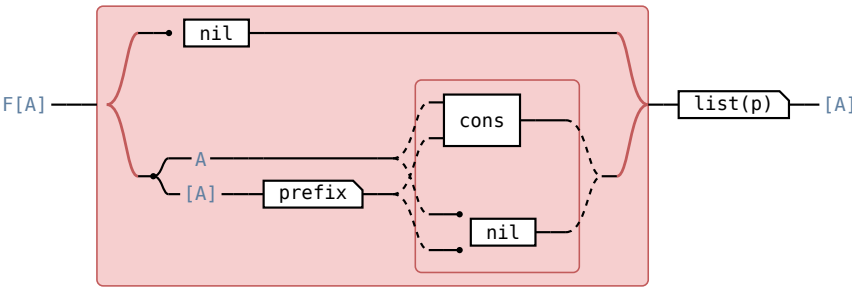
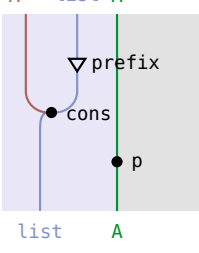
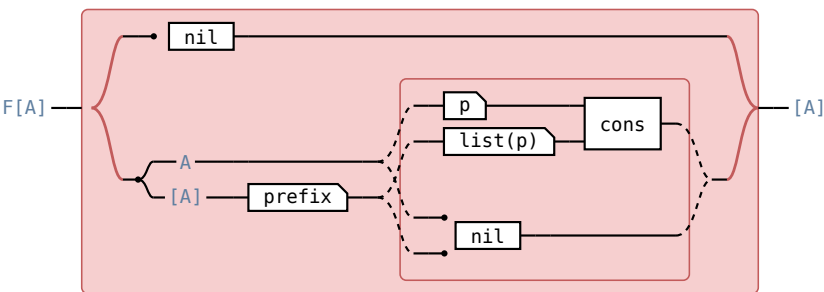
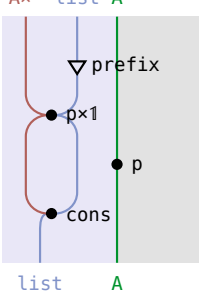
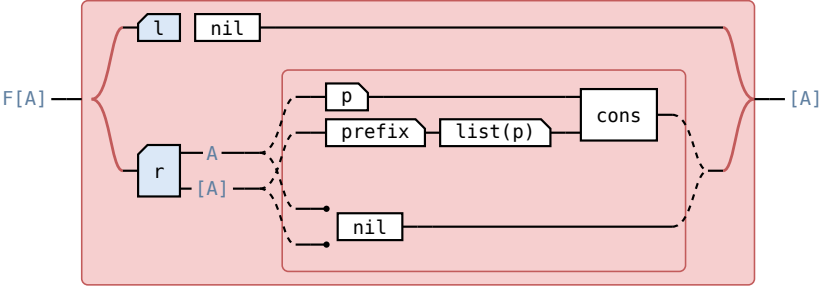
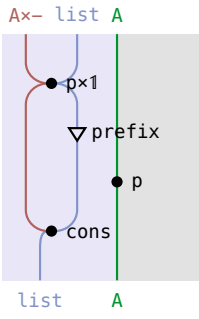
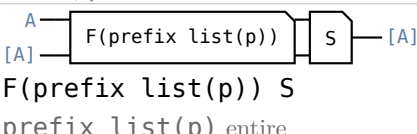
$\text{nil prefix } ys \iff ys = \text{nil}$
 $\text{cons}(a, xs') \text{ prefix } ys \iff ys = \text{nil} \vee \exists ys'. xs' \text{ prefix } ys' \wedge ys = \text{cons}(a, ys')$

a right fold: α° peels, $F(\text{prefix})$ recurses, the algebra runs on the way back; **init***
 $(\text{prefix} = ([\text{nil}, \text{cons } u \rightarrow \text{nil}]), \text{prefix} = \text{cat}^\circ \pi_1)$ is the tail-recursive form

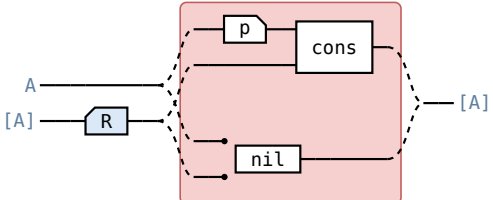
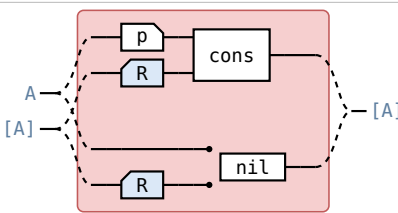
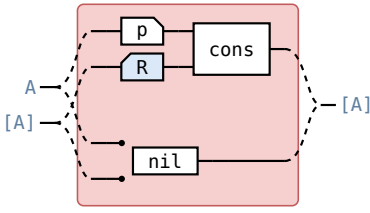
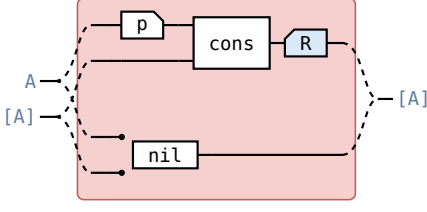
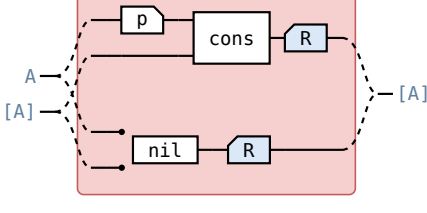
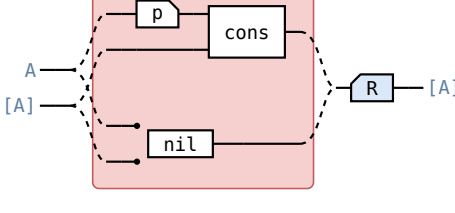
lax natural only, $\text{list}(R) \text{ prefix} \sqsubseteq \text{prefix}$
 $\text{list}(R)$: $\text{list}(R)$ first needs an R -image of every element, **prefix** first may have dropped the ones without



the algebra $[\text{nil}, \text{cons } u \rightarrow \text{nil}] : F([A]) \rightarrow [A]$ as a circuit: **nil** on the **1** branch; on a pair (a, ys') two outputs, **nil** and $\text{cons}(a, ys')$

$\text{Coreflexive}(p) \Rightarrow \alpha \text{ prefix list}(p) = F(\text{prefix list}(p)) S$ (7.2.4c)	
building the list and then keeping a p -passing prefix of it is keeping one of the tail first, and then building with S this same diagram is subseq 's: algebra $[\text{nil}, \text{cons } u \rightarrow \text{nil}]$, type $[A] \rightarrow [A]$	
circuit — the fork is $F([A]) = 1 + A \times [A]$: nil above, the pair below	Hinze–Marsden
 $\alpha \text{ prefix list}(p)$	 the cons branch alone, without $1 + \text{or } \rightarrow \text{nil}$
 $F(\text{prefix}) [\text{nil}, \text{cons } u \rightarrow \text{nil}] \text{list}(p)$ defining equation	 the cons operand of $\text{cons } u \rightarrow \text{nil}$
 $F(\text{prefix}) [\text{nil}, ((p \times \text{list}(p)) \text{cons}) u \rightarrow \text{nil}] \text{list}(p)$ list(p) through cons	 the $(p \times \text{list}(p)) \text{cons}$ operand of $(p \times \text{list}(p)) \text{cons}$ $u \rightarrow \text{nil}$
 $[\text{nil}, ((p \times (\text{prefix list}(p))) \text{cons}) u \rightarrow \text{nil}]$ relator, prefix entire	
 $F(\text{prefix list}(p)) S$ prefix list(p) entire	

$\triangleq$ reduce reads that off as $\text{prefix list}(p) = (S)$. fusion cannot: $\text{list}(p)$ is not entire, $(1 \times \text{list}(p)) \rightarrow \text{nil} \rightarrow \text{nil}$, and no algebra meets the side condition. ,

$\text{Coreflexive}(p) \Rightarrow (1 \times R^\circ) (((p \times 1) \text{ cons}) \cup \multimap \text{nil}) \sqsubseteq (((p \times 1) \text{ cons}) \cup \multimap \text{nil}) R^\circ$ (7.2.4d) shortening the tail and then taking the step lands inside taking the step and then shortening the result	
circuit — the cons branch of $F(R^\circ)S \sqsubseteq SR^\circ$	reason
 $(1 \times R^\circ) (((p \times 1) \text{ cons}) \cup \multimap \text{nil})$	
 $((p \times R^\circ) \text{ cons}) \cup ((1 \times R^\circ) \multimap \text{nil})$	each operand is reached on its own $(S \cup T) R = (S R) \cup (T R)$, $(R \cap R') / S = (R / S) \cap (R' / S)$ — and $(1 \times R^\circ)(p \times 1)$ is p and R° on the pair's two strands at once
 $((p \times R^\circ) \text{ cons}) \cup \multimap \text{nil}$	$\multimap$ is the greatest arrow into 1 , so $(1 \times R^\circ) \multimap \sqsubseteq \multimap$
 $((p \times 1) \text{ cons } R^\circ) \cup \multimap \text{nil}$	$\text{cons length} = (1 \times \text{length}) \pi_2$ succ with succ monotone — a shorter tail makes a shorter list
 $((p \times 1) \text{ cons } R^\circ) \cup (\multimap \text{nil } R^\circ)$	$\text{nil } R^\circ = \text{nil}$ (7.2.4a.11) — so the constant branch may carry the R° the other one already has
 $((((p \times 1) \text{ cons}) \cup \multimap \text{nil}) R^\circ)$	one R° past the join is the two inside it $(S \cup T) R = (S R) \cup (T R)$, $(R \cap R') / S = (R / S) \cap (R' / S)$

the nil branch, which no row above draws, is $\text{nil} \sqsubseteq \text{nil } R^\circ$.

$\text{Coreflexive}(p) \Rightarrow \frac{S}{\exists} \text{est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \multimap \text{nil})]$ (7.2.4e)	
the longest of the lists the algebra allows is the cons where the head passes p , and nil where it does not	
formula	reason
$ \begin{array}{c} \begin{array}{c} A \\ [A] \end{array} \rightarrow \boxed{\frac{1}{\exists}} \rightarrow \text{PF}[A] \rightarrow \text{E}(S) \rightarrow P[A] \rightarrow \boxed{\text{est}(R^\circ)} \rightarrow [A] \end{array} $	
$ \begin{array}{c} = \\ F[A] \rightarrow \left\{ \begin{array}{l} \begin{array}{c} \boxed{l} \end{array} \rightarrow \boxed{\frac{1}{\exists}} \rightarrow P1 \rightarrow \text{E}(\text{nil}) \rightarrow P[A] \rightarrow \boxed{\text{est}(R^\circ)} \rightarrow [A] \\ \begin{array}{c} \boxed{r} \end{array} \rightarrow \begin{array}{c} A \\ [A] \end{array} \rightarrow \boxed{\frac{1}{\exists}} \rightarrow P(A \times [A]) \rightarrow \text{E}(((p \times 1) \text{cons}) \cup \multimap \text{nil}) \rightarrow P[A] \rightarrow \boxed{\text{est}(R^\circ)} \rightarrow [A] \end{array} \right\} \end{array} $ $ \left[\frac{\text{nil}}{\exists} \text{est}(R^\circ), \frac{((p \times 1) \text{cons}) \cup \multimap \text{nil}}{\exists} \text{est}(R^\circ) \right] $	coproduct of maps
$ \begin{array}{c} = \\ F[A] \rightarrow \left\{ \begin{array}{l} \begin{array}{c} \boxed{l} \end{array} \rightarrow \text{nil} \rightarrow [A] \\ \begin{array}{c} \boxed{r} \end{array} \rightarrow \begin{array}{c} A \\ [A] \end{array} \rightarrow \boxed{\frac{1}{\exists}} \rightarrow P(A \times [A]) \rightarrow \text{E}(((p \times 1) \text{cons}) \cup \multimap \text{nil}) \rightarrow P[A] \rightarrow \boxed{\text{est}(R^\circ)} \rightarrow [A] \end{array} \right\} \end{array} $ $ \left[\text{nil}, \frac{((p \times 1) \text{cons}) \cup \multimap \text{nil}}{\exists} \text{est}(R^\circ) \right] $	singleton, R° reflexive
$ \begin{array}{c} = \\ F[A] \rightarrow \left\{ \begin{array}{l} \text{nil} \rightarrow [A] \\ \begin{array}{c} A \\ [A] \end{array} \rightarrow (\pi_1 p \rightarrow \text{cons}, \multimap \text{nil}) \rightarrow [A] \end{array} \right\} $	$\text{nil } R = T$

the set is $\{\text{nil}\}$ where p fails on the head and $\{\text{nil}, \text{cons}(a, xs)\}$ where it holds, and nil loses the second — $\triangleq \text{est}$ at a two-element set.

(7.2.4f)

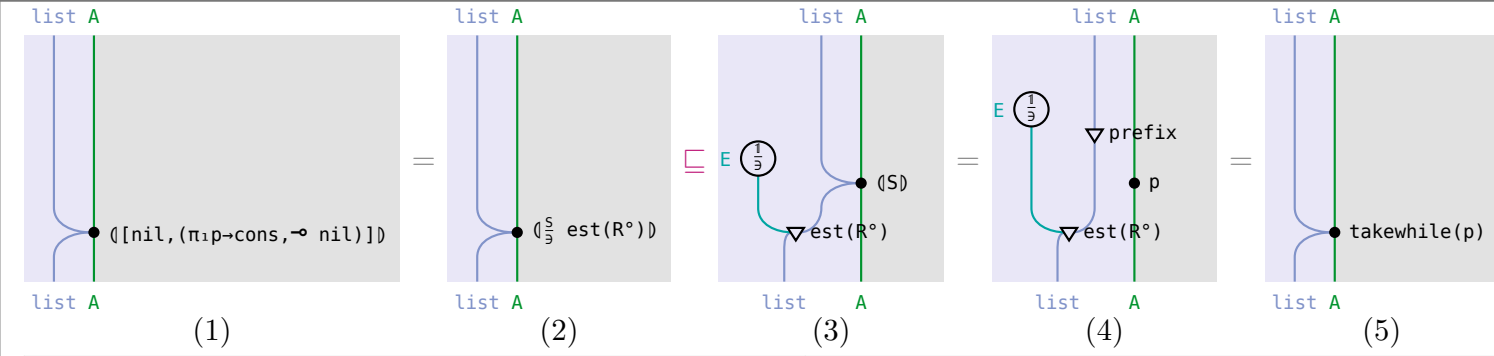
definition	type	example and in words
$S \triangleq [\text{nil}, ((p \times 1) \text{cons}) \cup \multimap \text{nil}]$	$F[A] \rightarrow [A]$	$(4, [2]) \ S \ [4, 2]$, and $(3, [2]) \ S \ \text{nil}$ only — prefix's algebra with one extra p — stop, or keep a head that passes p

$$\text{takewhile}(p) \triangleq \frac{\text{prefix list}(p)}{\exists} \text{est}(R^\circ)$$

Exercise 7.39

$$\text{Coreflexive}(p) \Rightarrow \text{takewhile}(p) = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})])$$

takewhile: $\text{takewhile}(p)(x)$ returns the longest prefix of x with the property that all its elements satisfy p ; the catamorphism is the standard implementation.



1	$[\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]$	(7.2.4e) $\text{Coreflexive}(p)$
2	$= ([\frac{S}{\exists} \text{est}(R^\circ)])$	$\Rightarrow \frac{S}{\exists} \text{est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]$
3	$\sqsubseteq [\frac{(S)}{\exists} \text{est}(R^\circ)]$	$\text{Coreflexive}(p)$
4	$= \frac{\text{prefix list}(p)}{\exists} \text{est}(R^\circ)$	$\Rightarrow ([\frac{S}{\exists} \text{est}(R^\circ)]) \sqsubseteq [\frac{(S)}{\exists} \text{est}(R^\circ)]$
5	$= \text{takewhile}(p)$	$\text{Coreflexive}(p)$
		$\Rightarrow \text{prefix list}(p) = (S)$

$$\text{takewhile}(p)^\circ \text{takewhile}(p) \sqsubseteq \text{prefix}^\circ \text{prefix} \cap R \cap R^\circ \sqsubseteq 1$$

$\text{takewhile}(p) \sqsubseteq \text{prefix list}(p)$ and $(\text{prefix list}(p))^\circ \text{takewhile}(p) \sqsubseteq R$ — (7.5) $\frac{S}{\exists} \text{est}(R) = S \cap (S^\circ \setminus R^\circ)$ at $\text{est}(R^\circ)$ — and two prefixes of one list of equal length are equal, so $\text{takewhile}(p)$ is simple: the longest, not a longest.

$$\frac{\text{prefix list}(p)}{\exists} \text{est}(R^\circ) \text{ entire}$$

nil is always a p -prefix and R is connected on the prefixes of one list, so the longest exists.

$$X \sqsubseteq Y, X \text{ entire}, Y \text{ simple} \Rightarrow X=Y$$

$[\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]$ is a reduce of maps, hence entire — what turns the $\sqsubseteq$ above into the heading's $=$.

§7.2.5. $\text{mss} = ([k] \pi_2 \text{est}(\geq))$ — mss is a fold followed by taking the largest

(7.2.5a)

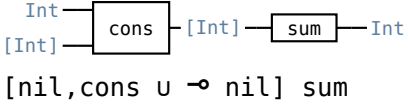
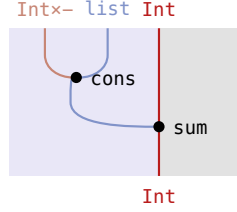
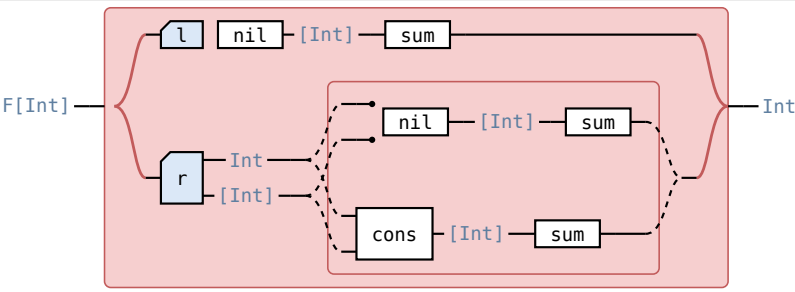
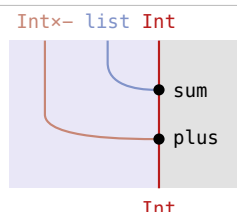
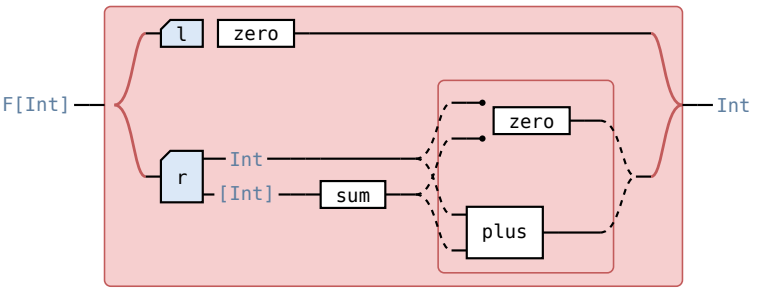
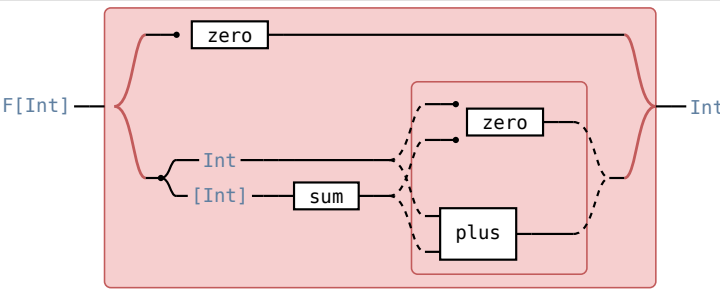
$\oplus \triangleq \frac{\neg \text{zero} \cup \text{plus}}{\exists} \text{est}(\geq)$ — $\oplus$ is the larger of 0 and $a+b$; B&dM write $\max(\wedge(\text{zero} \cup \text{plus}))$.

$\oplus \triangleq \frac{\neg \text{zero} \cup \text{plus}}{\exists} \text{est}(\geq)$ $\text{FX} = 1 + \text{Int} \times X$, $\alpha \triangleq [\text{nil}, \text{cons}]$, $\text{head} \triangleq \text{cons}^\circ \pi_1$, $\text{wrap} \triangleq (1, \neg \text{nil}) \text{cons}$.

$\text{mss} = \frac{\text{suffix}}{\exists} E\left(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)\right) \text{est}(\geq) \quad (7.2.5b)$	
maximum segment sum problem: using segment=suffix prefix , the specification is expressed in this form	
formula — one wire from [Int] to Int, its type written along it	reason
$[\text{Int}] \rightarrow \boxed{\frac{1}{\exists}} \rightarrow P[\text{Int}] \rightarrow \boxed{E(\text{segment sum})} \rightarrow P(\text{Int}) \rightarrow \boxed{\text{est}(\geq)} \rightarrow \text{Int}$	
$= [\text{Int}] \rightarrow \boxed{\frac{1}{\exists}} \rightarrow P[\text{Int}] \rightarrow \boxed{E(\text{suffix prefix sum})} \rightarrow P(\text{Int}) \rightarrow \boxed{\text{est}(\geq)} \rightarrow \text{Int}$	segment=suffix prefix segment = suffix prefix, $\triangleq$ mss
$= [\text{Int}] \rightarrow \boxed{\frac{1}{\exists}} \rightarrow \boxed{E(\text{suffix})} \rightarrow P[\text{Int}] \rightarrow \boxed{E(\text{prefix sum})} \rightarrow P(\text{Int}) \rightarrow \boxed{\text{est}(\geq)} \rightarrow \text{Int}$	absorption $\frac{S}{\exists} E(R) = \frac{S}{\exists} \frac{R}{\exists}$ — $\text{frac}(S, \exists)$ $E(R) = \text{frac}(SR, \exists)$ at $S := \text{suffix}, R := \text{prefix}$ sum
$= [\text{Int}] \rightarrow \boxed{\frac{1}{\exists}} \rightarrow \boxed{E(\text{suffix})} \rightarrow P[\text{Int}] \rightarrow \boxed{E\left(\frac{\text{prefix sum}}{\exists}\right)} \rightarrow PP(\text{Int}) \rightarrow \boxed{\text{union}} \rightarrow P(\text{Int}) \rightarrow \boxed{\text{est}(\geq)} \rightarrow \text{Int}$	$\frac{R}{\exists} \frac{\exists}{\exists} = R$ $\frac{R}{\exists} \frac{\exists}{\exists} = R$ $\text{frac}(R, \exists) \frac{\exists}{\exists} = R$ at $R := \text{prefix sum}$ and $E(R) \triangleq \frac{\exists}{\exists} R$; $\text{union} \triangleq \frac{\exists}{\exists} \frac{\exists}{\exists}$ $\text{union} \triangleq \text{frac}(\exists \exists, \exists)$; the middle equality is $\triangleq$ relator's $F(RS) = F(R)F(S)$ at $F := E$
$= [\text{Int}] \rightarrow \boxed{\frac{1}{\exists}} \rightarrow \boxed{E(\text{suffix})} \rightarrow P[\text{Int}] \rightarrow \boxed{E\left(\frac{\text{prefix sum}}{\exists}\right)} \rightarrow PP(\text{Int}) \rightarrow \boxed{E(\text{est}(\geq))} \rightarrow P(\text{Int}) \rightarrow \boxed{\text{est}(\geq)} \rightarrow \text{Int}$	$R R \subseteq R$ $\Rightarrow P(\text{est}(R)) \text{est}(R)$ $= P(\text{dom}(\text{est}(R)))$ $\text{union est}(R)$, the sets non-empty
$= [\text{Int}] \rightarrow \boxed{\frac{1}{\exists}} \rightarrow \boxed{E(\text{suffix})} \rightarrow P[\text{Int}] \rightarrow \boxed{E\left(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)\right)} \rightarrow P(\text{Int}) \rightarrow \boxed{\text{est}(\geq)} \rightarrow \text{Int}$	relator $\triangleq$ relator — $F(RS) = F(R)F(S)$ at $F := E$

The union step is $R R \subseteq R$

$\Rightarrow P(\text{est}(R)) \text{est}(R) = P(\text{dom}(\text{est}(R))) \text{union est}(R)$'s $P(\text{est}(R)) \text{est}(R) = P(\text{dom}(\text{est}(R))) \text{union est}(R)$ — every suffix has the empty prefix, so Dom is 1 here — and $P(f) = E(f)$ at the map it is applied to ($\text{Map}(f)$)
 $\Rightarrow P(f) = E(f)$.

$[nil, cons \ u \multimap nil] \text{ sum} = F(\text{sum}) \ S$ (7.2.5c)	
fusion: prefix expressed as a catamorphism on cons-lists, $([nil, cons \ u \multimap nil])$, and this is the fusion condition that expresses prefix sum as a catamorphism	
circuit — the fork is the bracket's case split $F([Int]) = 1 + Int \times [Int]: nil$ above, the pair and its u below	Hinze–Marsden
	
the cons operand of $cons \ u \multimap nil$	
	
	
the $(1 \times sum)$ plus operand of $\multimap zero \ u \ (1 \times sum)$ plus	
	
	

fusion at $R := [nil, cons \ u \multimap nil]$, $S := sum$: the side condition, so **prefix sum** $= ([zero, \multimap zero \ u \ plus])$. **prefix** is the reduce, **sum** the map fused into it — the intermediate list is gone.

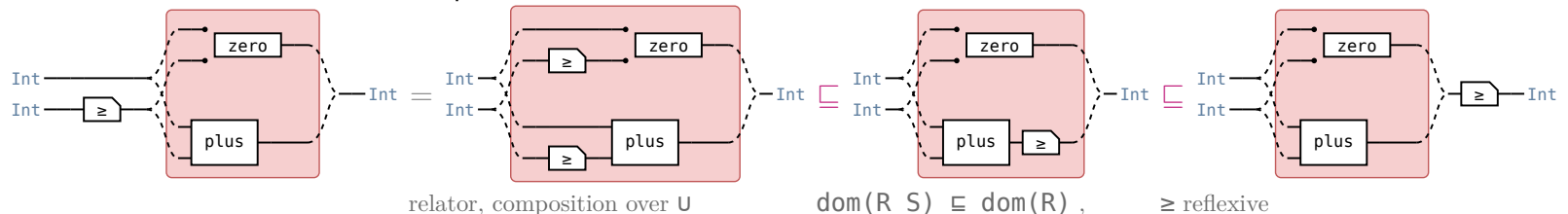
$$F(\geq) \ S \sqsubseteq S \geq$$

(7.2.5d)

monotonic algebra: an **F**-algebra **S** is monotonic on a relation **R** if $F(R)SES R$ — the **plus** branch of $F(\geq)SES\geq$; the **zero** branch is $zeroEzero\geq$

circuit — the head above, the running sum below; the tape is the **u**

$$(1 \times \geq) (\neg zero \cup plus) \quad ((1 \times \geq) \neg zero) \cup ((1 \times \neg zero \cup plus) \geq) \quad (\neg zero \cup plus) \geq$$



relator, composition over **u**

$\text{dom}(R \ S) \sqsubseteq \text{dom}(R)$, $\geq$ reflexive
 $(\geq \times \geq) \ plus \sqsubseteq plus \geq$; $(\leq \times \leq)$
 $plus \sqsubseteq plus \leq$ is $\triangleq$ monotonic,
 written $+$ there, and **plus** is a
 map, so it is monotonic on an
 order and on its opposite
 together, which carries it to $\geq$.

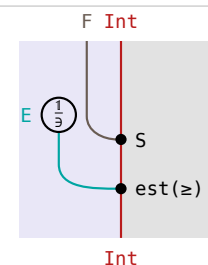
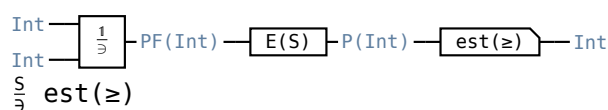
$$\frac{S}{\geq} \text{est}(\geq) = [\text{zero}, \oplus]$$

(7.2.5e)

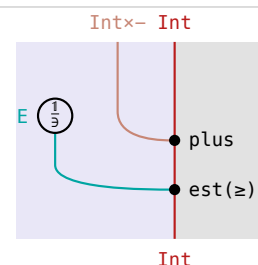
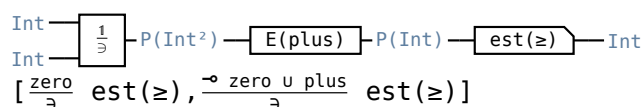
the largest sum the algebra offers is zero from nothing and, from a head and a running sum, the larger of zero and the head added to it

circuit — the tape is the coproduct: **zero**'s branch above, **plus**'s below

Hinze–Marsden

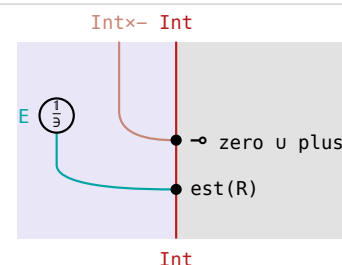
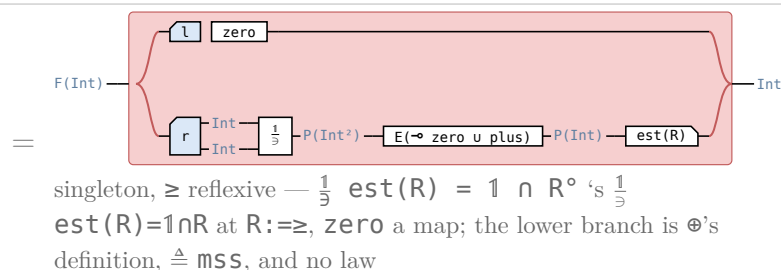


the **zero** arm of the bracket



$\left[\frac{zero}{\geq} \text{est}(\geq), \frac{\neg zero \cup plus}{\geq} \text{est}(\geq) \right]$
 = coproduct of maps — $[R, S] = \left[\frac{R}{\geq}, \frac{S}{\geq} \right] \ni$ at $T := [\text{zero}, \neg zero \cup plus]$, then $[U, V]Z = [UZ, VZ] \rightarrow [R, S] Z = [R Z, S Z]$, composition over **u**

the **plus** operand of the lower arm's $\neg zero \cup plus$, under its $\frac{1}{3}$
 $E(\dots)$ and $\text{est}(\geq)$



with $F(\geq) \ S \sqsubseteq S \geq$ the greedy theorem gives $\langle [\text{zero}, \oplus] \rangle \sqsubseteq \frac{\text{prefix sum}}{\geq} \text{est}(\geq)$ — and Exercise 7.40 $mss = \langle k \rangle \pi_2 \text{est}(\geq)$ makes it an equality.

$$[\text{nil}, \text{cons}] \langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\exists} E(\langle [\text{zero}, \oplus] \rangle) \rangle = F(\langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\exists} E(\langle [\text{zero}, \oplus] \rangle) \rangle) k \quad (7.2.5f)$$

$k \triangleq [\text{zero} \langle 1, \frac{1}{\exists} \rangle, \langle w, \langle w \frac{1}{\exists}, \pi_2 \pi_2 \rangle \text{cup} \rangle]$, $w \triangleq (1 \times \pi_1) \oplus$: the value at the whole list, paired with the set of the values at its suffixes, runs k 's recursion.

the equation at that branch

$$\text{nil}(g, \frac{\text{suffix}}{\exists} E(g)) = \text{zero} \langle 1, \frac{1}{\exists} \rangle$$

$$\text{cons}(g, \frac{\text{suffix}}{\exists} E(g)) = (1 \times \langle g, \frac{\text{suffix}}{\exists} E(g) \rangle) \langle w, \langle w \frac{1}{\exists}, \pi_2 \pi_2 \rangle \text{cup} \rangle$$

why

nil has one suffix, itself, and $g(\text{nil}) = \text{zero}$, so the set is the singleton $\{\text{zero}\}$

$g(\text{cons}(a, x)) = a \oplus (g(x))$, which is w reading $g(x)$ off π_1 ; and the suffixes of $\text{cons}(a, x)$ are $\text{cons}(a, x)$ itself, whose value is that same w , together with those of x , which $\pi_2 \pi_2$ carries — so the two sets meet at cup

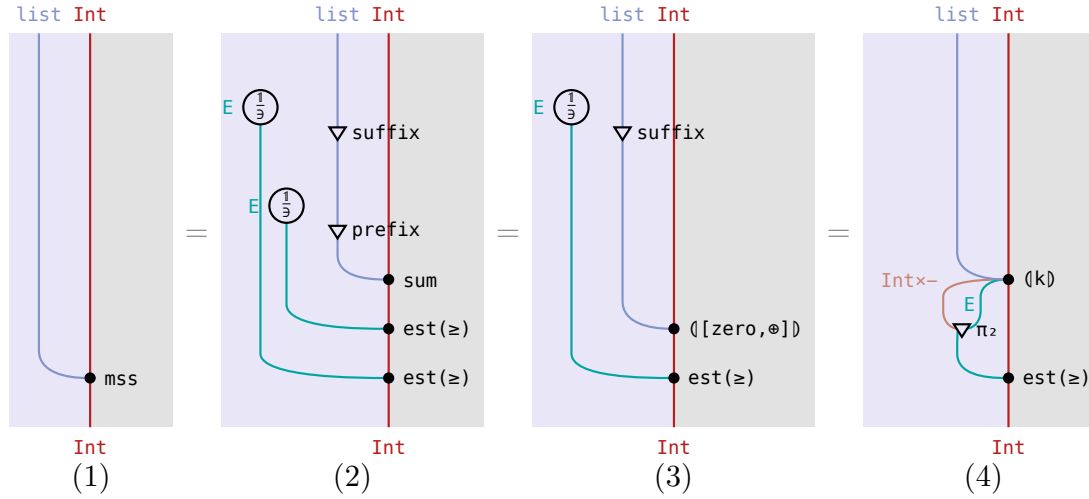
$$\text{mss} = \langle k \rangle \pi_2 \text{est}(\geq)$$

Exercise 7.40

maximum segment sum problem: $\frac{\text{suffix}}{\exists} E(\langle [\text{zero}, \oplus] \rangle)$ expressed as the catamorphism $\langle k \rangle$, hence

mss implemented by a linear-time algorithm, $\oplus \triangleq \text{zero} \vee \text{plus}$ $\text{est}(\geq) \triangleq \text{mss}$; k and w —

$$[\text{nil}, \text{cons}] \langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\exists} E(\langle [\text{zero}, \oplus] \rangle) \rangle = F(\langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\exists} E(\langle [\text{zero}, \oplus] \rangle) \rangle) k.$$



1	mss	$(7.2.5b) \text{mss} = \frac{\text{suffix}}{\exists} E(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)) \text{est}(\geq)$
2	$= \frac{\text{suffix}}{\exists} E(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)) \text{est}(\geq)$	$\frac{\text{prefix sum}}{\exists} \text{est}(\geq) = \langle [\text{zero}, \oplus] \rangle$
3	$= \frac{\text{suffix}}{\exists} E(\langle [\text{zero}, \oplus] \rangle) \text{est}(\geq)$	$\langle k \rangle \pi_2 = \frac{\text{suffix}}{\exists} E(\langle [\text{zero}, \oplus] \rangle)$
4	$= \langle k \rangle \pi_2 \text{est}(\geq)$	

one fold builds the $n+1$ running maxima and the final $\text{est}(\geq)$ reads them in one more pass, so mss is linear.

§7.2.6. Coreflexive(p) $\Rightarrow$ filter(p) = $\llbracket \text{nil}, (\pi_1 p \rightarrow \text{cons}, \pi_2) \rrbracket$ — filter(p) keeps the items satisfying p, as one fold

(7.2.6a)

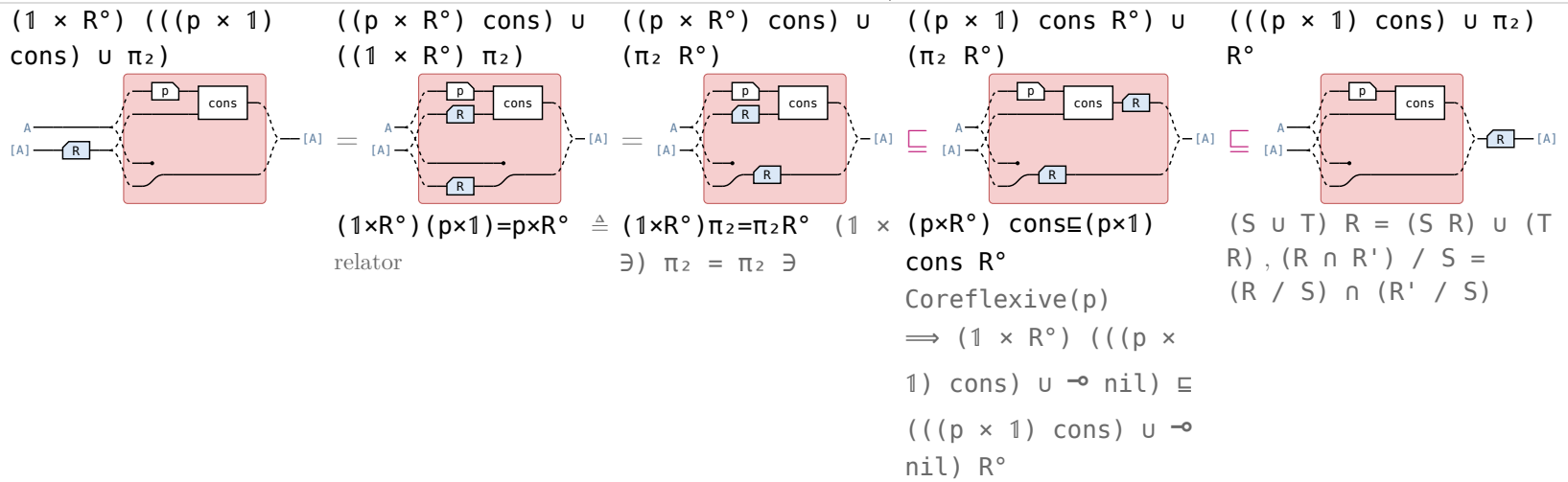
	definition	type	example and in words
1	$FX = L + E \times X$	$\mathcal{A} \rightarrow \mathcal{A}$	nothing, or a head and a tail
2	$\alpha \triangleq [\text{nil}, \text{cons}]$	$F[A] \rightarrow [A]$	both constructors as one map
3	$p \sqsubseteq 1$	$A \rightarrow A$	$p \triangleq \text{even} \rightarrow 2 \mid p \mid 2$, and $3 \notin \text{dom}(p)$ — a coreflexive: a passes p when a p a
4	$R \triangleq \text{length} \leq \text{length}^\circ$	$[A] \rightarrow [A]$	$[1] \mid R \mid [1, 2]$ — $xs \mid R \mid ys \iff \text{length}(xs) \leq \text{length}(ys)$
5	$\pi_2(a, x) \triangleq x$	$A \times [A] \rightarrow [A]$	$\pi_2(3, [1, 2]) = [1, 2]$ — drop the head, keep the tail
6	$\text{subseq} = \llbracket \text{nil}, \text{cons} \cup \pi_2 \rrbracket$	$[A] \rightarrow [A]$	$[3, 1, 2] \mid \text{subseq} \mid [3, 2]$ — $xs \mid \text{subseq} \mid ys \iff ys$ is xs with elements dropped — at each cons, keep the head or drop it
7	$S \triangleq [\text{nil}, ((p \times 1) \text{ cons}) \cup \pi_2]$	$F[A] \rightarrow [A]$	$(4, [2]) \mid S \mid [4, 2]$ and $(4, [2]) \mid S \mid [2]$, but $(3, [2]) \mid S \mid [2]$ only — subseq's algebra with one extra p — keep a head that passes p, or drop the head
8	$1 \sqsubseteq \pi_2 R \text{ cons}^\circ$		the tail is one shorter than the cons, so π_2 loses the $\text{est}(R^\circ)$ at every step — where $\rightarrow \text{nil} \triangleq \rightarrow \text{nil}$'s loser is nil

Coreflexive(p) $\Rightarrow F(R^\circ) S \sqsubseteq S R^\circ$

(7.2.6b)

shortening the tail and then taking the step lands inside taking the step and then shortening the result

formula — the cons branch of $F(R^\circ) S \sqsubseteq S R^\circ$; **reason** under each circuit



the nil branch: $\text{nil} \sqsubseteq \text{nil } R^\circ$.

$\text{Coreflexive}(p) \Rightarrow \frac{S}{\exists} \text{est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \pi_2)]$ (7.2.6c)	
the longest of the lists the algebra allows is the cons where the head passes p , and the tail where it does not	
formula	reason
$=$ $[\frac{\text{nil}}{\exists} \text{est}(R^\circ), \frac{((p \times 1) \text{ cons}) \cup \pi_2}{\exists} \text{est}(R^\circ)]$	$S \triangleq [\text{nil}, ((p \times 1) \text{ cons}) \cup \pi_2]$ — and the fraction of a coproduct of maps is the coproduct of their fractions $[R, S] = [\frac{R}{\exists}, \frac{S}{\exists}] \ni$
$=$ $[\text{nil}, \frac{((p \times 1) \text{ cons}) \cup \pi_2}{\exists} \text{est}(R^\circ)]$	$\frac{\text{nil}}{\exists}$ is the singleton $\{\text{nil}\}$, and $\text{est}(R^\circ)$ of a singleton is its element because R° is reflexive $\triangleq \text{est}$
$=$ $\frac{(p \times 1) \text{ cons} \cup \pi_2}{\exists} = \langle \frac{(p \times 1) \text{ cons}}{\exists}, \frac{\pi_2}{\exists} \rangle \text{ cup} \triangleq \text{cup}$	$\frac{(p \times 1) \text{ cons} \cup \pi_2}{\exists} = \langle \frac{(p \times 1) \text{ cons}}{\exists}, \frac{\pi_2}{\exists} \rangle \text{ cup} \triangleq \text{cup}$
$=$	$1 \sqsubseteq \pi_2 R \text{ cons}^\circ$ (7.2.6a.8) — $x s \ R \text{ cons}(a, x s)$, so $\text{est}(R^\circ)$ returns the cons where p a puts it in the set and xs where the set is $\{x s\} \triangleq \text{est}$

the head is dropped, not the whole tail: that is the one place π_2 shows against $\text{Coreflexive}(p)$

$$\Rightarrow \frac{S}{\exists} \text{est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})] \text{ 's } \neg \text{nil}.$$

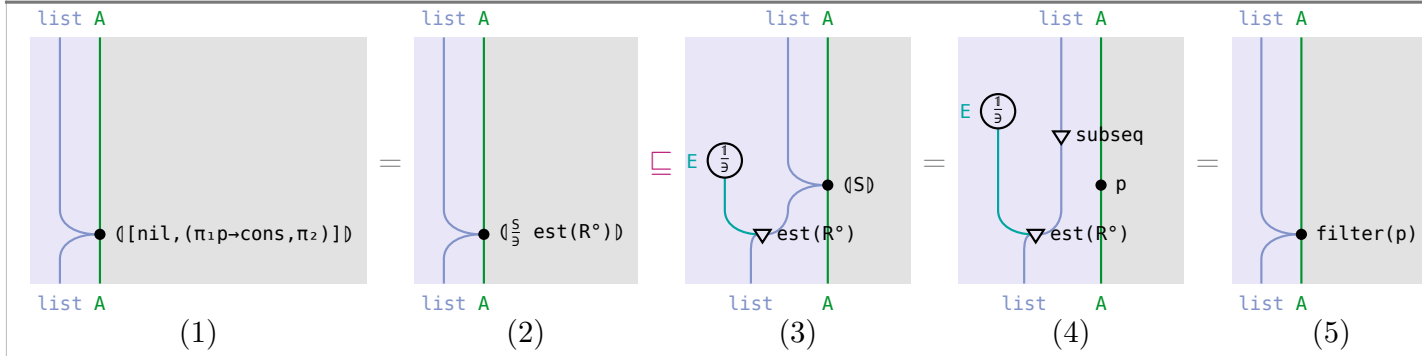
(7.2.6d)

definition	type	example and in words
$S \triangleq [\text{nil}, ((p \times 1) \text{ cons}) \cup \pi_2]$	$F[A] \rightarrow [A]$	$(4, [2]) \ S \ [4, 2]$ and $(4, [2]) \ S \ [2]$, but $(3, [2]) \ S \ [2]$ only — subseq's algebra with one extra p — keep a head that passes p , or drop the head

Coreflexive(p) $\Rightarrow$ filter(p) = $\langle [nil, (\pi_1 p \rightarrow cons, \pi_2)] \rangle$

Exercise 7.41

filter: filter(p) **x** returns the longest subsequence of **x** with the property that all its elements satisfy **p**; the catamorphism is the standard program; **R** a preorder.



1	$\langle [nil, (\pi_1 p \rightarrow cons, \pi_2)] \rangle$	(7.2.6c) Coreflexive(p) $\Rightarrow \bigvee_{\substack{\subseq}} \text{est}(R^\circ) = [nil, (\pi_1 p \rightarrow cons, \pi_2)]$
2	$= \langle \bigvee_{\substack{\subseq}} \text{est}(R^\circ) \rangle$	Coreflexive(p) $\Rightarrow \langle \bigvee_{\substack{\subseq}} \text{est}(R^\circ) \rangle \sqsubseteq \langle \bigvee_{\substack{\subseq}} \text{est}(R^\circ) \rangle$
3	$\sqsubseteq \langle \bigvee_{\substack{\subseq}} \text{est}(R^\circ) \rangle$	Coreflexive(p) $\Rightarrow \text{subseq list}(p) = \langle S \rangle$
4	$= \frac{\text{subseq list}(p)}{\subseq} \text{est}(R^\circ)$	
5	$= \text{filter}(p)$	

the catamorphism is entire and **filter(p)** simple, so $\sqsubseteq$ is $=$.

$(\text{subseq list}(p))^\circ (\text{subseq list}(p)) \cap R \cap R^\circ \sqsubseteq 1$ fails — two **p**-subsequences of one list can be of equal length and different — so §7.2.4's uniqueness argument does not transfer. What survives **est**(R°) is the one keeping **exactly** the passing elements: a subsequence that drops a passing element is beaten by the one that keeps it.

(7.2.6e)

	definition	type	note
1	segment = suffix prefix	$[A] \rightarrow [A]$	A contiguous stretch of xs : a suffix, then a prefix of that.
2	filter(p) $\triangleq \frac{\text{subseq list}(p)}{\subseq} \text{est}(R^\circ)$	$[A] \rightarrow [A]$	The longest subsequence of xs whose every element passes p . est (R°) is $\triangleq$ est
3	$R \triangleq \text{length} \leq \text{length}^\circ$	$[A] \rightarrow [A]$	The preorder filter and takewhile maximise over: the longer list wins. $\geq \triangleq \leq^\circ$
4	takewhile(p) $\triangleq \frac{\text{prefix list}(p)}{\subseq} \text{est}(R^\circ)$	$[A] \rightarrow [A]$	The same with prefix for subseq : the longest prefix whose every element passes p .
5	mss $\triangleq \frac{\text{segment sum}}{\subseq} \text{est}(\geq)$	$[\text{Int}] \rightarrow \text{Int}$	Maximum segment sum. segment=suffix prefix splits it into $\frac{\text{prefix sum}}{\subseq} \text{est}(\geq)$ on each suffix.

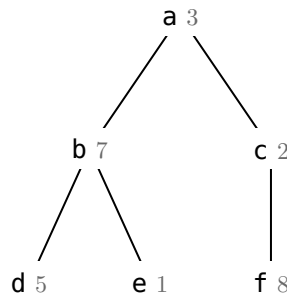
§7.3. Planning a company party

(7.3a)

	definition	type	note
1	$\text{tree } A ::= \text{node } (A, [\text{tree } A])$	$\mathcal{A} \rightarrow \mathcal{A}$	The company hierarchy: an employee, and the list of subtrees under them.
2	$F(A, B) = A \times [B]$	$\mathcal{A} \rightarrow \mathcal{A}$	The base functor tree folds: an employee beside the recursive position, one layer deep.
3	rating	$A \rightarrow \text{Int}$	What one employee is worth as a guest.
4	$\text{cost} = \text{list}(\text{rating}) \text{ sum}$	$[A] \rightarrow \text{Int}$	What a guest list is worth.
5	$R = \text{cost} \leq \text{cost}^\circ$	$[A] \rightarrow [A]$	The preorder the guest list is maximised over.
6	$\text{choose} \triangleq \pi_1 \cup \pi_2$	$[A] \times [A] \rightarrow [A]$	Takes one of the two parties a subtree returns.
7	$\text{include} = (1 \times (\text{list}(\pi_2) \text{ concat})) \text{ cons}$	$F(A, [A] \times [A]) \rightarrow [A]$	The party that invites the root, which puts every immediate subtree's root out. A map.
8	$\text{exclude} = (1 \times (\text{list}(\text{choose}) \text{ concat})) \pi_2$	$F(A, [A] \times [A]) \rightarrow [A]$	The party that leaves the root out, so each subtree is free to choose. Not a map.
9	$S \triangleq (\text{include}, \text{exclude})$	$F(A, [A] \times [A]) \rightarrow [A] \times [A]$	The algebra: one step returns both parties of a subtree at once.
10	$\text{party} = \langle S \rangle \text{ choose}$	$\text{tree}(A) \rightarrow [A]$	Every guest list the president's ruling allows.
11	the specification $\frac{\text{party}}{\exists} \text{ est}(R^\circ)$	$TA \rightarrow [A]$	A guest list of greatest total conviviality.

§7.3.1. include and exclude

(7.3.1a)



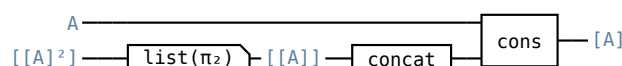
the small grey number is the employee's rating

(7.3.1b)

	node	the algebra's input $A \times [[A] \times [A]]$	include	exclude
1	d, e, f	$(d, [])$	$[d]$	$[]$
2	b	$(b, ([[d], []], ([e], []]))$	$[b]$	$[d, e], [d], [e], []$
3	c	$(c, ([[f], []]))$	$[c]$	$[f], []$
4	a	$(a, ([[b], \dots], ([c], \dots)))$	$[a] \# \text{exclude}(b) \# \text{exclude}(c)$	$\text{choose}(b) \# \text{choose}(c)$

with those ratings, $\text{est}(R^\circ)$ keeps $([b]=7, [d, e]=6)$ at **b**, where **include** wins, and $([c]=2, [f]=8)$ at **c**, where **exclude** wins, so at the root $\text{include}=[a, d, e, f]=17$ beats $\text{exclude}=[b, f]=15$, and **choose** takes 17.

(7.3.1c)



$$\begin{array}{c} A \\ \hline [[A]^2] \xrightarrow{\text{list}(\text{choose})} [[A]] \xrightarrow{\text{concat}} [A] \end{array}$$

(7.3.1d)

list(choose) [([d],[]), ([e],[])]
 1st item ([d],[]) choose $\rightarrow$ [d] or []
 2nd item ([e],[]) choose $\rightarrow$ [e] or []
 2x2=4 combinations: [[d],[e]] [[d],[]] [[],[e]] [[],[]]
 concat flattens: [d,e] [d] [e] []

(7.3.1e)

§7.3.2. list((R×R)^o)

$$\begin{array}{c} [([b],[d,e]) , ([c],[f])] \\ \downarrow \quad \downarrow \\ [([b],[d]) , ([c],[])] \end{array} \quad \text{list}((R \times R)^o), \text{ elementwise}$$

(7.3.2a)

([b],[d,e])
 | — exclude: the best party in b's subtree when b does not come
 — include: the best party in b's subtree when b comes

(7.3.2b)

element	1st component include	2nd component exclude
1 1st, ([b],[d,e])	[b] R ^o [b] 7 ≥ 7, reflexivity	[d,e] R ^o [d] 6 ≥ 5, and ([b],[d,e]) is the pair est (R ^o) keeps at b
2 2nd, ([c],[f])	[c] R ^o [c] 2 ≥ 2, reflexivity	[f] R ^o [] 8 ≥ 0, and ([c],[f]) is the pair est (R ^o) keeps at c

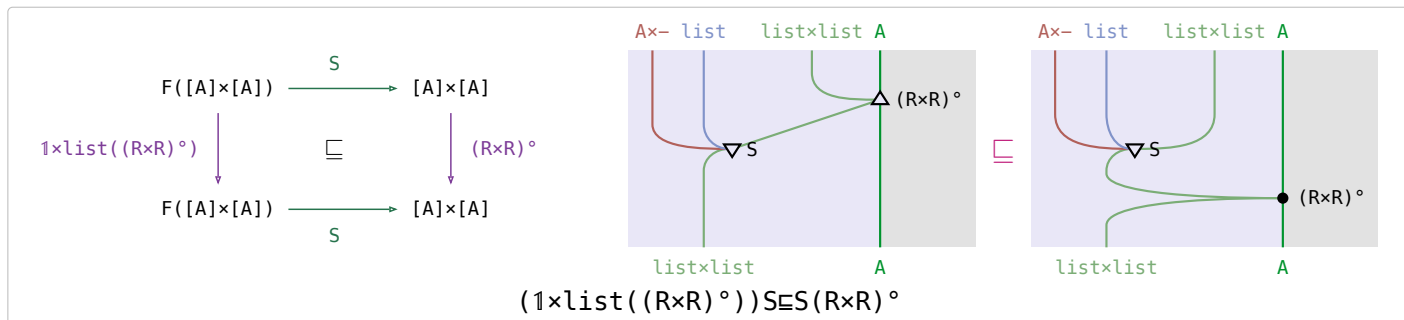
the first component of $1 \times \text{list}((R \times R)^o)$ is the root employee.

$R \triangleq \text{cost} \leq \text{cost}^o$ is a preorder, so the components that do not move are related by reflexivity — $(R \times R)^o$ demands a relation in **both** components, it cannot skip one.

R compares cost only, not contents: [f] R^o [] is legal though [f] has an element [] does not. That is why the three leaves of §7.3.3's proof all reduce to "cost is a sum".

§7.3.3. $(1 \times \text{list}((R \times R)^o)) S \sqsubseteq S (R \times R)^o$ — S is monotonic on $(R \times R)^o$

(7.3.3a)

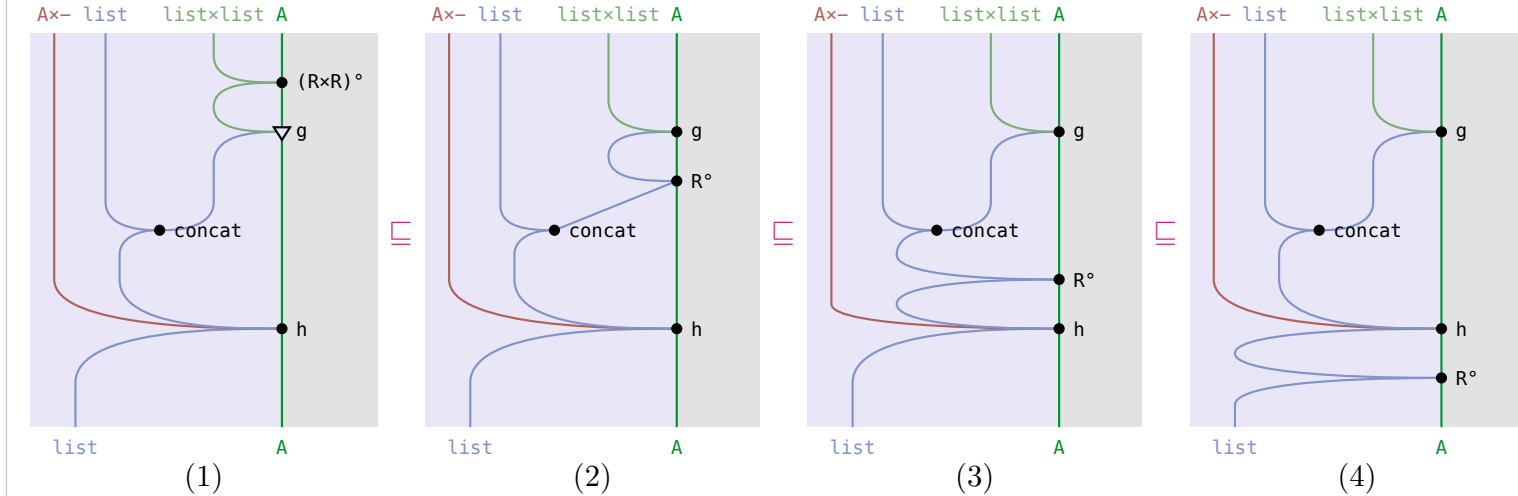


(7.3.3b)

$$\begin{array}{c} \text{tree}(A) \xrightarrow{\text{S} \text{ choose}} P[A] \quad \text{tree}(A) \xrightarrow{\text{S}} P([A] \times [A]) \xrightarrow{E(\text{choose})} P[A] \\ \text{S choose} = \text{S} E(\text{choose}) \quad \text{S} E(R) = \text{S} R, \text{ absorption,} \end{array}$$

$$\begin{aligned}
 S &\triangleq (\text{include}, \text{exclude}) \\
 (1 \times \text{list}((R \times R)^\circ)) \text{include} &\sqsubseteq \text{include } R^\circ \\
 (1 \times \text{list}((R \times R)^\circ)) \text{exclude} &\sqsubseteq \text{exclude } R^\circ \\
 (1 \times \text{list}((R \times R)^\circ)) S &\sqsubseteq S (R \times R)^\circ
 \end{aligned}
 \tag{7.3.3c}$$

bettering both parties of every subtree before the node's algebra runs gets no further than running it first and bettering the two parties it returns,



1	$(1 \times \text{list}((R \times R)^\circ)) (1 \times (\text{list}(g) \text{ concat})) h$	$(R \times R)^\circ g \sqsubseteq g R^\circ$
2	$\sqsubseteq (1 \times (\text{list}(g R^\circ) \text{ concat})) h$	$\text{list}(R^\circ) \text{ concat} \sqsubseteq \text{concat } R^\circ$
3	$\sqsubseteq (1 \times (\text{list}(g) \text{ concat } R^\circ)) h$	$(1 \times R^\circ) h \sqsubseteq h R^\circ$
4	$\sqsubseteq (1 \times (\text{list}(g) \text{ concat})) h R^\circ$	

g $g := \pi_2$ is $(R \times R)^\circ \pi_2 = (\text{dom}(\pi_1 R^\circ)) \pi_2 R^\circ \sqsubseteq \pi_2 R^\circ$, $g := \pi_1$ its mirror $(\text{dom}(\pi_2 R^\circ)) \pi_1 R^\circ \sqsubseteq \pi_1 R^\circ$ — $\triangleq \times$ and $\langle R, S \rangle \pi_1 = \text{dom}(S) R$, $\langle R, S \rangle \pi_2 = \text{dom}(R) S$, then $\text{dom} \sqsubseteq 1$; $g := \text{choose} \triangleq \pi_1 \cup \pi_2$ is the union of the two $\text{lax}(\varphi) \wedge \text{lax}(\psi) \Rightarrow G(R) (\varphi_B \cup \psi_B) \sqsubseteq (\varphi_A \cup \psi_A) F(R)$. list monotonic, $\triangleq$ relator

concat a LEAF: no law above it. cost is a sum, so $\text{cost}(\text{concat}(xss)) = \text{sum}(\text{list}(\text{cost})(xss))$ and a cheaper part makes a cheaper whole.

h a LEAF: $\text{cost}(\text{cons}(a, xs)) = \text{rating}(a) + \text{cost}(xs)$, so a cheaper tail makes a cheaper list. For $h := \pi_2$ it is an EQUALITY $(1 \times R^\circ) \pi_2 = (\text{dom}(\pi_1)) \pi_2 R^\circ = \pi_2 R^\circ$: π_1 is a map, hence entire, so $\text{dom}(\pi_1) = 1$ — $\text{Entire}(R) \triangleq \text{dom}(R) = 1$

$(1 \times (\text{list}(g) \text{ concat})) h$	g	h
include	π_2	cons
exclude	choose	π_2

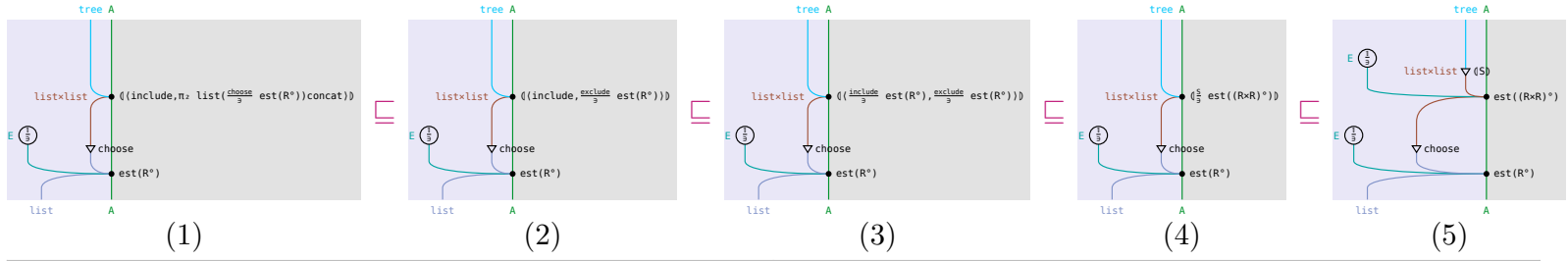
(7.3.3d)

1	$R \triangleq \{(\emptyset, \emptyset)\} : \{0, 1\} \rightarrow \{0, 1\}$	not entire — undefined at 1
2	$(R \times R) \text{choose}$ at $(\emptyset, 1)$	nothing — $R \times R$ needs BOTH components related, and R is undefined at 1
3	$\text{choose } R$ at $(\emptyset, 1)$	$\emptyset$ — choose picks the first component $\emptyset$, and $R \emptyset \emptyset$ holds
4	$((\emptyset, 1), \emptyset)$	in $\text{choose } R$, not in $(R \times R) \text{choose}$
5	$(R \times R) \text{choose} = \text{choose } R$	iff R is entire — $\text{dom} \sqsubseteq 1$ is the g row's only inequality step, and it is an equality iff $\text{dom}(R) = 1$

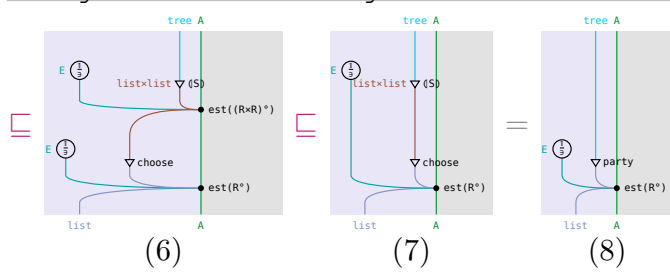
§7.3.4. Derivation

$$(\langle \langle \text{include}, \pi_2 \text{ list}(\frac{\text{choose}}{\exists} \text{ est}(R^\circ)) \text{ concat} \rangle \rangle \frac{\text{choose}}{\exists} \text{ est}(R^\circ) \sqsubseteq \frac{\text{party}}{\exists} \text{ est}(R^\circ)) \quad (7.3.4a)$$

the best of every guest list the president allows is one pass up the tree, each subtree handing up its best party
with its boss in and its best with the boss out, and **choose** taking the better of the two at the root



1	$(\langle \langle \text{include}, \pi_2 \text{ list}(\frac{\text{choose}}{\exists} \text{ est}(R^\circ)) \text{ concat} \rangle \rangle \frac{\text{choose}}{\exists} \text{ est}(R^\circ)$	$\pi_2 \text{ list}(\frac{\text{choose}}{\exists} \text{ est}(R^\circ)) \text{ concat} \sqsubseteq \frac{\text{exclude}}{\exists} \text{ est}(R^\circ)$
2	$\sqsubseteq (\langle \langle \text{include}, \frac{\text{exclude}}{\exists} \text{ est}(R^\circ) \rangle \rangle \frac{\text{choose}}{\exists} \text{ est}(R^\circ)$	$\text{include} \sqsubseteq \frac{\text{include}}{\exists} \text{ est}(R^\circ)$
3	$\sqsubseteq (\langle \langle \frac{\text{include}}{\exists} \text{ est}(R^\circ), \frac{\text{exclude}}{\exists} \text{ est}(R^\circ) \rangle \rangle \frac{\text{choose}}{\exists} \text{ est}(R^\circ)$	$\langle \frac{\text{include}}{\exists} \text{ est}(R^\circ), \frac{\text{exclude}}{\exists} \text{ est}(R^\circ) \rangle \sqsubseteq \frac{\exists}{\exists} \text{ est}((R \times R)^\circ)$
4	$\sqsubseteq (\langle \frac{\exists}{\exists} \text{ est}((R \times R)^\circ) \rangle \frac{\text{choose}}{\exists} \text{ est}(R^\circ)$	$(\langle \frac{\exists}{\exists} \text{ est}((R \times R)^\circ) \rangle \sqsubseteq \frac{(\exists)}{\exists} \text{ est}((R \times R)^\circ)$
5	$\sqsubseteq (\langle \frac{(\exists)}{\exists} \text{ est}((R \times R)^\circ) \rangle \frac{\text{choose}}{\exists} \text{ est}(R^\circ)$	



6	$\sqsubseteq (\langle \frac{(\exists)}{\exists} \text{ est}((R \times R)^\circ) \rangle \frac{\text{choose}}{\exists} \text{ est}(R^\circ)$	$\text{est}((R \times R)^\circ) \frac{\text{choose}}{\exists} \text{ est}(R^\circ) \sqsubseteq E(\text{choose}) \text{ est}(R^\circ)$
7	$\sqsubseteq (\langle \frac{(\exists)}{\exists} \text{ choose} \text{ est}(R^\circ) \rangle$	$\frac{(\exists)}{\exists} \text{ choose} = \frac{(\exists)}{\exists} E(\text{choose})$
8	$= \frac{\text{party}}{\exists} \text{ est}(R^\circ)$	$\text{party} = (\exists) \text{ choose}$

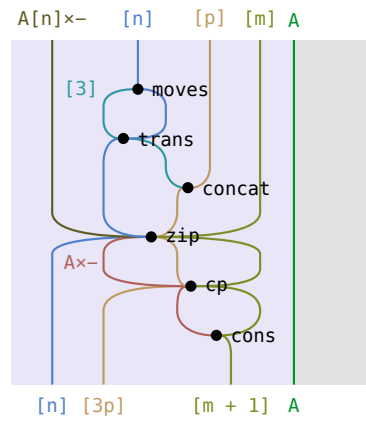
§7.4. Shortest paths on a cylinder

(7.4a)

	on $L = \text{list}^+$, 10.3.3	on Vec	note
1 F	$F(A, X) = A + A \times X$	$F(A, A[m]) = A[1] + A[m+1]$	A square alone, or in front of the path so far.
2 the input	$L(N\ A)$	$A[m][n] \triangleq (A[n])[m]$	m columns of n squares, outermost index first — the matrix itself.
3 a path	$L\ A$, a non-empty list	$A[1] \triangleq A$, $A[m+1] \triangleq A \times A[m]$	Through m columns, m squares: the count is in the type. A one-square path is the square; $\text{Vec}(m)$ is the functor $X \mapsto X[m]$.
4 the paths into a row	$P(L\ A)$	$A[p][m]$, $p = 3^{m-1}$	Three moves per column crossed, so this count is in the type too; a set forgets it.
5 α	$[\text{wrap}, \text{cons}]$ $F(A, LA) \rightarrow LA$	$\text{wrap} = 1 : A \rightarrow A[1]$ $\text{cons} : A \times A[m] \rightarrow A[m+1]$	wrap is an identity; cons is only an iso, so it stays a bead.
6 moves	$NA \rightarrow PNA$	$X[n] \rightarrow X[3][n]$	$\text{moves}(x) = (\text{up}(x), x, \text{down}(x))$, a $3 \times n$ matrix.
7 trans	$PNX \rightarrow NPX$	$X[3][n] \rightarrow X[n][3]$	The transpose.
8 union	$PPX \rightarrow PX$	$\text{concat} : X[j][k] \rightarrow X[jk]$	The j rows laid end to end; j is 3 inside gen and n inside paths .
9 zip	$F(NA, NB) \rightarrow NF(A, B)$	$F(A[n], B[n]) \rightarrow F(A, B)[n]$	Unchanged.
10 cp	$F(A, PB) \rightarrow P(F(A, B))$	$A \times B[p] \rightarrow (A \times B)[p]$	The square paired with each of the p candidates.
11 $\text{est}(R) \triangleq \exists n (\in \text{PPX}) \rightarrow X$		$X[k] \rightarrow X$, $\exists : X[k] \rightarrow X$	A component that is R -below every component; $\exists$ is “is some component” and $\in$ its converse.
12 $R \triangleq \text{sum} \leq \text{sum}^\circ$	$[N] \rightarrow [N]$	$N[m] \rightarrow N[m]$	The cost of a path, which the cheapest minimises — one order per length m .
13 gen	$F(1, \text{moves trans } N(\text{union}))$ $\text{zip } N(\text{cp } P(\alpha))$ $F(NA, N(P(LA)))$ $\rightarrow N(P(LA))$	$F(1, \text{moves trans } \text{Vec}(n)(\text{concat}))$ $\text{zip } \text{Vec}(n)(\text{cp } \text{Vec}(3p)(\text{cons}))$ $F(A[n], A[n][p][m]) \rightarrow A[n][3p][m+1]$	The same composite without α ; the type shown is the cons side, the wrap side is $1 : A[n] \rightarrow A[n]$.
14 (gen)	$L(N\ A) \rightarrow N(P(L\ A))$	$A[m][n] \rightarrow A[n][p][m]$	Columns in, rows out.
15 $\text{paths} \triangleq (\text{gen})$ concat	(gen) setify union $[A[n]] \rightarrow P[A]$	$A[m][n] \rightarrow A[np][m]$	The n rows of p paths laid end to end: every path of the cylinder, in one row.
16 Q , 7.4.3	$F(1, \text{moves trans } N(\text{est}(R)))$ $\text{zip } N(\alpha)$	$F(1, \text{moves trans } \text{Vec}(n)(\text{est}(R)))$ $\text{zip}, F(A[n], A[n][m]) \rightarrow A[n][m+1]$	The cheapest of three; no p anywhere. $(Q) : A[m][n] \rightarrow A[n][m]$ has the type of a transpose.
17 the specification paths $\text{est}(R)$	$[N[n]] \rightarrow [N]$	$A[m][n] \rightarrow A[m]$, $\text{est}(R) : X[np] \rightarrow X$	A cheapest path from the entry side to the exit side.

§7.4.1. $\text{gen} \triangleq (1 \times (\text{moves trans } \text{Vec}(\text{concat}))) \text{ zip } \text{Vec}(\text{cp } \text{Vec}(\text{cons}))$ — one more column: each square’s paths go to its three neighbours, then the square goes in front

(7.4.1a)



(7.4.1b)

a row is the outer index, a cell one path

$u = ((1,2,3,4), C)$

$C = 5$

6

7

8

$(1 \times (\text{moves trans Vec}(n)(\text{concat})))$

$\text{moves}(C)$

$= 6 \ 7 \ 8 \ 5$

5 6 7 8

8 5 6 7

$\text{trans}(\text{that})$

$= 6 \ 5 \ 8$

7 6 5

8 7 6

5 8 7

$\text{Vec}(n)(\text{concat})(\text{that}) = \text{that}$

$\text{zip}(\text{that})$

$= (1, 6 \ 5 \ 8)$

(2, 7 6 5)

(3, 8 7 6)

(4, 5 8 7)

$\text{Vec}(n)(\text{cp})(\text{that})$

$= 1 \ 6 \ 2 \ 7 \ 3 \ 8 \ 4 \ 5$

1 5 2 6 3 7 4 8

1 8 2 5 3 6 4 7

: $A[n] \times A[n]$

one candidate per row, one square long:

a column of squares

1 keeps the column, the candidates move down, unmoved, up: a $3 \times n$ matrix

: $A[3][n]$

the transpose: row k gets rows $k-1, k, k+1$

: $A[n][3]$

$\text{concat}: X[3][1] \rightarrow X[3]$ is 1

each row: its square, and its candidates

: $(A \times A[3])[n]$

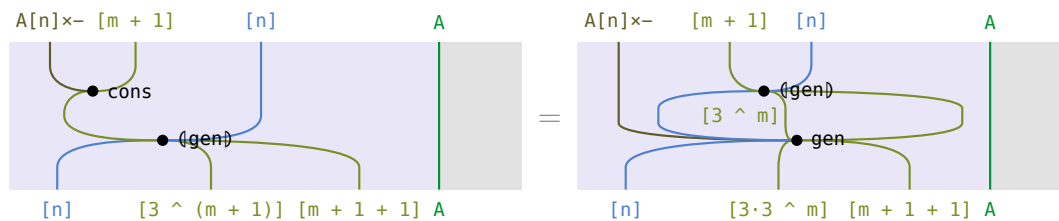
cp pairs the square with each candidate:

row k of that is the k -th 3×2 block, left to right; each line a two-square path

: $A[n][3][2]$

§7.4.2. $\text{cons } (\text{gen}) = (1 \times (\text{gen})) \text{ gen}$ — folding a matrix with one more column is folding the rest, then one gen

(7.4.2a)



the $[m+1]$ bracket puts the column back on the matrix, then the fold reads all of it

the fold reads the rest under F , then one gen puts the column in front

(7.4.2b)

$xs = 1 \ 2 \ 3 \ 4$

5 6 7 8

: $A[m][n], m=2$

$(\text{gen})(xs) = \text{gen}((1,2,3,4), (\text{gen})(5,6,7,8))$

$(\text{gen})(5,6,7,8)$

$= 5$

6

7

8

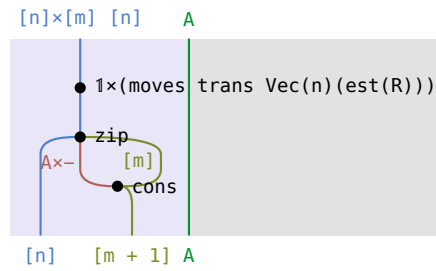
one column: each square its own one-square path

: $A[n][1][1]$

both halves are in $\text{gen} \triangleq (1 \times (\text{moves trans Vec}(\text{concat}))) \text{ zip Vec}(\text{cp Vec}(\text{cons}))$: the tail folds to the column 5 6 7 8, one path per row and each of them one square long, and gen on it is that walk.

§7.4.3. $Q \triangleq (1 \times (\text{moves trans Vec}(n)(\text{est}(R)))) \text{ zip Vec}(n)(\text{cons})$ — Q is gen with $\text{est}(R)$ choosing one path per square

(7.4.3a)



(7.4.3b)

$u = ((1,2,3,4), C)$
 $C = 5$
 6
 7
 8
 $(1 \times (\text{moves trans Vec}(n)(\text{est}(R))))$
 $\text{moves}(C)$
 $= 6 \ 7 \ 8 \ 5$
 $5 \ 6 \ 7 \ 8$
 $8 \ 5 \ 6 \ 7$
 $\text{trans}(\text{that})$
 $= 6 \ 5 \ 8$
 $7 \ 6 \ 5$
 $8 \ 7 \ 6$
 $5 \ 8 \ 7$
 $\text{Vec}(n)(\text{est}(R))(\text{that})$
 $= 5 \ 5 \ 6 \ 5$
 $\text{zip}(\text{that})$
 $= 1 \ 5$
 $2 \ 5$
 $3 \ 6$
 $4 \ 5$

$: A[n] \times A[n][m]$
 one path per row, one square long

 1 keeps the column, the paths move down, unmoved, up: a $3 \times n$ matrix

 $: A[3][n][m]$
 the transpose: row k gets rows $k-1, k, k+1$

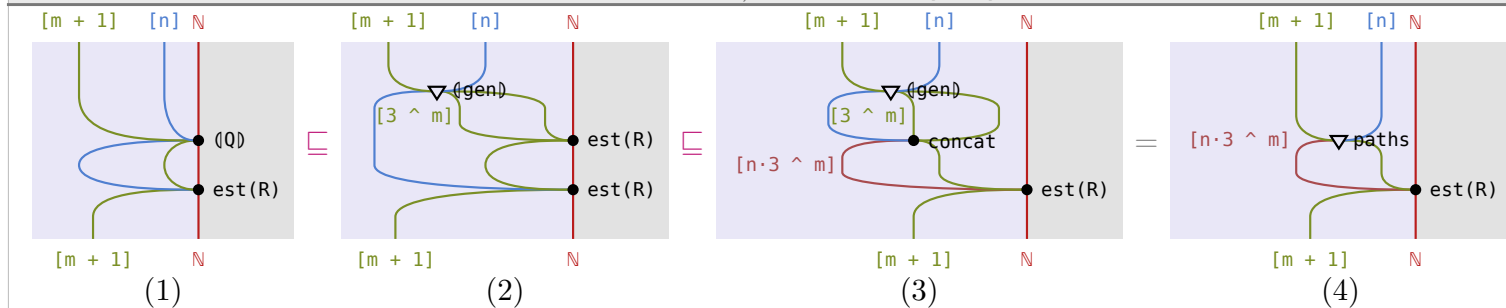
 $: A[n][3][m]$
 the cheapest into each row – chosen BEFORE
 $: A[n][m]$ the new square is put in front
 each row: its square, and the one predecessor that survived; the $[m+1]$ bracket is cons

 $: A[n][m+1]$

§7.4.4. $(Q) \text{ est}(R) \sqsubseteq \text{paths est}(R)$ — folding Q gives a smallest of paths

(7.4.4a)

a cheapest of all np paths of the cylinder is beaten by the greedy fold's one path per row and then a cheapest of those n , which costs $O(n \times m)$.



1	$(Q) \text{ est}(R)$	$(Q) \sqsubseteq (\text{gen}) \text{ Vec}(n)(\text{est}(R))$
2	$\sqsubseteq (\text{gen}) \text{ Vec}(n)(\text{est}(R)) \text{ est}(R)$	$R \ R \sqsubseteq R$
3	$\sqsubseteq (\text{gen}) \text{ concat est}(R)$	$f \ g = f \ g$
4	$= \text{paths est}(R)$	

$S \ S \sqsubseteq S \Rightarrow \text{Vec}(j)(\text{est}(S)) \text{ est}(S) \sqsubseteq \text{concat est}(S)$, R transitive

a cheapest of each of the j rows and then a cheapest of those j is a cheapest of all jk entries laid end to end.

$(Q) \sqsubseteq (\text{gen}) \text{ Vec}(n)(\text{est}(R))$

the one path per row the greedy fold keeps is one of the p that (gen) generates for that row, and a cheapest of them.

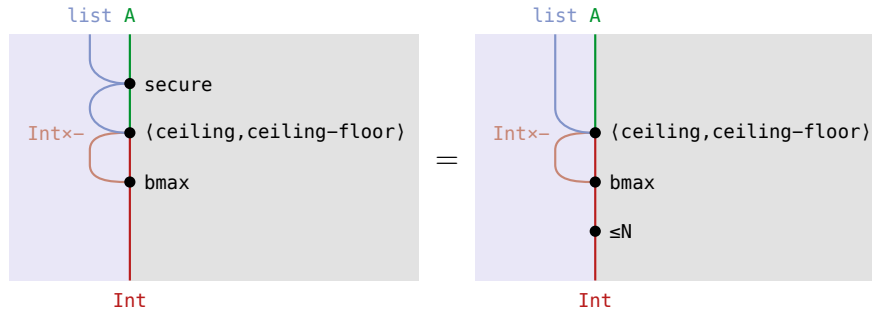
§7.5. The security van problem

(7.5a)

	definition	type	example
1	$R \triangleq \text{length} \leq \text{length}^\circ$	$[[A]] \rightarrow [[A]]$	$[[a, b, c]]$ to $[[a], [b, c]]$, and not back: $1 \leq 2$.
2	$\text{ceiling} \triangleq \frac{\text{prefix sum}}{\ni} \text{est}(\geq)$	$[A] \rightarrow \text{Int}$	$\text{ceiling}[a, b]$ is the largest of $0, a, a+b$.
3	$\text{floor} \triangleq \frac{\text{prefix sum}}{\ni} \text{est}(\leq)$	$[A] \rightarrow \text{Int}$	$\text{floor}[a, b]$ is the smallest of $0, a, a+b$, so $[a, b]$ carries ceiling-floor cash.
4	secure the coreflexive on x with $\text{bmax}(\text{ceiling } x, \text{ceiling } x - \text{floor } x) \leq N$	$[A] \rightarrow [A]$	secure keeps $[a]$ exactly when $-N \leq a \leq N$.
5	ok the coreflexive on (a, xs) with xs non-empty and $[a] \# \text{head}(xs)$ secure	$A \times [[A]] \rightarrow A \times [[A]]$	ok keeps $(a, [[b], [c]])$ exactly when $[a, b]$ is secure .
6	$\text{new} \triangleq (\text{wrap} \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{new}(a, [[b], [c]]) = [[a], [b], [c]]$.
7	$\text{glue} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } (\text{cons} \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{glue}(a, [[b], [c]]) = [[a, b], [c]]$.
8	$\text{old} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } ((\text{cons } \text{secure}) \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{old}(a, [[b], [c]]) = [[a, b], [c]]$ when $[a, b]$ is secure , and nothing otherwise.
9	partition = $([\text{nil}, \text{new} \cup \text{glue}])$ cutting the transactions into non-empty segments is one pass along them that either opens a segment for the transaction in hand or puts it on the front of the segment already open	$[A] \rightarrow [[A]]$	partition $[a, b]$ gives $[[a], [b]]$ and $[[a, b]]$.
10	$S \triangleq [\text{nil}, \text{new} \cup \text{old}]$	$F[[A]] \rightarrow [[A]]$	$S(a, [[b], [c]])$ gives $[[a], [b], [c]]$, and $[[a, b], [c]]$ when $[a, b]$ is secure .
11	$(\forall a. \text{secure}(N, [a])) \implies \text{partition}$ $\text{list}(\text{secure}) = (S)$ cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is $[\text{nil}, \text{new} \cup \text{old}]$	$[A] \rightarrow [[A]]$	Both take $[a, b]$ to $[[a], [b]]$ — each $[a]$ is secure — and to $[[a, b]]$ when $[a, b]$ is.
12	$H \triangleq (\text{head } \text{prefix}^\circ \text{ head}^\circ) \cup (\text{nil}^\circ \text{ nil})$	$[[A]] \rightarrow [[A]]$	$[[a], [b, c]]$ to $[[a, b], [c]]$: $[a]$ is a prefix of $[a, b]$.
13	$R; H \triangleq R \cap (R^\circ \Rightarrow H)$	$[[A]] \rightarrow [[A]]$	$[[a, b, c]]$ to $[[a], [b, c]]$ by $ R $, and $[[a], [b, c]]$ to $[[a, b], [c]]$ by $R \cap H$.
14	$R; H = R \cup R \cap H$ $ R $ is the strict part of R , strictly fewer segments; $R; H$ splits into it and $R \cap H$	$[[A]] \rightarrow [[A]]$	$[[a, b, c]]$ to $[[a], [b, c]]$ by $ R $: $1 < 2$.
15	the specification $\frac{\text{partition } \text{list}(\text{secure})}{\ni} \text{est}(R)$	$[A] \rightarrow [[A]]$	$[a, b]$ gives $[[a, b]]$ when $[a, b]$ is secure , and $[[a], [b]]$ otherwise.

§7.5.1. **secure prefix** $\sqsubseteq$ **prefix secure** — a prefix of a secure segment is secure

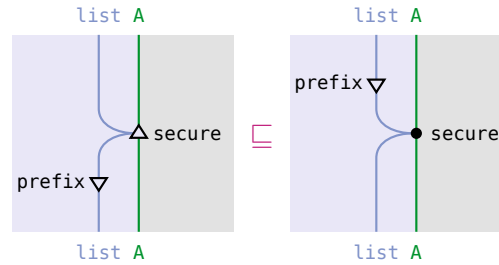
(7.5.1a)



$$\text{secure } \langle \text{ceiling}, \text{ceiling-floor} \rangle \text{ bmax} = \langle \text{ceiling}, \text{ceiling-floor} \rangle \text{ bmax } (\leq N)$$

a stretch passes **secure** before the test exactly where the test's own value passes $\leq N$ after it, which is $\text{secure} \triangleq \text{corefl}(\text{secure})$'s
 $\text{bmax}(\text{ceiling } x, \text{ceiling } x - \text{floor } x) \leq N$

Exercise 7.49



$$\text{secure prefix} \sqsubseteq \text{prefix secure}$$

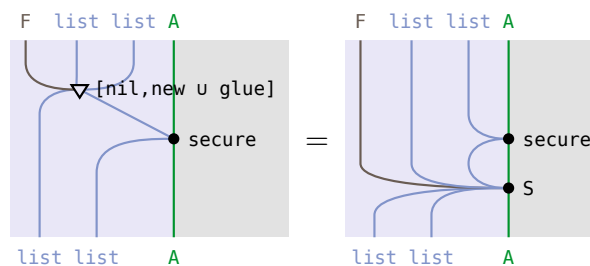
every pair **secure** then **prefix** gives, **prefix** then **secure** gives too — a prefix of a secure stretch is itself secure, which is the prefix-closure
 B&dM p.185 names

§7.5.2. $(\forall a. \text{secure}(N, [a])) \Rightarrow \text{partition list}(\text{secure}) = \langle S \rangle$ — partitioning into secure segments is one fold

(7.5.2a)

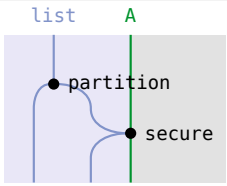
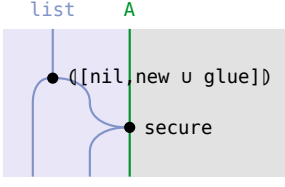
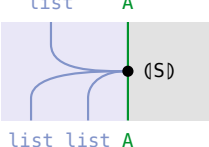
definition	Hinze–Marsden
1 new$\triangleq(\text{wrap} \times 1)$ cons wrap makes the transaction a segment on its own, and cons puts that segment at the front of the schedule: the van is called	
2 glue$\triangleq(1 \times \text{cons}^\circ)$ assocl $(\text{cons} \times 1)$ cons cons$^\circ$ splits the schedule into its first segment and the rest, the first cons puts the transaction at the front of that segment, and the second puts the segment back	
3 old$\triangleq(1 \times \text{cons}^\circ)$ assocl $((\text{cons secure}) \times 1)$ cons glue with secure on the segment the transaction has just joined: the van stays away only where the longer segment still passes	

(7.5.2b)

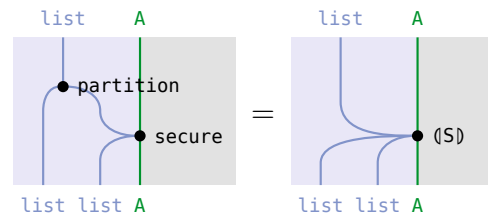


$$(\forall a. \text{secure}(N, [a])) \Rightarrow [\text{nil}, \text{new } u \text{ glue}] \text{ list}(\text{secure}) = F(\text{list}(\text{secure})) S$$

testing every segment for security after the algebra has run is testing it before it ran: a segment **glue** has extended is secure exactly where **old** would have let it be, and the one transaction **new** sets out on its own is secure whatever it is

$(\forall a. \text{secure}(N, [a])) \Rightarrow \text{partition list}(\text{secure}) = \langle S \rangle$ Exercise 7.51	
cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is $[\text{nil}, \text{new} \cup \text{old}]$	
step	Hinze–Marsden
partition list(secure) the specification: cut the transactions every way, then keep the cuts whose every segment is secure	
$= \langle [\text{nil}, \text{new} \cup \text{glue}] \rangle \text{list}(\text{secure})$ partition is the fold that either opens a segment for the transaction in hand or puts it on the front of the segment already open	
$= \langle [\text{nil}, \text{new} \cup \text{old}] \rangle$ fusion : the condition above moves $\text{list}(\text{secure})$ inside that fold, which is what turns glue into old	

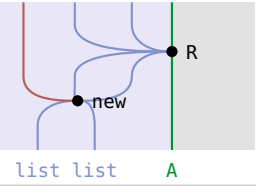
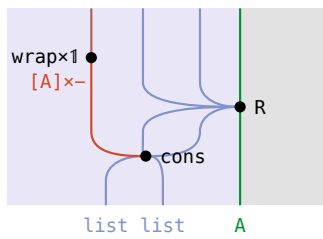
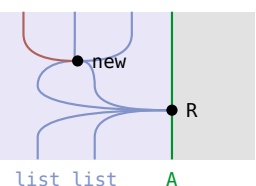
(7.5.2c)



$$(\forall a. \text{secure}(N, [a])) \Rightarrow \text{partition list}(\text{secure}) = \langle S \rangle$$

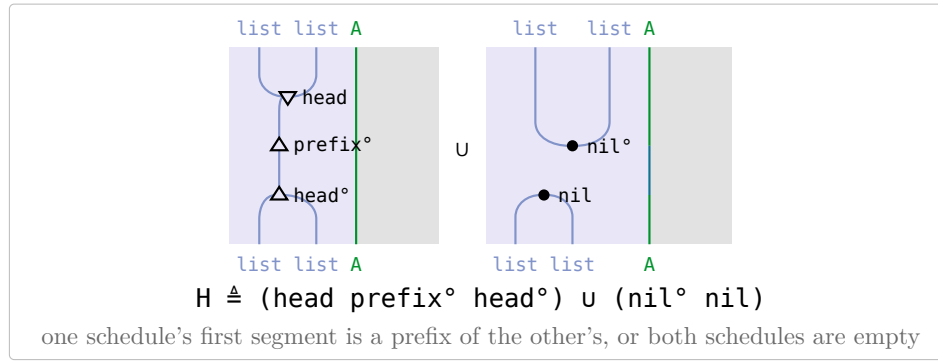
cutting the transactions every way and then keeping the cuts whose every segment is secure is one pass along them that either calls the van or extends the open segment while it stays secure

§7.5.3. $(1 \times R) \text{ new} \sqsubseteq (\text{new} \cup \text{old}) R$ — new on an R-related schedule is matched by new or old

$(1 \times R) \text{ new} \sqsubseteq (\text{new} \cup \text{old}) R$ (7.14)	
calling the van for the transaction on a no-longer schedule gets no further than calling it on this one and shortening the schedule afterwards	
step	Hinze–Marsden
$(1 \times R) \text{ new}$ the left side of (7.14) — $R = \text{clen cast} \leq \text{clen cast}^\circ$, $\text{new} \triangleq \text{new}$	
$= (\text{wrap} \times R) \text{ cons}$ $\text{new} \triangleq (\text{wrap} \times 1) \text{ cons}$ — $(\text{wrap} \times 1) \text{ cons}$ — and $(\text{wrap} \times 1) (1 \times R) = (\text{wrap} \times R)$	
$\sqsubseteq \text{new} R$ cons is monotonic on R — $(1 \times R) \text{ cons} \sqsubseteq \text{cons} R$, consing onto a no-longer schedule leaves it no longer — and $\text{new} \sqsubseteq \text{new} \cup \text{old}$	

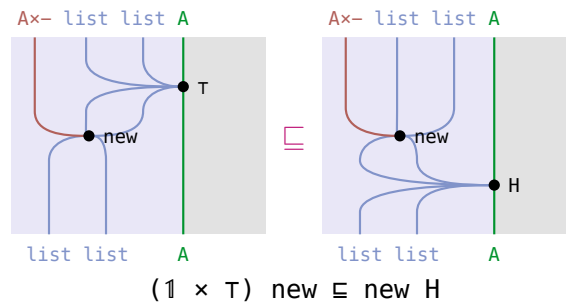
§7.5.4. $H \triangleq (\text{head prefix}^\circ \text{ head}^\circ) \cup (\text{nil}^\circ \text{ nil})$ — the right first segment is a prefix of the left one, or both are empty

(7.5.4a)



§7.5.5. $(1 \times (R;H)) \text{ new} \sqsubseteq (\text{new} \cup \text{old}) (R;H)$ — **new** on an $R;H$ -related schedule is matched by **new** or **old**

(7.18)



whatever schedule the van is called on, the result's first segment is the one transaction $[a]$, so any two results of **new** on that transaction stand in H

$(1 \times (R;H)) \text{ new} \sqsubseteq (\text{new} \cup \text{old}) (R;H)$

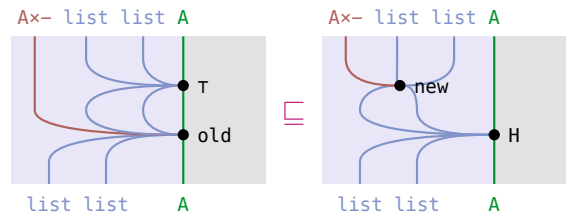
(7.16)

calling the van for the transaction on a $R;H$ -better schedule gets no further than calling it on this one and bettering the whole schedule afterwards

step	Hinze–Marsden
$(1 \times (R;H)) \text{ new}$ the left side of (7.16)	
$\sqsubseteq (1 \times R) \text{ new}$ $R;H \sqsubseteq R \text{ — } R;H = R \cap (R^\circ \Rightarrow H)$	
$\sqsubseteq \text{new } R \cap \text{new } H$ (7.14) as far as its new R line — (7.14) $(1 \times R) \text{ new} \sqsubseteq (\text{new} \cup \text{old}) R$ — and (7.18)	
$= \text{new } (R \cap H)$ new is a map, and a map distributes over $\cap$	
$\sqsubseteq \text{new } (R;H)$ $X \cap Y \sqsubseteq X;Y$, and $\text{new} \sqsubseteq \text{new} \cup \text{old}$	

§7.5.6. $(1 \times (R;H)) \text{ old} \sqsubseteq (\text{new} \cup \text{old}) (R;H)$ — **old** on an $R;H$ -related schedule is matched by **new** or **old**

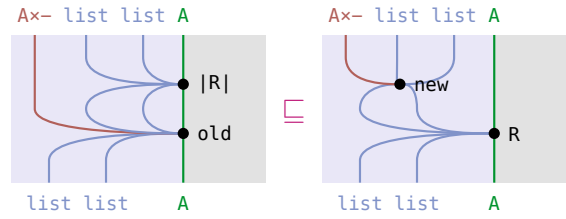
(7.19)



$$(1 \times \tau) \text{ old} \sqsubseteq \text{new H}$$

old leaves the transaction **[a]** at the front of the first segment, and **[a]** is what **new** makes that segment, so the two first segments are **prefix-related** whatever the schedules were

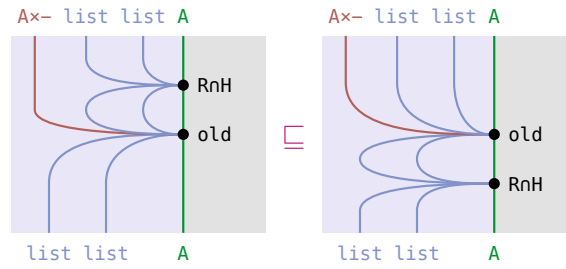
(7.20)



$$(1 \times |R|) \text{ old} \sqsubseteq \text{new R}$$

old keeps the schedule's length, so a strictly shorter one still comes out no longer than the one the van's own segment lengthens

(7.21)



$$(1 \times (R \cap H)) \text{ old} \sqsubseteq \text{old (R} \cap \text{H)}$$

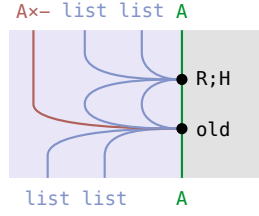
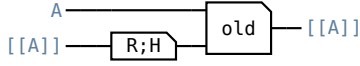
on a tie the one first segment is a prefix of the other, so prefix-closure of **secure** — Exercise 7.49 **secure prefix** $\sqsubseteq$ **prefix secure** — lets **old** fire on this side too, and it keeps both the length and the prefix

$$(1 \times (R;H)) \text{ old} \sqsubseteq (\text{new} \cup \text{old}) (R;H) \quad (7.17)$$

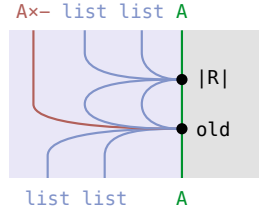
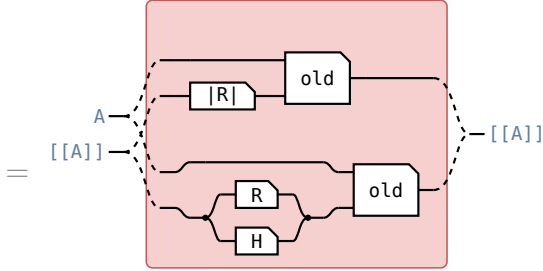
gluing the transaction onto a better schedule for the rest gets no further than gluing it on, or calling the van, and
bettering the whole schedule after,

circuit — the old branch of each union

Hinze–Marsden

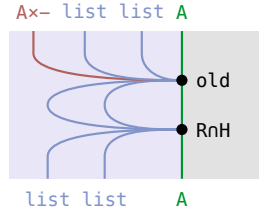
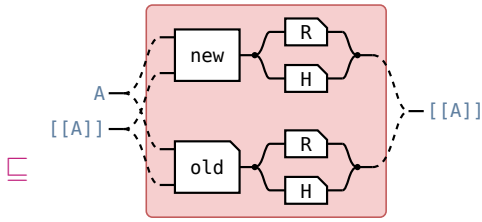


the **old** operand of **new** $\cup$ **old**, in every row



$$((1 \times |R|) \text{ old}) \cup ((1 \times (R \cap H)) \text{ old})$$

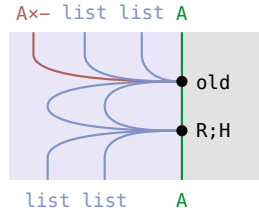
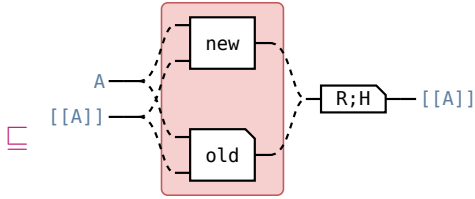
$$R;H = |R| \cup (R \cap H) \text{ — } R;H = |R| \cup R \cap H, \cup \text{ distributes,}$$



$$(\text{new} (R \cap H)) \cup (\text{old} (R \cap H))$$

$$(7.19) \text{ and } (7.20) \text{ on } |R| \text{ — } (7.19) (1 \times \tau) \text{ old} \sqsubseteq \text{new } H, (7.20) (1 \times |R|) \text{ old}$$

$$\sqsubseteq \text{new } R \text{ — and } (7.21) \text{ on } R \cap H \text{ — } (7.21) (1 \times (R \cap H)) \text{ old} \sqsubseteq \text{old} (R \cap H)$$



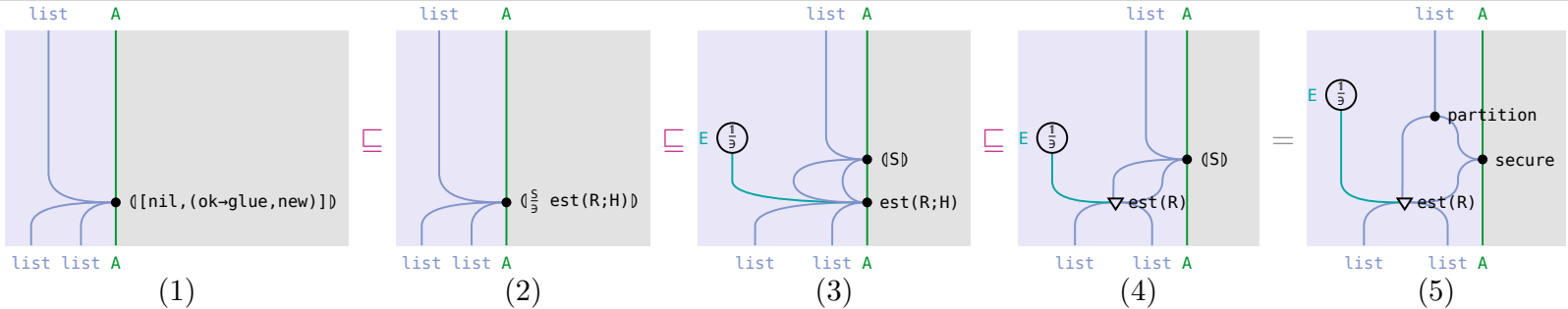
$$(\text{new} \cup \text{old}) (R;H)$$

$$X \cap Y \sqsubseteq X; Y, \text{ converses}$$

§7.5.7. Derivation

$$(\forall a. \text{secure}(N, [a])) \Rightarrow ([\text{nil}, (\text{ok} \rightarrow \text{glue}, \text{new})]) \sqsubseteq \frac{\text{partition list}(\text{secure})}{3} \text{est}(R) \quad (7.5.7a)$$

the fewest secure segments the transactions can be cut into are one pass along them, the next transaction glued
onto the open segment wherever that segment stays secure and the van called where it does not



1	$[\text{nil}, (\text{ok} \rightarrow \text{glue}, \text{new})]$	$[\text{nil}, (\text{ok} \rightarrow \text{glue}, \text{new})] \sqsubseteq \frac{S}{3} \text{est}(R;H)$
2	$\sqsubseteq ([\frac{S}{3} \text{est}(R;H)])$	$([\frac{S}{3} \text{est}(R;H)]) \sqsubseteq \frac{(S)}{3} \text{est}(R;H)$
3	$\sqsubseteq \frac{(S)}{3} \text{est}(R;H)$	$R;H \sqsubseteq R$
4	$\sqsubseteq \frac{(S)}{3} \text{est}(R)$	Exercise 7.51 $(\forall a. \text{secure}(N, [a]))$
5	$= \frac{\text{partition list}(\text{secure})}{3} \text{est}(R)$	$\Rightarrow \text{partition list}(\text{secure}) = (S)$

§8. Thinning Algorithms

§8.1. Thinning

(8.1)

$\text{thin}(Q) \triangleq \sup_n (\epsilon \setminus (Q^\circ \epsilon))$ — $\text{thin}(Q)$ takes a set ys to a subset xs that keeps a Q -lower bound of every $a \in ys$.

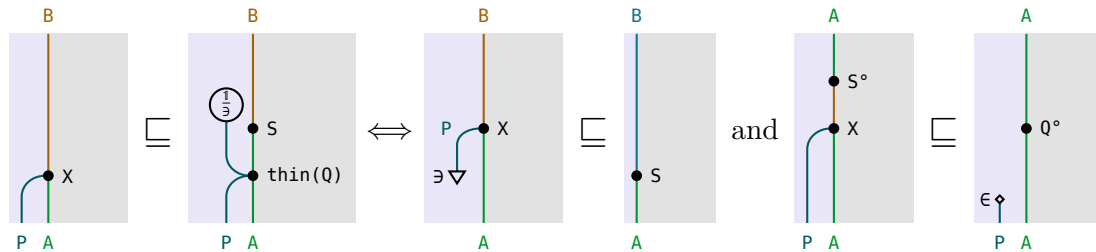
$\text{thin}(\leq) \ ys \ \text{thin}(\leq) \ xs \iff xs \subseteq ys \wedge (\forall a \in ys. \exists b \in xs. b \leq a).$

(8.1a)

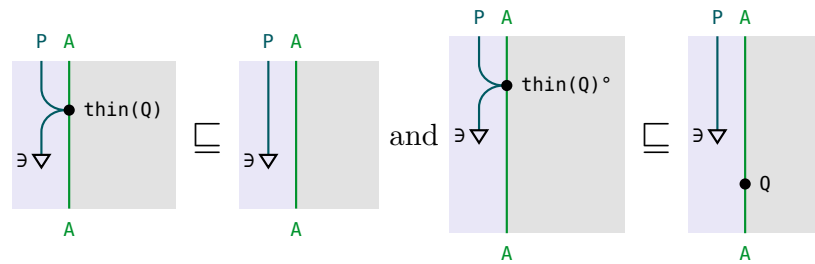
the law	what it says
1 $X \sqsubseteq \frac{S}{\sup} \text{thin}(Q) \iff X \ni \sqsubseteq S \wedge S^\circ X \sqsubseteq Q^\circ \epsilon$	S is the whole algebra: it produces every candidate that $\text{thin}(Q)$ thins. Everything kept is an S -value, and every S -value has a Q -lower bound among the kept ones
2 $\text{thin}(Q) \ni \sqsubseteq \ni$	everything a thinning keeps was in the set
3 $\text{thin}(Q)^\circ \ni \sqsubseteq \ni Q$	every element of the set has a Q -lower bound among the kept ones
4 $Q \sqsubseteq R \implies \text{thin}(Q) \sqsubseteq \text{thin}(R)$	the fewer pairs Q relates, the fewer subsets count as thinnings
5 $1 \sqsubseteq Q \implies 1 \sqsubseteq \text{thin}(Q)$ $Q \sqsubseteq Q \implies \text{thin}(Q) \text{thin}(Q) \sqsubseteq \text{thin}(Q)$	keeping everything is always a legal thinning
6 $Q \sqsubseteq R \wedge 1 \sqsubseteq Q \wedge R R \sqsubseteq R \implies \text{est}(R) = \text{thin}(Q) \text{est}(R)$ $1 \sqsubseteq Q$ $\implies \text{est}(R) \sqsubseteq \text{thin}(Q) \text{est}(R)$	thin-introduction: thinning first cannot lose an R -minimum
7 $\text{thin}(Q) \supseteq \text{est}(Q) \frac{1}{\sup} \quad (8.2) \text{ — } (8.2) \text{est}(Q) \frac{1}{\sup} \sqsubseteq \text{thin}(Q)$	thin-elimination: keeping one element is a thinning, but its domain is the sets $\text{est}(Q)$ is defined on
8 $\frac{S}{\sup} \text{thin}(Q) \supseteq \frac{S}{\sup} \text{est}(R) \frac{1}{\sup}$ (8.3), $Rn(S^\circ S) \sqsubseteq Q$ — thin-elimination variant	the usable variant: R need only refine Q between values S gives one argument
9 $P(\text{thin}(Q)) \text{ union} \sqsubseteq \text{union thin}(Q) \quad (8.4) \text{ — } (8.4)$ $P(\text{thin}(Q)) \text{ union} \sqsubseteq \text{union thin}(Q)$	thinning each member set is a thinning of the union

§8.1.1. $X \sqsubseteq \frac{S}{\sup} \text{thin}(Q) \iff X \ni \sqsubseteq S \wedge S^\circ X \sqsubseteq Q^\circ \epsilon$ — the definition of $\frac{S}{\sup} \text{thin}(Q)$

(8.1.1a)



(8.1.1b)

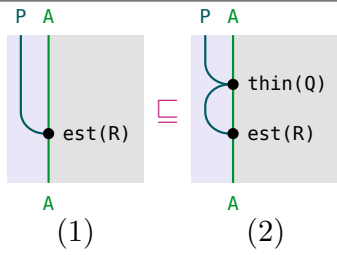


$X \sqsubseteq \frac{S}{\sup} \text{thin}(Q) \iff X \ni \sqsubseteq S \wedge S^\circ X \sqsubseteq Q^\circ \epsilon$ at $X \triangleq \text{thin}(Q)$, $S \triangleq \ni$: the left side is $\text{thin}(Q) \sqsubseteq \text{thin}(Q)$, since $\frac{\ni}{\sup} = \wedge(\ni) = 1$

§8.1.2. $Q \sqsubseteq R \wedge 1 \sqsubseteq Q \wedge R R \sqsubseteq R \implies \text{est}(R) = \text{thin}(Q) \text{est}(R)$ — thinning first, then taking the smallest, is the same as taking the smallest

$$1 \sqsubseteq Q \Rightarrow \text{est}(R) \sqsubseteq \text{thin}(Q) \text{ est}(R) \quad (8.1.2a)$$

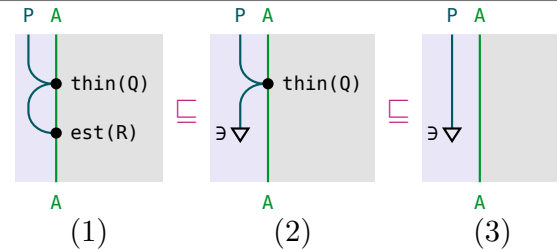
the $\sqsubseteq$ half: keeping everything is a thinning — $1 \sqsubseteq Q$



1	$\text{est}(R)$	$1 \sqsubseteq \text{thin}(Q) \text{ — } 1 \sqsubseteq Q$ $\Rightarrow 1 \sqsubseteq \text{thin}(Q), Q$ $Q \sqsubseteq Q$ $\Rightarrow \text{thin}(Q) \text{ thin}(Q)$ $\sqsubseteq \text{thin}(Q)$
2	$\sqsubseteq \text{thin}(Q) \text{ est}(R)$	

$$\text{thin}(Q) \text{ est}(R) \sqsubseteq \exists \quad (8.1.2b)$$

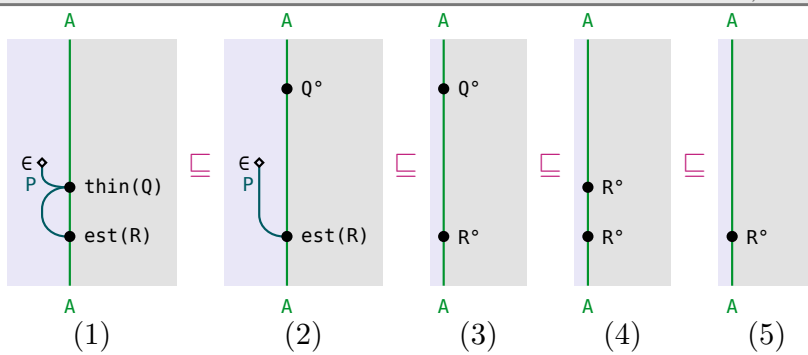
the $\sqsubseteq$ half, first condition of the UP of est at $X \triangleq \text{thin}(Q) \text{ est}(R) \text{ — } X \sqsubseteq \text{est}(R) \Leftrightarrow X \sqsubseteq \exists \wedge \in X \sqsubseteq R^\circ$



1	$\text{thin}(Q) \text{ est}(R)$	$R \cap S \sqsubseteq R$
2	$\sqsubseteq \text{thin}(Q) \exists$	$\text{thin}(Q) \exists \sqsubseteq \exists$
3	$\sqsubseteq \exists$	

$$Q \sqsubseteq R \wedge R R \sqsubseteq R \Rightarrow \in \text{thin}(Q) \text{ est}(R) \sqsubseteq R^\circ \quad (8.1.2c)$$

the $\sqsubseteq$ half, second condition



1	$\in \text{thin}(Q) \text{ est}(R)$	$\in \text{thin}(Q) \sqsubseteq Q^\circ \in$
2	$\sqsubseteq Q^\circ \in \text{est}(R)$	$\in \text{est}(R) \sqsubseteq R^\circ$
3	$\sqsubseteq Q^\circ R^\circ$	$Q \sqsubseteq R$
4	$\sqsubseteq R^\circ R^\circ$	$R R \sqsubseteq R$ $\Rightarrow R^\circ R^\circ \sqsubseteq R^\circ$
5	$\sqsubseteq R^\circ$	

§8.1.3. Properties of thinning

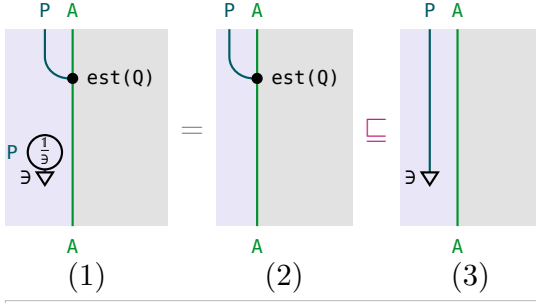
$$\mathbf{est}(Q) \stackrel{1}{\exists} \sqsubseteq \mathbf{thin}(Q)$$

(8.2)

the singleton holding a Q -least member of a set is a thinning of that set

$$\mathbf{est}(Q) \stackrel{1}{\exists} \ni \sqsubseteq \ni$$

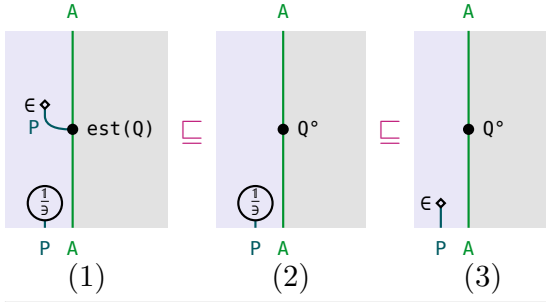
every member of the singleton is a member of the set — $\triangleq \mathbf{thin}, \ni/\ni$ half



1	$\mathbf{est}(Q) \stackrel{1}{\exists} \ni$	$\stackrel{1}{\exists} \ni = 1$
2	$= \mathbf{est}(Q)$	$R \cap S \sqsubseteq R$
3	$\sqsubseteq \ni$	

$$\in \mathbf{est}(Q) \stackrel{1}{\exists} \sqsubseteq Q^\circ \in$$

the singleton's member stands in Q to every member of the set — $\triangleq \mathbf{thin}, \in/(Q^\circ \in)$ half

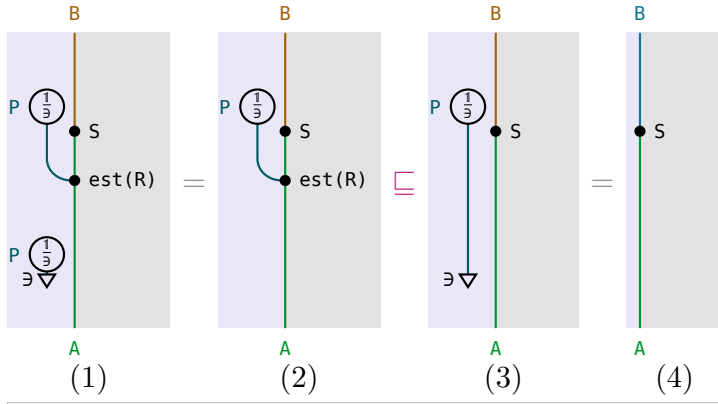


1	$\in \mathbf{est}(Q) \stackrel{1}{\exists}$	$\in \mathbf{est}(R) \sqsubseteq R^\circ$
2	$\sqsubseteq Q^\circ \stackrel{1}{\exists}$	$\stackrel{1}{\exists} \sqsubseteq \in$
3	$\sqsubseteq Q^\circ \in$	

$$R \cap (S^\circ S) \subseteq Q \Rightarrow \frac{S}{S} \text{est}(R) \frac{1}{S} \subseteq \frac{S}{S} \text{thin}(Q) \quad (8.3)$$

$$\frac{S}{S} \text{est}(R) \frac{1}{S} \ni \in S$$

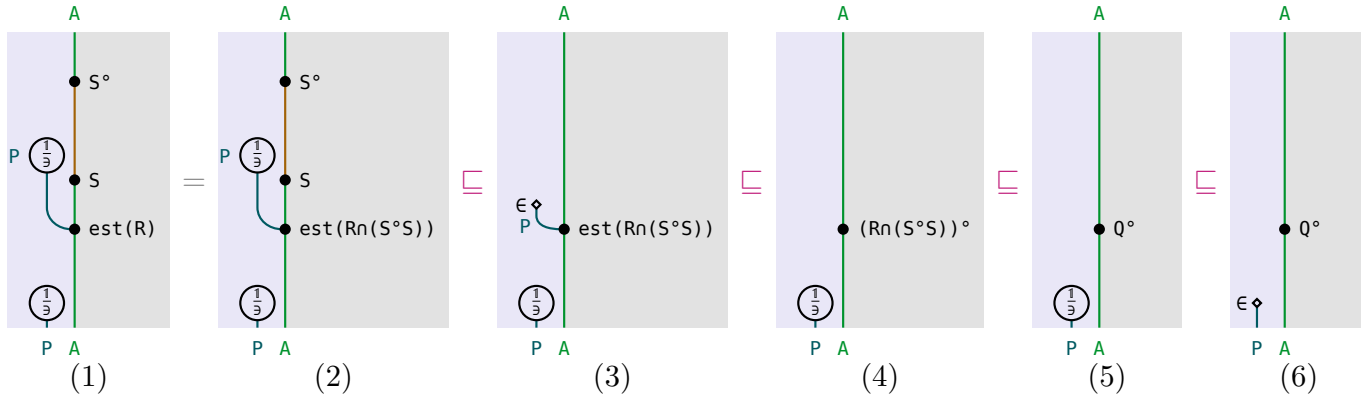
every member of the singleton is a value S returns — $X \subseteq \frac{S}{S} \text{thin}(Q) \Leftrightarrow X \ni \in S \wedge S^\circ X \subseteq Q^\circ \in$, first condition



1	$\frac{S}{S} \text{est}(R) \frac{1}{S} \ni$	$\frac{1}{S} \ni = 1$
2	$= \frac{S}{S} \text{est}(R)$	$R \cap S \subseteq R$
3	$\subseteq \frac{S}{S} \ni$	$\frac{R}{S} \ni = R$
4	$= S$	

$$R \cap (S^\circ S) \subseteq Q \Rightarrow S^\circ \frac{S}{S} \text{est}(R) \frac{1}{S} \subseteq Q^\circ \in$$

the singleton's member stands in Q to every value S returns at the same argument — $X \subseteq \frac{S}{S} \text{thin}(Q) \Leftrightarrow X \ni \in S \wedge S^\circ X \subseteq Q^\circ \in$, second condition



1	$S^\circ \frac{S}{S} \text{est}(R) \frac{1}{S}$	$\frac{S}{S} \text{est}(R \cap (S^\circ S)) = \frac{S}{S} \text{est}(R)$
2	$= S^\circ \frac{S}{S} \text{est}(R \cap (S^\circ S)) \frac{1}{S}$	$T^\circ \frac{1}{S} \subseteq \in$
3	$\subseteq \in \text{est}(R \cap (S^\circ S)) \frac{1}{S}$	$\in \text{est}(R) \subseteq R^\circ$
4	$\subseteq (R \cap (S^\circ S))^\circ \frac{1}{S}$	$R \cap (S^\circ S) \subseteq Q$
5	$\subseteq Q^\circ \frac{1}{S}$	$\frac{1}{S} \subseteq \in$
6	$\subseteq Q^\circ \in$	

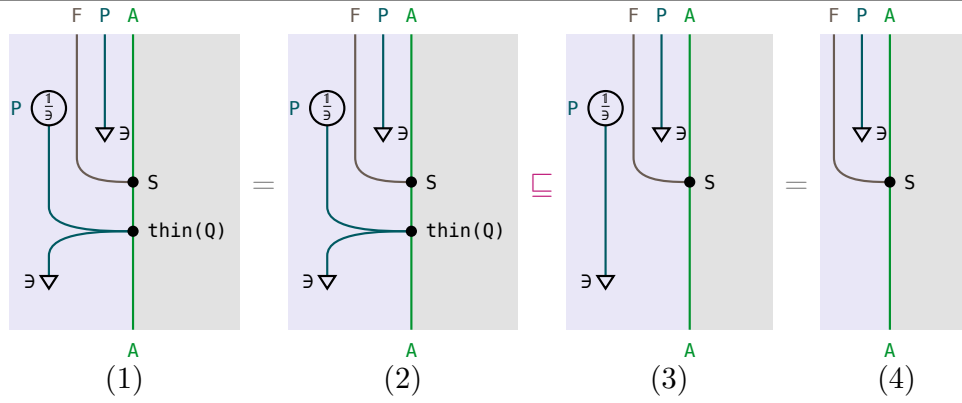
$$\text{preorder}(Q) \wedge S \text{ Pres } Q \Rightarrow (\frac{F(\exists) S}{\exists} \text{thin}(Q)) \sqsubseteq (\frac{S}{\exists} \text{thin}(Q))$$

Theorem 8.1

thinning at every step of the reduce is a thinning of the whole candidate set

$$(\frac{F(\exists) S}{\exists} \text{thin}(Q)) \ni \sqsubseteq (S)$$

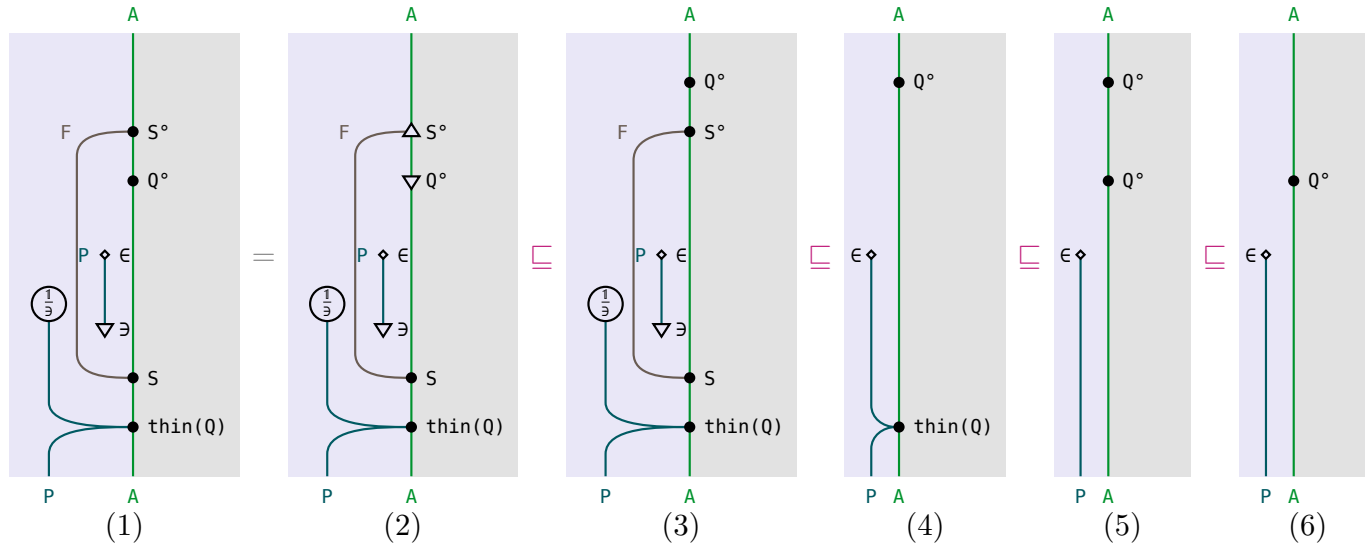
everything the thinning fold keeps is a value of the plain fold — $X \sqsubseteq \frac{S}{\exists} \text{thin}(Q) \iff X \ni \sqsubseteq S \wedge S^\circ X \sqsubseteq Q^\circ \in$, first condition, by fusion from the chain below



1	$\frac{F(\exists) S}{\exists} \text{thin}(Q) \ni$	
2	$= \frac{F(\exists) S}{\exists} \text{thin}(Q) \ni$	$\text{thin}(Q) \ni \sqsubseteq \exists$
3	$\sqsubseteq \frac{F(\exists) S}{\exists} \ni$	$\frac{R}{\exists} \ni = R$
4	$= F(\exists) S$	

$$\text{preorder}(Q) \wedge S \text{ Pres } Q \Rightarrow (S)^\circ (\frac{F(\exists) S}{\exists} \text{thin}(Q)) \sqsubseteq Q^\circ \in$$

every value of the plain fold is Q -above one the thinning fold keeps — $X \sqsubseteq \frac{S}{\exists} \text{thin}(Q) \iff X \ni \sqsubseteq S \wedge S^\circ X \sqsubseteq Q^\circ \in$, second condition, by hylomorphism theorem from the chain below



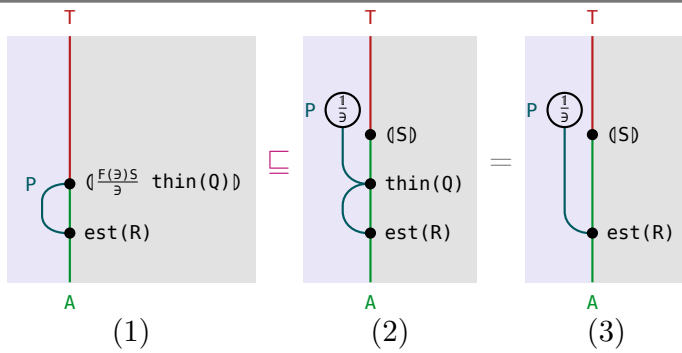
1	$S^\circ F(Q^\circ \in) \frac{F(\exists) S}{\exists} \text{thin}(Q)$	
2	$= S^\circ F(Q^\circ) F(\epsilon) \frac{F(\exists) S}{\exists} \text{thin}(Q)$	$S \text{ Pres } Q$
3	$\sqsubseteq Q^\circ S^\circ F(\epsilon) \frac{F(\exists) S}{\exists} \text{thin}(Q)$	$\implies S^\circ F(Q^\circ) \sqsubseteq Q^\circ S^\circ$
4	$\sqsubseteq Q^\circ \in \text{thin}(Q)$	$S^\circ F(\epsilon) \frac{F(\exists) S}{\exists} \sqsubseteq \epsilon$
5	$\sqsubseteq Q^\circ Q^\circ \in$	$\epsilon \text{thin}(Q) \sqsubseteq Q^\circ \in$
6	$\sqsubseteq Q^\circ \in$	

(8.1.3a)

step	type	example	what it does
S:F(A)→A is 8.1's algebra; example F(X)=V+V×X, S=[wrap,cons], candidate a=cons(v,p)			
S°	A→FA	cons(v,p) ↦ r(v,p)	takes a apart into the input of its last step
F(Q°∈)	FA→F(PA)	r(v,p) ↦ r(v,ps), some p'∈ps with p' Q p	replaces each recursive component by a set holding a Q-no-worse one; v stays
$\frac{F(\exists)S}{\exists}$	F(PA)→PA	r(v,ps) ↦ {cons(v,t) t∈ps}	picks one element from each set, combines by S, collects every result
thin(Q)	PA→PA	{cons(v,t) t∈ps} ↦ ys	keeps a subset; every dropped element is Q-beaten by a kept one
$\sqsubseteq Q^\circ \in$	A→PA	ys holds y with y Q cons(v,p') Q cons(v,p) = a	the thinned set still holds an element Q-no-worse than a (S monotonic on Q, Q transitive)

$Q \sqsubseteq R \wedge \text{preorder}(Q) \wedge \text{preorder}(R) \wedge S \text{ Pres } Q \Rightarrow (\frac{F(\exists) S}{\exists} \text{thin}(Q)) \Downarrow \text{est}(R) \sqsubseteq \frac{(S)}{\exists} \text{est}(R)$ Corollary 8.1

the thinning fold refines the optimisation problem itself



1	$(\frac{F(\exists) S}{\exists} \text{thin}(Q)) \Downarrow \text{est}(R)$	Theorem 8.1 $\text{preorder}(Q) \wedge S \text{ Pres } Q$
2	$\sqsubseteq \frac{(S)}{\exists} \text{thin}(Q) \text{est}(R)$	$\Rightarrow (\frac{F(\exists) S}{\exists} \text{thin}(Q)) \sqsubseteq \frac{(S)}{\exists} \text{thin}(Q)$
3	$= \frac{(S)}{\exists} \text{est}(R)$	

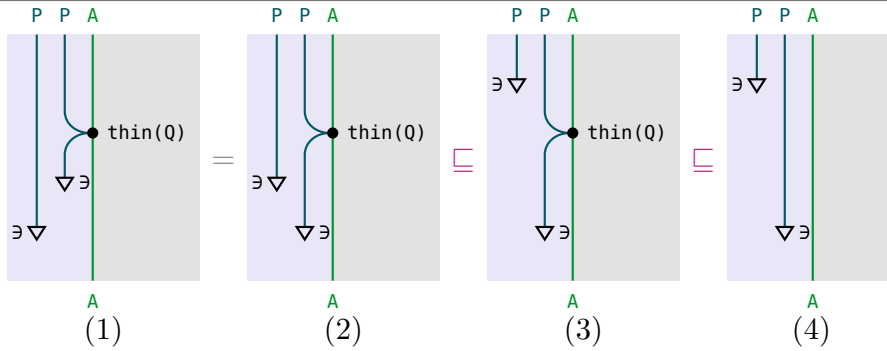
$P(\text{thin}(Q)) \text{ union} \sqsubseteq \text{union thin}(Q)$

(8.4)

thinning each member set, then taking the union, is a thinning of the union

$P(\text{thin}(Q)) \text{ union} \exists \sqsubseteq \exists$

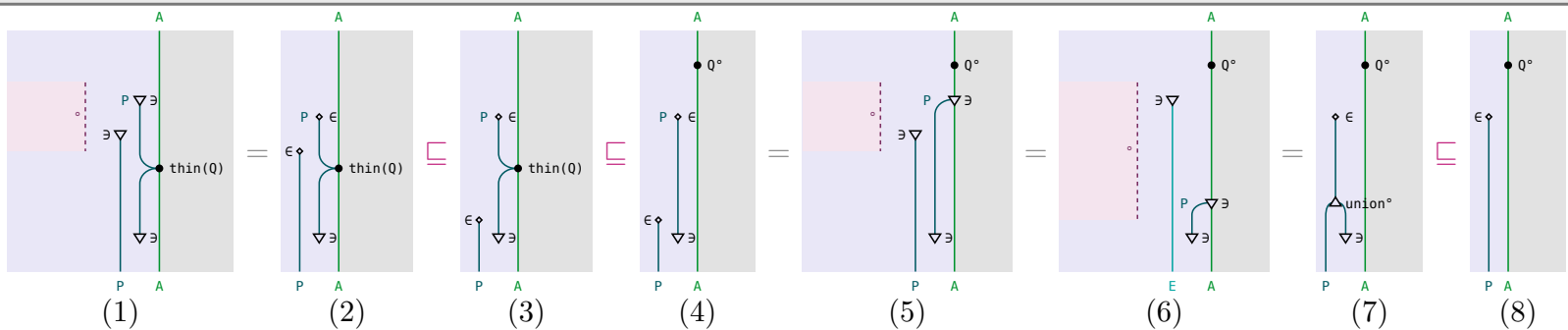
every member of the result is a member of a member set — $X \sqsubseteq \frac{S}{\exists} \text{thin}(Q) \iff X \exists \sqsubseteq S \wedge S^\circ X \sqsubseteq Q^\circ \in$, first condition



1	$P(\text{thin}(Q)) \text{ union} \exists$	$\text{union} \exists = \exists \exists$
2	$= P(\text{thin}(Q)) \exists \exists$	$P(R) \exists \sqsubseteq \exists R$
3	$\sqsubseteq \exists \text{thin}(Q) \exists$	$\text{thin}(Q) \exists \sqsubseteq \exists$
4	$\sqsubseteq \exists \exists$	

$(\exists \exists)^\circ P(\text{thin}(Q)) \text{ union} \sqsubseteq Q^\circ \in$

every member of a member set has a Q -lower bound in the result — $X \sqsubseteq \frac{S}{\exists} \text{thin}(Q) \iff X \exists \sqsubseteq S \wedge S^\circ X \sqsubseteq Q^\circ \in$, second condition



1	$(\exists \exists)^\circ P(\text{thin}(Q)) \text{ union}$	$(R S)^\circ = S^\circ R^\circ$
2	$= \in \in P(\text{thin}(Q)) \text{ union}$	$\in P(R) \sqsubseteq R \in$
3	$\sqsubseteq \in \text{thin}(Q) \in \text{union}$	$\in \text{thin}(Q) \sqsubseteq Q^\circ \in$
4	$\sqsubseteq Q^\circ \in \in \text{union}$	$(R S)^\circ = S^\circ R^\circ$
5	$= Q^\circ (\exists \exists)^\circ \text{union}$	$\text{union} \exists = \exists \exists$
6	$= Q^\circ (\text{union} \exists)^\circ \text{union}$	$(R S)^\circ = S^\circ R^\circ$
7	$= Q^\circ \in \text{union}^\circ \text{union}$	$\text{union} = E(\exists)$
8	$\sqsubseteq Q^\circ \in$	

§8.2. Paths in a layered network

(8.2a)

	name	type	definition	meaning
1	F	$\mathcal{A} \times \mathcal{A} \rightarrow \mathcal{A}$	$F(B, X) = B + (B \times X)$	a path is either one vertex a , or a vertex a followed by a path xs
2	α	$F(A, LA) \rightarrow LA$	$\alpha = [\text{wrap}, \text{cons}]$	build a path: wrap one vertex as $[a]$, or put a in front of xs
3	wrapz	$A \rightarrow LA \times \mathbb{N}$	$\text{wrapz} \triangleq \langle \text{wrap}, \text{zero} \rangle, a \mapsto ([a], 0)$	a path of one vertex, no edge, cost 0
4	consw	$A \times (LA \times \mathbb{N}) \rightarrow LA \times \mathbb{N}$	$(a, (xs, n)) \mapsto ([a] \# xs, \text{wt}(a, \text{head}(xs)) + n)$	put a in front of the path; the cost adds the weight of the new edge $a \rightarrow \text{head}(xs)$
5	cost	$LA \rightarrow \mathbb{N}$	$\text{cost} \triangleq \langle [\text{wrapz}, \text{consw}] \rangle \pi_2$	the cost of $[a_0, \dots, a_n]$ is $\text{wt}(a_0, a_1) + \dots + \text{wt}(a_{n-1}, a_n)$
6	R	$LA \rightarrow LA$	$R \triangleq \text{cost} \leq \text{cost}^\circ$	xs is cheaper than ys
7	$L(R)$	$LA \rightarrow LB$	$\langle F(\exists, 1) \alpha \rangle = L(\exists)$	$L(\exists)$ relates a list of layers to every path that chooses one vertex from each layer
8	minpath	$L(PA) \rightarrow LA$	$\text{minpath} \sqsubseteq \frac{L(\exists)}{\exists} \text{est}(R)$	the input $[x_0, \dots, x_n] : L(PA)$ is a list of layers, each a set of vertices; a path chooses one vertex from each layer, and minpath returns a cheapest one
9	Q	$LA \rightarrow LA$	$Q \triangleq R \cap (\text{head} \text{ head}^\circ)$	like R , and the two paths also start at the same vertex: $\text{head}(xs) = \text{head}(ys)$
10	l	$B \rightarrow B + C$	$a \mapsto l(a)$	mark a value as the left alternative
11	r	$C \rightarrow B + C$	$b \mapsto r(b)$	mark a value as the right alternative
12	cpl	$PB \times A \rightarrow P(B \times A)$	$\text{cpl} \triangleq \text{cp}(1 \times A)$	pair every element of the set with the value beside it
13	cpr	$A \times PB \rightarrow P(A \times B)$	$\text{cpr} \triangleq \text{cp}(A \times 1)$	pair the value beside the set with every element of the set
14	$\frac{F(\exists, 1)}{\exists}$	$F(PB, X) \rightarrow PF(B, X)$	$\frac{F(\exists, 1)}{\exists} = [P(l), \text{cpl } P(r)]$	cpl transpose: from layer x and a set of paths ps , choose a vertex $a \in x$
15	$\frac{F(1, \exists)}{\exists}$	$F(B, PX) \rightarrow PF(B, X)$	$\frac{F(1, \exists)}{\exists} = [\frac{1}{\exists} P(l), \text{cpr } P(r)]$	cpr transpose: from a vertex a and a set of paths ps , choose a tail $xs \in ps$
16	step	$A \times P(LA) \rightarrow LA$	$\text{step} \triangleq \text{cpr } P(\text{cons}) \text{est}(R)$	put a in front of every path in ps and keep a cheapest one

(8.2b)

the law	what it says
1 $F(\exists, Q) \alpha \sqsubseteq F(\exists, 1) \alpha Q$	$F(\exists, 1) \alpha$ is monotonic on Q ; on R it is not, since the next edge can cost arbitrarily much
2 $S \text{ head} \sqsubseteq [1, \pi_1] \quad S \triangleq F(1, \exists) \alpha$	S head is simple, which gives $R \cap (S^\circ S) \sqsubseteq Q$: between two paths S builds from one argument, equal cost and equal head already means Q

§8.2.1. Derivation

$\frac{F(\exists, 1) \quad P(\frac{F(1, \exists) \alpha}{\exists} \text{ est}(R))}{\exists} \text{ est}(R) \sqsubseteq \frac{F(\exists, 1) \alpha}{\exists} \text{ est}(R) \quad (8.2.1a)$ a least-cost path in a layered network, as a fold over the layers	
- Why it is needed: $\text{est}(R)$ keeps one cheapest w of the set $\Lambda(S)(x)$; $\text{thin}(Q)$ needs every dropped z to be Q-beaten by a kept one. For $\{w\}$ alone to be a thinning we need $w Q z$ for every $z \in \Lambda(S)(x)$. - We have $w R z$ (w is cheapest) and $w (S^\circ S) z$ (both are built by S from the same argument x); the hypothesis turns the two into $w Q z$: among candidates from one argument, cheaper already means Q-better.	
(1) (2) (3)	
1 $\frac{F(\exists, 1) \quad P(\frac{F(1, \exists) \alpha}{\exists} \text{ est}(R))}{\exists} \text{ est}(R)$	$\frac{F(\exists, 1) \quad P(\frac{F(1, \exists) \alpha}{\exists} \text{ est}(R))}{\exists} \sqsubseteq \frac{F(\exists, \exists) \alpha}{\exists} \text{ thin}(Q)$
2 $\sqsubseteq \frac{F(\exists, \exists) \alpha}{\exists} \text{ thin}(Q) \text{ est}(R)$	$\frac{F(\exists, \exists) \alpha}{\exists} \text{ thin}(Q) \text{ est}(R) \sqsubseteq \frac{F(\exists, 1) \alpha}{\exists} \text{ est}(R)$
3 $\sqsubseteq \frac{F(\exists, 1) \alpha}{\exists} \text{ est}(R)$	

$S \triangleq F(1, \exists) \alpha, R \cap (S^\circ S) \sqsubseteq Q \Rightarrow \frac{F(\exists, 1) \quad P(\frac{F(1, \exists) \alpha}{\exists} \text{ est}(R))}{\exists} \sqsubseteq \frac{F(\exists, \exists) \alpha}{\exists} \text{ thin}(Q) \quad (8.2.1b)$ thinning the algebra of a layered network costs no more than taking the program's two cases	
(1) (2) (3) (4) (5) (6) (7)	
1 $\frac{F(\exists, 1) \quad P(\frac{F(1, \exists) \alpha}{\exists} \text{ est}(R))}{\exists}$	
2 $= \frac{F(\exists, 1) \quad P(\frac{F(1, \exists) \alpha}{\exists} \text{ est}(R))}{\exists}$	$P(\frac{1}{\exists}) E(\exists) = 1$
3 $= \frac{F(\exists, 1) \quad P(\frac{F(1, \exists) \alpha}{\exists} \text{ est}(R) \frac{1}{\exists})}{\exists} E(\exists)$	(8.3) $R \cap (S^\circ S) \sqsubseteq Q$
4 $\sqsubseteq \frac{F(\exists, 1) \quad P(\frac{F(1, \exists) \alpha}{\exists}) \quad P(\text{thin}(Q)) \quad E(\exists)}{\exists}$	$\Rightarrow \frac{S}{\exists} \text{ est}(R) \frac{1}{\exists} \sqsubseteq \frac{S}{\exists} \text{ thin}(Q)$
5 $\sqsubseteq \frac{F(\exists, 1) \quad P(\frac{F(1, \exists) \alpha}{\exists}) \quad E(\exists) \quad \text{thin}(Q)}{\exists}$	(8.4) $P(\text{thin}(Q)) \text{ union} \sqsubseteq \text{union thin}(Q)$
6 $= \frac{F(\exists, 1) \quad F(1, \exists) \quad \alpha}{\exists} \text{ thin}(Q)$	$\bigvee \frac{S}{\exists} = \bigvee \frac{P(\frac{S}{\exists})}{\exists} \text{ union}$
7 $= \frac{F(\exists, \exists) \alpha}{\exists} \text{ thin}(Q)$	$F(R, 1) F(1, S) = F(R, S)$
$= \text{cp}(F) P(\alpha) \text{ est}(R) = [\text{wrap}, \text{step}]$ at the layered network $(F(A, X) = A + A \times X, B \text{ the paths } V^+, \alpha = [\text{wrap}, \text{cons}]): \frac{F(1, \exists)}{\exists} P(\alpha) \text{ est}(R) = [\text{wrap}, \text{step}] - \text{step} \triangleq \text{cpr}$ $P(\text{cons}) \text{ est}(R)$ $= \frac{F(\exists, 1)}{\exists} = 1 + \text{cpl} - \frac{F(\exists, 1)}{\exists} = [P(l), \text{cpl } P(r)]$	

$$\begin{array}{ccccc}
 F(A, X) & = & A & + & A \times X \\
 & & | & & | \quad | \\
 & & R & & R \quad S \\
 & & \downarrow & & \downarrow \quad \downarrow \\
 F(A', X') & = & A' & + & A' \times X'
 \end{array}$$

(8.2.1c)

u $F(R, S) v$ holds iff one of:

- u, v are both in l and $d R d'$: the leaf moves by R ;

- u, v are both inr and $p_1 \ R \ q_1 \wedge p_2 \ S \ q_2$: the vertex moves by R , the rest by S .
- An inl never relates to an inr : $R+S \models [R \ \text{inl}, S \ \text{inr}]$ returns to the summand it came from, and $F(1)=1$ allows no crossing.
- $F(\exists, \exists)$ relates $\text{inr}(\{a, b\}, \{p, q\})$ to $\text{inr}(a, p), \text{inr}(a, q), \text{inr}(b, p), \text{inr}(b, q)$, and $\text{inl}(\{a, b\})$ to $\text{inl}(a), \text{inl}(b)$.

§8.3. Implementing thin

(8.3a)

	name	type	definition	meaning
1	setify	$[A] \rightarrow PA$	$(\text{setify } \exists)(xs, x) \iff x \in xs$	the set of a list's elements
2	cup	$PA \times PA \rightarrow PA$	$\text{cup} \triangleq \frac{(\pi_1 \exists) \cup (\pi_2 \exists)}{\exists}$	the union of two sets
3	cp(F)	$F(PA) \rightarrow PFA$	$\text{cp}(F) \triangleq \frac{F(\exists)}{\exists}$	every F-shaped value whose parts are chosen from the given sets
4	listcp(F,A,X)	$\text{relator}(\text{list}(X)) \rightarrow \text{list}(\text{relator}(X))$	$\text{listcpFn}(\text{zer}, x) = []$, $\text{listcpFn}(\text{PolyC.one}, u) = [u, ()]$, $\text{listcpFn}(\text{const}(a), c) = [c]$, $\text{listcpFn}(\text{arg}_1, a) = [a]$, $\text{listcpFn}(\text{arg}_2, xs) = xs$, $\text{listcpFn}(l \oplus a, l(u)) =$ $\text{list}(\text{inl}, \text{listcpFn}(l, u))$, $\text{listcpFn}(a \oplus r, r(v)) =$ $\text{list}(\text{inr}, \text{listcpFn}(r, v))$, $\text{listcpFn}(l \otimes r, (u, v)) =$ $\text{cppFn}(\text{listcpFn}(l, u), \text{listcpFn}(r, v))$	by the shape of F: a constant or arg_1 is a one-element list, arg_2 the list itself, a sum lists its summand, a product pairs every element of one list with every element of the other
5	linear	$P(\text{PolyC}(\imath))$	$\text{linear}(\text{zer}) = \text{true}$, $\text{linear}(\text{PolyC.one}) = \text{true}$, $\text{linear}(\text{const}(a)) = \text{true}$, $\text{linear}(\text{arg}_1) = \text{true}$, $\text{linear}(\text{arg}_2) = \text{true}$, $\text{linear}(l \oplus r) = \text{linear}(l) \wedge$ $\text{linear}(r)$, $\text{linear}(l \otimes r) = \text{linear}(l) \wedge \text{linear}(r) \wedge (\neg$ $\text{hasArg}_2(l) \vee \neg \text{hasArg}_2(r))$	no product in F has the argument on both sides
6	sort($\leq$)	$PA \rightarrow LA$	$\text{sort}(\leq) \triangleq \text{setify}^\circ \text{ordered}(\leq)$	read a set back as one of its $\leq$ -ordered listings
7	bump(Q)	$A \times [A] \rightarrow [A]$	$(a, []) \mapsto [a]$, $\text{bump}(Q)((a, [b] \# xs), ys) \iff Q(a, b) \wedge ys =$ $[a] \# xs \vee Q(b, a) \wedge ys = [b] \# xs \vee \neg Q(a, b) \wedge \neg Q(b, a) \wedge ys =$ $[a] \# [b] \# xs$	put a in front of a thinned list, dropping whichever of a and the old head the other beats
8	thinlist(Q)	$[A] \rightarrow [A]$	$\text{thinlist}(Q) \triangleq (\text{setify } \text{thin}(Q) \text{ setify}^\circ) \text{ n subseq}$	only drop elements, and on the set of the elements be a thinning by Q
9			$T \sqsubseteq \text{thinlist}(Q) \iff T \sqsubseteq \text{subseq} \wedge T \text{ setify} \sqsubseteq \text{setify } \text{thin}(Q)$	what a list implementation of $\text{thin}(Q)$ must do: only drop elements, and thin the set
10			$\text{preorder}(Q) \implies ([\text{nil}, \text{bump}(Q)]) \sqsubseteq \text{thinlist}(Q)$	an implementation: thin a list in one pass, bumping each element into the thinned rest
11	minlist(Q)	$[A] \rightarrow A$	$\text{minlist}(Q) \triangleq \text{setify } \text{est}(Q)$	a Q-least element of the list

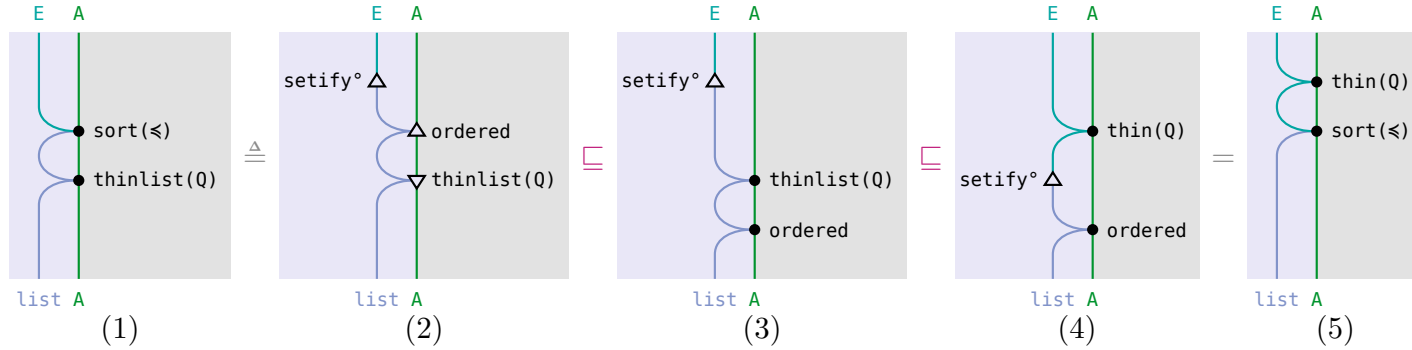
§8.3.1. Sorting sets

(8.3.1a)

the law	what it says
1 $\text{preorder}(Q) \wedge \text{connected}(Q) \Rightarrow$ $\text{bumpFold}(Q)([a]\#xs,ys) \Leftarrow (\exists w. \text{minlist}(Q)([a]\#xs,w)$ $\wedge ys = [w])$ (8.5)	what thinning should come to when it can: one element
2 $\text{sort}(\leq) \text{thinlist}(Q) \sqsubseteq \text{thin}(Q) \text{sort}(\leq)$ (8.6) — (8.6) $\text{sort}(\leq) \text{thinlist}(Q) \sqsubseteq \text{thin}(Q) \text{sort}(\leq)$	the thinning may run after the sort
3 $\text{sort}(\leq) \text{minlist}(R) \sqsubseteq \text{est}(R)$ (8.7)	a minimum of the sorted list is a minimum of the set
4 $\text{sort}(f \leq f^\circ) \text{list}(f) \sqsubseteq P(f) \text{sort}(\leq)$ (8.8)	shunt a function through a sort
5 $\text{Coreflexive}(p) \Rightarrow \text{sort}(\leq) \text{filter}(p) \sqsubseteq E(p) \text{sort}(\leq)$ (8.9) $p : A \rightarrow A$	filtering a sorted list sorts the restricted set
6 $\text{preorder}(\leq) \wedge \text{connected}(\leq) \Rightarrow (\text{sort}(\leq) \times \text{sort}(\leq))$ $\text{merge}(\leq) \sqsubseteq \text{cup sort}(\leq)$ (8.10)	merging two sorted lists sorts their union
7 $F(\text{sort}(\leq)) \text{listcp} \sqsubseteq \text{cp}(F) \text{sort}(F(\leq))$ (8.11), $FX=L+E \times X$	listcp is the list implementation of the cartesian product $\text{cp}(F)$
8 $\text{linear}(F) \Rightarrow \text{relator}(\text{sort}(\leq)) \text{listcp}(F,A,X) \sqsubseteq$ $\text{cp}(\text{relator}(F,A)) \text{sort}(\text{relator}(\leq))$ (8.11), F polynomial and linear	the same for every linear polynomial F

sort($\leq$) thinlist(Q) $\sqsubseteq$ thin(Q) sort($\leq$)

(8.6)

every list got by sorting the set and then thinning it by Q is also got by thinning the set by Q and then sorting.

1 $\text{sort}(\leq) \text{thinlist}(Q)$	$\text{sort}(\leq) \triangleq \text{setify}^\circ \text{ordered}(\leq)$
2 $\triangleq \text{setify}^\circ \text{ordered thinlist}(Q)$	$\text{ordered thinlist}(Q) \sqsubseteq \text{thinlist}(Q) \text{ordered}$
3 $\sqsubseteq \text{setify}^\circ \text{thinlist}(Q) \text{ordered}$	$\text{setify}^\circ \text{thinlist}(Q) \sqsubseteq \text{thin}(Q) \text{setify}^\circ$
4 $\sqsubseteq \text{thin}(Q) \text{setify}^\circ \text{ordered}$	
5 $= \text{thin}(Q) \text{sort}(\leq)$	

$\text{Coreflexive}(p) \wedge f \text{ Pres} \leq \Rightarrow F(\text{sort}(\leq)) \text{ listcp list}(f) \text{ filter}(p) \sqsubseteq \frac{F(\exists) f p}{\exists} \text{ sort}(\leq)$ Lemma 8.1	
sorting each argument set ($F(\text{sort}(\leq))$), forming every combination of their elements (listcp), applying f to each ($\text{list}(f)$) and keeping those that pass p ($\text{filter}(p)$) gives one of the sorted lists of the set of all $f(u)$ that pass p , with u drawn from the argument sets.	
1	$F(\text{sort}(\leq)) \text{ listcp list}(f) \text{ filter}(p)$
2	$\sqsubseteq \text{cp}(F) \text{ sort}(F(\leq)) \text{ list}(f) \text{ filter}(p)$
3	$\sqsubseteq \text{cp}(F) \text{ sort}(f \leq f^\circ) \text{ list}(f) \text{ filter}(p)$
4	$\sqsubseteq \text{cp}(F) P(f) \text{ sort}(\leq) \text{ filter}(p)$
5	$\sqsubseteq \text{cp}(F) E(f p) \text{ sort}(\leq)$
6	$= \frac{F(\exists) f p}{\exists} \text{ sort}(\leq)$
	$F(\text{sort}(\leq)) \text{ listcp} \sqsubseteq \text{cp}(F) \text{ sort}(F(\leq))$ $\text{Map}(f)$ $\Rightarrow f \text{ Pres } R \Leftrightarrow F(R) \sqsubseteq f R f^\circ$ $\text{sort}(f \leq f^\circ) \text{ list}(f) \sqsubseteq P(f) \text{ sort}(\leq)$ $\text{Coreflexive}(p)$ $\Rightarrow \text{sort}(\leq) \text{ filter}(p) \sqsubseteq E(p) \text{ sort}(\leq)$ $\text{Map}(f)$ $\Rightarrow P(f) = E(f)$ $\frac{S}{\exists} E(R) = \frac{S R}{\exists}$

(8.3.1b)

At $F(X)=L+Ex$, $f(r(e,x))=e+x$, $\leq$ is $\leq$, p passes the odd numbers, input $r(3,\{4,1,2\})$.

left side	what it does	result
1 input	an $r(e, xs)$, xs a set	$r(3, \{4, 1, 2\})$
2 $F(\text{sort}(\leq))$	sorts the set in the X place; e is untouched	$r(3, [1, 2, 4])$
3 listcp	pairs e with each element, in list order	$[r(3, 1), r(3, 2), r(3, 4)]$
4 $\text{list}(f)$	f on each	$[4, 5, 7]$
5 $\text{filter}(p)$	keeps the odd ones, order kept	$[5, 7]$

right side	what it does	result
1 input		$r(3, \{4, 1, 2\})$
2 $F(\exists)$	relates $r(3, xs)$ to each $r(3, x)$ with $x \in xs$	$r(3, 4), r(3, 1), r(3, 2)$
3 f		7, 4, 5
4 p	only odd values pass	7, 5
5 $\wedge(\dots)$	collects every result into one set	$\{5, 7\}$
6 $\text{sort}(\leq)$		$[5, 7]$

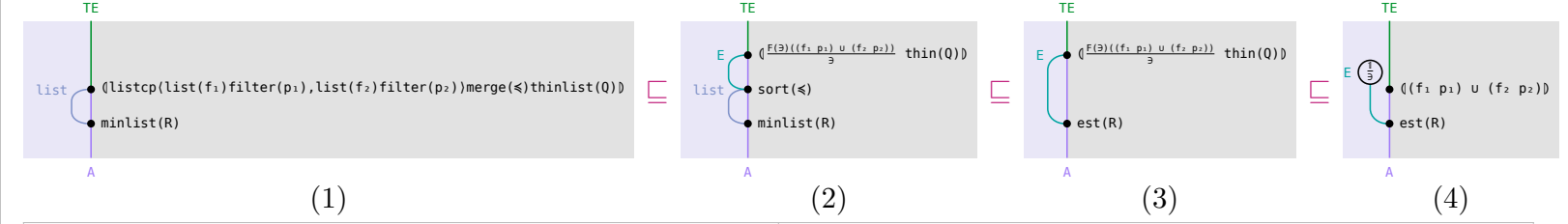
- $\sqsubseteq$: the left's $[5, 7]$ is one result of the right; with ties $\text{sort}(\leq)$ has several orders.
- f preserving $\leq$ is used at $\text{list}(f)$: $[1, 2, 4]$ is sorted, so $[4, 5, 7]$ is; $\text{filter}(p)$ only drops elements. With $f(r(e, x))=e-x$ the left gives $[1, -1]$ after p , not sorted, so $\sqsubseteq$ fails.

§8.3.2. Binary thinning

(8.3.2a)

Binary thinning data: Q a preorder with $Q \sqsubseteq R$ and both $f_1 p_1, f_2 p_2$ monotonic on Q ; $\leq$ a connected preorder with both f_1, f_2 monotonic on $\leq$; R a preorder.

Theorem 8.2
 $\text{Coreflexive}(p_1) \wedge \text{Coreflexive}(p_2) \wedge Q \sqsubseteq R \wedge \text{preorder}(Q) \wedge \text{preorder}(R) \wedge (f_1 \text{ Pres } \leq \wedge f_2 \text{ Pres } \leq) \implies$
 $(\text{listcp}(\text{list}(f_1) \text{ filter}(p_1), \text{list}(f_2) \text{ filter}(p_2)) \text{ merge}(\leq) \text{ thinlist}(Q)) \text{ minlist}(R) \sqsubseteq$
 $\frac{((f_1 \text{ p}_1) \cup (f_2 \text{ p}_2))}{\exists} \text{ est}(R)$
 a fold on sorted lists of partial solutions, thinned at every step, refines the thinning specification — at binary thinning data.



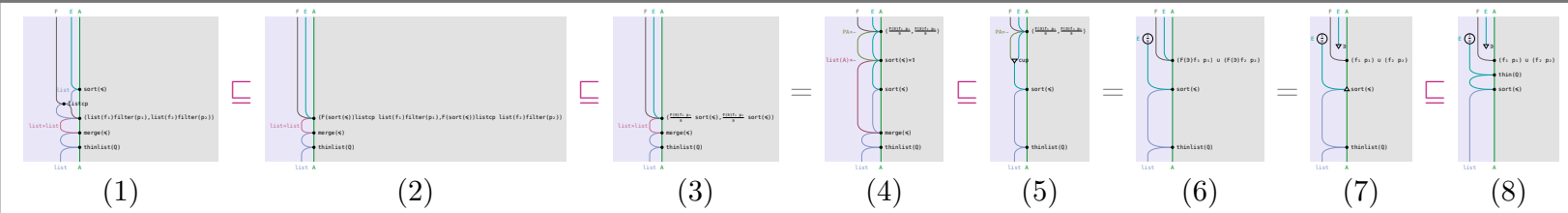
1	$(\text{listcp}(\text{list}(f_1) \text{ filter}(p_1), \text{list}(f_2) \text{ filter}(p_2)) \text{ merge}(\leq) \text{ thinlist}(Q)) \text{ minlist}(R)$	(8.3.2c) $\text{Coreflexive}(p_1) \wedge \text{Coreflexive}(p_2) \wedge \text{preorder}(\leq) \wedge \text{connected}(\leq) \wedge f_1 \text{ Pres } \leq \wedge f_2 \text{ Pres } \leq$ $\implies F(\text{sort}(\leq)) \text{ listcp}(\text{list}(f_1) \text{ filter}(p_1), \text{list}(f_2) \text{ filter}(p_2)) \text{ merge}(\leq) \text{ thinlist}(Q) \sqsubseteq \frac{F(\exists) ((f_1 \text{ p}_1) \cup (f_2 \text{ p}_2))}{\exists} \text{ thin}(Q) \text{ sort}(\leq)$
2	$\sqsubseteq \frac{F(\exists) ((f_1 \text{ p}_1) \cup (f_2 \text{ p}_2))}{\exists} \text{ thin}(Q) \text{ sort}(\leq) \text{ minlist}(R)$	$\text{sort}(\leq) \text{ minlist}(R) \sqsubseteq \text{est}(R)$
3	$\sqsubseteq \frac{F(\exists) ((f_1 \text{ p}_1) \cup (f_2 \text{ p}_2))}{\exists} \text{ thin}(Q) \text{ est}(R)$	$S_1 \text{ Pres } Q \wedge S_2 \text{ Pres } Q$ $\implies F(Q) (S_1 \cup S_2) \sqsubseteq (S_1 \cup S_2) Q$
4	$\sqsubseteq \frac{((f_1 \text{ p}_1) \cup (f_2 \text{ p}_2))}{\exists} \text{ est}(R)$	

At $F(X)=L+E \times X$, $f_1(\text{inl}(l))=f_2(\text{inl}(l))=0$, $f_1(r(e,x))=x$, $f_2(r(e,x))=e+x$, p_1 passes all, p_2 passes values ≤ 6 , $\leq$ is $\leq$, Q is $=$, R is $\geq$, input $[3,3,1]$. (8.3.2b)

input	$\text{list}(f_1) \text{ filter}(p_1)$	$\text{list}(f_2) \text{ filter}(p_2)$	$\text{merge}(\leq)$	$\text{thinlist}(Q)$
1 $\text{inl}(l)$	$[0]$	$[0]$	$[0,0]$	$[0]$
2 $r(1,[0])$	$[0]$	$[1]$	$[0,1]$	$[0,1]$
3 $r(3,[0,1])$	$[0,1]$	$[3,4]$	$[0,1,3,4]$	$[0,1,3,4]$
4 $r(3,[0,1,3,4])$	$[0,1,3,4]$	$[3,4,6]$	$[0,1,3,3,4,4,6]$	$[0,1,3,4,6]$
5 $\text{minlist}(R)$				6

- $\text{merge}(\leq)$ is one pass: both copies arrive $\leq$ -sorted (f_i monotonic on $\leq$, $\text{filter}(p_i)$ only drops), so it only compares their two heads.
- $\text{thinlist}(Q)$ compares neighbours only; the list is $\leq$ -sorted, so equal values are neighbours and one pass drops every repeat.

Theorem 8.3c
 $\text{Coreflexive}(p_1) \wedge \text{Coreflexive}(p_2) \wedge \text{preorder}(\leq) \wedge \text{connected}(\leq) \wedge f_1 \text{ Pres } \leq \wedge f_2 \text{ Pres } \leq \implies$
 $F(\text{sort}(\leq)) \text{ listcp}(\text{list}(f_1) \text{ filter}(p_1), \text{list}(f_2) \text{ filter}(p_2)) \text{ merge}(\leq) \text{ thinlist}(Q) \sqsubseteq$
 $\frac{F(\exists) ((f_1 \text{ p}_1) \cup (f_2 \text{ p}_2))}{\exists} \text{ thin}(Q) \text{ sort}(\leq)$
 sorting the candidate set turns the thinning algebra into an algebra on sorted lists.



1	$F(\text{sort}(\leq)) \text{ listcp}(\text{list}(f_1) \text{ filter}(p_1), \text{list}(f_2) \text{ filter}(p_2)) \text{ merge}(\leq) \text{ thinlist}(Q)$	$X \langle R, S \rangle \sqsubseteq \langle X R, X S \rangle$
2	$\sqsubseteq (F(\text{sort}(\leq)) \text{ listcp}(\text{list}(f_1) \text{ filter}(p_1), F(\text{sort}(\leq)) \text{ listcp}(\text{list}(f_2) \text{ filter}(p_2)) \text{ merge}(\leq) \text{ thinlist}(Q))$	Lemma 8.1 $\text{Coreflexive}(p) \wedge f \text{ Pres } \leq$ $\implies F(\text{sort}(\leq)) \text{ listcp}(\text{list}(f) \text{ filter}(p)) \sqsubseteq \frac{F(\exists) f p}{\exists} \text{ sort}(\leq)$
3	$\sqsubseteq \left(\frac{F(\exists) f_1 p_1}{\exists} \text{ sort}(\leq), \frac{F(\exists) f_2 p_2}{\exists} \text{ sort}(\leq) \right) \text{ merge}(\leq) \text{ thinlist}(Q)$	$\langle X, Y \rangle \langle R \times S \rangle = \langle X R, Y S \rangle$
4	$= \left(\frac{F(\exists) f_1 p_1}{\exists}, \frac{F(\exists) f_2 p_2}{\exists} \right) (\text{sort}(\leq) \times \text{sort}(\leq)) \text{ merge}(\leq) \text{ thinlist}(Q)$	$\text{preorder}(\leq) \wedge \text{connected}(\leq)$ $\implies (\text{sort}(\leq) \times \text{sort}(\leq)) \text{ merge}(\leq) \sqsubseteq \text{cup sort}(\leq)$
5	$\sqsubseteq \left(\frac{F(\exists) f_1 p_1}{\exists}, \frac{F(\exists) f_2 p_2}{\exists} \right) \text{ cup sort}(\leq) \text{ thinlist}(Q)$	$\frac{R \cup S}{\exists} = \left(\frac{R}{\exists}, \frac{S}{\exists} \right) \text{ cup}$
6	$= \frac{F(\exists) ((f_1 \text{ p}_1) \cup (f_2 \text{ p}_2))}{\exists} \text{ sort}(\leq) \text{ thinlist}(Q)$	$R (S \cup T) = (R S) \cup (R T)$
7	$= \frac{F(\exists) ((f_1 \text{ p}_1) \cup (f_2 \text{ p}_2))}{\exists} \text{ sort}(\leq) \text{ thinlist}(Q)$	(8.6) $\text{sort}(\leq) \text{ thinlist}(Q) \sqsubseteq \text{thin}(Q) \text{ sort}(\leq)$
8	$\sqsubseteq \frac{F(\exists) ((f_1 \text{ p}_1) \cup (f_2 \text{ p}_2))}{\exists} \text{ thin}(Q) \text{ sort}(\leq)$	

§8.4. The knapsack problem

(8.4a)

	name	type	definition	meaning
1	<code>total(f,xs)</code>	<code>Int</code>	$\text{total}(f,xs) \triangleq \text{sum}(\text{list}(f,xs))$, $\text{total } f = \text{list}(f) \text{ sum}$	add up f over the items of the list
2	<code>subseq</code>	<code>[A]→[A]</code>	$xs \text{ subseq } ys \iff \text{subseq}(ys,xs)$	every subsequence of the list
3	<code>[nil,cons]</code>	<code>F[Item]→[Item]</code>	$[\text{nil},\text{cons}] = [\text{nil},\text{cons}]$, $[\text{nil},\text{cons}](\text{l}(u)) = []$, $[\text{nil},\text{cons}](\text{r}(a,xs)) = [a]\#xs$	keep the item: the list constructor
4	<code>[nil,π₂]</code>	<code>Unit + Item × [Item]→[Item]</code>	$[\text{nil},\pi_2](\text{l}(D)) = []$, $[\text{nil},\pi_2](\text{r}(p)) = \pi_2(p)$, $[\text{nil},\pi_2] = [\text{nil},\pi_2]$	drop the item: keep the tail
5	<code>within(w)</code>	<code>[Item]→[Item]</code>	$\text{within}(w)(xs,ys) \iff xs = ys \wedge \text{total}(\text{wt},xs) \leq w$	the packings whose total weight is at most w
6	$([\text{nil},\text{cons}]\text{within}(w)) \cup [\text{nil},\pi_2]$	<code>F[Item]→[Item]</code>	$([\text{nil},\text{cons}]\text{within}(w)) \cup [\text{nil},\pi_2] \triangleq ([\text{nil},\text{cons}]\text{within}(w)) \cup [\text{nil},\pi_2]$, $([\text{nil},\text{cons}]\text{within}(w)) \cup [\text{nil},\pi_2] = [\text{nil},(\text{cons within}(w)) \cup \pi_2]$, $([\text{nil},\text{cons}]\text{within}(w))(u,rs) \iff rs = [\text{nil},\text{cons}](u) \wedge \text{total}(\text{wt},rs) \leq w$	at each item, keep it if the packing still fits, or drop it
7	<code>R</code>	<code>[Item]→[Item]</code>	$R = \text{total vol} \geq \text{total vol}^\circ$, $R(xs,ys) \iff \text{total}(\text{vol},ys) \leq \text{total}(\text{vol},xs)$	x is worth at least as much as y
8	<code>Q(vol,wt)</code>	<code>[Item]→[Item]</code>	$Q(\text{vol},\text{wt}) = R \cap (\text{total wt} \leq \text{total wt}^\circ)$, $Q(\text{vol},\text{wt})(xs,ys) \iff \text{total}(\text{vol},ys) \leq \text{total}(\text{vol},xs) \wedge \text{total}(\text{wt},xs) \leq \text{total}(\text{wt},ys)$	x is worth at least as much as y and weighs no more

(8.4b)

	the law	what it says
1	$(1 \times R) (\text{cons } (\text{within } w)) \sqsubseteq \text{cons } (\text{within } w) R$ FALSE	not monotonic on R: a selection of greater value need not still fit once one more item goes in
2	$F(Q(\text{vol},\text{wt})) [\text{nil},\text{cons}] \text{within}(w) \sqsubseteq [\text{nil},\text{cons}] \text{within}(w) Q(\text{vol},\text{wt})$ $F(Q(\text{vol},\text{wt})) [\text{nil},\pi_2] \sqsubseteq [\text{nil},\pi_2] Q(\text{vol},\text{wt})$,	both halves are monotonic on Q once ties in value are broken by weight
3	$F(R) [\text{nil},\text{cons}] \sqsubseteq [\text{nil},\text{cons}] R$ $F(R) [\text{nil},\pi_2] \sqsubseteq [\text{nil},\pi_2] R$	both algebras are monotonic on R itself, so R is the sort order P

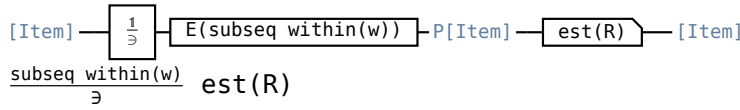
§8.4.1. Derivation

$$0 \leq w \wedge (\forall i. 0 \leq \text{wt}(i)) \Rightarrow (\text{listcp } \langle g_1, g_2 \rangle \text{ merge}(R) \text{ thinlist}(Q(\text{vol}, \text{wt}))) \text{ minlist}(R) \sqsubseteq \text{est}(R) \quad (8.4.1a)$$

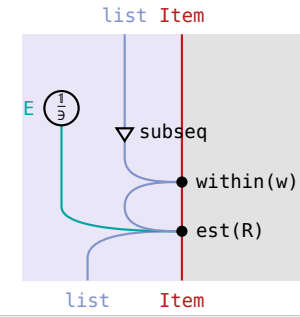
$\frac{\text{subseq within}(w)}{\exists}$

the knapsack problem, as a fold that thins the packings kept at each item

circuit

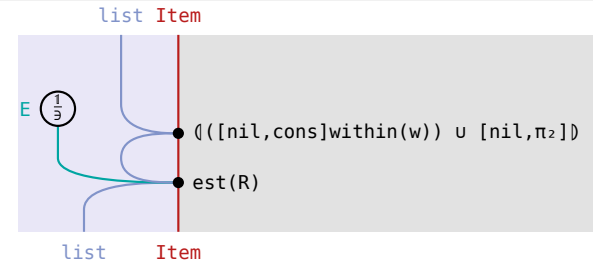


Hinze–Marsden



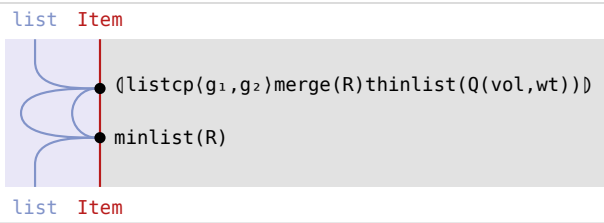
$$= \frac{\text{E}(\text{Q}([\text{nil}, \text{cons}] \text{ within}(w)) \cup [\text{nil}, \pi_2])}{\exists} \text{ est}(R)$$

fusion, weights non-negative.



$$= \frac{\text{listcp } \langle g_1, g_2 \rangle \text{ merge}(R) \text{ thinlist}(Q(\text{vol}, \text{wt}))}{\exists} \text{ minlist}(R)$$

binary thinning theorem, at $P \triangleq R$, F linear, Q from $F(Q(\text{vol}, \text{wt}))$
 $[\text{nil}, \text{cons}] \text{ within}(w) \sqsubseteq [\text{nil}, \text{cons}] \text{ within}(w) \text{ } Q(\text{vol}, \text{wt})$,
 $F(Q(\text{vol}, \text{wt})) [\text{nil}, \pi_2] \sqsubseteq [\text{nil}, \pi_2] \text{ } Q(\text{vol}, \text{wt})$



$$= \text{listcp} = \text{wrap} + \text{cpr}, g_i = [\text{list}(\text{nil}), h_i] - [\text{nil}, \text{cons}] =$$

$[\text{nil}, \text{cons}], [\text{nil}, \pi_2](\text{l}(D)) = [], [\text{nil}, \pi_2](\text{r}(p)) = \pi_2(p)$;
 $\text{minlist}(R)$ is head, packings coming out in descending value

§8.5. The paragraph problem

(8.5a)

	name	type	definition	meaning
1	Line	Type	$\text{Line} \triangleq \text{L}(\text{Word})$	a line is a non-empty list of words
2	Para	Type	$\text{Para} \triangleq \text{L}(\text{Line})$	a paragraph is a non-empty list of lines
3	$\text{new}(a, \text{xss})$	Para	$\text{new}(a, \text{xss}) \triangleq [[a]] \# \text{xss}$	start a new first line holding only the word a
4	$\text{glue}(a)$	$\text{Para} \rightarrow \text{Para}$	$\text{glue}(a, [\text{xs}]) = [[a] \# \text{xs}]$, $\text{glue}(a, [\text{xs}] \# a') = [[a] \# \text{xs}] \# a'$	put the word a at the front of the first line
5	$[\text{wrap wrap}, \text{new}]$	$\text{F}(\text{Para}) \rightarrow \text{Para}$	$[\text{wrap wrap}, \text{new}](\text{l}(a)) = [[a]]$, $[\text{wrap wrap}, \text{new}](\text{r}(q)) = \text{new}(\pi_1(q), \pi_2(q))$	one word is a one-line paragraph; each further word starts a new line
6	$[\text{wrap wrap}, \text{glue}]$	$\text{F}(\text{Para}) \rightarrow \text{Para}$	$[\text{wrap wrap}, \text{glue}](\text{l}(a)) = [[a]]$, $[\text{wrap wrap}, \text{glue}](\text{r}(q)) = \text{glue}(\pi_1(q), \pi_2(q))$	one word is a one-line paragraph; each further word joins the first line
7	$[\text{wrap wrap}, \text{new}] \cup [\text{wrap wrap}, \text{glue}]$	$\text{F}(\text{Para}) \rightarrow \text{Para}$	$[\text{wrap wrap}, \text{new}] \cup [\text{wrap wrap}, \text{glue}] \triangleq [\text{wrap wrap}, \text{new}] \cup [\text{wrap wrap}, \text{glue}]$	each further word either starts a new line or joins the first line
8	partition	$\text{L}(\text{Word}) \rightarrow \text{Para}$	$\text{partition} \triangleq ([\text{wrap wrap}, \text{new}] \cup [\text{wrap wrap}, \text{glue}])$, $\text{partition}([a], \text{yss}) \iff \text{yss} = [[a]]$, $\text{partition}([a] \# \text{xs}, \text{yss}) \iff (\exists \text{xss}. \text{partition}(\text{xs}, \text{xss}) \wedge (\text{yss} = \text{new}(a, \text{xss}) \vee \text{yss} = \text{glue}(a, \text{xss})))$	every way of breaking the words into lines
9	width	$\text{Line} \rightarrow \text{Int}$	$\text{width}[a] = \text{len}(a)$, $\text{width}([a] \# \text{xs}) = (\text{len}(a) + \text{width}(\text{xs})) + 1$	the lengths of the words plus one space between neighbours
10	head	$\text{Para} \rightarrow \text{Line}$	$\text{head}[\text{xs}] = \text{xs}$, $\text{head}([\text{xs}] \# a) = \text{xs}$	the first line of the paragraph
11	$\text{fits}(w)$	$\text{P}(\text{Para})$	$\text{fits}(w, [\text{xs}]) = \text{width}(\text{xs}) \leq w$, $\text{fits}(w, [\text{xs}] \# a) = \text{width}(\text{xs}) \leq w \wedge \text{fits}(w, a)$	every line is at most w wide
12	$\text{fits}(w)$	$\text{Para} \rightarrow \text{Para}$	$\text{fits}(w)(\text{xss}, \text{yss}) \iff \text{xss} = \text{yss} \wedge \text{fits}(w, \text{xss})$	the paragraphs whose every line is at most w wide
13	$\text{ok}(w)$	$\text{Para} \rightarrow \text{Para}$	$\text{ok}(w)(\text{xss}, \text{yss}) \iff \text{xss} = \text{yss} \wedge \text{width}(\text{head}(\text{xss})) \leq w$	the paragraphs whose first line is at most w wide
14	$\text{sqr}(n)$	Int	$\text{sqr}(n) \triangleq nn$	n squared

	name	type	definition	meaning
15	<code>waste(w)</code>	<code>Para→Int</code>	$\text{waste}(w, [xs]) = 0, \text{waste}(w, [xs]\#a) = \text{sqr}(w - \text{width } xs) + \text{waste}(w, a)$	the squared white space left on every line but the last, added up
16	<code>R</code>	<code>Para→Para</code>	$R = \text{waste } w \leq \text{waste } w^\circ, R(xss, yss) \iff \text{waste}(w, xss) \leq \text{waste}(w, yss)$	p wastes no more than q
17	<code>Q(len, w)</code>	<code>Para→Para</code>	$Q(\text{len}, w) = R \cap (\text{head } \text{head}^\circ), Q(\text{len}, w)(xss, yss) \iff \text{waste}(w, xss) \leq \text{waste}(w, yss) \wedge \text{head}(xss) = \text{head}(yss)$	p wastes no more than q and has the same first line
18	<code>start(a)</code>	<code>[Para]</code>	$\text{start}(a) \triangleq [[a]]$	the one candidate for a single word: the paragraph of one line holding that word

(8.5b)

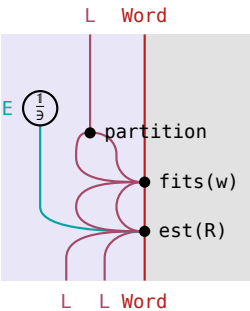
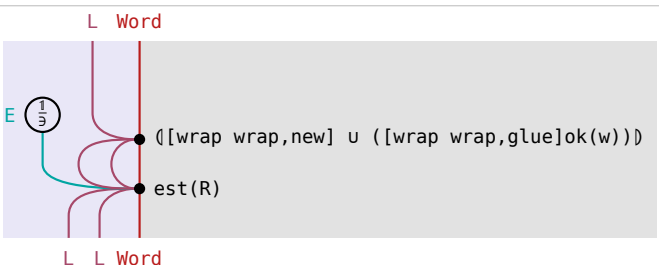
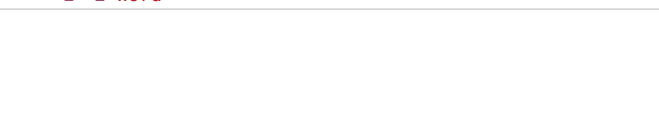
	the law	what it says
1	$(1 \times R) \text{ glue } \sqsubseteq \text{ glue } R \quad \text{FALSE}$	glue is not monotonic on R : the waste of a paragraph depends on its whole first line, so no greedy algorithm solves this
2	$F(Q(\text{len}, w)) \text{ [wrap wrap, new] } \sqsubseteq \text{ [wrap wrap, new] } Q(\text{len}, w)$ $(\forall a. 0 \leq \text{len}(a)) \implies F(Q(\text{len}, w)) \text{ [wrap wrap, glue] } \text{ ok}(w) \sqsubseteq \text{ [wrap wrap, glue] } \text{ ok}(w) Q(\text{len}, w)$ $\text{cons monotonic on collect} \leq \text{collect}^\circ.$	both halves are monotonic on Q once ties in waste are broken by the first line
3	$\text{merge}(\tau) = \text{cat}$	τ needs no sorting at all
4	$F(\tau) \text{ [wrap wrap, new] } \sqsubseteq \text{ [wrap wrap, new] } \tau$ $F(\tau) \text{ [wrap wrap, glue] } \sqsubseteq \text{ [wrap wrap, glue] } \tau$	both algebras are monotonic on τ , so τ is the sort order P

§8.5.1. Derivation

$$(\forall a. 0 \leq \text{len}(a)) \wedge (\forall a. \text{len}(a) \leq w) \Rightarrow (\text{listcp } \langle g_1, g_2 \rangle \text{ merge}(\tau) \text{ thinlist}(Q(\text{len}, w))) \quad (8.5.1a)$$

$$\text{minlist}(R) \sqsubseteq \frac{\text{partition fits}(w)}{\exists} \text{ est}(R)$$

a paragraph laid out as a fold that thins the layouts kept at each word

circuit	Hinze–Marsden
$\text{L}(\text{Word}) \xrightarrow{\frac{1}{\exists}} \text{P}(\text{L}(\text{Word})) \xrightarrow{\text{E}(\text{partition fits}(w))} \text{P}(\text{L}(\text{Line})) \xrightarrow{\text{est}(R)} \text{L}(\text{Line})$ $\frac{\text{partition fits}(w)}{\exists} \text{ est}(R)$	
$\text{L}(\text{Word}) \xrightarrow{\frac{1}{\exists}} \text{P}(\text{L}(\text{Word})) \xrightarrow{\text{E}(\frac{1}{\exists} [\text{wrap wrap, new}] \cup ([\text{wrap wrap, glue}] \text{ok}(w)))} \text{P}(\text{L}(\text{Line})) \xrightarrow{\text{est}(R)} \text{L}(\text{Line})$ $= \frac{1}{\exists} [\text{wrap wrap, new}] \cup ([\text{wrap wrap, glue}] \text{ok}(w)) \text{ est}(R)$ fusion, every word fits on a line by itself.	
$\text{L}(\text{Word}) \xrightarrow{\frac{1}{\exists}} \text{P}(\text{L}(\text{Word})) \xrightarrow{\text{E}(\frac{1}{\exists} [\text{wrap wrap, new}] \cup ([\text{wrap wrap, glue}] \text{ok}(w)))} \text{P}(\text{L}(\text{Line})) \xrightarrow{\text{est}(R)} \text{L}(\text{Line})$ $= \frac{1}{\exists} [\text{wrap wrap, new}] \cup ([\text{wrap wrap, glue}] \text{ok}(w)) \text{ est}(R)$ the algebra as $(f_1 p_1) \cup (f_2 p_2)$, $p_1 \triangleq 1$ — binary thinning theorem.	
$\text{L}(\text{Word}) \xrightarrow{\frac{1}{\exists}} \text{P}(\text{L}(\text{Word})) \xrightarrow{\text{E}(\frac{1}{\exists} [\text{wrap wrap, new}] \cup ([\text{wrap wrap, glue}] \text{ok}(w)))} \text{P}(\text{L}(\text{Line})) \xrightarrow{\text{est}(R)} \text{L}(\text{Line})$ $= \frac{1}{\exists} [\text{wrap wrap, new}] \cup ([\text{wrap wrap, glue}] \text{ok}(w)) \text{ est}(R)$ the algebra as $(f_1 p_1) \cup (f_2 p_2)$, $p_1 \triangleq 1$ — binary thinning theorem. $\text{listcp } \langle g_1, g_2 \rangle \text{ merge}(\tau) \text{ thinlist}(Q(\text{len}, w))$ $\text{minlist}(R)$ binary thinning theorem, at $P \triangleq T$ with $\text{merge } \tau = \text{cat}$ ($\text{merge}(\tau) = \text{cat}$), Q from $F(Q(\text{len}, w))$ $[\text{wrap wrap, new}] \sqsubseteq [\text{wrap wrap, new}] Q(\text{len}, w)$, $(\forall a. 0 \leq \text{len}(a)) \Rightarrow F(Q(\text{len}, w)) [\text{wrap wrap, glue}] \text{ok}(w) \sqsubseteq [\text{wrap wrap, glue}] \text{ok}(w) Q(\text{len}, w)$	
$Q[\text{start, cpr } \langle h_1, h_2 \rangle \text{ cat thinlist}(Q)] \text{ minlist}(R)$ $\text{listcp} = \text{wrap} + \text{cpr}$, g_i along the coproduct — $[\text{wrap wrap, new}]$ $= (\text{l}(a)) = [[a]], [\text{wrap wrap, new}](r(q)) = \text{new}(\pi_1(q), \pi_2(q)), [\text{wrap wrap, glue}](\text{l}(a)) = [[a]], [\text{wrap wrap, glue}](r(q)) = \text{glue}(\pi_1(q), \pi_2(q))$	

§8.6. Bitonic tours

(8.6a)

	name	type	definition	meaning
1	Journey(City)	Type	$\text{Journey}(\text{City}) \triangleq \mu F$	a list of at least two cities, the last two held as a pair
2	Tour(City)	Type	$\text{Tour}(\text{City}) \triangleq \text{Journey}(\text{City}) \times \text{Journey}(\text{City})$	the outward journey and the return journey
3	head	$\text{Journey}(\text{City}) \rightarrow \text{City}$	$\text{head}(\text{wrap}(p)) = \pi_1(p), \text{head}([a]\#a') = a$	the first city of the journey
4	next	$\text{Journey}(\text{City}) \rightarrow \text{City}$	$\text{next}(\text{wrap}(p)) = \pi_2(p), \text{next}([a]\#a') = \text{head}(a')$	the second city of the journey
5	$[a]\#\text{tail}(\cdot)$	$\text{Journey}(\text{City}) \rightarrow \text{Journey}(\text{City})$	$[a]\#\text{tail}(\text{wrap}(p)) = \text{wrap}(a, \pi_2(p)), [a]\#\text{tail}([a']\#a'') = [a]\#a''$	replace the first city of the journey by a
6	outcost	$\text{Journey}(\text{City}) \rightarrow \text{Int}$	$\text{outcost}(\text{wrap}(p)) = \text{tc}(p), \text{outcost}([a]\#a') = \text{tc}(a, \text{head}(a')) + \text{outcost}(a')$	the cost of travelling the journey forwards
7	incost	$\text{Journey}(\text{City}) \rightarrow \text{Int}$	$\text{incost}(\text{wrap}(p)) = \text{tc}(\pi_2(p), \pi_1(p)), \text{incost}([a]\#a') = \text{tc}(\text{head}(a'), a) + \text{incost}(a')$	the cost of travelling the journey backwards
8	cost(t)	Int	$\text{cost}(t) \triangleq \text{outcost}(\pi_1(t)) + \text{incost}(\pi_2(t))$	the outward journey forwards plus the return journey backwards
9	start	$\text{City} \times \text{City} \rightarrow \text{Tour}(\text{City})$	$\text{start}(a) \triangleq (\text{wrap}(a), \text{wrap}(a))$	the tour of the last two cities, both journeys being that one edge
10	dropl(a,t)	Tour(City)	$\text{dropl}(a, t) \triangleq ([a]\#\text{tail}(\pi_1(t)), [a]\#\pi_2(t))$	a replaces the first city of the outward journey and is put in front of the return
11	dropr(a,t)	Tour(City)	$\text{dropr}(a, t) \triangleq ([a]\#\pi_1(t), [a]\#\text{tail}(\pi_2(t)))$	a is put in front of the outward journey and replaces the first city of the return
12	$[\text{start}, \text{dropl}]$	$F(\text{Tour}(\text{City})) \rightarrow \text{Tour}(\text{City})$	$[\text{start}, \text{dropl}](l(p)) = \text{start}(p), [\text{start}, \text{dropl}](r(q)) = \text{dropl}(\pi_1(q), \pi_2(q))$	start at the last two cities, then add each city by dropl

	name	type	definition	meaning
13	$[\text{start}, \text{dropr}]$	$F(\text{Tour}(\text{City})) \rightarrow \text{Tour}(\text{City})$	$[\text{start}, \text{dropr}](l(p)) = \text{start}(p),$ $[\text{start}, \text{dropr}](r(q)) = \text{dropr}(\pi_1(q), \pi_2(q))$	start at the last two cities, then add each city by dropr
14	$[\text{start}, \text{dropl}] \cup [\text{start}, \text{dropr}]$	$F(\text{Tour}(\text{City})) \rightarrow \text{Tour}(\text{City})$	$[\text{start}, \text{dropl}] \cup [\text{start}, \text{dropr}] \triangleq$ $[\text{start}, \text{dropl}] \cup [\text{start}, \text{dropr}],$ $([\text{start}, \text{dropl}] \cup [\text{start}, \text{dropr}])(u, t) = t =$ $[\text{start}, \text{dropl}](u) \vee t = [\text{start}, \text{dropr}](u)$	each further city is added by dropl or by dropr
15	$\langle [\text{start}, \text{dropl}] \cup [\text{start}, \text{dropr}] \rangle$	$\mu F \rightarrow \text{Tour}(\text{City})$	$\langle [\text{start}, \text{dropl}] \cup [\text{start}, \text{dropr}] \rangle \triangleq$ $\langle [\text{start}, \text{dropl}] \cup [\text{start}, \text{dropr}] \rangle$	every bitonic tour of the cities
16	R	$\text{Tour}(\text{City}) \rightarrow \text{Tour}(\text{City})$	$R = \text{cost} \leq \text{cost}^\circ, R(t, t') \iff \text{cost}(t) \leq \text{cost}(t')$	t costs no more than t'
17	$\text{next2}(t)$	$\text{City} \times \text{City}$	$\text{next2}(t) \triangleq (\text{next}(\pi_1(t)), \text{next}(\pi_2(t)))$	the second cities of both journeys
18	$\text{head2}(t)$	$\text{City} \times \text{City}$	$\text{head2}(t) \triangleq (\text{head}(\pi_1(t)), \text{head}(\pi_2(t)))$	the first cities of both journeys
19	Q	$\text{Tour}(\text{City}) \rightarrow \text{Tour}(\text{City})$	$Q = R \cap (\text{next2 next2}^\circ) \cap (\text{head2 head2}^\circ),$ $Q(t, t') \iff \text{cost}(t) \leq \text{cost}(t') \wedge$ $(\text{next}(\pi_1(t)) = \text{next}(\pi_1(t')) \wedge \text{next}(\pi_2(t)) = \text{next}(\pi_2(t'))) \wedge$ $\text{head}(\pi_1(t)) = \text{head}(\pi_1(t')) \wedge \text{head}(\pi_2(t)) = \text{head}(\pi_2(t'))$	t costs no more than t' , and both journeys of t and t' share their first two cities

(8.6b)

	the law	what it says
1	$(1 \times R) \text{ dropl} \sqsubseteq \text{dropl} \quad R \quad \text{FALSE}$ $(1 \times R) \text{ dropr} \sqsubseteq \text{dropr} \quad R \quad \text{FALSE}$	neither drop is monotonic on R : the two edges it adds and removes depend on head and next of both lists
2	$F(Q) \text{ [start, dropl]} \sqsubseteq \text{[start, dropl]} \quad Q$ $F(Q) \text{ [start, dropr]} \sqsubseteq \text{[start, dropr]} \quad Q,$	both are, once ties in cost are broken by the two second cities — the heads already agree among tours of one input
3	$F(T) \text{ [start, dropl]} \sqsubseteq \text{[start, dropl]} \quad T$ $F(T) \text{ [start, dropr]} \sqsubseteq \text{[start, dropr]} \quad T$	both algebras are monotonic on T , so T is the sort order P

§8.6.1. Derivation

$(\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \text{ minlist}(R) \sqsubseteq \frac{(\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \sqcup \text{est}(R)}{3} \text{ est}(R) \quad (8.6.1a)$ a least-cost bitonic tour, as a fold that thins the tours kept at each city	
circuit	Hinze–Marsden
$\mu F \xrightarrow{\frac{1}{3}} P(\mu F) \xrightarrow{E((\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \sqcup \text{est}(R))} P(\text{Journey}(\text{City})^2) \xrightarrow{\text{est}(R)} \text{Journey}(\text{City})$ $\frac{(\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \sqcup \text{est}(R)}{3}$	
$\mu F \xrightarrow{\frac{1}{3}} P(\mu F) \xrightarrow{E((\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \sqcup \text{est}(R))} P(\text{Journey}(\text{City})^2) \xrightarrow{\text{est}(R)} \text{Journey}(\text{City})$ $= \frac{(\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \sqcup \text{est}(R)}{3}$ $(\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \triangleq (\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \sqcup \text{est}(R)$	
$\mu F \xrightarrow{\frac{1}{3}} P(\mu F) \xrightarrow{E((\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \sqcup \text{est}(R))} P(\text{Journey}(\text{City})^2) \xrightarrow{\text{est}(R)} \text{Journey}(\text{City})$ $(\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \text{ minlist}(R)$ binary thinning theorem, at $P \triangleq T$ with merge $T = \text{cat}$, Q from $F(Q)$ $[\text{start}, \text{dropl}] \sqsubseteq [\text{start}, \text{dropl}] \sqcup Q, F(Q) [\text{start}, \text{dropr}] \sqsubseteq [\text{start}, \text{dropr}] \sqcup Q$	
$(\text{listcp } \langle g_1, g_2 \rangle \text{ cat thinlist}(Q)) \text{ minlist}(R)$ $\text{listcp} = \text{wrap} + \text{cpr}, g_i = [\text{list}(\text{start}), \text{list}(\text{drop}_i)] \text{ —}$ $= [\text{start}, \text{dropl}](l(p)) = \text{start}(p), [\text{start}, \text{dropl}](r(q)) = \text{dropl}(\pi_1(q), \pi_2(q)), [\text{start}, \text{dropr}](l(p)) = \text{start}(p),$ $[\text{start}, \text{dropr}](r(q)) = \text{dropr}(\pi_1(q), \pi_2(q)); \text{quadratic, two tours added per step}$	

§9. Dynamic Programming

§9.1. Theory

§9.1.1. Example: segmenting a list

- (9.1.1a)
- $\langle T \rangle = \text{concat}$
 $T=[\text{nil}, \text{cat}]$ takes a non-empty first list, so T° cuts a non-empty prefix off a list every way, and folding with T flattens a list of segments
 - $\langle h \rangle = \text{list}(\text{sum})$
 $h=[\text{nil}, \text{cons}(\text{sum} \times 1)]$ puts a segment's sum in front, so folding with h sums every segment
 - $H = \text{partition list}(\text{sum})$
 H segments a list every way, then sums each segment; with R comparing the largest entry, M cuts the list so that the largest segment sum is as small as possible

(9.1.1b)

	x	h(x)
1	$l(())$	$[]$
2	$r([c], [])$	$[c]$
3	$r([a, b], [c])$	$[a + b, c]$
4	$r([a], [b + c])$	$[a, b + c]$

- (9.1.1c)
- $H([a, b, c], ys) \iff ys = [a + b + c] \vee ys = [a, b + c] \vee ys = [a + b, c] \vee ys = [a, b, c]$
 H sends $[a, b, c]$ to the sums of its four segmentations and nothing else
 - $T^\circ F(H) h = H$
cut once, solve the rest, put the pieces back together

(9.1.1d)

	name	type	definition	meaning
1	H	A→B	$H \triangleq \langle T \rangle^\circ \langle h \rangle$	take the input apart by the T-decompositions, and put it together by h
2	M	A→B	$M \triangleq \frac{H}{\exists} \text{ est}(R)$	the problem to be solved: an answer of H that is best under R

$\text{Map}(h) \wedge h \text{ Pres } R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \implies (\mu X : \frac{T^\circ}{\exists} P(F(X) h) \text{ est}(R)) \sqsubseteq \frac{H}{\exists} \text{ est}(R)$

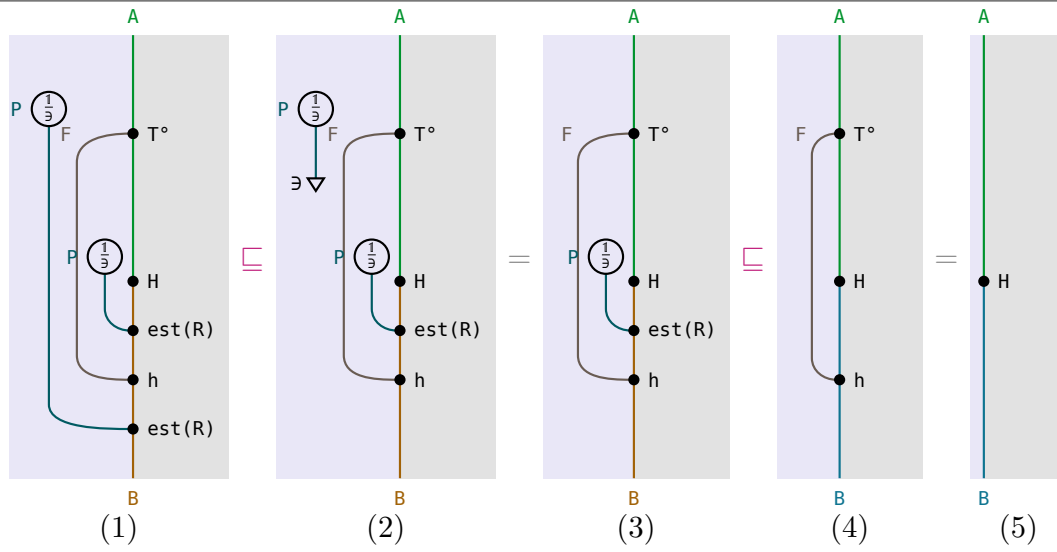
Theorem 9.1

every answer the recursion returns is an optimal one

- $\frac{T^\circ}{\exists}$ takes the input apart one step every way; $F(X)$ solves each part by the recursion X ; h assembles each candidate; $\text{est}(R)$ keeps a best one
- $\sqsubseteq M$: that never misses the optimum, though all candidates are never listed as H would
- h monotonic on R : a better part never makes the whole worse
- $\sqsubseteq$, not $=$: some optima may not be returned; that the recursion returns anything at all needs T° to stop taking apart — dynamic programming theorem

$$\frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R) \sqsubseteq H \quad (9.2)$$

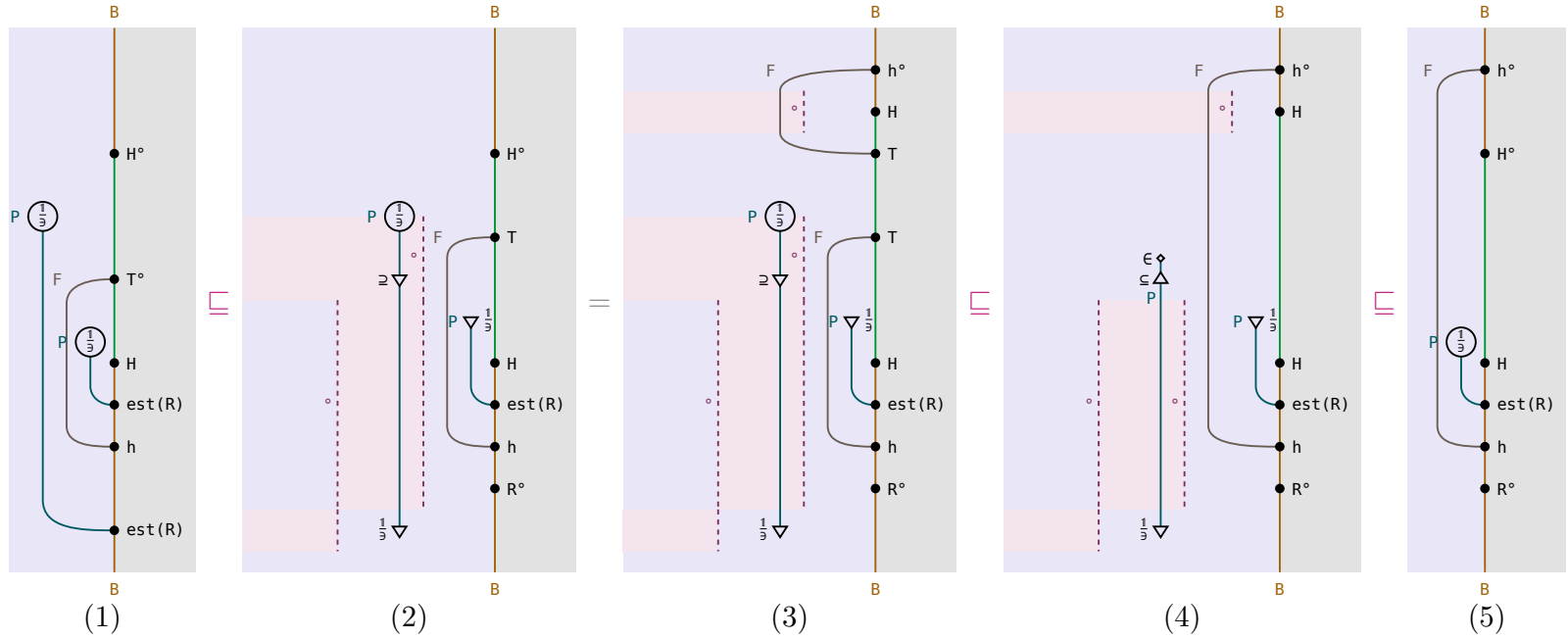
taking the input apart every way T allows ($\frac{T^\circ}{\exists}$), solving each part by M and keeping an optimum ($P(F(M)h) \text{est}(R)$)
returns only what H returns



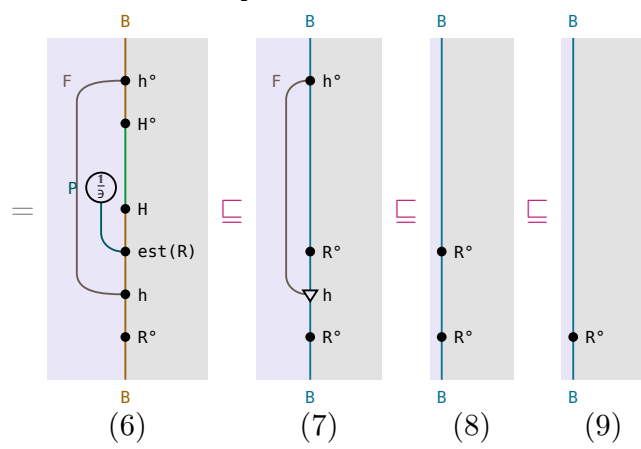
1	$\frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \exists S$
2	$\sqsubseteq \frac{T^\circ}{\exists} \exists F(\frac{H}{\exists} \text{est}(R)) h$	$\frac{R}{\exists} \exists = R$
3	$= T^\circ F(\frac{H}{\exists} \text{est}(R)) h$	$\frac{S}{\exists} \text{est}(R) \sqsubseteq S$
4	$\sqsubseteq T^\circ F(H) h$	$T^\circ F(H) h = H$
5	$= H$	

$$\text{Map}(h) \wedge h \text{ Pres } R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \implies H^\circ \frac{T^\circ}{3} P(F(\frac{H}{3} \text{ est}(R)) h) \text{ est}(R) \sqsubseteq R^\circ \quad (9.3)$$

for every b that H returns from an input, the step $\frac{T^\circ}{3} P(F(M)h) \text{ est}(R)$ returns from that input only b' with R relating b' to b



1	$H^\circ \frac{T^\circ}{3} P(F(\frac{H}{3} \text{ est}(R)) h) \text{ est}(R)$	$P(S) \text{ est}(R) \sqsubseteq \epsilon \setminus (S R^\circ)$
2	$\sqsubseteq H^\circ \frac{T^\circ}{3} (\epsilon \setminus (F(\frac{H}{3} \text{ est}(R)) h R^\circ))$	$H^\circ = h^\circ F(H^\circ) T$
3	$= h^\circ F(H^\circ) T \frac{T^\circ}{3} (\epsilon \setminus (F(\frac{H}{3} \text{ est}(R)) h R^\circ))$	$T \frac{T^\circ}{3} \sqsubseteq \epsilon$
4	$\sqsubseteq h^\circ F(H^\circ) \epsilon (\epsilon \setminus (F(\frac{H}{3} \text{ est}(R)) h R^\circ))$	$S (S \setminus R) \sqsubseteq R$
5	$\sqsubseteq h^\circ F(H^\circ) F(\frac{H}{3} \text{ est}(R)) h R^\circ$	

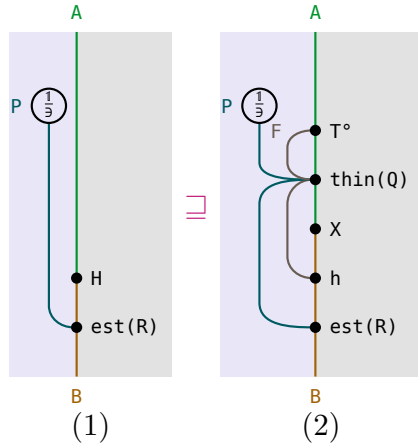


6	$= h^\circ F(H^\circ \frac{H}{3} \text{ est}(R)) h R^\circ$	$S^\circ \frac{S}{3} \text{ est}(R) \sqsubseteq R^\circ$
7	$\sqsubseteq h^\circ F(R^\circ) h R^\circ$	$\text{Map}(f)$
8	$\sqsubseteq R^\circ R^\circ$	$\implies f \text{ Pres } R \iff f^\circ F(R) f \sqsubseteq R$
9	$\sqsubseteq R^\circ$	$R^\circ R^\circ \sqsubseteq R^\circ$

Map(h) $\wedge$ h Pres $R^\circ \wedge R^\circ R^\circ \subseteq R^\circ \wedge \text{ThinCondition}(T, h, R, Q) \Rightarrow (\mu X : \frac{T^\circ}{S} \text{thin}(Q) P(F(X) h) \text{est}(R)) \subseteq \frac{H}{S} \text{est}(R)$ Theorem 9.2

an optimum over everything **H** returns is reached by taking the input apart every way **T** allows, dropping the parts that can never win, solving each of the rest and keeping one optimum

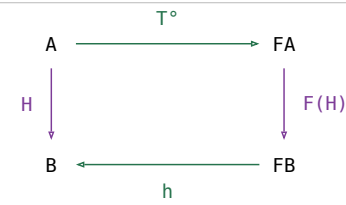
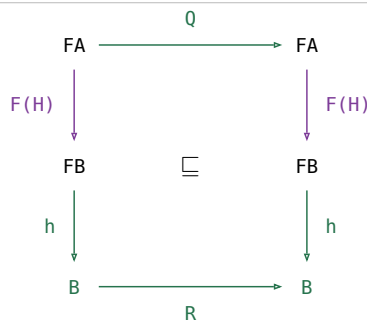
$$H \triangleq (T)^\circ (h) \text{ — } H \triangleq (T)^\circ (h)$$



$$1 \quad \frac{H}{S} \text{est}(R)$$

$$2 \quad \sqsubseteq \frac{T^\circ}{S} \text{thin}(Q) P(F(X) h) \text{est}(R)$$

$QF(H)h \subseteq F(H)hR$, h monotonic on $R \text{ — } Q \subseteq R \wedge 1 \subseteq Q \wedge R$
 $R \subseteq R$
 $\Rightarrow \text{est}(R) = \text{thin}(Q) \text{est}(R)$, $X \subseteq \text{est}(R) \Leftrightarrow X \subseteq \exists$
 $\wedge \in X \subseteq R^\circ$

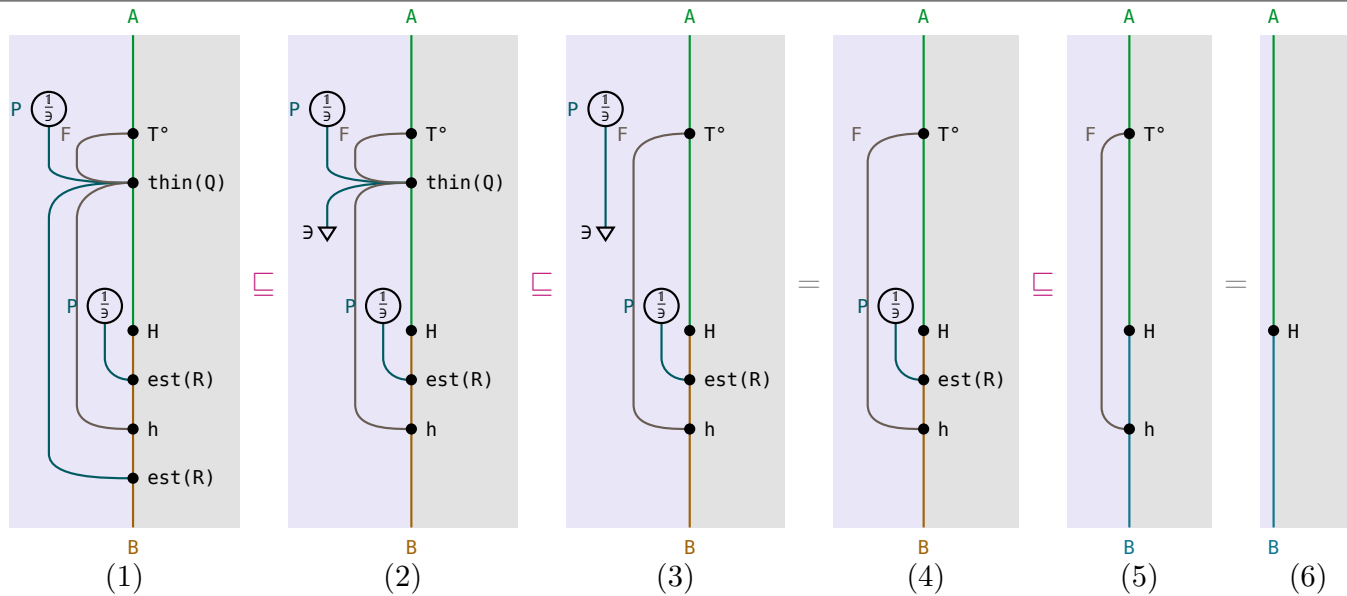


$T^\circ F(H)h = H$: taking the input apart once by T° , solving the parts by $F(H)$ and assembling by h returns exactly what H returns

$QF(H)h \subseteq F(H)hR$: every result of thinning by Q , solving the parts by $F(H)$ and assembling by h is reached by R from a result of solving by $F(H)$ and assembling by h without Q

$\frac{T^\circ}{S} \text{thin}(Q) P(F(\frac{H}{S} \text{est}(R)) h) \text{est}(R) \subseteq H$ (9.1.1e)

taking the input apart every way **T** allows ($\frac{T^\circ}{S}$), dropping the parts **thin(Q)** rejects, solving each rest by **M** and keeping an optimum returns only what **H** returns



$$1 \quad \frac{T^\circ}{S} \text{thin}(Q) P(F(\frac{H}{S} \text{est}(R)) h) \text{est}(R)$$

$$2 \quad \sqsubseteq \frac{T^\circ}{S} \text{thin}(Q) \exists F(\frac{H}{S} \text{est}(R)) h$$

$$3 \quad \sqsubseteq \frac{T^\circ}{S} \exists F(\frac{H}{S} \text{est}(R)) h$$

$$4 \quad = T^\circ F(\frac{H}{S} \text{est}(R)) h$$

$$5 \quad \sqsubseteq T^\circ F(H) h$$

$$6 \quad = H$$

$$P(S) \text{est}(R) \subseteq \exists S$$

$$\text{thin}(Q) \exists \subseteq \exists$$

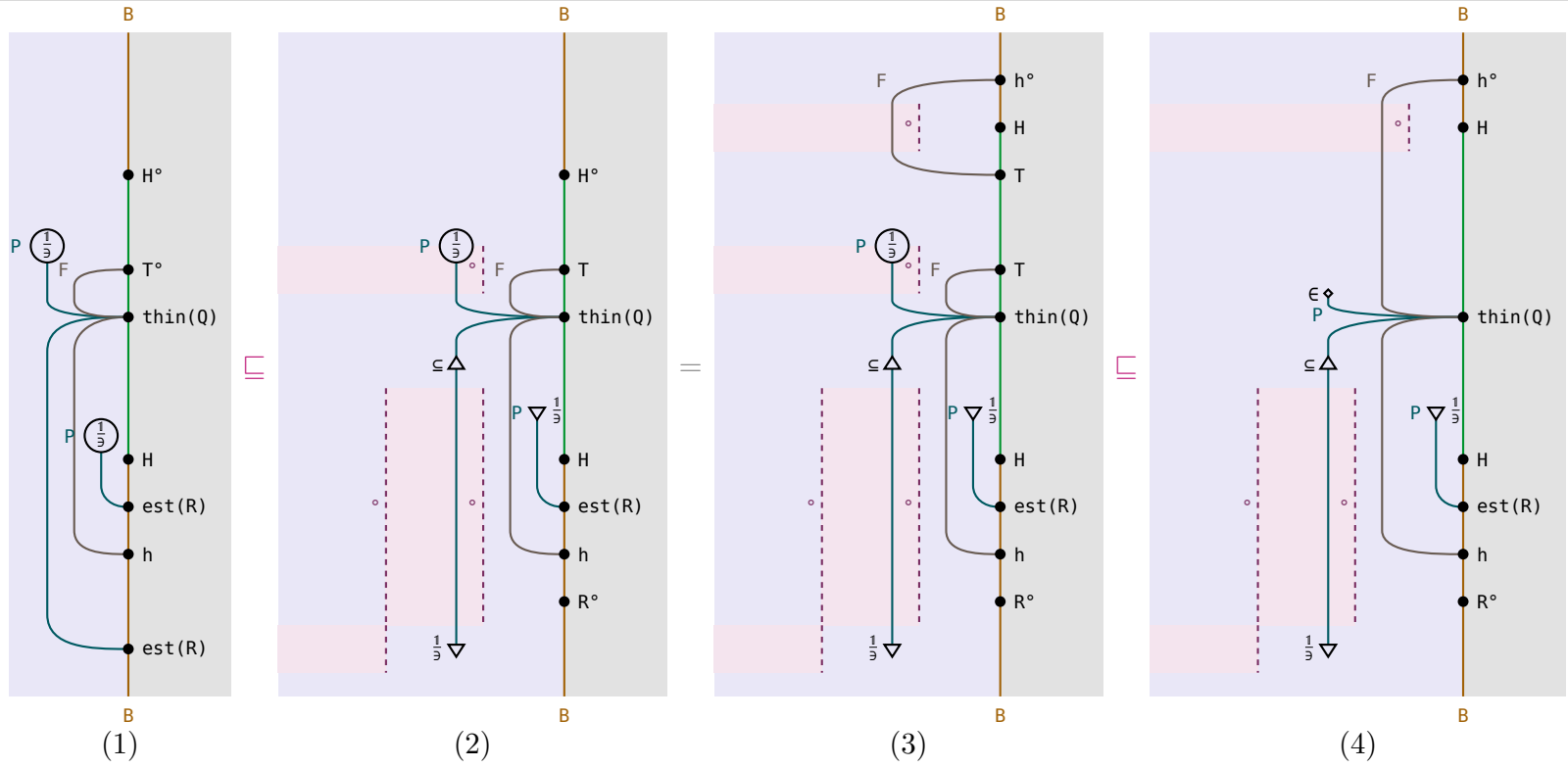
$$\frac{R}{S} \exists = R$$

$$\frac{S}{S} \text{est}(R) \subseteq S$$

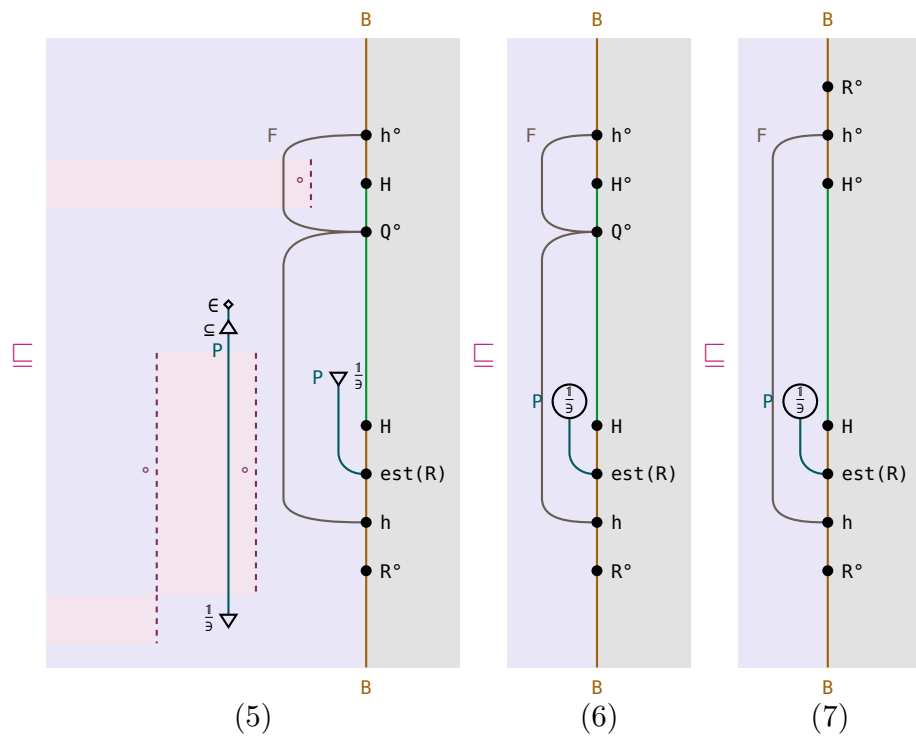
$$T^\circ F(H) h = H$$

$$\text{Map}(h) \wedge h \text{ Pres } R^\circ \wedge R^\circ R^\circ \sqsubseteq R^\circ \wedge \text{ThinCondition}(T, h, R, Q) \Rightarrow H^\circ \frac{T^\circ}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R) \sqsubseteq R^\circ \quad (9.1.1f)$$

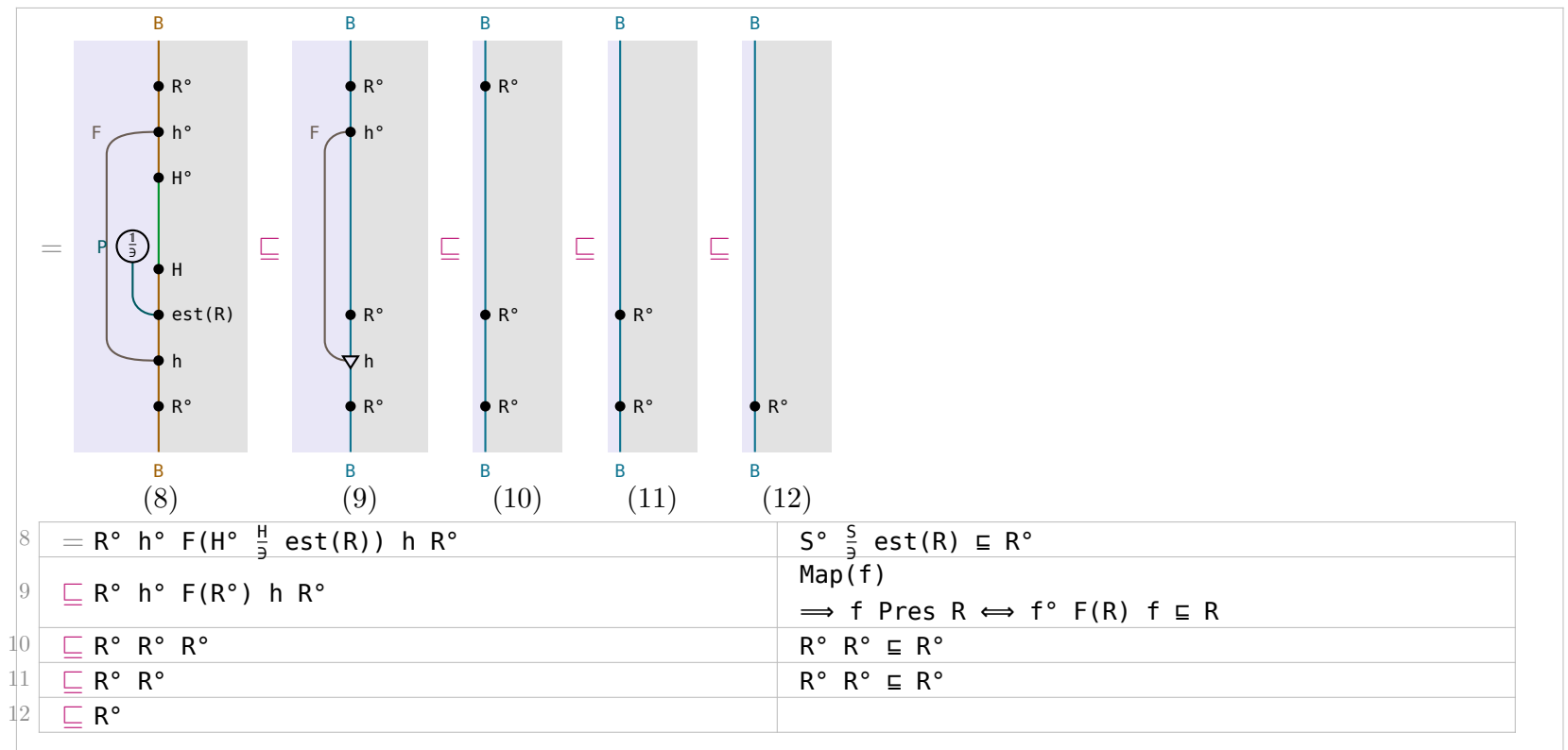
H° followed by the body at M is $\mathbb{E}R^\circ$: an answer of the body is never worse than an answer of H to the same input



1	$H^\circ \frac{T^\circ}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \mathbb{E} \setminus (S R^\circ)$
2	$\sqsubseteq H^\circ \frac{T^\circ}{3} \text{thin}(Q) (\mathbb{E} \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$	$H^\circ = h^\circ F(H^\circ) T$
3	$= h^\circ F(H^\circ) T \frac{T^\circ}{3} \text{thin}(Q) (\mathbb{E} \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$	$T \frac{T^\circ}{3} \sqsubseteq \mathbb{E}$
4	$\sqsubseteq h^\circ F(H^\circ) \mathbb{E} \text{thin}(Q) (\mathbb{E} \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$	$\mathbb{E} \text{thin}(Q) \sqsubseteq Q^\circ \mathbb{E}$



5	$\sqsubseteq h^\circ F(H^\circ) Q^\circ \mathbb{E} (\mathbb{E} \setminus (F(\frac{H}{3} \text{est}(R)) h R^\circ))$	$S (S \setminus R) \sqsubseteq R$
6	$\sqsubseteq h^\circ F(H^\circ) Q^\circ F(\frac{H}{3} \text{est}(R)) h R^\circ$	$\text{ThinCondition}(T, h, R, Q)$
7	$\sqsubseteq R^\circ h^\circ F(H^\circ) F(\frac{H}{3} \text{est}(R)) h R^\circ$	$\Rightarrow h^\circ F(H^\circ) Q^\circ \sqsubseteq R^\circ h^\circ F(H^\circ)$

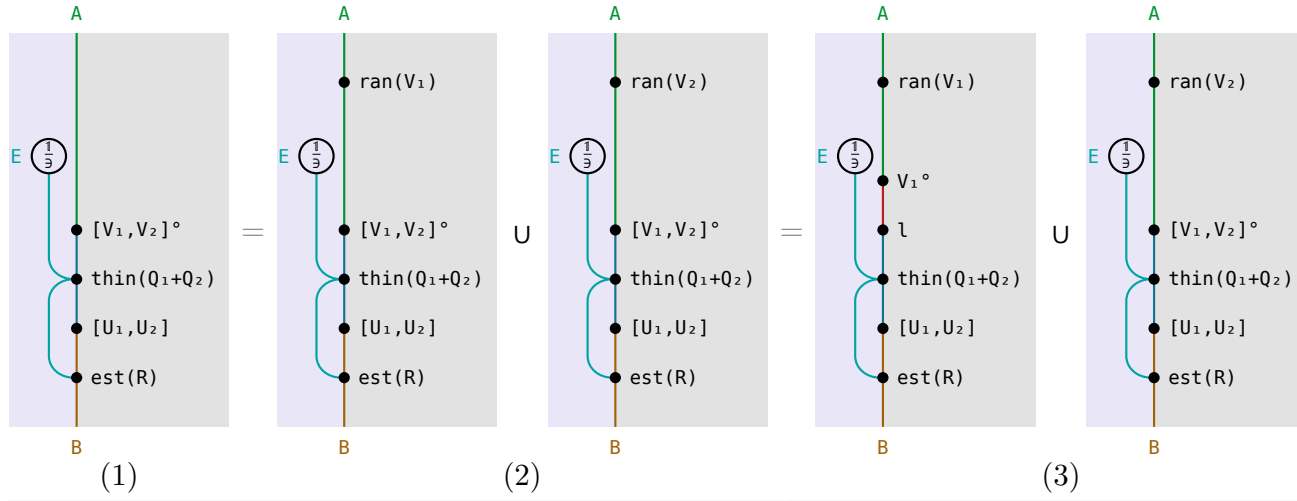


§9.1.2. Checking the conditions

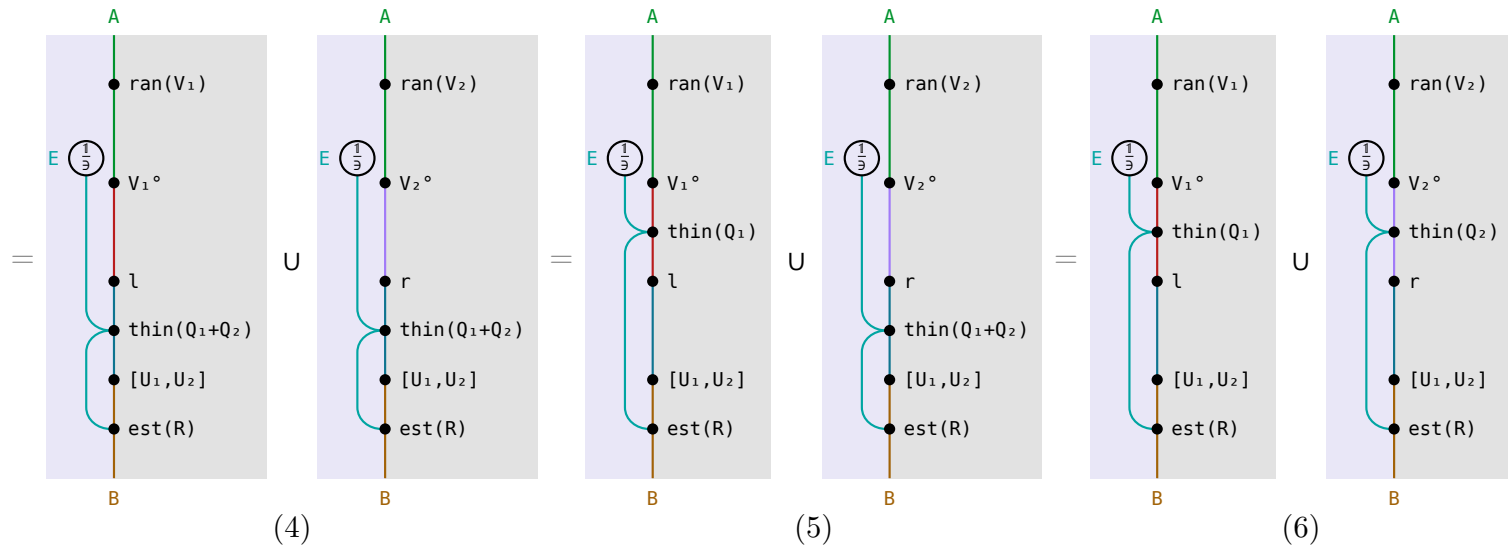
Proposition 9.1

$$V_2 \ V_1^\circ = \emptyset \implies \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R) = (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \text{thin}(Q_2) \ P(U_2) \ \text{est}(R))$$

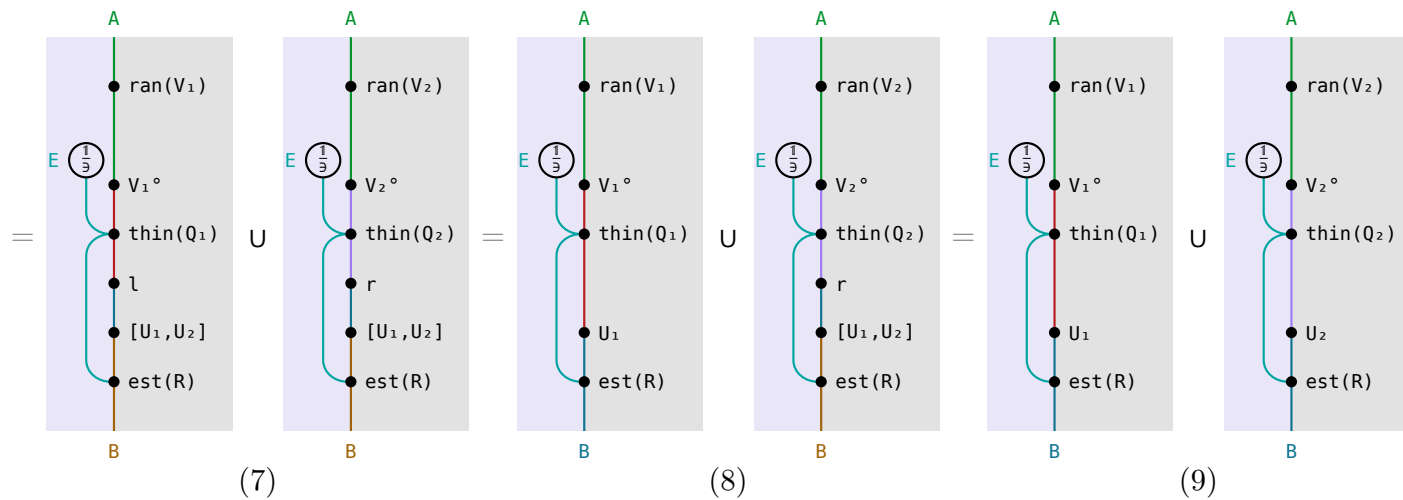
when V_1 and V_2 have disjoint ranges, the body over $[V_1, V_2]$ runs the V_1 problem on inputs V_1 reaches and the V_2 problem on inputs V_2 reaches



1	$\frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)$	$\frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R) = (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$
2	$= (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l)$
3	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r)$



4	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$P(l) \ \text{thin}(Q_1 + Q_2) = \text{thin}(Q_1) \ P(l)$
5	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ \text{est}(R))$	$P(r) \ \text{thin}(Q_1 + Q_2) = \text{thin}(Q_2) \ P(r)$
6	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$	



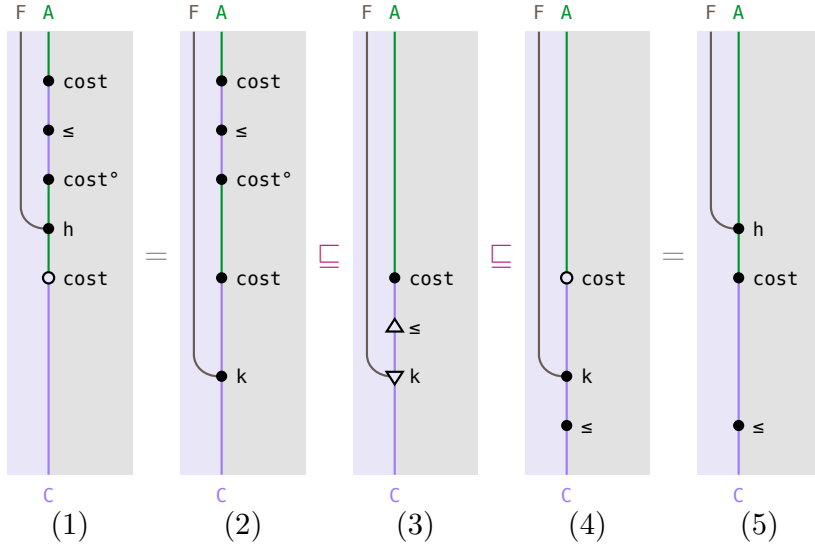
7	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$	$l \ [R, S] = R$
8	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ \text{est}(R))$	$r \ [R, S] = S$
9	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ \text{est}(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(U_2) \ \text{est}(R))$	

Map(cost) $\wedge$ h cost = F(cost) k $\wedge$ F($\leq$) k $\sqsubseteq$ k $\leq \implies$ F(cost $\leq$ cost $^\circ$) h $\sqsubseteq$ h cost $\leq$ cost $^\circ$ Proposition 9.2

if h then **cost** equals **F(cost)** then a **k** monotonic on $\leq$, then **h** is monotonic on **cost $\leq$ cost $^\circ$** ,
which compares two values by comparing their **costs** under $\leq$

Map(cost) $\wedge$ h cost = F(cost) k $\wedge$ F($\leq$) k $\sqsubseteq$ k $\leq \implies$ F(cost $\leq$ cost $^\circ$) h cost $\sqsubseteq$ h cost $\leq$

by shunting at the map **cost**, **F(cost $\leq$ cost $^\circ$)h $\sqsubseteq$ h cost $\leq$ cost $^\circ$** is **F(cost $\leq$ cost $^\circ$)h cost $\sqsubseteq$ h cost $\leq$** , which the chain proves



1	$F(\text{cost} \leq \text{cost}^\circ) \text{ h cost}$	$\text{h cost} = F(\text{cost}) \text{ k}$
2	$= F(\text{cost} \leq \text{cost}^\circ) F(\text{cost}) \text{ k}$	$\text{Map}(f) \wedge R = f S f^\circ$
3	$\sqsubseteq F(\text{cost}) F(\leq) \text{ k}$	$\implies R f \sqsubseteq f S$
4	$\sqsubseteq F(\text{cost}) \text{ k} \leq$	$F(\leq) \text{ k} \sqsubseteq \text{ k} \leq$
5	$= \text{h cost} \leq$	$\text{h cost} = F(\text{cost}) \text{ k}$

(9.1.2a)

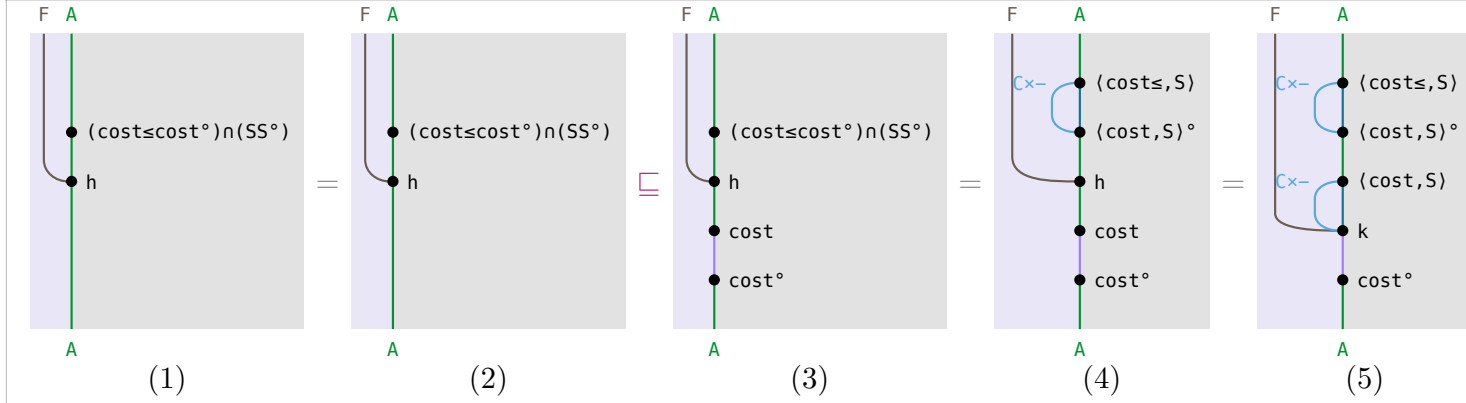
	the condition	how it is discharged
1	$F(R) \text{ h} \sqsubseteq \text{h} R$ Proposition 9.2, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $\text{h cost} = F(\text{cost}) \text{ k}$, $F(\leq) \text{ k} \sqsubseteq \text{ k} \leq$	monotonicity when the cost is itself a fold with a step k monotonic on $\leq$
2	$F(R \text{ n } (H^\circ H)) \text{ h} \sqsubseteq \text{h} R$ Proposition 9.3, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $\text{h cost} = F(\langle \text{cost}, H^\circ \rangle) \text{ k}$, $F(\leq \times 1) \text{ k} \sqsubseteq \text{ k} \leq$, H° simple;	monotonicity in context : k may also read the input the part was built from
3	$QF(H) \text{ h} \sqsubseteq F(H) \text{ h} R$ at $Q \triangleq F(U, V)$ Proposition 9.4, U, V preorders, $F(U, R) \text{ h} \sqsubseteq \text{h} R$, $V H \sqsubseteq H R$	both conditions at once, split along the two arguments of a bifunctor

Map(cost) $\wedge$ Simple(S) $\wedge$ h cost = F((cost,S)) k $\wedge$ F($\leq \times 1$) k $\sqsubseteq$ k $\leq \Rightarrow$

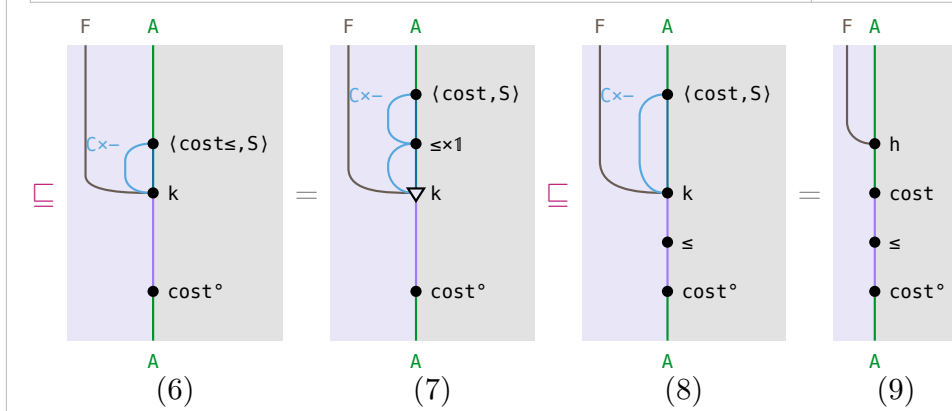
Proposition 9.3

F((cost $\leq$ cost $^\circ$) $\cap$ (S S $^\circ$)) h $\sqsubseteq$ h cost $\leq$ cost $^\circ$

improving each part by **R** within its **S**-context, then assembling by **h**, is below **hR**



1	F((cost $\leq$ cost $^\circ$) $\cap$ (S S $^\circ$)) h	
2	= F((cost $\leq$ cost $^\circ$) $\cap$ (S S $^\circ$)) h	Map(f)
3	$\sqsubseteq$ F((cost $\leq$ cost $^\circ$) $\cap$ (S S $^\circ$)) h cost cost $^\circ$	$\Rightarrow 1 \sqsubseteq f f^\circ$
4	= F((cost $\leq$, S) (cost, S) $^\circ$) h cost cost $^\circ$	$\langle X, Y \rangle (R, S)^\circ = (X R^\circ) \cap (Y S^\circ)$
5	= F((cost $\leq$, S) (cost, S) $^\circ$ (cost, S)) k cost $^\circ$	h cost = F((cost, S)) k



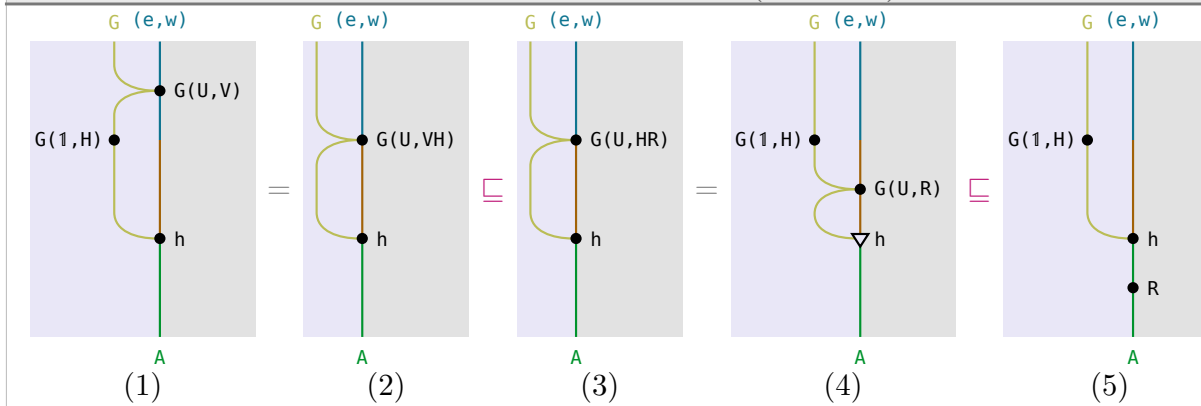
6	$\sqsubseteq$ F((cost $\leq$, S)) k cost $^\circ$	$\langle X, Y \rangle (R \times 1) = \langle X R, Y \rangle$
7	= F((cost, S)) F($\leq \times 1$) k cost $^\circ$	F($\leq \times 1$) k $\sqsubseteq$ k $\leq$
8	$\sqsubseteq$ F((cost, S)) k $\leq$ cost $^\circ$	h cost = F((cost, S)) k
9	= h cost $\leq$ cost $^\circ$	

G(U, R) h $\sqsubseteq$ h R $\wedge$ V H $\sqsubseteq$ H R $\Rightarrow$ G(U, V) G(1, H) h $\sqsubseteq$ G(1, H) h R

Proposition 9.4

thinning the parts by **U** in the first argument and by **V** in the second (G(U, V)), then solving by **H** and assembling by **h**, gives only what solving and assembling and then improving by **R** gives

(G(1, H) h R)



1	G(U, V) G(1, H) h	
2	= G(U, V H) h	V H $\sqsubseteq$ H R
3	$\sqsubseteq$ G(U, H R) h	
4	= G(1, H) G(U, R) h	G(U, R) h $\sqsubseteq$ h R
5	$\sqsubseteq$ G(1, H) h R	

§9.2. The string edit problem

(9.2a)

	name	type	definition	meaning
1	Op	Type	$(\exists a. o = \text{cpy}(a)) \vee (\exists a. o = \text{del}(a)) \vee (\exists a. o = \text{ins}(a))$	an edit operation copies, deletes or inserts one character
2			$FX = 1 + (Op \times X)$	$F(Op, X)$ is the empty sequence or one operation paired with an X
3	edit	$[Op] \rightarrow [Char] \times [Char]$	$\text{edit} = ([\text{base}, \text{step}])$, $\text{edit}([]) = ([], [])$, $\text{edit}([op] \# es) = \text{step}((op, \text{edit}(es)))$	the two strings an edit sequence turns one into the other
4			$\text{length} = ([\text{zero}, \pi_2 \text{ succ}])$	the number of operations in a sequence
5	R	$[Op] \rightarrow [Op]$	$R = \text{length} \leq \text{length}^\circ$, $R(es, fs) \iff \text{length}(es) \leq \text{length}(fs)$	es is no longer than fs
6	V	$[Char] \times [Char] \rightarrow [Char] \times [Char]$	$V = \text{suffix}^\circ \times \text{suffix}^\circ$, $V(p, q) \iff \text{suffix}(\pi_1(p), \pi_1(q)) \wedge \text{suffix}(\pi_2(p), \pi_2(q))$	each string of the first pair is a suffix of the matching string of the second
7	Q	$F([Char] \times [Char]) \rightarrow F([Char] \times [Char])$	$l(val) \ Q \ l(val') \iff \text{true}$, $r(p) \ Q \ r(q) \iff V(\pi_2(p), \pi_2(q))$, $(\forall val. \forall val'. u = l(val) \implies v = l(val') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false}) \implies (u \ Q \ v \iff \text{false})$, $Q(l(u), l(u))$, $Q(r(p), r(q)) \iff V(\pi_2(p), \pi_2(q))$	the identity on the base summand; on the step summand any two operations, and the two pairs of strings by V
8	empty	$[Char] \times [Char] \rightarrow [Char] \times [Char]$	$\text{empty}(p, q) \iff p = q \wedge p = ([], [])$	holds only at the pair of two empty strings
9	unstep	$[Char] \times [Char] \rightarrow [Op] \times [Char] \times [Char]$	$q \in \text{unstep}(p) \implies \text{step}(q) = p$, $\text{unstep}([], []) = []$, $\text{unstep}([a] \# xs, []) = [(\text{del}(a), (xs, []))]$, $\text{unstep}([], [b] \# ys) = [(\text{ins}(b), ([], ys))]$, $\text{unstep}([a] \# xs, [b] \# ys) = \text{if } a = b \text{ then } [(\text{cpy } a, xs, ys)] \text{ else } [(\text{del } a, xs, [b] \# ys), (\text{ins } b, [a] \# xs, ys)]$	every split of a pair of strings into one operation and a smaller pair; step puts it back
10	[nil, cons]	$\text{Unit} + Op \times [Op] \rightarrow [Op]$	$[\text{nil}, \text{cons}](l(u)) = []$, $[\text{nil}, \text{cons}](r(o, es)) = [o] \# es$	[nil, cons]: the empty edit sequence, or one operation in front of a sequence
11	[base, step]	$\text{Unit} + Op \times [Char] \times [Char] \rightarrow [Char] \times [Char]$	$\text{base} = ([], [])$, $\text{step}((\text{cpy}(a), (xs, ys))) = ([a] \# xs, [a] \# ys)$, $\text{step}((\text{del}(a), (xs, ys))) = ([a] \# xs, ys)$, $\text{step}((\text{ins}(a), (xs, ys))) = (xs, [a] \# ys)$	[base, step]: the two empty strings, or one operation added to the front of the pair

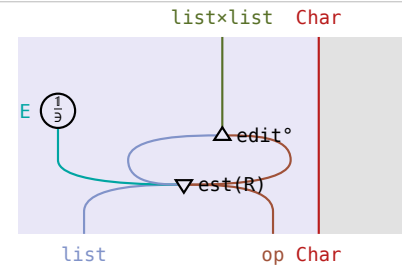
$$\begin{aligned} \text{mle} = (\text{empty} \rightarrow \text{nil}, \text{unstep list}((1 \times \text{mle}) \text{cons}) \text{minlist}(R)), \quad (\mu X : \frac{[\text{base}, \text{step}]^\circ}{3} \text{thin}(Q) \text{P}(F(X) \\ [\text{nil}, \text{cons}]) \text{est}(R)) \sqsubseteq \frac{\text{edit}^\circ}{3} \text{est}(R) \end{aligned} \quad (9.2b)$$

a shortest edit sequence from which both strings can be reconstituted is one pass over the two of them, each step copying, deleting or inserting one character and the best sequence for what is left taken from the entries already computed

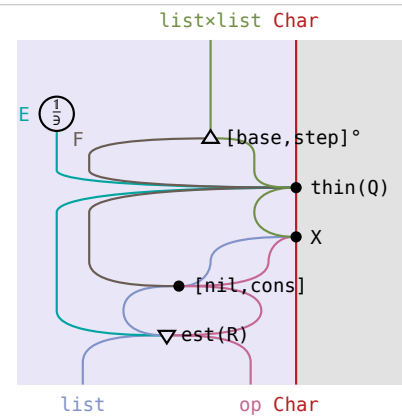
circuit

$[\text{Char}] \xrightarrow{\frac{1}{3}} P[\text{Char}]^2 \xrightarrow{E(\text{edit}^\circ)} P[0p] \xrightarrow{\text{est}(R)} [0p]$
the specification — $\text{edit}([\text{ }]) = ([\text{ }], [\text{ }])$, $\text{edit}([op] \# es) = \text{step}((op, \text{edit}(es)))$

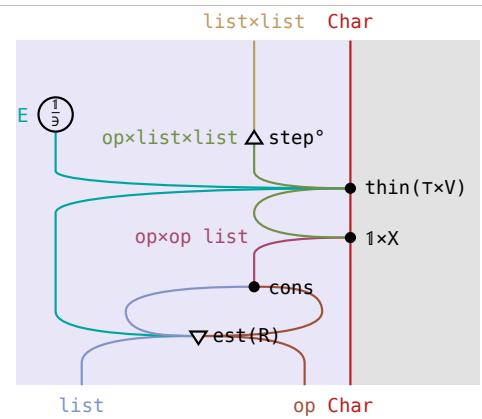
Hinze–Marsden



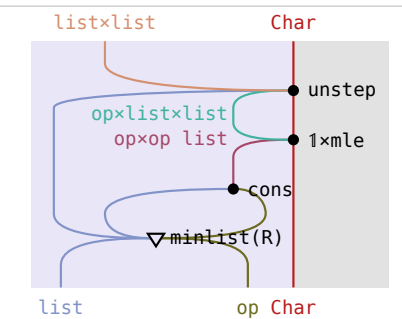
$[\text{Char}] \xrightarrow{\frac{1}{3}} P[\text{Char}]^2 \xrightarrow{E([\text{base}, \text{step}]^\circ)} \text{thin}(Q) \text{P}[F(X) [\text{nil}, \text{cons}]] \xrightarrow{P[0p]} \text{est}(R) [0p]$
at $Q \triangleq 1 + (U \times V)$. Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is Proposition 9.2 at $\text{length} \triangleq ([\text{zero}, \pi_2 \text{succ}])$ with succ monotonic on $\leq$, so cons is monotonic on R . The thinning condition $\sqsubseteq QF(1, \text{edit}^\circ) \alpha \sqsubseteq F(1, \text{edit}^\circ) \alpha R$ over $F(0p, [\text{Char}] \times [\text{Char}])$ is Proposition 9.4 at $U \triangleq T$ — $F(T, R) \alpha \sqsubseteq \alpha R$, left as an exercise, so any two operations may be compared — and $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, — $\text{edit}(1 \times \text{suffix}) \sqsubseteq R^\circ \text{edit}$, $\text{edit}(\text{suffix} \times 1) \sqsubseteq R^\circ \text{edit}$. $\text{edit.suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \Rightarrow BA^* \sqsubseteq C^*B$ cut those to one step each: drop the operation that produced the head, or weaken its cpy to a del , never lengthening the sequence



$[\text{Char}] \xrightarrow{\frac{1}{3}} P[\text{Char}]^2 \xrightarrow{E(\text{step}^\circ)} \text{thin}(T \times V) \text{P}(0p \times [\text{Char}] \times [\text{Char}]) \xrightarrow{P((1 \times X) \text{cons})} P[0p] \xrightarrow{\text{est}(R)} [0p]$
= Proposition 9.1: base and step have disjoint ranges, so the fixed point splits into one branch per summand — empty , the coreflexive on (xs, ys) with both lists empty, is where base returns



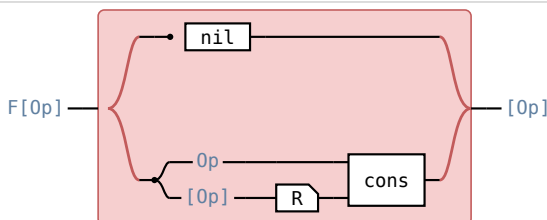
$[\text{Char}] \xrightarrow{\text{unstep}} [0p \times [\text{Char}] \times [\text{Char}]] \xrightarrow{\text{list}((1 \times \text{mle}) \text{cons})} [[0p]] \xrightarrow{\text{minlist}(R)} [0p]$
 unstep implements $\frac{\text{step}^\circ}{3} \text{thin}(U \times V)$ — at most two decompositions survive, a copy beating a delete or an insert wherever it is available — and $\text{minlist}(R)$ implements $\text{est}(R)$. The same subproblem is solved many times over, so the running time is exponential in the two lengths



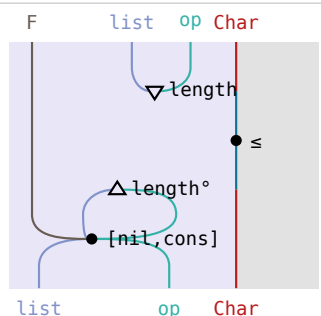
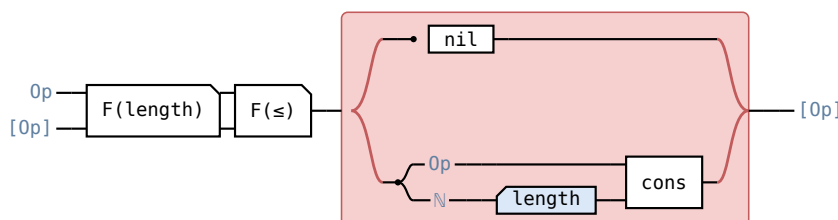
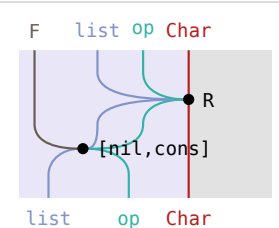
$$F(R) [\text{nil}, \text{cons}] \sqsubseteq [\text{nil}, \text{cons}] R \quad (9.2c)$$

comparing the tails by R and then putting an operation in front ($F(R) \alpha$) relates only sequences that R also relates once the operation is in front (αR)

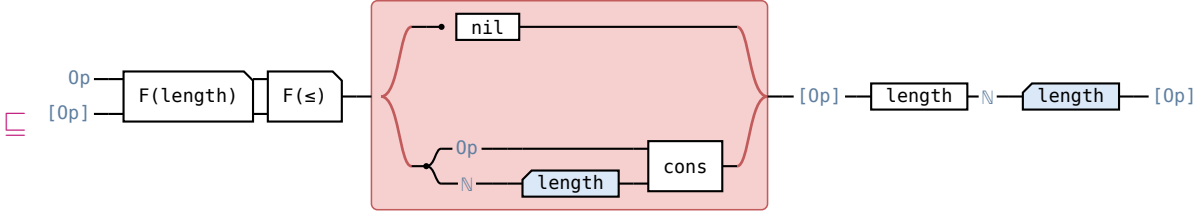
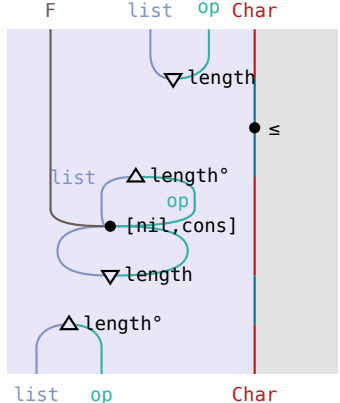
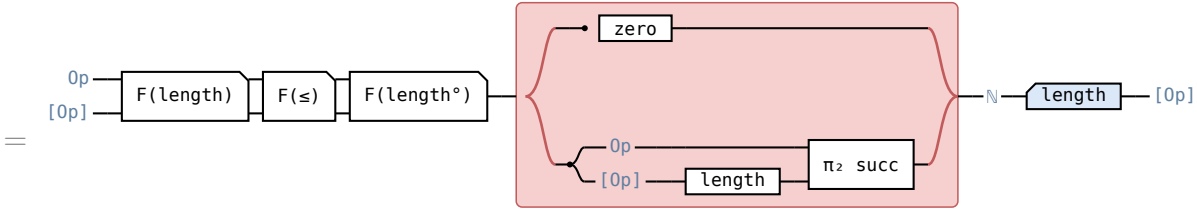
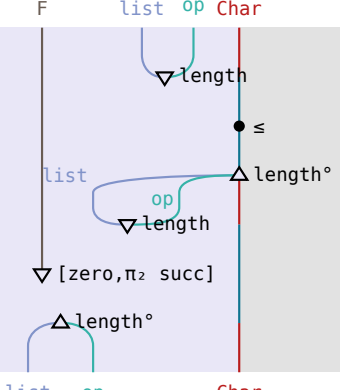
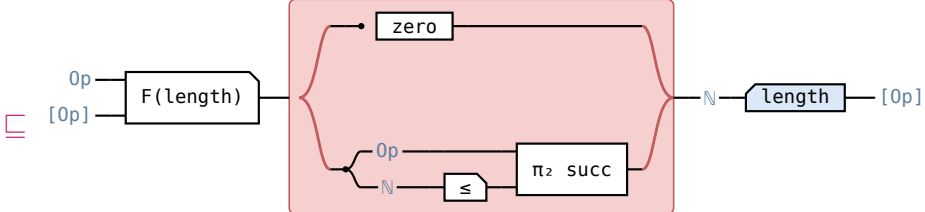
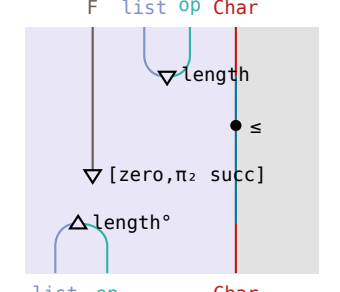
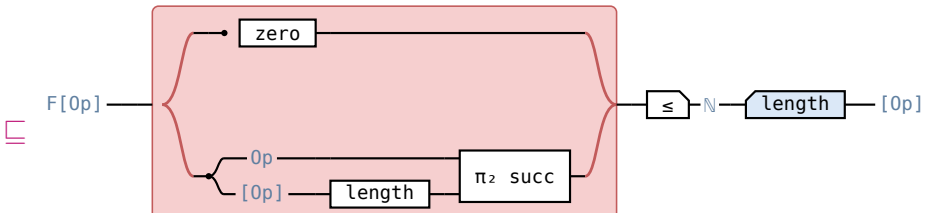
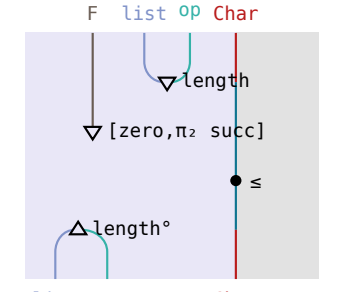
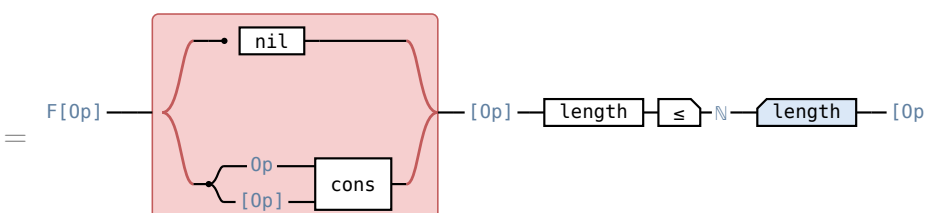
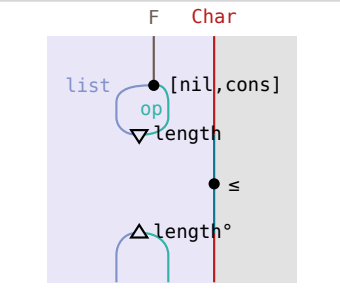
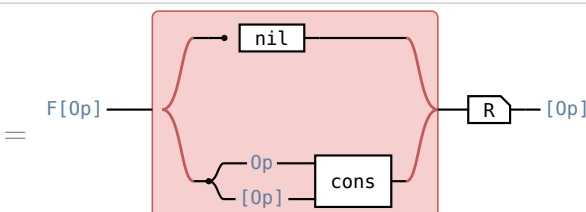
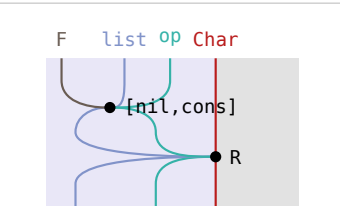
circuit



Hinze–Marsden

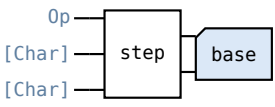
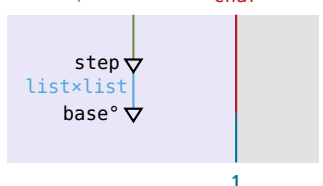
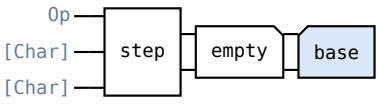
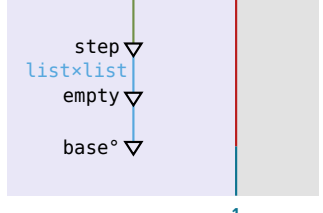
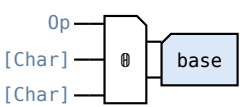
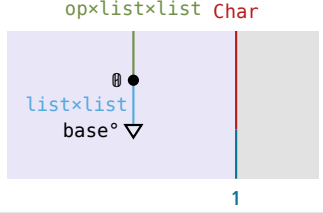
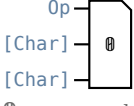
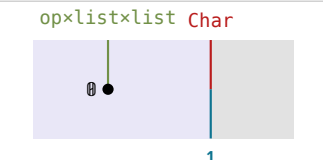


$R \triangleq \text{length} \leq \text{length}^\circ$ — $es \ R \ fs \iff \text{length}(es) \leq \text{length}(fs)$; F preserves composition

circuit	Hinze–Marsden
 length is a map, so entire: $1 \leq \text{length} \text{ length}^\circ$	
 $\alpha \text{ length} = F(\text{length})[\text{zero}, \pi_2 \text{ succ}]: \text{length} \neq ([\text{zero}, \pi_2 \text{ succ}]) - \text{es R fs} \Leftrightarrow \text{length}(\text{es}) \leq \text{length}(\text{fs})$	
 length is a map, so simple, and F is monotonic: $F(\text{length}^\circ)F(\text{length}) \leq F(1) = 1$	
 succ is monotonic on $\leq$: $F(\leq)[\text{zero}, \pi_2 \text{ succ}] \leq [\text{zero}, \pi_2 \text{ succ}] \leq$	
 $F(\text{length})[\text{zero}, \pi_2 \text{ succ}] = \alpha \text{ length again}$	
 $R \neq \text{length} \leq \text{length}^\circ - \text{es R fs} \Leftrightarrow \text{length}(\text{es}) \leq \text{length}(\text{fs})$	

$Q \vdash F(\text{edit}^\circ) [\text{nil}, \text{cons}] \sqsubseteq F(\text{edit}^\circ) [\text{nil}, \text{cons}] R$ (9.2d) every edit sequence built from a decomposition Q puts above a given one — any operation, each string lengthened at the front — is at least as long as one built from the given one	
circuit	Hinze–Marsden
$=$ $Q \triangleq 1 + (U \times V)$ is $F(U, V)$ at $U \triangleq \tau - l(\text{val}) \ Q \ l(\text{val}') \iff \text{true}$, $r(p) \ Q \ r(q) \iff V(\pi_2(p), \pi_2(q))$, $(\forall \text{val}. \forall \text{val}'. u = l(\text{val}) \implies v = l(\text{val}') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false})$ $\implies (u \ Q \ v \iff \text{false})$; and the bifunctor F preserves composition: $F(U, V)F(1, \text{edit}^\circ) = F(U, V \text{ edit}^\circ)$	
$\sqsubseteq$ Proposition 9.4's second condition $V \text{ edit}^\circ \sqsubseteq \text{edit}^\circ R - \text{edit } V^\circ \sqsubseteq R^\circ \text{ edit}$	
$=$ F preserves composition: $F(U, \text{edit}^\circ R) = F(1, \text{edit}^\circ)F(U, R)$	
$\sqsubseteq$ Proposition 9.4's first condition $F(\tau, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: cons adds one to both lengths whatever the two operations are	

$\text{edit } V^\circ \sqsubseteq R^\circ \text{ edit}$ (9.2e) cutting a front off either string an edit sequence produces ($\text{edit } V^\circ$) leaves a pair that a sequence no longer produces ($R^\circ \text{ edit}$)	
circuit	Hinze–Marsden
$=$ $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, and $\times$ preserves composition	
$\sqsubseteq$ $\text{edit } (\text{suffix} \times 1) \sqsubseteq R^\circ \text{ edit}$: drop the operation that produced the head, or weaken its cpy to an ins . Proved by induction at suffix directly, not through $\text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \implies BA^* \sqsubseteq C^*B$ as the book does	
$\sqsubseteq$ $\text{edit } (1 \times \text{suffix}) \sqsubseteq R^\circ \text{ edit}$, the mirror image: cpy weakens to a del	
$\sqsubseteq$ R° is transitive, $\leq$ being so	

$\text{step } \text{base}^\circ = \emptyset$ (9.2f) no pair of strings is both a result of step and the result $([], [])$ of base	
circuit	Hinze–Marsden
	
 base returns only $([], [])$, so $\text{base}^\circ = \text{empty } \text{base}^\circ$	
 $\text{step } \text{empty} = \emptyset$: cpy and del put a character on the left string, ins one on the right	
 $\emptyset$ composed with anything is $\emptyset$	

§9.2.1. Tabulation

(9.2.1a)

1	$\text{mle}(xs, ys) = \text{head}(\text{column}(xs, ys))$, $\text{column}(xs, ys) = [\text{mle}(u, ys) \mid u \leftarrow \text{tails}(xs)]$ $\text{column}(xs) = ([\text{fstcol}(xs), \text{nextcol}(xs)])$, $\text{fstcol} = \text{list}(\text{del}) \text{ tails}$ the tabulation: $\text{mle}(xs, ys)$ needs $\text{mle}(u, v)$ for every tail u of xs and v of ys , so the columns are built right to left
2	$\text{column}(xs, [b] \# ys) = \text{nextcol}(xs, (b, \text{column}(xs, ys)))$ $\text{nextcol}(xs)(b, us) = ([\text{base}(b, \text{last}(us)), \text{step}(b)] \# \text{us})$, $xus = \text{zip}(xs, \text{zip}(\text{init}(us), \text{tail}(us)))$ each column is a fold built bottom to top, over xs zipped with the adjacent pairs of the column to its right
3	$\text{base}(b, u) = [[\text{ins}(b)] \# u]$ $\text{step}(b)((a, (u, v)), ws) = (a \rightarrow b \rightarrow [[\text{cpy}(a)] \# v] \# ws, [\text{bmin}(R)([\text{del}(a)] \# w, [\text{ins}(b)] \# u)] \# ws)$ $w = \text{head}(ws)$; these base , step are not edit 's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths

§9.3. Optimal bracketing

(9.3a)

	name	type	definition	meaning
1	<code>tree(A)</code>	Type	$\text{tree}(A) \triangleq \text{tip}(A) \mid \text{bin}(\text{tree}(A), \text{tree}(A))$	a tree is a tip holding one value, or a node joining two trees; $\mathbf{h} \triangleq [\text{tip}, \text{bin}]$
2	<code>flatten</code>	$\text{tree}(A) \rightarrow \text{LA}$	$\langle [\text{wrap}, \text{cat}] \rangle = \text{flatten}, \text{flatten}(\text{tip}(a)) = [a],$ $\text{flatten}(\text{bin}(xs, ys)) = \text{cat}(\text{flatten}(xs), \text{flatten}(ys))$	the tips of a tree, left to right; $\mathbf{H} = \text{flatten}^\circ$ builds every tree over a list
3	<code>opb(p)</code>	$\text{Int} \times \text{S}$	$\text{opb}(p) \triangleq ((\text{cb}(\pi_2(\pi_1(p)), \pi_2(\pi_2(p))) + \pi_1(\pi_1(p))) + \pi_1(\pi_2(p)), \text{sb}(\pi_2(\pi_1(p)), \pi_2(\pi_2(p))))$	joining two subtrees costs cb of their sizes plus the cost of each
4	$\langle \text{cost}, \text{size} \rangle$	$\text{tree}(A) \rightarrow \text{Int} \times \text{S}$	$\langle \text{cost}, \text{size} \rangle(\text{tip}(a)) = (0, \text{st}(a)),$ $\langle \text{cost}, \text{size} \rangle(\text{bin}(xs, ys)) = \text{opb}(\langle \text{cost}, \text{size} \rangle(xs), \langle \text{cost}, \text{size} \rangle(ys))$	the cost of evaluating a bracketing, paired with the size of its result
5	<code>sz</code>	$\text{LA} \rightarrow \text{S}$	$\text{sz}([a]) = \text{st}(a), \text{sz}([a] \# xs) = \text{sb}(\text{st}(a), \text{sz}(xs)),$ $\text{Assoc}(\text{sb}) \implies \pi_2(\langle \text{cost}, \text{size} \rangle(xs)) = \text{sz}(\text{flatten}(xs))$	sb associative, so the size depends only on the flattened list
6	<code>R</code>	$\text{tree}(A) \rightarrow \text{tree}(A)$	$R = \text{cost} \leq \text{cost}^\circ, R(xs, ys) \iff \text{cost}(xs) \leq \text{cost}(ys)$	t costs no more than t'
7	<code>g</code>	$\text{F}(\text{Int} \times \text{LA}) \rightarrow \text{Int}$	$g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1], g(\text{l}(a)) = 0,$ $g(\text{r}((cx, xs), cy, ys)) = (\text{cb}(\text{sz}(xs), \text{sz}(ys)) + cx) + cy$	the cost of the top step alone: zero at a tip, cb of the two sizes at a node
8	<code>nonsingle</code>	$\text{LA} \rightarrow \text{LA}$	$xs \text{ nonsingle } ys \iff xs = ys \wedge (\exists a. \exists zs. xs = [a] \# zs)$	holds at the lists of two or more elements
9	<code>splits</code>	$\text{LA} \rightarrow [\text{L } A \times \text{L } A]$	$\text{splits} = (\text{inits}^+, \text{tails}^+) \text{ zip}, \text{splits}(xs) = \text{zip}(((\text{inits}^+)(xs), (\text{tails}^+)(xs)))$	every way to cut a list into two non-empty parts — an implementation of $\frac{\text{cat}^\circ}{\exists}$
10	<code>array</code>	$\text{LA} \rightarrow [\text{tree } A]$	$\text{array} \triangleq \text{inits list}(\text{row}), \text{array}(xs, xss) \iff xss = \text{list}(\text{row}, \text{inits}(xs))$	the best trees of every segment, held as rows
11	<code>row</code>	$\text{LA} \rightarrow [\text{tree } A]$	$\text{row} = \text{tails list}(\text{mct}), \text{row}(xs, xss) \iff xss = \text{list}(\text{mct}, \text{tails}(xs))$	the best tree of every non-empty suffix
12	<code>col</code>	$\text{LA} \rightarrow [\text{tree } A]$	$\text{col} = \text{inits list}(\text{mct}), \text{col}(xs, xss) \iff xss = \text{list}(\text{mct}, \text{inits}(xs))$	the best tree of every non-empty prefix
13	<code>mix</code>	$[\text{tree } A] \times [\text{tree } A] \rightarrow \text{tree}(A)$	$\text{mix} = \text{zip list}(\text{bin}) \text{ minlist}(R), \text{mix}(p, xs) \iff xs = \text{minlist}(R) (\text{list bin zip})$	join the matching prefix and suffix trees and keep a cheapest
14	<code>next</code>	$[\text{tree } A] \times [\text{tree } A] \rightarrow [\text{tree } A]$	$\text{next} = (\pi_1, \text{mix}) \text{ snoc}, \text{next}(p, xss) \iff xss = \text{snoc}((\pi_1(p), \text{minlist}(R) (\text{list bin zip})))$	add the next best tree to the end of the column

15

name	type	definition	meaning
process	$A \times [[\text{tree } A]] \rightarrow [\text{tree } A]$	$\text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next}) ,$ $\text{process}((a, xsss), xss) \Leftrightarrow$ $\text{loop}(\text{next})(([\text{tip } a], xsss), xss)$	start the column at a tip and run next over the rows

Assoc(sb) $\Rightarrow$ F((cost, flatten)) g = [tip, bin] cost (9.5) pairing each subtree with its cost and its flattening and then applying g gives what building the tree by [tip, bin] and taking its cost gives	
circuit	Hinze–Marsden
$F(\text{tree}(A)) \rightarrow \boxed{F((\text{cost}, \text{flatten}))} \rightarrow F(\text{Int} \times \text{LA}) \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \text{zero} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{flatten} \rightarrow \text{LA} \rightarrow \text{sz} \end{array} \right\} \\ \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{flatten} \rightarrow \text{LA} \rightarrow \text{sz} \end{array} \right\} \end{array} \right\} \rightarrow \text{opb} \rightarrow \text{Int}$ definition of g; coproducts and products — $g \triangleq [\multimap 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \text{zero} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{size} \end{array} \right\} \\ \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{size} \end{array} \right\} \end{array} \right\} \rightarrow \text{opb} \rightarrow \text{Int}$ flatten sz=size, since sb is associative — $\text{sz}([a]) = \text{st}(a)$, $\text{sz}([a] \# xs) = \text{sb}(\text{st}(a), \text{sz}(xs))$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \text{zero} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{bin}} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \rightarrow \text{Int} \\ \text{size} \rightarrow S \end{array} \right\} \end{array} \right\} \rightarrow \text{opb} \rightarrow \text{Int}$ $(\text{cost}, \text{size}) \triangleq ([\text{opt}, \text{opb}])$, at a node $\text{bin}(\text{cost}, \text{size}) = (\text{cost}, \text{size})^2 \text{ opb}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \boxed{\text{tip}} \rightarrow \text{tree}(A) \rightarrow \text{cost} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{bin}} \rightarrow \text{tree}(A) \rightarrow \text{cost} \end{array} \right\} \rightarrow \text{Int}$ tip cost=zero; products, $(\text{cost}, \text{size}) \pi_1 = \text{cost}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \boxed{l} \rightarrow A \rightarrow \boxed{\text{tip}} \\ \boxed{r} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{bin}} \end{array} \right\} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{cost}} \rightarrow \text{Int}$ coproducts, $[\text{tip cost}, \text{bin cost}] = [\text{tip}, \text{bin}] \text{ cost}$	

$F(\leq \times 1) \quad g \sqsubseteq g \leq$ (9.6)	
raising the cost of either subtree ($\leq \times 1$ in both slots) and then applying g returns a value at least what g returns on the costs before raising ($g \leq$)	
circuit	Hinze–Marsden
$F(\text{Int} \times \text{LA}) \rightarrow \boxed{F(\leq \times 1)} \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$ $=$ definition of $g \rightarrow g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$	
$A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$ $\sqsubseteq$ definition of opb , and $+$ monotonic	
$A + \text{Int} \times \text{LA} \times \text{Int} \times \text{LA} \rightarrow$ $\sqsubseteq$ $\leq$ reflexive, so $\text{zero} \sqsubseteq \text{zero} \leq$; coproducts	
$F(\text{Int} \times \text{LA}) \rightarrow \boxed{g} \rightarrow \boxed{\leq} \rightarrow \text{Int}$ $=$ definition of $g \rightarrow g \triangleq [\neg 0, (1 \times \text{sz})^2 \text{ opb } \pi_1]$	

$$\text{mct}=(\text{single}\rightarrow\text{head tip},\{\text{init col},\text{tail row}\}\text{mix}), \text{ Assoc}(\text{sb}) \Rightarrow (\mu X : \frac{[\text{wrap},\text{cat}]^\circ}{\mathfrak{g}} \text{P}([\text{tip},(X \times X) \text{bin}]) \text{est}(\text{R})) \sqsubseteq \frac{\text{flatten}^\circ}{\mathfrak{g}} \text{est}(\text{R}) \quad (9.3b)$$

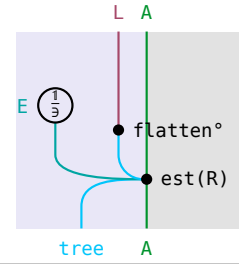
a least-cost bracketing of $a_1 \oplus \dots \oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

circuit

$$\text{LA} \rightarrow \frac{1}{\mathfrak{g}} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}(\text{flatten}^\circ) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

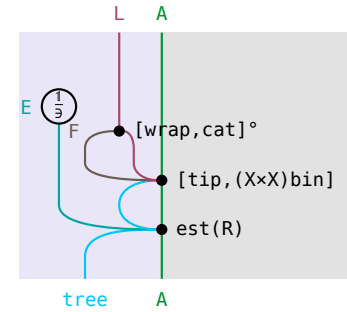
the specification — $\text{flatten}(\text{tip}(\text{a})) = [\text{a}]$, $\text{flatten}(\text{bin}(\text{xs},\text{ys})) = \text{cat}(\text{flatten}(\text{xs}),\text{flatten}(\text{ys}))$

Hinze–Marsden



$$\text{LA} \rightarrow \frac{1}{\mathfrak{g}} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}([\text{wrap},\text{cat}]^\circ) \rightarrow \text{PF}(\text{LA}) \rightarrow \text{P}([\text{tip},(X \times X)\text{bin}]) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

split the list in every way, bracket both halves, join. The condition is monotonicity **in context**,
 $\sqsubseteq \text{F}(\text{Rn}(\text{flatten} \text{ flatten}^\circ)) \text{h} \sqsubseteq \text{hR}$ — only trees with the same flattening are compared — which is Proposition 9.3 at $\text{H}^\circ = \text{flatten}$, a map, with (9.5) $[\text{tip},\text{bin}] \text{ cost} = (1 + (\text{cost}, \text{flatten})^2) \text{g}$ (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) $(1 + (\leq \times 1)^2) \text{g} \sqsubseteq \text{g} \leq \text{g}$ (g monotonic on $\leq$ in its two cost arguments)

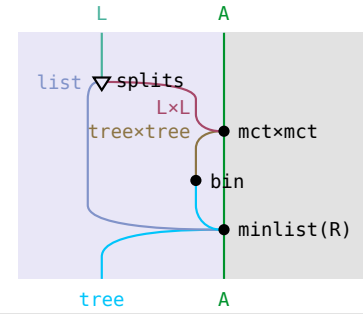


$$\text{LA} \rightarrow \frac{1}{\mathfrak{g}} \rightarrow \text{P}(\text{LA}) \rightarrow \text{E}(\text{cat}^\circ) \rightarrow \text{P}(\text{LA}^2) \rightarrow \text{P}((X \times X)\text{bin}) \rightarrow \text{P}(\text{tree}(\text{A})) \rightarrow \text{est}(\text{R}) \rightarrow \text{tree}(\text{A})$$

Proposition 9.1: **wrap** and **cat** have disjoint ranges, and **single** is the coreflexive on singleton lists, where **wrap** returns

$$\text{LA} \rightarrow \text{splits} \rightarrow [\text{LA}^2] \rightarrow \text{list}((\text{mct} \times \text{mct})\text{bin}) \rightarrow [\text{tree}(\text{A})] \rightarrow \text{minlist}(\text{R}) \rightarrow \text{tree}(\text{A})$$

$\text{splits} \triangleq \langle \text{inits}^+, \text{tails}^+ \rangle$ zip implements $\frac{\text{cat}^\circ}{\mathfrak{g}}$ and $\text{minlist}(\text{R})$ implements $\text{est}(\text{R})$. Exponential, since the segments of one list overlap



$$\text{mct}=(\text{single}\rightarrow\text{head tip},\{\text{init col},\text{tail row}\}\text{mix}) \quad (9.7)$$

the tabulation: mct xs is needed for every non-empty segment xs , so the values are held as an array of rows, $\text{array} \triangleq \text{inits list}(\text{row}), \text{row} \triangleq \text{tails list}(\text{mct}), \text{col} \triangleq \text{inits list}(\text{mct}), \text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(\text{R})$

$$\text{col}=(\text{single}\rightarrow\text{head tip wrap},\{\text{init col},\text{tail row}\}\text{next}) \quad (9.8)$$

$$\text{cons col}=(1 \times \text{array}) \text{ process} \quad (9.9)$$

$$\text{row}=(\text{single}\rightarrow\text{head tip wrap},\{\text{mct},\text{tail row}\}\text{cons}) \quad (9.10)$$

a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, $\text{next} \triangleq \langle \pi_1, \text{mix} \rangle \text{ snoc}, \text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next})$

$$\text{array}=(\text{fstcol},\text{addcol}), \text{fstcol} \triangleq \text{tip wrap wrap}$$

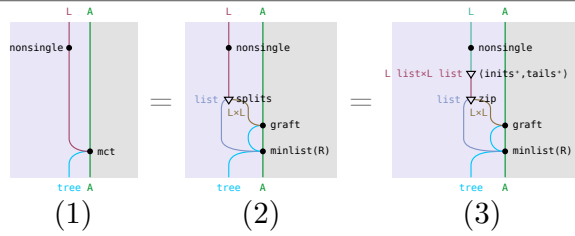
$$\text{addcol} \triangleq \langle \pi_1 \text{ tip wrap}, \text{step} \rangle \text{ cons}, \text{step} \triangleq \langle \text{process tail}, \pi_2 \rangle \text{ zip list}(\text{cons})$$

the program: one fold building the array column by column, cubic in the length of the input

nonsingle mct = nonsingle (init col,tail row) mix

(9.7)

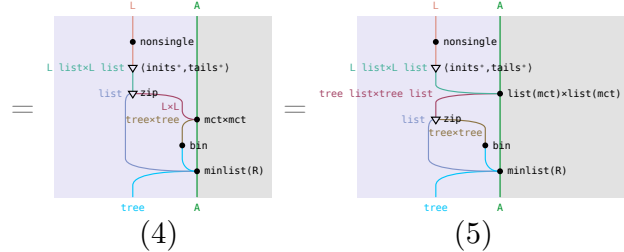
on a list of two or more elements, **mct** gives what **mix** gives on the column of its **init** and the row of its **tail**



(2)

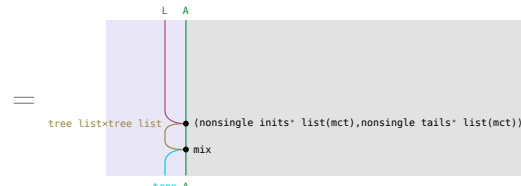
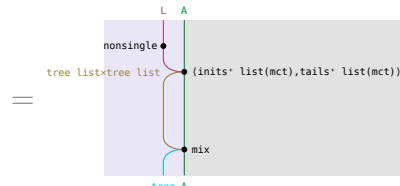
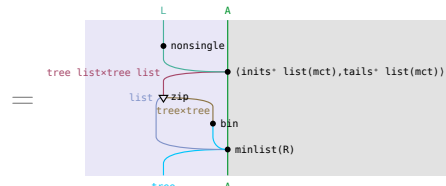
(3)

1	nonsingle mct	nonsingle splits list(graft) minlist(R)
2	= nonsingle splits list(graft) minlist(R)	splits = (inits ⁺ ,tails ⁺) zip
3	= nonsingle (inits ⁺ ,tails ⁺) zip list(graft) minlist(R)	graft $\triangleq$ (mct $\times$ mct) bin

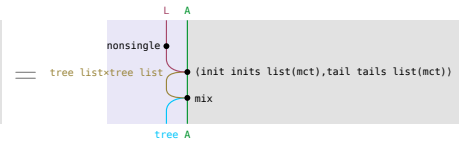
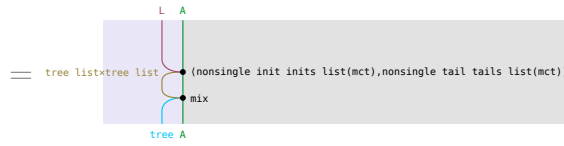
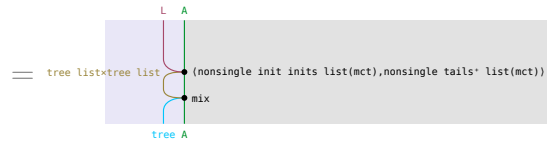


(5)

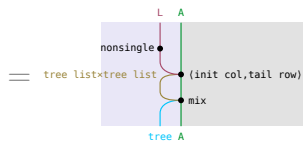
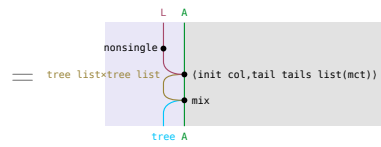
4	= nonsingle (inits ⁺ ,tails ⁺) zip list(mct $\times$ mct) list(bin) minlist(R)	zip list(f $\times$ g) M = (list(f) $\times$ list(g)) zip M
5	= nonsingle (inits ⁺ ,tails ⁺) (list(mct) $\times$ list(mct)) zip list(bin) minlist(R)	(R,S) (U $\times$ V) = (R U,S V)



6	= nonsingle (inits ⁺ list(mct),tails ⁺ list(mct)) zip list(bin) minlist(R)	mix = zip list(bin) minlist(R)
7	= nonsingle (inits ⁺ list(mct),tails ⁺ list(mct)) mix	nonsingle (U,V) = (nonsingle U,nonsingle V)
8	= (nonsingle inits ⁺ list(mct),nonsingle tails ⁺ list(mct)) mix	nonsingle inits ⁺ M = nonsingle init inits M



9	= (nonsingle init inits list(mct),nonsingle tails ⁺ list(mct)) mix	nonsingle tails ⁺ M = nonsingle tail tails M
10	= (nonsingle init inits list(mct),nonsingle tail tails list(mct)) mix	nonsingle (U,V) = (nonsingle U,nonsingle V)
11	= nonsingle (init inits list(mct),tail tails list(mct)) mix	col = inits list(mct)

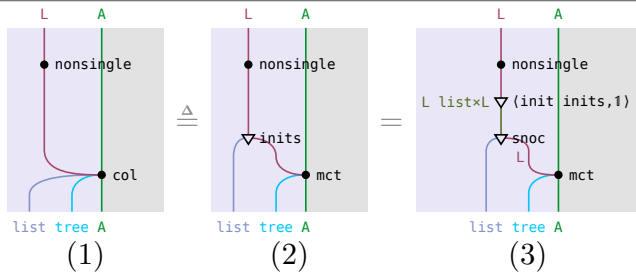


12	= nonsingle (init col,tail tails list(mct)) mix	row = tails list(mct)
13	= nonsingle (init col,tail row) mix	

nonsingle col = nonsingle (init col,tail row) next

(9.8)

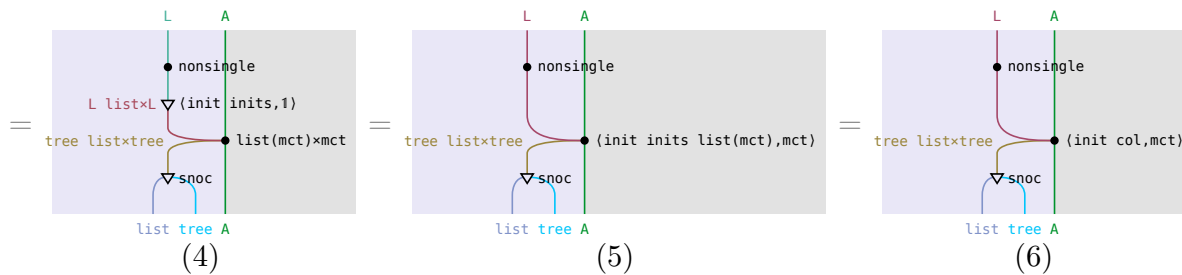
on a list of two or more elements, the column is what **next** makes of the column of its **init** and the row of its **tail**



(2)

(3)

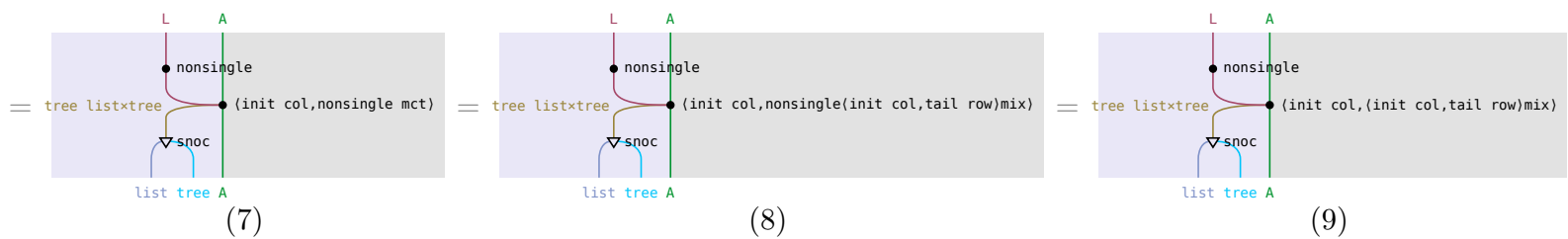
1	nonsingle col	col $\triangleq$ inits list(mct)
2	$\triangleq$ nonsingle inits list(mct)	nonsingle inits M = nonsingle (init inits,1) snoc M
3	= nonsingle (init inits,1) snoc list(mct)	snoc list(f) = (list(f) $\times$ f) snoc



(5)

(6)

4	= nonsingle (init inits,1) (list(mct) $\times$ mct) snoc	$\langle R,S \rangle (U \times V) = \langle R U, S V \rangle$
5	= nonsingle (init inits list(mct),mct) snoc	col = inits list(mct)
6	= nonsingle (init col,mct) snoc	nonsingle $\langle U,V \rangle =$ nonsingle $\langle U,$ nonsingle $V \rangle$



(8)

(9)

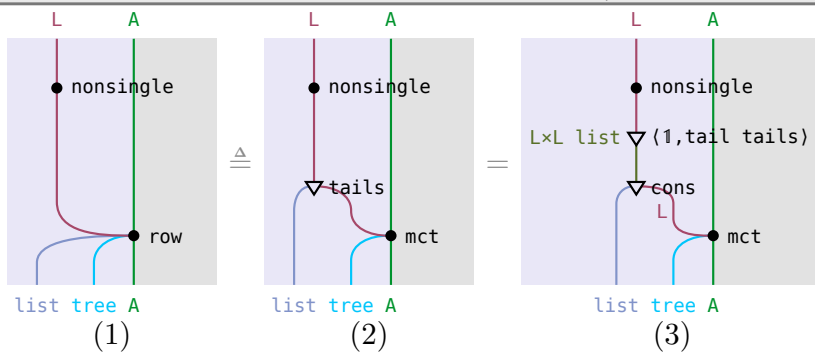
7	= nonsingle (init col,nonsingle mct) snoc	(9.7) nonsingle mct = nonsingle (init col,tail row) mix
8	= nonsingle (init col,nonsingle (init col,tail row) mix) snoc	nonsingle $\langle U,V \rangle =$ nonsingle $\langle U,$ nonsingle $V \rangle$
9	= nonsingle (init col,(init col,tail row) mix) snoc	$\langle \text{init col}, V \rangle \langle \pi_1, M \rangle = \langle \text{init col}, \langle \text{init col}, V \rangle M \rangle$



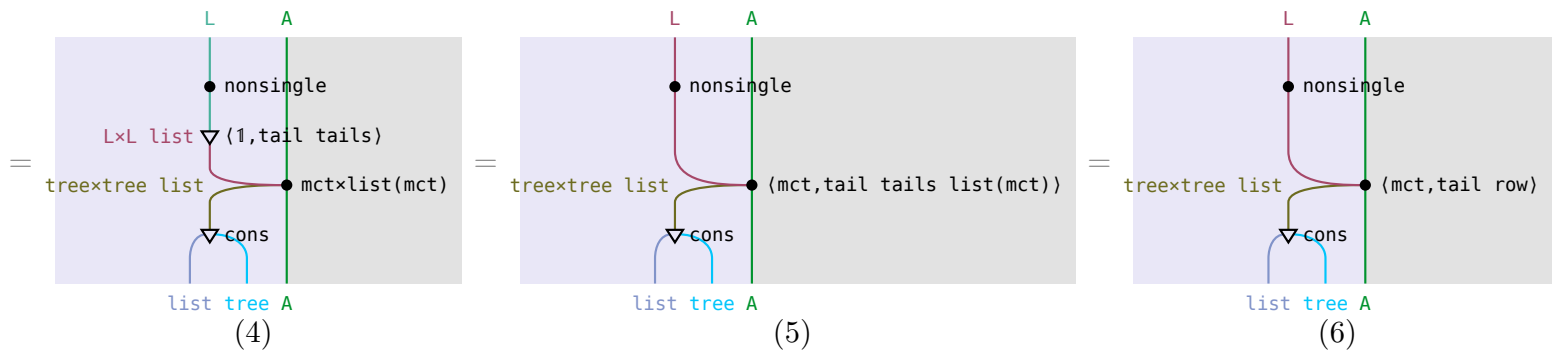
(11)

10	= nonsingle (init col,tail row) $\langle \pi_1, \text{mix} \rangle$ snoc	next = $\langle \pi_1, \text{mix} \rangle$ snoc
11	= nonsingle (init col,tail row) next	

on a list of two or more elements, the row is **mct** of the whole list consed onto the row of its **tail**



1	nonsingle row	row $\triangleq$ tails list(mct)
2	$\triangleq$ nonsingle tails list(mct)	nonsingle tails M = nonsingle (1,tail tails) cons M
3	= nonsingle (1,tail tails) cons list(mct)	cons list(f) = (f $\times$ list(f)) cons

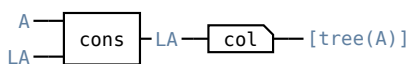


4	= nonsingle (1,tail tails) (mct × list(mct)) cons	(R,S) (U × V) = (R U,S V)
5	= nonsingle (mct,tail tails list(mct)) cons	row = tails list(mct)
6	= nonsingle (mct,tail row) cons	

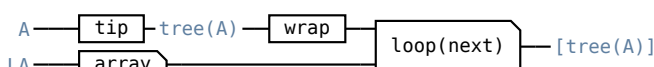
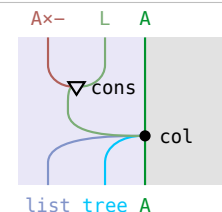
```
cons col = (1 x array) process
```

the column of **a** consed onto **x** is what **process** makes of **a** and the array of **x**

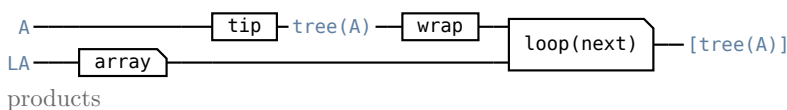
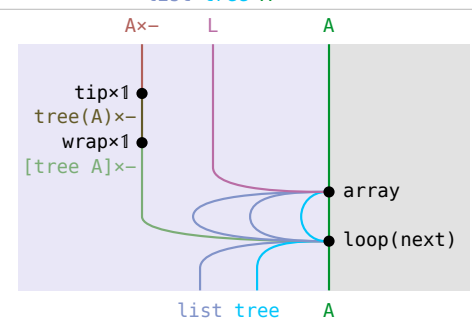
circuit



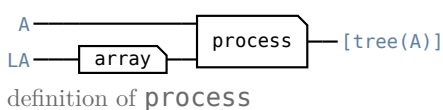
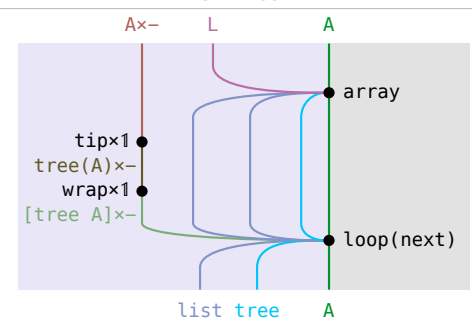
Hinze–Marsden



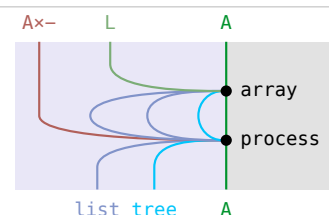
= Exercise 9.13 with `kcons col, gtip wrap, hrow, fnext`: its two equations are (9.8) — (9.8) `nonsingle col = nonsingle (init col, tail row) next` — at `a:[b]` and at `a:(u++[b])`, with `[a] col=[tip(a)]`

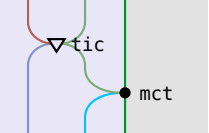
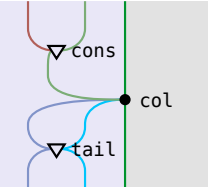
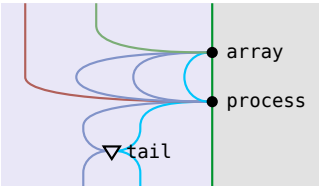


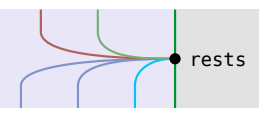
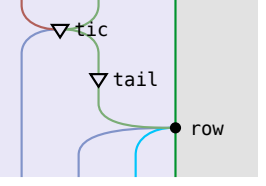
products

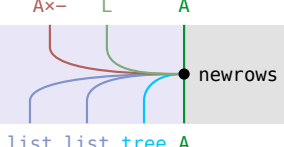
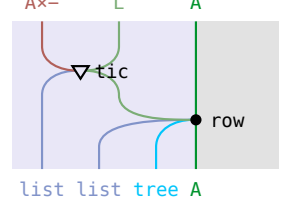
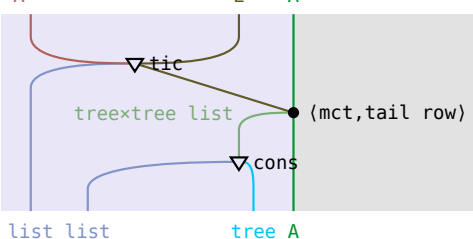
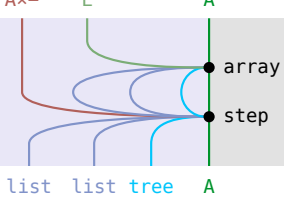


definition of process



tops = (1 × array) process tail (9.3c) the trees at the top of the new rows are what process makes of a and the array of x , less its first	
circuit	Hinze–Marsden
	 list tree A
$= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{inits} \text{tail} \text{LA} \text{list(mct)} \text{tree(A)}$ definition of tops$\triangleq$tic list(mct), our name	 list tree A
$= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{col} \text{tail} \text{tree(A)}$ tail list(f)=list(f) tail; definition of col	 list tree A
$= \begin{array}{c} A \\ LA \end{array} \text{array} \text{process} \text{tail} \text{tree(A)}$ (9.9) — (9.9) cons col = (1 × array) process	 list tree A

rests = π_2 array (9.3d) the rest of each new row is the array of x	
circuit	Hinze–Marsden
	 list list tree A
$= \begin{array}{c} A \\ LA \end{array} \text{cons} \text{LA} \text{inits} \text{tail} \text{LA} \text{list(tail row)} \text{tree(A)}$ definition of rests$\triangleq$tic list(tail row), our name	 list list tree A
$= \begin{array}{c} A \\ LA \end{array} \text{array} \text{tree(A)}$ tic list(tail)=π_2 inits; definition of array	 list list tree A

$\text{newrows} = (1 \times \text{array}) \text{ step}$ (9.3e) the rows of the array of $\mathbf{a}:\mathbf{x}$ below its first are what step makes of $\mathbf{a}$ and the array of $\mathbf{x}$	
circuit	Hinze–Marsden
	
$= \text{LA} \rightarrow \text{cons} \rightarrow \text{LA} \rightarrow \text{inits} \rightarrow \text{tail} \rightarrow [\text{LA}] \rightarrow \text{list}(\text{row}) \rightarrow [[\text{tree}(\text{A})]]$ definition of $\text{newrows} \triangleq \text{tic list}(\text{row})$, our name	
$= \text{LA} \rightarrow \text{cons} \rightarrow \text{LA} \rightarrow \text{inits} \rightarrow \text{tail} \rightarrow [\text{LA}] \rightarrow \text{list}(\text{mct, tail row cons}) \rightarrow [[\text{tree}(\text{A})]]$ (9.10) — (9.10) nonsingle row = nonsingle (mct, tail row) cons — on the non-singleton tic lists	
$= \text{LA} \rightarrow \text{tops} \rightarrow \text{rests} \rightarrow \text{zip} \rightarrow [\text{tree}(\text{A}) \times [\text{tree}(\text{A})]] \rightarrow \text{list}(\text{cons}) \rightarrow [[\text{tree}(\text{A})]]$ $\text{list}(f,g) = (\text{list}(f), \text{list}(g))$ zip; products; definitions of tops and rests	
$= \text{LA} \rightarrow \text{array} \rightarrow \text{step} \rightarrow [[\text{tree}(\text{A})]]$ $\text{tops} = (1 \times \text{array})$ process tail and rests = π_2 array; products; definition of step	

$\text{cons array} = (1 \times \text{array}) \text{ addcol}$ (9.3f)	
the array of a consed onto x is what addcol makes of a and the array of x	
circuit	Hinze–Marsden
$=$ definition of array	
$=$ $\text{cons inits} = (\pi_1 \text{ wrap, tic}) \text{ cons}$, abbreviating cons inits tail by tic	
$=$ $\text{cons list}(f) = (f \times \text{list}(f)) \text{ cons}$; definition of newrows	
$=$ $\text{wrap row} = \text{tip wrap}$	
$=$ $\text{newrows} = (1 \times \text{array}) \text{ step}$; products; definition of addcol	

§9.4. Data compression

(9.4a)

	name	type	definition	meaning
1	String	Type	$\text{String} \triangleq [\text{Char}]$	a string is a snoc-list of characters
2	LA	Type	$\text{LA} \triangleq [\text{A}]$	a non-empty snoc-list: one value, then any number more appended
3	Code	Type	$\text{Code} \triangleq \text{sym}(\text{Char}) \mid \text{ptr}(\text{String}, \text{String})$	a code is one character, or a pointer back to a repeated piece
4	extend	$\text{String} \times \text{Code} \rightarrow [\text{Char}]$	$\text{extend} \triangleq \text{extend} \ , \ \text{extend}((\text{xs}, \text{sym}(\text{a})), \text{x}) = \text{x} = \text{snoc}(\text{xs}, \text{a}) \ , \ \text{extend}((\text{xs}, \text{ptr}(\text{ys}, \text{zs})), \text{x}) = \text{x} = \text{cat}(\text{xs}, \text{zs}) \wedge \text{zs} \neq [] \wedge (\text{init}^+)(\text{cat}(\text{ys}, \text{zs}), \text{cat}(\text{xs}, \text{zs}))$	a symbol appends its character; a pointer appends zs when $\text{ys} \# \text{zs}$ is a proper prefix of $\text{xs} \# \text{zs}$
5	decode	$[\text{Code}] \rightarrow [\text{Char}]$	$\text{decode} \triangleq ([\text{nil}, \text{extend}])$	rebuild the string from its codes, left to right; $\text{H} = \text{decode}^\circ$
6	bytes	$\text{Code} \rightarrow \mathbb{N}$	$\text{bytes}(\text{sym}(\text{a})) = \text{c} \ , \ \text{bytes}(\text{ptr}(\text{a}, \text{a}')) = \text{p}$	a symbol costs c , a pointer costs p
7	size	$[\text{Code}] \rightarrow \mathbb{N}$	$([\text{nil}, \text{distr} \ [1 \times \text{c}, 1 \times \text{p}] \text{ plus}]) = \text{size} \ , \ \text{size}(\text{wrap}(\text{a})) = 0 \ , \ \text{size}(\text{snoc}(\text{cs}, \text{e})) = \text{size}(\text{cs}) + \text{bytes}(\text{e})$	the total cost of a code sequence
8	R	$[\text{Code}] \rightarrow [\text{Code}]$	$\text{R} = \text{size} \leq \text{size}^\circ \ , \ \text{R}(\text{xs}, \text{ys}) \iff \text{size}(\text{xs}) \leq \text{size}(\text{ys})$	u costs no more than v
9	prefix	$[\text{Char}] \rightarrow [\text{Char}]$	$\text{xs prefix ys} \iff (\text{prefix}^\circ)(\text{ys}, \text{xs})$	ys is a prefix of x
10	$(\text{T} + \text{T})$	$\text{Code} \rightarrow \text{Code}$	$(\text{T} + \text{T}) = [\text{sym}, \text{ptr}]^\circ \ (\text{T} + \text{T}) \ [\text{sym}, \text{ptr}] \ , \ \text{sym}(\text{a}) \ (\text{T} + \text{T}) \ \text{sym}(\text{a}') \iff \text{true} \ , \ \text{ptr}(\text{a}, \text{a}') \ (\text{T} + \text{T}) \ \text{ptr}(\text{a}'', \text{a}''') \iff \text{true} \ , \ (\forall \text{a}. \forall \text{a}'. \text{e} = \text{sym}(\text{a}) \implies \text{f} = \text{sym}(\text{a}') \implies \text{false}) \wedge (\forall \text{a}. \forall \text{a}'. \forall \text{a}'' . \forall \text{a}''' . \text{e} = \text{ptr}(\text{a}, \text{a}') \implies \text{f} = \text{ptr}(\text{a}'', \text{a}''') \implies \text{false}) \implies (\text{e} \ (\text{T} + \text{T}) \ \text{f} \iff \text{false})$	any symbol to any symbol, any pointer to any pointer
11	Q	$\text{F}[\text{Char}] \rightarrow \text{F}[\text{Char}]$	$\text{Q} = \text{F}((\text{T} + \text{T}), \text{prefix}^\circ) \ , \ \text{l}(\text{val}) \ \text{Q} \ \text{l}(\text{val}') \iff \text{true} \ , \ \text{r}(\text{p}) \ \text{Q} \ \text{r}(\text{q}) \iff (\text{prefix}^\circ)(\pi_1(\text{p}), \pi_1(\text{q})) \wedge (\text{T} + \text{T})(\pi_2(\text{p}), \pi_2(\text{q})) \ , \ (\forall \text{val}. \forall \text{val}' . \text{u} = \text{l}(\text{val}) \implies \text{v} = \text{l}(\text{val}') \implies \text{false}) \wedge (\forall \text{p}. \forall \text{q}. \text{u} = \text{r}(\text{p}) \implies \text{v} = \text{r}(\text{q}) \implies \text{false}) \implies (\text{u} \ \text{Q} \ \text{v} \iff \text{false})$	compare two decompositions by the prefix order on the strings and U on the codes
12	reduce	$[\text{Char}] \rightarrow [\text{String} \times \text{Code}]$	$\text{reduce} \triangleq \text{reduce} \ , \ \text{q} \in \text{reduce}(\text{ws}) \iff \text{extend}(\text{q}, \text{ws})$	the last code of a string: one symbol, or a pointer to its longest repeated tail

$(\mu X : \frac{[nil, extend]^{\circ}}{\exists} \text{thin}(Q) P([nil, (X \times 1) snoc]) \text{est}(R)) \sqsubseteq \frac{decode^{\circ}}{\exists} \text{est}(R) , \quad (9.4b)$ $\text{encode} = (\text{null} \rightarrow nil, \text{reduce } list((\text{encode} \times 1) snoc) \text{minlist}(R))$ a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded	
circuit	Hinze–Marsden
$[Char] \xrightarrow{\frac{1}{\exists}} P[Char] \xrightarrow{E(decode^{\circ})} P[Code] \xrightarrow{\text{est}(R)} [Code]$ the specification — $decode \triangleq \lambda [nil, extend]$	
$[Char] \xrightarrow{\frac{1}{\exists}} P[Char] \xrightarrow{E([nil, extend]^{\circ})} \text{thin}(Q) \xrightarrow{P([Char])} P([nil, (X \times 1) snoc]) \xrightarrow{P[Code]} \text{est}(R) \xrightarrow{[Code]}$ at $Q \triangleq F(T+T, prefix^{\circ}) = 1 + (prefix^{\circ} \times (T+T))$, the two T on symbols and on pointers. Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is routine, the two costs being constants: $(T+T)[c, p] = [c, p]$. $\sqsubseteq$ Proposition 9.4 splits the thinning condition in two — $F(T+T, R) \alpha \sqsubseteq \alpha R$, and $decode$ $prefix \sqsubseteq R^{\circ} decode$, for which $decode \text{init} \sqsubseteq R^{\circ} decode$ is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins	
$[Char] \xrightarrow{\frac{1}{\exists}} P[Char] \xrightarrow{E(extend^{\circ})} \text{thin}(prefix^{\circ} \times (T+T)) \xrightarrow{P([Char] \times Code)} P((X \times 1) snoc) \xrightarrow{P[Code]} \text{est}(R) \xrightarrow{[Code]}$ $\sqsubseteq$ Proposition 9.1: nil and $extend$ have disjoint ranges. The decompositions of one string are $\frac{extend^{\circ}}{\exists} (ws \# [a]) = \{(ws, \text{sym } a)\} \cup \{(xs, \text{ptr } (ys, zs)) \mid xs \# zs = ws \# [a], ys \# zs \text{ a proper prefix of } ws\}$ — take the last character as a symbol, or end with a pointer	
$[Char] \xrightarrow{\text{reduce}} [[Char] \times Code] \xrightarrow{list((\text{encode} \times 1) snoc)} [[Code]] \xrightarrow{\text{minlist}(R)} [Code]$ $\sqsubseteq$ reduce implements $\frac{extend^{\circ}}{\exists} \text{thin}(prefix^{\circ} \times (T+T))$: thinning leaves at most two, the symbol and the pointer of the longest repeated tail lrt. Again exponential; the book gives no tabulation for it	

§9.4.1. Derivation

$Q \vdash F(\text{decode}^\circ) [\text{nil}, \text{snoc}] \sqsubseteq F(\text{decode}^\circ) [\text{nil}, \text{snoc}] R$ (9.4.1a)	
every code sequence built from a decomposition Q puts above a given one — a code element of the same kind, a longer front string — costs at least as much as one built from the given one	
circuit	Hinze–Marsden
$=$ $Q \triangleq 1 + (\text{prefix}^\circ \times (T+T))$ is $F(T+T, \text{prefix}^\circ) - l(\text{val}) \ Q \ l(\text{val}') \iff \text{true}, r(p) \ Q \ r(q) \iff (\text{prefix}^\circ)(\pi_1(p), \pi_1(q)) \wedge (T+T)(\pi_2(p), \pi_2(q)), (\forall \text{val}. \forall \text{val}'. u = l(\text{val}) \implies v = l(\text{val}') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false})$ $\implies (u \ Q \ v \iff \text{false})$	
$=$ the bifunctor F preserves composition: $F(U, \text{prefix}^\circ) F(1, \text{decode}^\circ) = F(U, \text{prefix}^\circ \text{decode}^\circ)$	
$\sqsubseteq$ Proposition 9.4's second condition $\text{prefix}^\circ \text{decode}^\circ \sqsubseteq \text{decode}^\circ R$: dropping the last character drops the last sym, or shortens or drops the last pointer. Proved at prefix directly by induction, not through init	
$=$ F preserves composition: $F(U, \text{decode}^\circ R) = F(1, \text{decode}^\circ) F(U, R)$	
$\sqsubseteq$ Proposition 9.4's first condition $F(T+T, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: snoc adds the cost of the last element to both sides, and $[c, p](T+T) = [c, p]$	

extend nil° = 0 (9.4.1b) no string is both a result of extend and the result [] of nil	
circuit	Hinze–Marsden
nil returns only [], so $\text{nil}^\circ = \text{null } \text{nil}^\circ$	
$\text{extend } \text{null} = 0$: a symbol ends the string with a character, a pointer with its non-empty ZS	
0 composed with anything is 0	

§10. Greedy Algorithms

§10.1. Theory

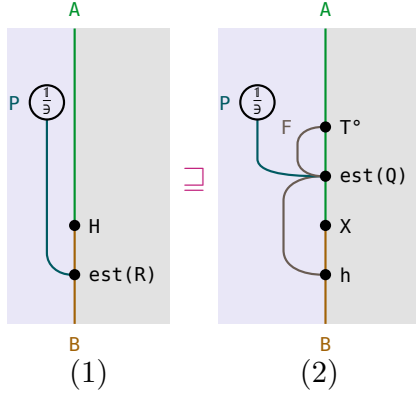
(10.1a)

h, T, R, H, M as in $M \triangleq \frac{H}{\exists} \text{ est}(R)$; additionally Q a **connected** preorder on the sets $\frac{T^\circ}{\exists}$ returns, so that $\frac{T^\circ}{\exists} \text{ est}(Q)$ is entire.

$\text{Map}(h) \wedge h \text{ Pres } R \wedge R \in R \wedge \text{ThinCondition}(T, h, R, Q) \Rightarrow (\mu X : \frac{T^\circ}{\exists} \text{ est}(Q) F(X) h) \sqsubseteq \frac{H}{\exists} \text{ est}(R)$ Theorem 10.1

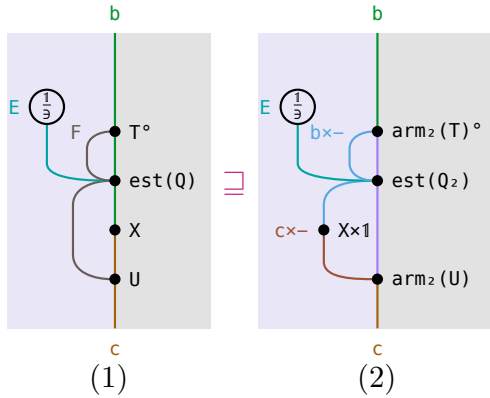
the same optimum reached by keeping ONE decomposition at each step, so that no set is ever carried and the recursion runs on values alone

$H \triangleq (T)^\circ (h) \triangleq H, Q$



1	$\frac{H}{\exists} \text{ est}(R)$	
2	$\sqsubseteq \frac{T^\circ}{\exists} \text{ est}(Q) F(X) h$	

the body at T a coproduct of two arms

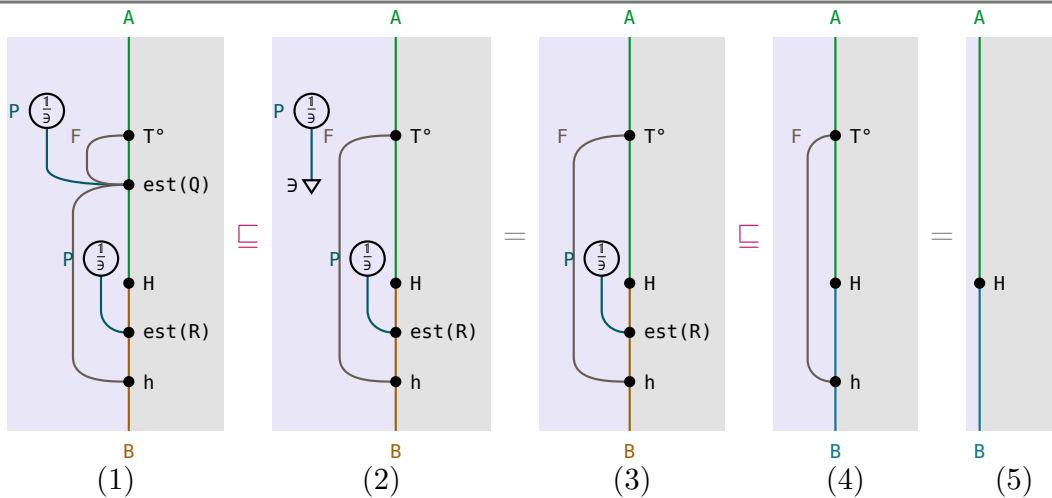


1	$\frac{T^\circ}{\exists} \text{ est}(Q) F(X) U$	Proposition 10.1, $\forall_2 \forall_1^\circ = \emptyset$
2	$\sqsubseteq \frac{\text{arm}_2(T)^\circ}{\exists} \text{ est}(Q_2) (X \times 1) \text{ arm}_2(U)$	

$\frac{T^\circ}{\exists} \text{ est}(Q) F(\frac{H}{\exists} \text{ est}(R)) h \sqsubseteq H$

(10.1b)

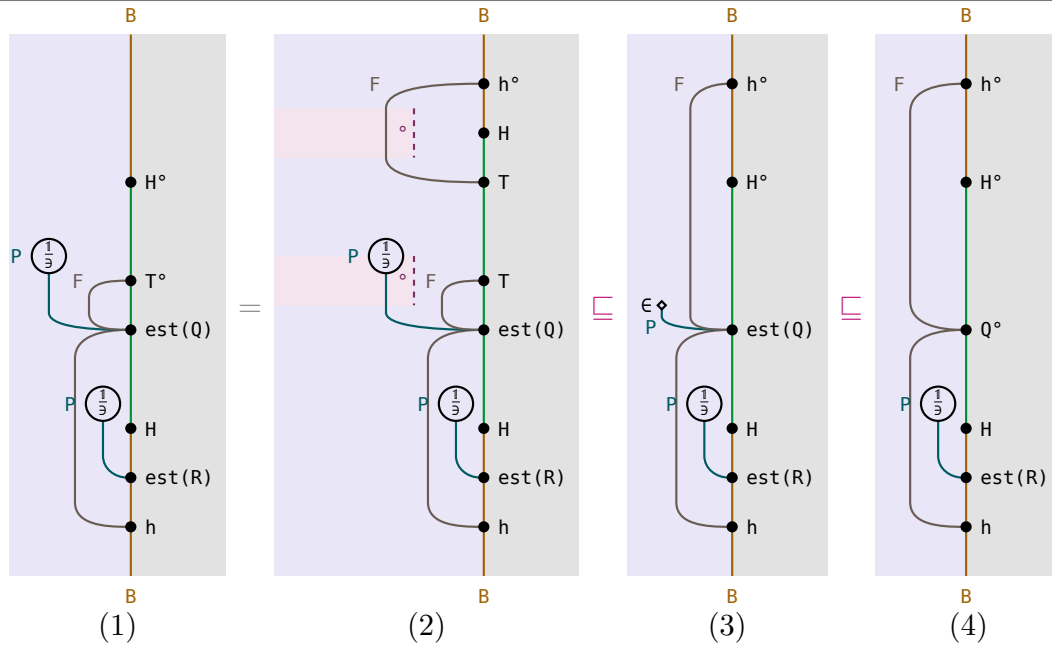
taking the input apart in one Q -extreme way, solving the parts by M and assembling by h returns only what H returns



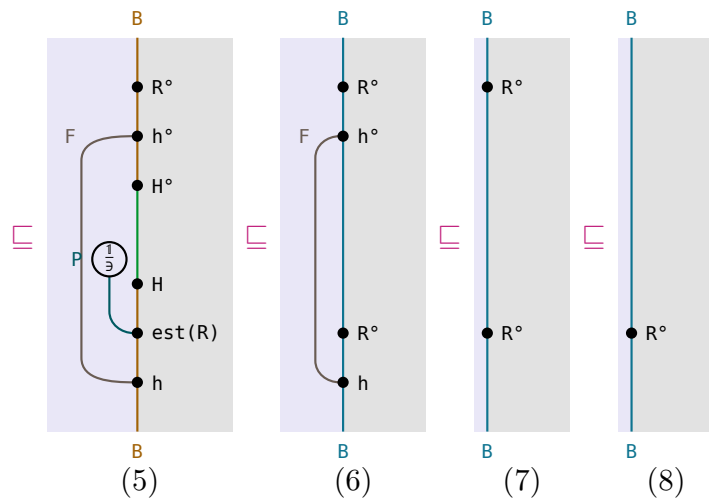
1	$\frac{T^\circ}{\exists} \text{ est}(Q) F(\frac{H}{\exists} \text{ est}(R)) h$	$\text{est}(Q) \sqsubseteq \exists$
2	$\sqsubseteq \frac{T^\circ}{\exists} \exists F(\frac{H}{\exists} \text{ est}(R)) h$	$\frac{R}{\exists} \exists = R$
3	$= T^\circ F(\frac{H}{\exists} \text{ est}(R)) h$	$\frac{\text{est}(R)}{\exists} \sqsubseteq S$
4	$\sqsubseteq T^\circ F(H) h$	$T^\circ F(H) h = H$
5	$= H$	

Map(h) $\wedge$ h Pres R $\wedge$ R R $\subseteq$ R $\wedge$ ThinCondition(T,h,R,Q) $\Rightarrow$ $H^\circ \xrightarrow{T^\circ} \text{est}(Q) F(\frac{H}{S} \text{est}(R)) h \subseteq R^\circ$ (10.1c)

H° followed by the greedy body at M is $\subseteq R^\circ$: an answer of the body is never worse than an answer of H to the same input



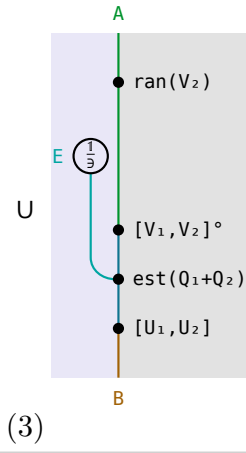
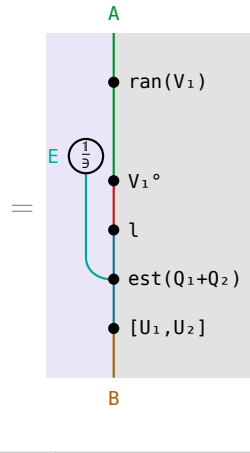
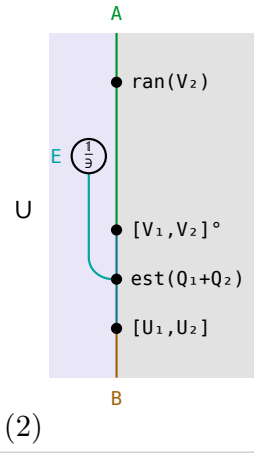
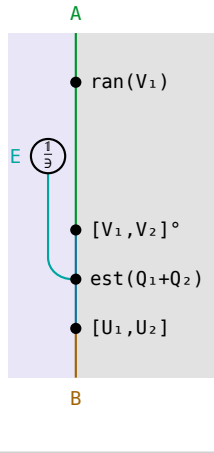
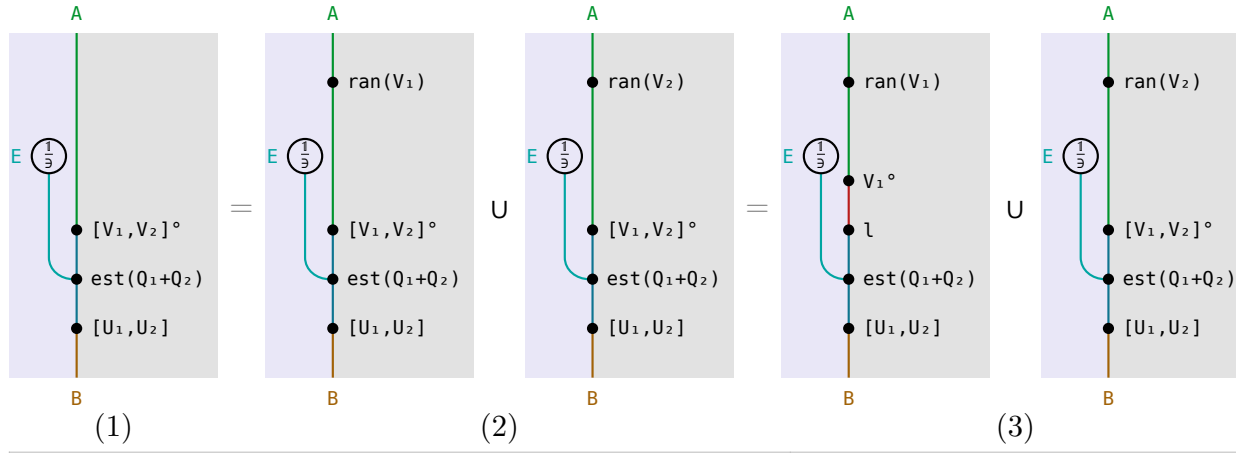
1	$H^\circ \xrightarrow{T^\circ} \text{est}(Q) F(\frac{H}{S} \text{est}(R)) h$	$H^\circ = h^\circ F(H^\circ) T$
2	$= h^\circ F(H^\circ) T \xrightarrow{T^\circ} \text{est}(Q) F(\frac{H}{S} \text{est}(R)) h$	$T \xrightarrow{T^\circ} \subseteq \epsilon$
3	$\subseteq h^\circ F(H^\circ) \epsilon \text{est}(Q) F(\frac{H}{S} \text{est}(R)) h$	$\epsilon \text{est}(R) \subseteq R^\circ$
4	$\subseteq h^\circ F(H^\circ) Q^\circ F(\frac{H}{S} \text{est}(R)) h$	ThinCondition(T,h,R,Q) $\Rightarrow h^\circ F(H^\circ) Q^\circ \subseteq R^\circ h^\circ F(H^\circ)$



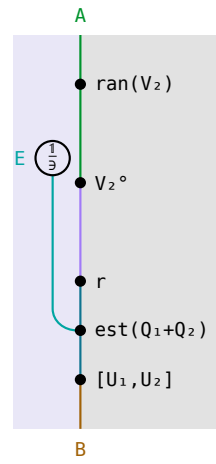
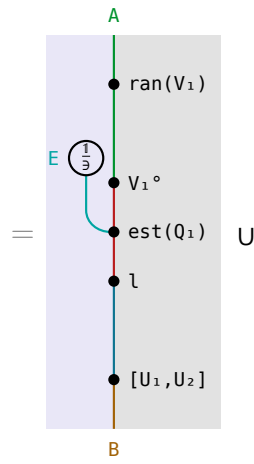
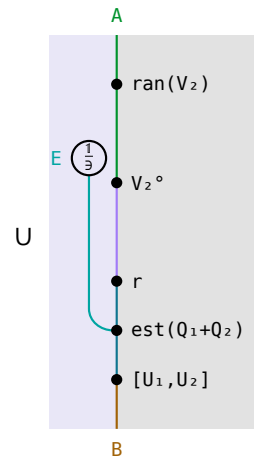
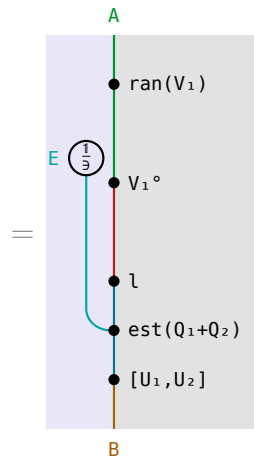
5	$\subseteq R^\circ h^\circ F(H^\circ \xrightarrow{H} \text{est}(R)) h$	$S^\circ \xrightarrow{S} \text{est}(R) \subseteq R^\circ$
6	$\subseteq R^\circ h^\circ F(R^\circ) h$	Map(h) $\wedge$ h Pres R $\Rightarrow h^\circ F(R^\circ) h \subseteq R^\circ$
7	$\subseteq R^\circ R^\circ$	R R $\subseteq$ R $\Rightarrow R^\circ R^\circ \subseteq R^\circ$
8	$\subseteq R^\circ$	

$$V_2 \ V_1^\circ = \emptyset \implies \frac{[V_1, V_2]^\circ}{3} \text{est}(Q_1 + Q_2) [U_1, U_2] = (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \text{est}(Q_1) U_1) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \text{est}(Q_2) U_2) \quad \text{Proposition 10.1}$$

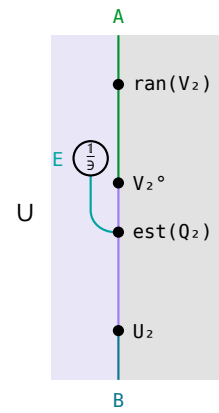
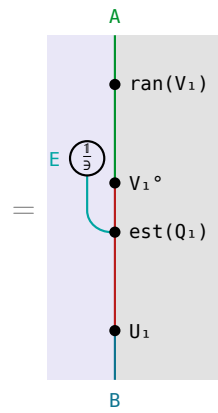
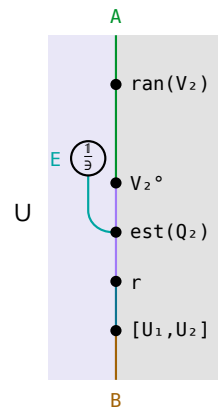
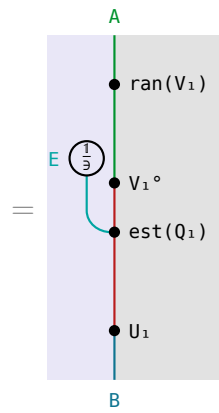
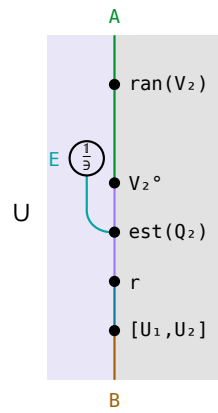
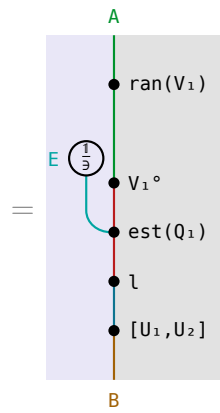
when V_1 and V_2 have disjoint ranges, the greedy step over $[V_1, V_2]$ runs the V_1 step on inputs V_1 reaches and the V_2 step on inputs V_2 reaches



1	$\frac{[V_1, V_2]^\circ}{3} \text{est}(Q_1 + Q_2) [U_1, U_2]$	$\frac{[V_1, V_2]^\circ}{3} \text{est}(Q_1 + Q_2) [U_1, U_2] = (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{est}(Q_1 + Q_2) [U_1, U_2]) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{est}(Q_1 + Q_2) [U_1, U_2])$
2	$= (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \text{est}(Q_1 + Q_2) [U_1, U_2]) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{est}(Q_1 + Q_2) [U_1, U_2])$	$V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_1) \ \frac{V_1^\circ}{3} P(l)$
3	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} P(l) \text{est}(Q_1 + Q_2) [U_1, U_2]) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \text{est}(Q_1 + Q_2) [U_1, U_2])$	$V_2 \ V_1^\circ = \emptyset \implies \text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} = \text{ran}(V_2) \ \frac{V_2^\circ}{3} P(r)$



4	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} P(l) \text{est}(Q_1 + Q_2) [U_1, U_2]) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} P(r) \text{est}(Q_1 + Q_2) [U_1, U_2])$	$P(l) \text{est}(Q_1 + Q_2) = \text{est}(Q_1) \ l$
5	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \text{est}(Q_1) \ l [U_1, U_2]) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} P(r) \text{est}(Q_1 + Q_2) [U_1, U_2])$	$P(r) \text{est}(Q_1 + Q_2) = \text{est}(Q_2) \ r$



6	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \text{est}(Q_1) \ l [U_1, U_2]) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \text{est}(Q_2) \ r [U_1, U_2])$	$l [R, S] = R$
7	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \text{est}(Q_1) U_1) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \text{est}(Q_2) \ r [U_1, U_2])$	$r [R, S] = S$
8	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \text{est}(Q_1) U_1) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \text{est}(Q_2) U_2)$	

§10.2. The detab-entab problem

§10.2.1. Detab

(10.2.1a)

	name	type	definition	meaning
1	detab	$\text{String} \rightarrow \text{String}$	$\text{detab}([nil, expand]) = \text{detab}$, $\text{detab}(\text{wrap}(a)) = []$, $\text{detab}(\text{snoc}(x, a)) = \text{expand}(\text{detab}(x), a)$	replace every tab by blanks up to the next tab stop; $H = \text{detab}^\circ$
2	$\text{expand}(xs, a)$	String	$\text{expand}(xs, a) \triangleq \text{if } a = \text{tb} \text{ then fill } xs \text{ else snoc } xs \ a$	a tab fills the line to the next stop; any other character is appended
3	$\text{fill}(xs)$	String	$\text{fill}(xs) \triangleq xs ++ \text{blanks } (n - \text{col } xs \% n)$	append blanks up to the next multiple of n columns
4	$\text{col}()$	$\text{String} \rightarrow \mathbb{N}$	$\text{col}(\text{wrap}(a)) = 0$, $\text{col}(\text{snoc}(x, a)) = \text{if } a = \text{nl} \text{ then } 0 \text{ else } \text{col } x + 1$	the column the string ends at: characters since the last newline
5	R	$[\text{Char}] \rightarrow [\text{Char}]$	$xs \ R \ ys \iff \text{length}(xs) \leq \text{length}(ys)$	x is no longer than y
6	U	$\text{Char} \rightarrow \text{Char}$	$a \ U \ b \iff a = \text{tb} \vee a = b$	a is the tab or $a=b$
7	V	$[\text{Char}] \rightarrow [\text{Char}]$	$xs \ V \ ys \iff \text{prefix}(xs, ys) \wedge \text{fill}(xs) = \text{fill}(ys)$	a prefix that fills to the same string
8	Q	$F[\text{Char}] \rightarrow F[\text{Char}]$	$Q = 1 + (V \times U)$, $l(\text{val}) \ Q \ l(\text{val}') \iff \text{true}$, $r(p) \ Q \ r(q) \iff V(\pi_1(p), \pi_1(q)) \wedge U(\pi_2(p), \pi_2(q))$, $(\forall \text{val}. \forall \text{val}'. u = l(\text{val}) \implies v = l(\text{val}') \implies \text{false}) \wedge (\forall p. \forall q. u = r(p) \implies v = r(q) \implies \text{false}) \implies (u \ Q \ v \iff \text{false})$	compare two decompositions by V on the strings and U on the characters
9	$\text{unfill}()$	$\text{String} \rightarrow \text{String}$	$\text{unfill}(\text{wrap}(a)) = []$, $\text{unfill}(\text{snoc}(x, a)) = \text{if } a = \text{blank} \wedge \text{col } (\text{snoc } x \ a) \% n \neq 0 \text{ then } \text{unfill } x \text{ else snoc } x \ a$	the shortest prefix that fills to the same string
10	$\text{tbc}()$	$\text{String} \rightarrow \mathbb{N}$	$\text{tbc}(\text{wrap}(a)) = 0$, $\text{tbc}(\text{snoc}(x, a)) = \text{if } a = \text{blank} \wedge \text{col } (\text{snoc } x \ a) \% n \neq 0 \text{ then } \text{tbc } x + 1 \text{ else } 0$	the number of trailing blanks
11	triple	$[\text{Char}] \rightarrow \text{String} \times (\mathbb{N} \times \mathbb{N})$	$\text{triple} \triangleq (\text{unfill } \text{entab}, (\text{tbc}, \text{col}))$	what entab keeps: the unfilled output, its trailing blank count and its column

§10.2.2. Derivation

$$0 < n \Rightarrow \text{expand } V^\circ \sqsubseteq \text{expand } u \ (\pi_1 V^\circ)$$

(10.2.2a)

shortening the output of one **expand** step to a V -smaller string either keeps the whole step (**expand**) or drops its character and shortens its input string ($\pi_1 V^\circ$)

(1)	(2)	(3)	(4)
1 expand V° 2 $= ((\text{istab } \pi_1 \text{ fill}) \cup (\text{nottab } \text{snoc})) V^\circ$ 3 $= (\text{istab } \pi_1 \text{ fill } V^\circ) \cup (\text{nottab } \text{snoc } V^\circ)$		$\text{expand} = (\text{istab } \pi_1 \text{ fill}) \cup (\text{nottab } \text{snoc})$ $(S \cup T) R = (S R) \cup (T R)$ $0 < n \wedge \text{blank} \neq n\ell$ $\Rightarrow \text{fill } V^\circ = \text{fill}$	
(5)	(6)	(7)	(8)
4 $= (\text{istab } \pi_1 \text{ fill}) \cup (\text{nottab } \text{snoc } V^\circ)$ 5 $\sqsubseteq (\text{istab } \pi_1 \text{ fill}) \cup (\text{nottab } (\text{snoc } \cup (\pi_1 V^\circ)))$ 6 $\sqsubseteq \text{expand } \cup (\pi_1 V^\circ)$		$0 < n \wedge \text{blank} \neq n\ell$ $\Rightarrow \text{snoc } V^\circ \sqsubseteq \text{snoc } \cup (\pi_1 V^\circ)$ $(\text{istab } \pi_1 \text{ fill}) \cup (\text{nottab } (\text{snoc } \cup X)) \sqsubseteq \text{expand} \cup X$	

$$0 < n \Rightarrow \text{detab } V^\circ \sqsubseteq R^\circ \text{ detab}$$

(10.2.2b)

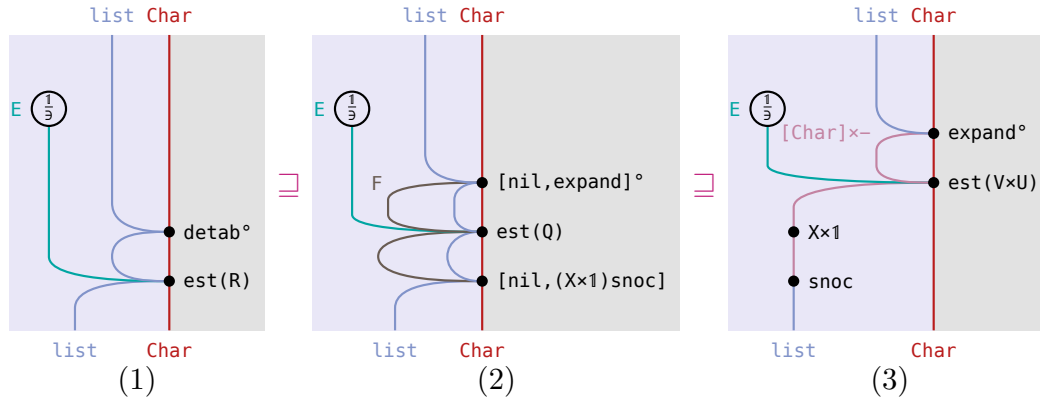
any V -shortening of **detab**'s output is **detab**'s output on an input no longer than the given one

(1)	(2)	(3)	(4)
1 detab V° 2 $= [\text{nil}, \text{snoc}]^\circ F(\text{detab}) [\text{nil}, \text{expand}] V^\circ$ 3 $= [\text{nil}, \text{snoc}]^\circ F(\text{detab}) [\text{nil } V^\circ, \text{expand } V^\circ]$ 4 $= [\text{nil}, \text{snoc}]^\circ F(\text{detab}) [\text{nil}, \text{expand } V^\circ]$		$\text{detab} = [\text{nil}, \text{snoc}]^\circ F(\text{detab}) [\text{nil}, \text{expand}]$ $[R, S] Z = [R Z, S Z]$ $\text{nil } V^\circ = \text{nil}$ (10.2.2a) $0 < n$ $\Rightarrow \text{expand } V^\circ \sqsubseteq \text{expand } \cup (\pi_1 V^\circ)$	
(5)	(6)	(7)	(8)
5 $\sqsubseteq [\text{nil}, \text{snoc}]^\circ F(\text{detab}) [\text{nil}, \text{expand } \cup (\pi_1 V^\circ)]$ 6 $= \text{detab } \cup (\text{snoc}^\circ (\text{detab} \times 1) \pi_1 V^\circ)$ 7 $= \text{detab } \cup (\text{snoc}^\circ \pi_1 \text{ detab } V^\circ)$ 8 $\sqsubseteq R^\circ \text{ detab}$		$[\text{nil}, \text{snoc}]^\circ F(\text{detab}) [\text{nil}, \text{expand } \cup X] = \text{detab } \cup (\text{snoc}^\circ (\text{detab} \times 1) X)$ $\text{snoc}^\circ (\text{detab} \times 1) \pi_1 Y = \text{snoc}^\circ \pi_1 \text{ detab } Y$ $0 < n$ $\Rightarrow \text{detab } \cup (\text{snoc}^\circ \pi_1 \text{ detab } V^\circ) \sqsubseteq R^\circ \text{ detab}$	

$$0 < n \Rightarrow (\mu X : \frac{[nil, expand]^{\circ}}{3} \text{ est}(Q) F(X) [nil, snoc]) \sqsubseteq \frac{detab^{\circ}}{3} \text{ est}(R) \quad (10.2.2c)$$

the shortest input **detab** expands to the given output is one pass along that output, holding each blank back and caching the held blanks in for a tab wherever the column reaches a tab stop

the specification — $detab(wrap(a)) = []$, $detab(snoc(x,a)) = expand(detab(x),a)$; $detab \text{ entab} = 1$ and nothing shorter does



$$1 \quad \frac{detab^{\circ}}{3} \text{ est}(R)$$

at $Q \triangleq 1 + (V \times U)$: one character of input is decided at each step. $F(T, R) \alpha \sqsubseteq \alpha R$. $detab \text{ prefix} \sqsubseteq R^{\circ} \text{ detab}$ is FALSE, — at $n=8$, $detab [a,b,c,d,e,TB] = [a,b,c,d,e,BL,BL,BL]$, whose prefix $[a,b,c,d,e,BL,BL]$ is longer than any input giving it, and $detab V^{\circ} \sqsubseteq R^{\circ} \text{ detab}$ holds. $expand V^{\circ} \sqsubseteq expand \cup (\pi_1 V^{\circ})$ — shortening the output either leaves the last step alone or discards it

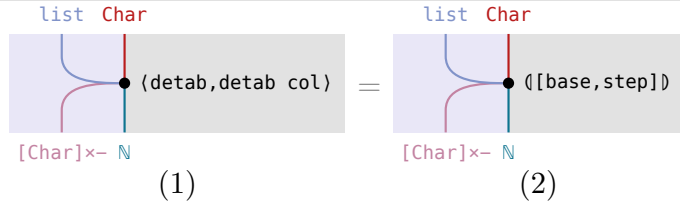
$$2 \quad \sqsubseteq \frac{[nil, expand]^{\circ}}{3} \text{ est}(Q) F(X) [nil, snoc]$$

Proposition 10.1: **nil** and **expand** have disjoint ranges. The greedy step is to emit a tab whenever a tab is legal, consuming all the blanks back to the previous tab stop

$$3 \quad \sqsubseteq \frac{expand^{\circ}}{3} \text{ est}(V \times U) (X \times 1) snoc$$

$$\langle detab, detab \text{ col} \rangle = \langle [base, step] \rangle \quad (10.2.2d)$$

one pass over the input carries the output so far together with its column



$$1 \quad \langle detab, detab \text{ col} \rangle$$

both components are folds over the same input; **step** on $(detab \ x, col(detab \ x))$ is $(detab, col \ detab)$ one character further — not a tabulated row

$$2 \quad = \langle [base, step] \rangle$$

Exercise 10.1

$$\bullet \quad \langle [base, step] \rangle (\text{convert}(xs), r) \Leftrightarrow r = \text{loop}(\text{step}, ([], 0), xs)$$

the fold of $[base, step]$ over the input converted to a snoc-list is the left loop of **step** from **base**

$$\bullet \quad \pi_1(\text{loop}(\text{step}, ([], c_0), xs)) = \text{loop}'((f, g), (c_0, xs))$$

Exercise 10.1: when **step** only appends $f(c, a)$ to the output and moves the state by g , the output of the loop is $\text{loop}'(f, g)$, which never carries the output

$$\bullet \quad a \neq tb \Rightarrow \text{expand}(ys, b) = \text{snoc}(xs, a) \Leftrightarrow \text{fill}(ys) = \text{snoc}(xs, a) \wedge b = tb \vee (ys, b) = (xs, a)$$

expand produces $x \# [a]$ from a tab after a string that fills to it, or from x and a

$$\bullet \quad 0 < n \Rightarrow (\exists ys. \text{fill}(ys) = \text{snoc}(xs, a)) \Leftrightarrow a = \text{blank} \wedge col(\text{snoc } xs \ a) \% n = 0$$

a string is a **fill** exactly when it ends in a blank on a tab stop

(10.2.2e)

$$\bullet \quad \text{entab}(xs) = \text{entab}(\text{unfill } xs) ++ \text{blanks}(\text{tbc } xs)$$

(10.1): the output of **entab** is that of **unfill** x followed by the trailing blanks held back

(10.1)

$$\bullet \quad \text{tbc}[] = 0$$

the empty string has no trailing blanks

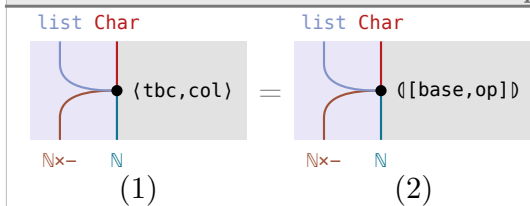
$$\bullet \quad \text{tbc}(\text{snoc}(xs, a)) = \text{if } a = \text{blank} \wedge col(\text{snoc } xs \ a) \% n \neq 0 \text{ then } \text{tbc } xs + 1 \text{ else } 0$$

a blank off a tab stop adds one held blank; anything else releases them all

(tbc,col) = ([base,op])

(10.2.2f)

one pass counts the held blanks and the column together



1 (tbc,col)

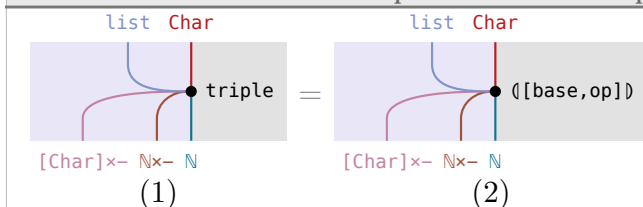
2 = ([base,op])

op on (tbc x,col x) is (tbc,col) one character further,
 from the equations for tbc — (10.1) entab(xs) = entab
 (unfill xs) ++ blanks (tbc xs), tbc[] = 0,
 tbc(snoc(xs,a)) = if a = blank ∧ col (snoc xs
 a) % n ≠ 0 then tbc xs + 1 else 0

triple = ([base,op])

(10.2.2g)

one pass carries the output up to the held blanks, their count and the column



1 triple

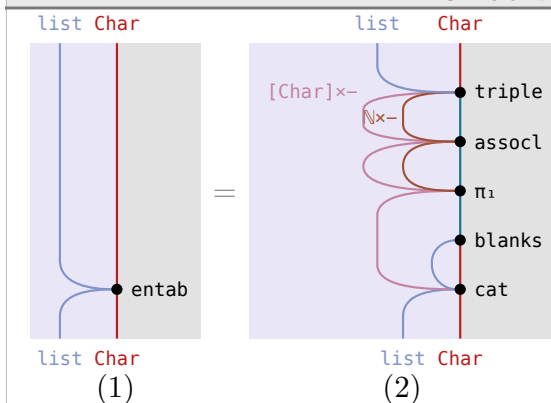
2 = ([base,op])

op on triple x is triple one character further: (10.1) where
 the blanks are released — (10.1) entab(xs) = entab
 (unfill xs) ++ blanks (tbc xs), tbc[] = 0,
 tbc(snoc(xs,a)) = if a = blank ∧ col (snoc xs
 a) % n ≠ 0 then tbc xs + 1 else 0

entab = triple assocl π₁ (1 × blanks) cat

(10.2.2h)

entab is triple with the held blanks appended to its output



1 entab

2 = triple assocl π₁ (1 × blanks) cat

(10.1) — (10.1) entab(xs) = entab (unfill xs) ++
 blanks (tbc xs), tbc[] = 0, tbc(snoc(xs,a)) = if
 a = blank ∧ col (snoc xs a) % n ≠ 0 then tbc xs
 + 1 else 0

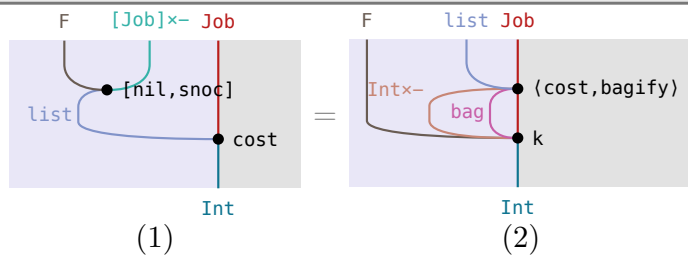
§10.3. The minimum tardiness problem

(10.3a)

	name	type	definition	meaning
1	bag(Job)	$\mathcal{A}$	$\text{bag}(\text{Job}) \triangleq \text{Quotient}(\text{perm})$	a bag of jobs: a list up to reordering
2	snag(p)	bag(Job)	$\text{snag}(p) \triangleq \pi_2 p :: \pi_1 p$	put one job into a bag
3	bagify	[Job]→bag(Job)	$\langle [\text{nil}, \text{snag}] \rangle = \text{bagify} , \text{bagify}(\text{wrap}(u)) = \text{nil} ,$ $\text{bagify}(\text{snoc}(xs, a)) = \text{snag}(\text{bagify}(xs), a)$	forget the order of a schedule; $H = \text{bagify}^\circ$ lists every schedule of a bag
4	penalty(xs, j)	Int	$\text{penalty}(xs, j) \triangleq (\text{sum}(\text{list}(\text{ct})) \text{ xs} + \text{ct } j - \text{dt } j) \text{wt}(j)$	how late job j finishes after the schedule xs, times its weight
5	cost	[Job]→Int	$\text{cost}(\text{wrap}(a)) = 0 , \text{cost}(\text{snoc}(x, a)) =$ $\text{bmax}(\text{cost}(x), \text{penalty}(x, a))$	the largest penalty of any job in the schedule
6	bmax(a, b)	Int	$\text{bmax}(a, b) \triangleq \text{if } a \leq b \text{ then } b \text{ else } a$	the larger of two numbers
7	R	[Job]→[Job]	$R = \text{cost} \leq \text{cost}^\circ , R(xs, ys) \iff \text{cost}(xs) \leq \text{cost}(ys)$	xs costs no more than ys
8	add	[Job]×Job→[Job]	$p \text{ add } ys \iff \text{add}(\pi_2(p), ys)$	insert j anywhere in xs; $\text{perm} \triangleq \text{bagify}$ $\text{bagify}^\circ = \langle [\text{nil}, \text{add}] \rangle$
9	k	F(Int×bag(Job))→Int	$k \triangleq [\text{nil}, \text{assocr } (1 \times ((\text{bagify}^\circ \times 1) \text{ penalty})) \text{ bmax}] ,$ $k(l(u)) = 0 , k(r((c, xs), j)) =$ $\text{bmax}(c, (\text{bagify}^\circ \times 1) \text{ penalty}((xs, j)))$	the cost of a schedule from the cost of its front and the penalty of its last job
10	f	F(bag(Job))→Int	$f(l(\text{val})) = 0 , f(r(p)) = (\text{bagify}^\circ \times 1) \text{ penalty}(p)$	the penalty of the last job, read off the bag before it
11	Q	F(bag(Job))→F(bag(Job))	$u Q v \iff f(u) \leq f(v)$	the last job of u gets no larger penalty than that of v
12	Q'	bag(Job)×Job→bag(Job)×Job	$p Q' q \iff (\text{bagify}^\circ \times 1) \text{ penalty}(p) \leq$ $(\text{bagify}^\circ \times 1) \text{ penalty}(q)$	Q on the snag summand: compare the penalties of the two last jobs
13	g	F[Job]→Int	$g \triangleq [\text{zero}, \text{penalty}] , g(s, c) \iff c = g(s) , g(l(u)) = 0 ,$ $g(r(xs, j)) = \text{penalty}(xs, j)$	(10.5): the penalty of the last job, zero for the empty schedule
14	m	F[Job]→Int	$m \triangleq [\text{zero}, \pi_1 \text{ cost}]$	(10.6), B&dM's h renamed as in (Vj. $0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j))$ $\implies (\mu X : \frac{[\text{nil}, \text{snag}]^\circ}{\exists} \text{ est}(Q) F(X) [\text{nil}, \text{snoc}]) \sqsubseteq \frac{\text{bagify}^\circ}{\exists} \text{ est}(R) :$ the cost of the schedule before the last job

$$[\text{nil}, \text{snoc}] \text{ cost} = F(\langle \text{cost}, \text{bagify} \rangle) k \quad (10.3b)$$

the cost of a schedule is got from the cost and the bag of the schedule before its last job, and that job



1 $[\text{nil}, \text{snoc}] \text{ cost}$

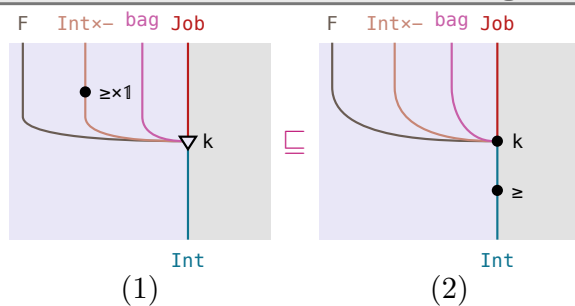
2 $= F(\langle \text{cost}, \text{bagify} \rangle) k$

$\text{cost}(xs \# [j]) = \text{bmax}(\text{cost } xs, \text{penalty}(xs, j))$, and $\text{penalty}(xs, j)$ reads only the bag of xs — $\text{penalty}(xs, j) \triangleq (\text{sum}(\text{list}(ct)) \text{ } xs + ct \text{ } j - dt \text{ } j) \text{wt}(j)$

$$F(\geq \times 1) k \sqsubseteq k \geq$$

Exercise 10.5

a larger cost before the last job gives a larger cost after it



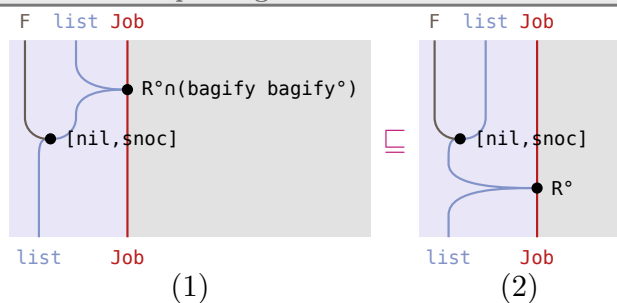
1 $F(\geq \times 1) k$

2 $\sqsubseteq k \geq$

bmax is monotone in its first argument

$$F(R^\circ \text{ } n \text{ } (\text{bagify } \text{bagify}^\circ)) [\text{nil}, \text{snoc}] \sqsubseteq [\text{nil}, \text{snoc}] R^\circ \quad (10.2)$$

improving the schedule before the last job, among schedules of the same bag, improves the whole schedule



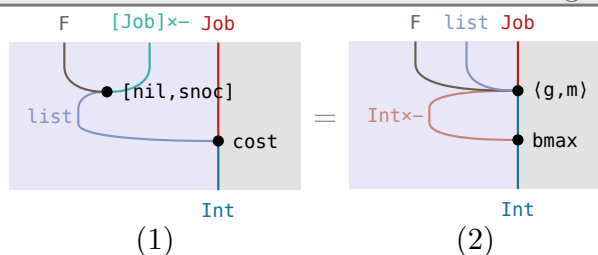
1 $F(R^\circ \text{ } n \text{ } (\text{bagify } \text{bagify}^\circ)) [\text{nil}, \text{snoc}]$

2 $\sqsubseteq [\text{nil}, \text{snoc}] R^\circ$

Proposition 9.3 at $S \triangleq \text{bagify}$, $\leq \triangleq$ — Proposition 9.3
 $\text{Map}(\text{cost}) \wedge \text{Simple}(S) \wedge h \text{ cost} = F(\langle \text{cost}, S \rangle) k$
 $\wedge F(\leq \times 1) k \sqsubseteq k \leq$
 $\Rightarrow F((\text{cost} \leq \text{cost}^\circ) \text{ } n \text{ } (S \text{ } S^\circ)) h \sqsubseteq h \text{ cost} \leq \text{cost}^\circ$, with $[\text{nil}, \text{snoc}] \text{ cost} = F(\langle \text{cost}, \text{bagify} \rangle) k$
 and Exercise 10.5 $F(\geq \times 1) k \sqsubseteq k \geq$

$$[\text{nil}, \text{snoc}] \text{ cost} = \langle g, m \rangle \text{ bmax} \quad (10.4)$$

the cost of a schedule is the larger of the penalty of its last job and the cost of the schedule before it



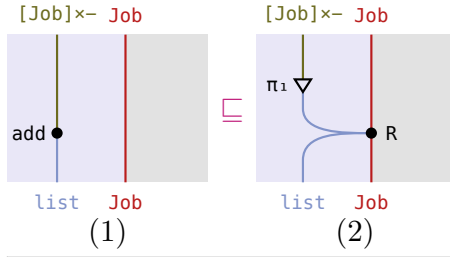
1 $[\text{nil}, \text{snoc}] \text{ cost}$

2 $= \langle g, m \rangle \text{ bmax}$

$\text{cost}(xs \# [j]) = \text{bmax}(\text{cost } xs, \text{penalty}(xs, j))$ and bmax commutes — $\text{cost}(\text{wrap}(a)) = 0$,
 $\text{cost}(\text{snoc}(x, a)) = \text{bmax}(\text{cost}(x), \text{penalty}(x, a))$

$$(\forall j. 0 \leq ct(j)) \wedge (\forall j. 0 \leq wt(j)) \Rightarrow add \sqsubseteq \pi_1 R \quad (10.7)$$

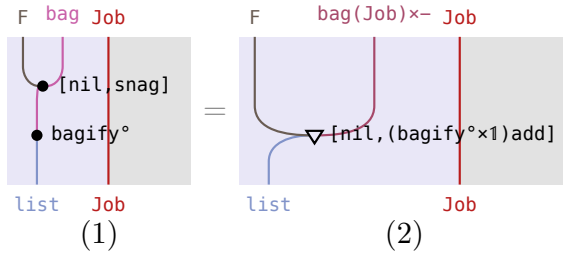
putting one more job anywhere into a schedule never lowers its cost



1	add	induction on where j goes: every job after it starts later, so no penalty falls
2	$\sqsubseteq \pi_1 R$	

$$[nil, snag] \text{ bagify}^\circ = [nil, (\text{bagify}^\circ \times 1) \text{ add}] \quad (10.8)$$

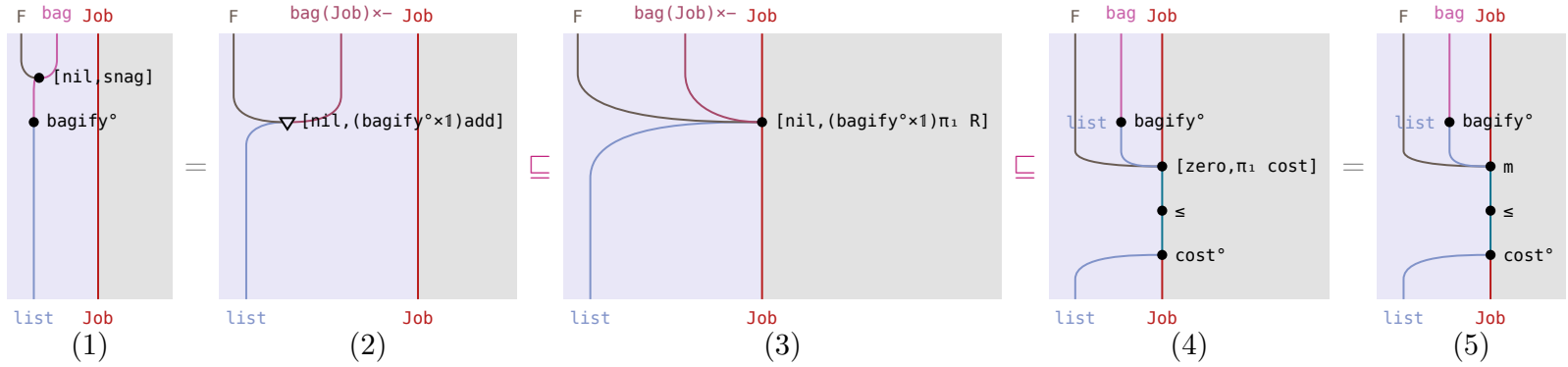
the orderings of a bag with one more job are the orderings of the bag with that job put in anywhere



1	$[nil, snag] \text{ bagify}^\circ$	take out the last copy of j and put it back — $p \text{ add } ys \iff \text{add}(\pi_2(p), ys)$
2	$= [nil, (\text{bagify}^\circ \times 1) \text{ add}]$	

$$(\forall j. 0 \leq ct(j)) \wedge (\forall j. 0 \leq wt(j)) \Rightarrow [nil, snag] \text{ bagify}^\circ \sqsubseteq F(\text{bagify}^\circ) m \leq \text{cost}^\circ \quad (10.3c)$$

an ordering of a bag with one more job costs at least the cost of the ordering of the bag before its last job



1	$[nil, snag] \text{ bagify}^\circ$	(10.8) $[nil, snag] \text{ bagify}^\circ = [nil, (\text{bagify}^\circ \times 1) \text{ add}]$
2	$= [nil, (\text{bagify}^\circ \times 1) \text{ add}]$	(10.7) $(\forall j. 0 \leq ct(j)) \wedge (\forall j. 0 \leq wt(j)) \Rightarrow \text{add} \sqsubseteq \pi_1 R$
3	$\sqsubseteq [nil, (\text{bagify}^\circ \times 1) \pi_1 R]$	$[nil, \pi_1 R] \sqsubseteq [zero, \pi_1 \text{ cost}] \leq \text{cost}^\circ$
4	$\sqsubseteq F(\text{bagify}^\circ) [zero, \pi_1 \text{ cost}] \leq \text{cost}^\circ$	
5	$= F(\text{bagify}^\circ) m \leq \text{cost}^\circ$	

$$(\forall j. 0 \leq ct(j)) \wedge (\forall j. 0 \leq wt(j)) \Rightarrow (Q \cap ([nil, snag] [nil, snag]^\circ)) F(bagify^\circ) [nil, snoc] \sqsubseteq F(bagify^\circ) [nil, snoc] R \quad (10.3)$$

a schedule of a bag ending in a job of least penalty is no worse than any schedule of the same bag

(1)	(2)	$\begin{aligned} 1 & (Q \cap ([nil, snag] [nil, snag]^\circ)) F(bagify^\circ) [nil, snoc] \\ 2 & \sqsubseteq (Q F(bagify^\circ) [nil, snoc]) \cap ([nil, snag] [nil, snag]^\circ F(bagify^\circ) [nil, snoc]) \\ 3 & = (Q F(bagify^\circ) [nil, snoc]) \cap ([nil, snag] bagify^\circ) \\ 4 & \sqsubseteq (Q F(bagify^\circ) [nil, snoc]) \cap (F(bagify^\circ) m \leq cost^\circ) \end{aligned}$	$\begin{aligned} (S \cap T) R & \sqsubseteq (S R) \cap (T R) \\ [nil, snag]^\circ F(bagify^\circ) [nil, snoc] & = bagify^\circ \\ (10.3c) (\forall j. 0 \leq ct(j)) \wedge (\forall j. 0 \leq wt(j)) & \Rightarrow [nil, snag] bagify^\circ \sqsubseteq F(bagify^\circ) m \leq cost^\circ \\ (Q F(bagify^\circ) [nil, snoc]) \cap (F(bagify^\circ) M) & \sqsubseteq F(bagify^\circ) ((F(bagify) Q F(bagify^\circ)) \cap (M [nil, snoc]^\circ)) [nil, snoc] \end{aligned}$
(5)	(6)	$\begin{aligned} 5 & \sqsubseteq F(bagify^\circ) ((F(bagify) Q F(bagify^\circ)) \cap (m \leq cost^\circ [nil, snoc]^\circ)) [nil, snoc] \\ 6 & = F(bagify^\circ) ((g \leq g^\circ) \cap (m \leq cost^\circ [nil, snoc]^\circ)) [nil, snoc] \\ 7 & = F(bagify^\circ) ((g \leq g^\circ) \cap (m \leq ([nil, snoc] cost)^\circ)) [nil, snoc] \\ 8 & = F(bagify^\circ) \langle g \leq, m \leq \rangle \langle g, [nil, snoc] cost \rangle^\circ [nil, snoc] \\ 9 & \sqsubseteq F(bagify^\circ) [nil, snoc] R \end{aligned}$	(7) (8) (9) $\begin{aligned} F(bagify) Q F(bagify^\circ) & = g \leq g^\circ \\ (R S)^\circ & = S^\circ R^\circ \\ \langle X, Y \rangle \langle R, S \rangle^\circ & = \langle X R^\circ \rangle \cap \langle Y S^\circ \rangle \\ (10.3d) \langle g \leq, m \leq \rangle \langle g, [nil, snoc] cost \rangle^\circ & [nil, snoc] \sqsubseteq [nil, snoc] R \end{aligned}$

$$\langle g \leq, m \leq \rangle \langle g, [nil, snoc] cost \rangle^\circ [nil, snoc] \sqsubseteq [nil, snoc] R \quad (10.3d)$$

a job whose penalty is at most the last penalty, put after a schedule costing at most the cost before the last job, gives a schedule costing at most the whole

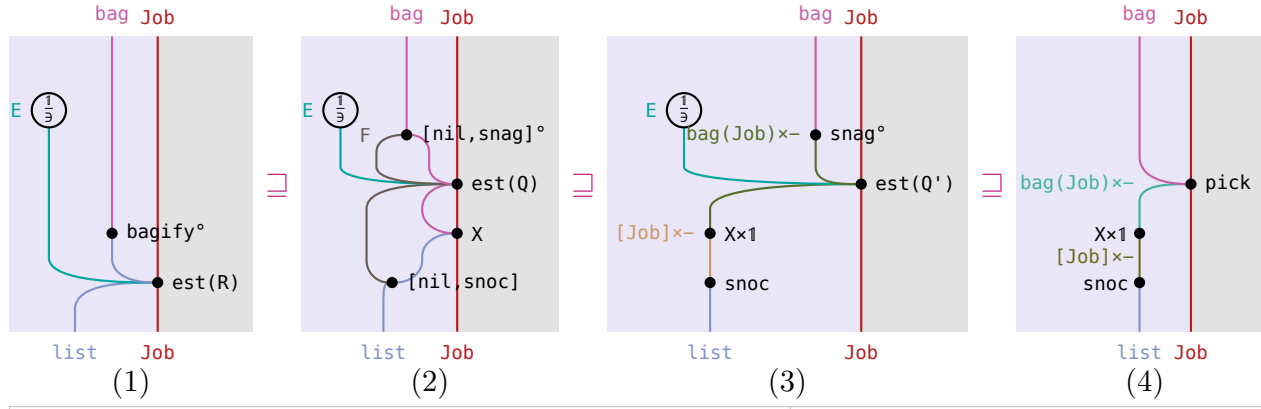
(1)	(2)	$\begin{aligned} 1 & \langle g \leq, m \leq \rangle \langle g, [nil, snoc] cost \rangle^\circ [nil, snoc] \\ 2 & \sqsubseteq \langle g \leq, m \leq \rangle \langle g, [nil, snoc] cost \rangle^\circ [nil, snoc] cost cost^\circ \\ 3 & = \langle g \leq, m \leq \rangle \langle g, [nil, snoc] cost \rangle^\circ \langle g, [nil, snoc] cost \rangle bmax cost^\circ \end{aligned}$	(3) $\begin{aligned} 1 & \sqsubseteq cost cost^\circ \\ [nil, snoc] cost & = \langle g, [nil, snoc] cost \rangle bmax \\ \langle g, [nil, snoc] cost \rangle^\circ \langle g, [nil, snoc] cost \rangle & \sqsubseteq 1 \end{aligned}$
(4)	(5)	$\begin{aligned} 4 & \sqsubseteq \langle g \leq, m \leq \rangle bmax cost^\circ \\ 5 & \sqsubseteq \langle g, m \rangle bmax \leq cost^\circ \\ 6 & = [nil, snoc] cost \leq cost^\circ \\ 7 & = [nil, snoc] R \end{aligned}$	(6) (7) $\begin{aligned} \langle g \leq, m \leq \rangle bmax & \sqsubseteq \langle g, m \rangle bmax \leq \\ (10.4) [nil, snoc] cost & = \langle g, m \rangle bmax \\ R & = cost \leq cost^\circ \end{aligned}$

$$\text{schedule} = (\text{null} \rightarrow \text{nil}, \text{pick} (\text{schedule} \times 1) \text{ snoc}), (\forall j. 0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j)) \Rightarrow (\mu X : \quad (10.3e)$$

$$\frac{[\text{nil}, \text{snag}]^\circ}{\exists} \text{ est}(Q) \text{ F}(X) [\text{nil}, \text{snoc}] \sqsubseteq \frac{\text{bagify}^\circ}{\exists} \text{ est}(R)$$

an ordering of the given bag with least maximum penalty is got by taking a job of least penalty out of the bag, putting it last, and scheduling what is left the same way

the specification — $\text{bagify} \triangleq \text{bagify}$



1	$\frac{\text{bagify}^\circ}{\exists} \text{ est}(R)$	No greedy reduce exists — one would also solve every prefix of the input, and the best schedule of a prefix need not extend to a best schedule of the whole
2	$\sqsubseteq \frac{[\text{nil}, \text{snag}]^\circ}{\exists} \text{ est}(Q) \text{ F}(X) [\text{nil}, \text{snoc}]$	Proposition 10.1: nil and snag have disjoint ranges
3	$\sqsubseteq \frac{\text{snag}^\circ}{\exists} \text{ est}(Q') (X \times 1) \text{ snoc}$	$\text{pick} \sqsubseteq \frac{\text{snag}^\circ}{\exists} \text{ est}(Q')$, a partial function, quadratic in the number of jobs
4	$\sqsubseteq \text{pick} (X \times 1) \text{ snoc}$	

§10.4. The TeX problem

(10.4a)

	name	type	definition	meaning
1	intern	$[\text{Digit}] \rightarrow [0, 2^{16})$	$\text{intern} \triangleq \text{val round}$	the value of a decimal fraction, rounded to 16 binary places
2	val	$[\text{Digit}] \rightarrow \text{Real}$	$\text{val} \triangleq ([\text{zero}, \text{shift}])$	the real number a digit list denotes after the decimal point
3	shift	$\text{Digit} \times \text{Real} \rightarrow \text{Real}$	$\text{shift} \triangleq \text{shift}, \text{shift}((d, r)) \triangleq (d + r)/10$	put digit d in front of r: $(d+r)/10$
4	round	$\text{Real} \rightarrow [0, 2^{16})$	$r \text{ round } n \iff (2n-1)/w < r \wedge r < (2n+1)/w, \text{round}^\circ = \text{interval inrange}$	the nearest integer to $2^{16}r$: $\text{round } r = n$ iff $2n-1 < 2^{17}r < 2n+1$
5	interval	$[0, 2^{16}) \rightarrow \text{Interval}$	$\text{interval}(n) \triangleq ((2n-1)/w, (2n+1)/w)$	the open interval $((2n-1)/2^{17}, (2n+1)/2^{17})$ of the reals that round to n
6	inrange	$\text{Interval} \rightarrow \text{Real}$	$p \text{ inrange } r \iff \text{lo } p < r \wedge r < \text{hi } p$	r lies strictly inside (a,b)
7	Legal(p)	Prop	$\text{Legal}(p) \triangleq 0 < \text{hi } p \wedge \text{hi } p < 1 \wedge \text{lo } p < \text{hi } p$	(10.9): $0 < b < 1$ and $a < b$; Interval is the legal pairs
8	arb	$1 \rightarrow \text{Interval}$	$x \text{ arb } p \iff \text{lo } p < 0 \wedge 0 < \text{hi } p$	any legal interval
9	step	$\text{Digit} \times \text{Interval} \rightarrow \text{Interval}$	$\text{step}(p) \triangleq (\text{shift}((\pi_1(p), \text{lo}(\pi_2(p)))), \text{shift}((\pi_1(p), \text{hi}(\pi_2(p)))))$	put digit d in front of both ends: $((d+a)/10, (d+b)/10)$
10	H	$\text{Interval} \rightarrow [\text{Digit}]$	$H \triangleq ([\text{arb}, \text{step}])^\circ$	every digit list whose interval the given one is
11	R	$[\text{Digit}] \rightarrow [\text{Digit}]$	$xs \text{ R } ys \iff \text{length}(xs) \leq \text{length}(ys)$	x is no longer than y
12	l	$1 \rightarrow \text{FX}$	$l \triangleq \text{inl}$	the injection of the empty case into $\text{FX} = 1 + (\text{Digit} \times X)$
13	r	$\text{Digit} \times X \rightarrow \text{FX}$	$r \triangleq \text{inr}$	the injection of the digit case into $\text{FX} = 1 + (\text{Digit} \times X)$
14	!	$\text{Digit} \times \text{Interval} \rightarrow 1$	$! \triangleq \multimap ()$	the one map to 1
15	Q	$F(\text{Interval}) \rightarrow F(\text{Interval})$	$Q \triangleq (l^\circ !^\circ r) \cup 1, Q(u, z) \iff (\exists t. u = l(t) \wedge (\exists s. z = r(s))) \vee u = z$	u is the empty case and z a digit case, or $u=z$
16	w	Int	$w \triangleq 131072$	2^{17}

$$([arb, step]) \sqsubseteq val \text{ inrange}^\circ$$

(10.4b)

every interval got by folding a decimal's digits with $[arb, step]$ has the decimal's value strictly inside it

	=		$\sqsubseteq$		=		=	
(1)		(2)		(3)		(4)		(5)

1	$F(\text{inrange}^\circ) [arb, step]$	$F(S) [T, U] = [T, (1 \times S) U]$
2	$= [arb, (1 \times \text{inrange}^\circ) step]$	$(1 \times \text{inrange}^\circ) step \sqsubseteq \text{shift inrange}^\circ$
3	$\sqsubseteq [arb, \text{shift inrange}^\circ]$	$\text{zero inrange}^\circ = arb$
4	$= [\text{zero inrange}^\circ, \text{shift inrange}^\circ]$	$[R, S] Z = [R Z, S Z]$
5	$= [\text{zero}, \text{shift}] \text{ inrange}^\circ$	

$$Q F(X) \alpha \sqsubseteq F(X) \alpha R$$

(10.4c)

choosing by Q among the one-step decompositions before building with $F(X)$ and α yields only decimals that are R -related to one built without choosing

	$\triangleq$		=		U	
(1)		(2)		(3)		(4)

1	$Q F(X) \alpha$	$Q \triangleq (!^\circ !^\circ r) u 1$
2	$\triangleq (!^\circ !^\circ r) u 1 F(X) \alpha$	$(S u T) R = (S R) u (T R)$
3	$= (!^\circ !^\circ r F(X) \alpha) u (F(X) \alpha)$	$1 \sqsubseteq R$

	$\sqsubseteq$		U		$\sqsubseteq$		U	
(5)		(6)		(7)		(8)		(9)

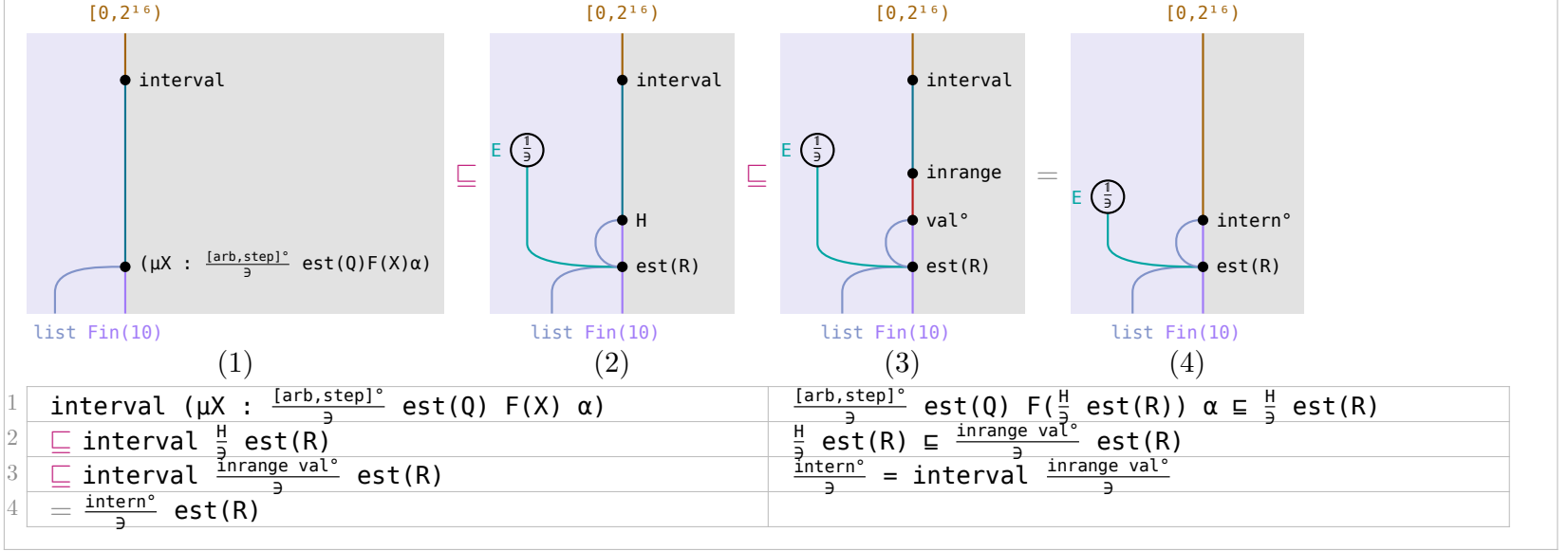
4	$\sqsubseteq (!^\circ !^\circ r F(X) \alpha) u (F(X) \alpha R)$	$r F(X) \alpha \sqsubseteq !^\circ l \alpha R$
5	$\sqsubseteq (!^\circ !^\circ !^\circ l \alpha R) u (F(X) \alpha R)$	$!^\circ !^\circ \sqsubseteq 1$
6	$\sqsubseteq (!^\circ l \alpha R) u (F(X) \alpha R)$	$l F(X) = l$

	=		$\sqsubseteq$		U		=	
(10)		(11)		(12)		(13)		(14)

7	$= (!^\circ l F(X) \alpha R) u (F(X) \alpha R)$	$!^\circ l \sqsubseteq 1$
8	$\sqsubseteq (F(X) \alpha R) u (F(X) \alpha R)$	$R u R = R$
9	$= F(X) \alpha R$	

$$\text{extern}(n)=f(2n-1,2n+1), \quad \text{interval } (\mu X : \frac{[\text{arb}, \text{step}]^\circ}{3} \text{est}(Q) F(X) \alpha) \sqsubseteq \frac{\text{intern}^\circ}{3} \text{est}(R) \quad (10.4d)$$

a shortest decimal whose internal representation is the given multiple of 2^{-16} is got by emitting the one digit the interval of admissible reals allows, until that interval contains zero and the empty decimal will do



- $f(a,b) = xs \iff a < 0 \wedge xs = [] \vee \neg a < 0 \wedge (\exists d. \exists ys. d = [10b] \wedge f(10a-d, 10b-d) = ys \wedge xs = [d] \# ys)$
 f gives (a,b) the empty decimal when $a < 0$, and otherwise the digit $d = [10b]$ followed by what it gives $(10a-d, 10b-d)$ (10.4e)
- $(\mu X : \frac{[\text{arb}, \text{step}]^\circ}{3} \text{est}(Q) F(X) \alpha) = f$
the least solution of the greedy recursion of $\text{interval } (\mu X : \frac{[\text{arb}, \text{step}]^\circ}{3} \text{est}(Q) F(X) \alpha) \sqsubseteq \frac{\text{intern}^\circ}{3} \text{est}(R)$ is f
- $d = [10b] \wedge e = [10b] \implies d = e$
Exercise 10.12: two digits d and e with $0 < 10b-d < 1$ and $0 < 10b-e < 1$ are equal
- $(0 < q-p \wedge q < w) \wedge p < 0 \implies f(p,q) = []$
the program's first clause: $f(p,q) = []$ when $p < 0$
- $(0 < q-p \wedge q < w) \wedge \neg p < 0 \implies f(p,q) = [10q/w] \# f(10p-w \cdot (10q/w), 10q-w \cdot (10q/w))$
the program's second clause: with $d = (10q) \text{ div } w$, $f(p,q)$ is d followed by $f(10p-w \cdot d, 10q-w \cdot d)$
- $(0 < q-p \wedge q < w) \wedge \text{Reach}(p,q) \implies f(p/w, q/w) = f(p,q)$
on the pairs the program reaches from $\text{interval } n$, the integer f at (p,q) is a value of the rational f at $(p/w, q/w)$
- $\text{interval } f = \text{extern}$
 f after interval is the program extern , whose value at n is $f(2n-1, 2n+1)$

§10. Appendix

§10.1. $P(S) \text{ est}(R) = (\exists S) \cap (\epsilon \setminus (SR^\circ))$

(10.1a)

(p, q) tabulates $W \triangleq (\exists S) \cap (\epsilon \setminus (SR^\circ))$, and $y \triangleq \frac{(p \exists S) \cap (qR)}{\exists}$, a map.

(10.1b)

$$\begin{array}{c} \boxed{\begin{array}{c} W \\ P(S) \text{ est}(R) \end{array}} \xleftarrow{p^\circ q = W, 1 \sqsubseteq yy^\circ} \begin{array}{c} \boxed{\begin{array}{c} p^\circ y \\ P(S) \end{array}} \\ \boxed{\begin{array}{c} y^\circ q \\ \text{est}(R) \end{array}} \end{array}$$

(10.1c)

$$\begin{array}{c} \boxed{\begin{array}{c} y^\circ q \\ \exists \end{array}} \xleftarrow{f^\circ \cdot \neg f^\circ, \cdot \exists \neg \exists} \boxed{\begin{array}{c} q \\ (p \exists S) \cap (qR) \end{array}} \xleftarrow{\Delta \neg \cap, f^\circ \cdot \neg f^\circ} \begin{array}{c} \boxed{\begin{array}{c} p^\circ q \\ \exists S \end{array}} \\ \boxed{\begin{array}{c} 1 \\ R \end{array}} \end{array}$$

R reflexive

$$\boxed{\begin{array}{c} \epsilon y^\circ q \\ R^\circ \end{array}} \xleftarrow{\cdot \exists \neg \exists, ^\circ, \text{meet}} \boxed{\begin{array}{c} R^\circ q^\circ q \\ R^\circ \end{array}} \quad \text{q simple}$$

(10.1d)

$$\begin{array}{c} \boxed{\begin{array}{c} p^\circ y \\ P(S) \end{array}} \xleftarrow{\Delta \neg \cap, \cdot T \neg / T, ^\circ, f^\circ \cdot \neg f^\circ} \begin{array}{c} \boxed{\begin{array}{c} p^\circ y \exists \\ \exists S \end{array}} \\ \boxed{\begin{array}{c} p \exists \\ y \exists S^\circ \end{array}} \end{array}$$

$$\boxed{\begin{array}{c} p^\circ y \exists \\ \exists S \end{array}} \xleftarrow{\cdot \exists \neg \exists, \text{meet}} \boxed{\begin{array}{c} p^\circ p \exists S \\ \exists S \end{array}} \quad \text{p simple}$$

$$\boxed{\begin{array}{c} p \exists \\ y \exists S^\circ \end{array}} \xleftarrow{\text{modular law}} \boxed{\begin{array}{c} p \exists \\ (p \exists) \cap (qRS^\circ) \end{array}} \xleftarrow{1 \sqsubseteq qq^\circ} \boxed{\begin{array}{c} q^\circ p \exists \\ RS^\circ \end{array}} \xleftarrow{^\circ, T \cdot \neg T \setminus} \boxed{\begin{array}{c} W \\ \epsilon \setminus (SR^\circ) \end{array}} \quad \text{W's right half}$$

§10.2. $P(\text{est}(R)) \text{ est}(R) = P(\text{dom}(\text{est}(R))) \text{ union est}(R)$

(10.2a)

$$\boxed{P(\text{est}(R)) \text{ est}(R)} \xrightarrow{\text{dom}(\text{est}(R)) \text{ est}(R) = \text{est}(R)} \boxed{P(\text{dom}(\text{est}(R)) \text{ est}(R)) \text{ est}(R)}$$

$$\begin{array}{ccc} \xrightarrow{\text{P a relator}} \boxed{P(\text{dom}(\text{est}(R)))P(\text{est}(R)) \text{ est}(R)} & \xrightarrow{(7.11) \ R \ R \sqsubseteq R} & \boxed{P(\text{dom}(\text{est}(R))) \text{ union est}(R)} \\ \Rightarrow P(\text{est}(R)) \text{ est}(R) \sqsubseteq \text{union est}(R) & & \end{array}$$

(10.2b)

$$\begin{array}{ccc} \boxed{P(\text{dom}(\text{est}(R))) \text{ union est}(R)} & \xleftarrow{\text{P}(\text{dom}(\text{est}(R))) \text{ union est}(R) \sqsubseteq \text{est}(R)} & \boxed{P(\text{dom}(\text{est}(R))) \text{ union est}(R)} \\ \boxed{P(\text{est}(R)) \text{ est}(R)} & \xRightarrow{\text{P}(S) \text{ est}(R) = (\exists S) \cap (\epsilon \setminus (S R^\circ)) \text{ at } S := \text{est}(R), \Delta \vdash n, T \vdash \neg T \setminus} & \boxed{\epsilon P(\text{dom}(\text{est}(R))) \text{ union est}(R)} \\ & & \boxed{\text{est}(R) R^\circ} \end{array}$$

$$\boxed{P(\text{dom}(\text{est}(R))) \text{ union est}(R)} \xleftarrow{\text{P}(\text{dom}(\text{est}(R))) \sqsubseteq 1} \boxed{\text{union est}(R)}$$

$$\begin{array}{ccc} \xleftarrow{\text{T}(\text{UnV}) \sqsubseteq \text{TUnTV}, \cdot \exists \vdash \frac{\cdot}{\cdot}, (7.3) \text{ union } (\epsilon \setminus R) = \epsilon \setminus (\epsilon \setminus R)} & \boxed{(\exists \exists) n (\epsilon \setminus (\epsilon \setminus R^\circ))} & \xleftarrow{\text{modular law}} \boxed{\exists (\exists n (\epsilon \setminus (\epsilon \setminus R^\circ)))} \\ & \boxed{\exists \text{est}(R)} & \boxed{\exists (\exists n (\epsilon \setminus R^\circ))} \\ & & \text{count of } T \cdot \neg T \setminus \end{array}$$

$$\boxed{\epsilon P(\text{dom}(\text{est}(R))) \text{ union est}(R)} \xleftarrow{\epsilon \text{ lax natural}} \boxed{\text{dom}(\text{est}(R)) \epsilon \text{ union est}(R)}$$

$$\begin{array}{ccc} \xleftarrow{\text{dom}(\text{est}(R)) \sqsubseteq \text{est}(R) \text{ est}(R)^\circ} & \boxed{\text{est}(R) \text{ est}(R)^\circ \epsilon \text{ union est}(R)} & \xleftarrow{\text{est}(R)^\circ \sqsubseteq \epsilon, \epsilon \epsilon \text{ union} \sqsubseteq \epsilon} \boxed{\text{est}(R) \epsilon \text{est}(R)} \\ & \boxed{\text{est}(R) R^\circ} & \boxed{\text{est}(R) R^\circ} \\ & & \text{UP of est} \end{array}$$

§10.3. Shortest paths on a cylinder, on lists

(10.3a)

	definition	type	note
1	$F(A,X)=A+A \times X$	$\mathcal{A} \rightarrow \mathcal{A}$	The base functor: one square of the new column, alone or in front of the path so far.
2	$L = \text{list}^+$	$\mathcal{A} \rightarrow \mathcal{A}$	A path is a non-empty list of squares, one per column crossed.
3	α the initial algebra	$F(\mu F) \rightarrow \mu F$	$\alpha(5)=[5]$ and $\alpha(1,[5])=[1,5]$: a path is started by one square, or extended by one.
4	N the n-tuple relator	$\mathcal{A} \rightarrow \mathcal{A}$	One component per row of a column, so the fold carries n answers at once.
5	$R \triangleq \text{sum} \leq \text{sum}^\circ$	$[\mathbb{N}] \rightarrow [\mathbb{N}]$	The cost of a path, which the cheapest minimises.
6	setify	$A[n] \rightarrow PA$	$\text{setify}(1,2,3,4)=\{1,2,3,4\}$ — which row a component came from is forgotten.
7	moves	$A[n] \rightarrow P(A[n])$	$\text{moves}(x)=\{\text{up}(x), x, \text{down}(x)\}$ — rotated up, unrotated, rotated down.
8	trans	$P(A[n]) \rightarrow (P\ A)[n]$	$\text{trans}\{(a,b,c), (x,y,z)\} = (\{a,x\}, \{b,y\}, \{c,z\})$ — component k of the result is the set of the k-th components.
9	zip	$F(a[n]) \rightarrow (F\ a)[n]$	$\text{zip}((1,2,3,4), (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})) = ((1, \{[5]\}), (2, \{[6]\}), (3, \{[7]\}), (4, \{[8]\}))$.
10	$\text{cp} \triangleq \frac{F(1, \exists)}{\exists}$	$F(PA) \rightarrow PFA$	$\text{cp}(1, \{[5], [6], [8]\}) = ((1, [5]), (1, [6]), (1, [8]))$.
11	$\text{gen} \triangleq F(1, \text{moves trans } N(\text{union}))$ $\text{zip } N(\text{cp}(F(A, -))\ E(\alpha))$	$F(NA, N(P(TA))) \rightarrow N(P(TA))$	$\text{gen}((1,2,3,4), (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\}))$ is worked out in $\text{gen} \triangleq F(1, \text{moves trans } N(\text{union})) \text{ zip } N(\text{cp}(F(A, -))\ E(\alpha))$.
12	$\text{paths} \triangleq (\text{gen})\ \text{setify}\ \text{union}$	$T(NA) \rightarrow P(TA)$	$\text{paths}[(1,2,3,4), (5,6,7,8)]$ is the union of $\text{gen} \triangleq F(1, \text{moves trans } N(\text{union})) \text{ zip } N(\text{cp}(F(A, -))\ E(\alpha))$'s four sets: 12 paths, 3 from each entry row.
13	the specification $\text{paths est}(R)$	$[\mathbb{N}[n]] \rightarrow [\mathbb{N}]$	A cheapest path from the entry side to the exit side.

§10.3.1. $N(E(L\ A))$

(10.3.1a)

```

1  5
2  6      [(1,2,3,4), (5,6,7,8)] : L N Nat
3  7
4  8

```

(10.3.1b)

```
gen ≐ F(1, moves trans N(union)) zip N(cp(F(A, -)) E(α))
```

```
(gen)[(5,6,7,8)] = ({[5]}, {[6]}, {[7]}, {[8]})
```

```
gen((1,2,3,4), ({[5]}, {[6]}, {[7]}, {[8]})) =
( {[1,5]}, {[1,6]}, {[1,8]} ,
  {[2,5]}, {[2,6]}, {[2,7]} ,
  {[3,6]}, {[3,7]}, {[3,8]} ,
  {[4,5]}, {[4,7]}, {[4,8]} )
```

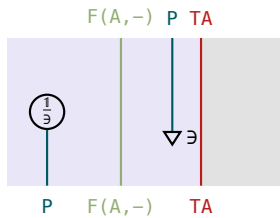
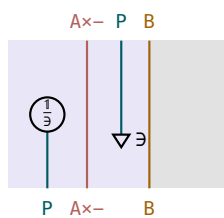
(10.3.1c)

1	N	the 4-tuple, one component per row of the entry column
2	E	the set of paths that can start in that row
3	L	a path is the non-empty list of the squares it crosses

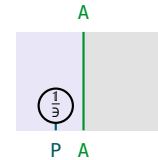
row 1's set has no $[1,7]$ — from row 1 only rows 4, 1 and 2 are reachable — and it does have $[1,8]$, because the cylinder glues the bottom row to the top. That is $\text{moves trans } N(\text{union})$ of $\text{gen} \triangleq F(1, \text{moves trans } N(\text{union})) \text{ zip } N(\text{cp}(F(A, -)) E(\alpha))$: component k collects the paths of rows $k-1, k, k+1$.

§10.3.2. The cross product $\text{cp} \triangleq \frac{F(1, \exists)}{\exists}$

(10.3.2a)

cp on all of $F(A, E(L A))$ 

the $A \times -$ summand: $\exists$ picks one path, $1\% \exists$ collects the results



the A summand, no E to distribute: $1\% \exists$ alone, $a \mapsto \{a\}$

(10.3.2b)

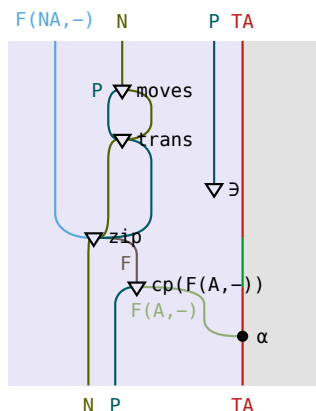
type	note
1 $F(A, -) = A + A \times -$	one wire carries the whole functor, both summands with it; the $\exists$ under it is $F(1, \exists)$
2 $\exists : E(L A) \rightarrow L A$	the $\exists$ inside $F(1, \exists)$: a set of paths, one of them — $L A$ is one path, A one square
3 $F(1, \exists) = 1 + 1 \times \exists$	$: A + A \times E(L A) \rightarrow A + A \times L A$, the relator acting on each summand
4 $\exists : E(A + A \times L A) \rightarrow A + A \times L A$	the $\exists$ under the bar: a set of cells, one of them — a different $\exists$
5 $\text{cp} : A + A \times E(L A) \rightarrow E(A + A \times L A)$	$\frac{R}{\exists}$ turns $R : X \rightarrow Y$ into $X \rightarrow E Y$
6 $\text{cp} = \frac{1}{\exists} E(F(1, \exists))$	what the picture draws: the unit makes the outer E , leaving one $\exists$ — $\frac{R}{\exists} = \frac{1}{\exists} E(R)$

(10.3.2c)

$w = (1, \{[5], [6], [8]\}) : A \times E(L A)$ the right summand
 $1 \times \exists : A \times E(L A) \rightarrow A \times L A$ 1 keeps the square 1, $\exists$ picks one path
 $w \mapsto (1, [5]), (1, [6]), (1, [8]) : A \times L A$ one output per path in w
 $\text{cp}(w) = \{(1, [5]), (1, [6]), (1, [8])\} : E(A + A \times L A)$ the three of them, collected

§10.3.3. $\text{gen} = F(1, \text{moves trans } N(\text{union})) \text{ zip } N(\text{cp } P(\alpha))$

(10.3.3a)

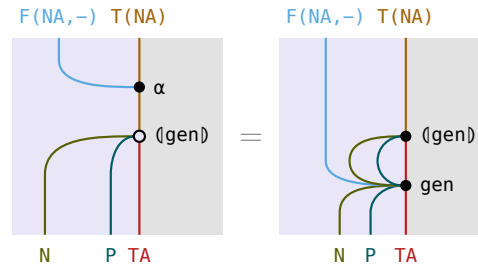


(10.3.3b)

$\text{gen} \triangleq F(1, \text{moves trans } N(\text{union})) \text{ zip } N(\text{cp}(F(A, -)) E(\alpha))$
 $u = ((1, 2, 3, 4), (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})) : F(N A, N(E(L A)))$
 $F(1, \text{moves trans } N(\text{union}))$ 1 keeps the column, the path SETS move
 $\text{moves}(\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})$
 $= (\{[6]\}, \{[7]\}, \{[8]\}, \{[5]\}),$
 $(\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\}),$
 $(\{[8]\}, \{[5]\}, \{[6]\}, \{[7]\})$: $E(N(E(L A)))$ down, unmoved, up
 $\text{trans}(\text{that})$
 $= (\{[6]\}, \{[5]\}, \{[8]\}, \{[7]\}, \{[6]\}, \{[5]\}),$
 $\{[8]\}, \{[7]\}, \{[6]\}, \{[5]\}, \{[8]\}, \{[7]\})$: $N(E(E(L A)))$ row k gets rows k-1, k, k+1
 $N(\text{union})(\text{that})$
 $= (\{[5], [6], [8]\}, \{[5], [6], [7]\},$
 $\{[6], [7], [8]\}, \{[5], [7], [8]\})$: $N(E(L A))$ every path into row k, none dropped
 $\text{zip}(\text{that})$
 $= ((1, \{[5], [6], [8]\}), (2, \{[5], [6], [7]\}),$
 $(3, \{[6], [7], [8]\}), (4, \{[5], [7], [8]\}))$: $N(F(A, E(L A)))$ each row: its square, and the paths it may be put in front of
 $N(\text{cp } P(\alpha))(\text{that})$
 $= (\{[1, 5], [1, 6], [1, 8]\}, \{[2, 5], [2, 6], [2, 7]\},$
 $\{[3, 6], [3, 7], [3, 8]\}, \{[4, 5], [4, 7], [4, 8]\})$: $N(E(L A))$ cp pairs the square with each path, α prefixes it: $\alpha(1, [5]) = [1, 5]$

§10.3.4. gen is an F-algebra; $(\text{gen}) : \alpha(\text{gen}) = F(1, (\text{gen})) \text{gen}$

(10.3.4a)

 α puts the column back on the list, then the fold reads all of itthe fold reads the rest under F , then one gen puts the column in front

$$xs = [(1, 2, 3, 4), (5, 6, 7, 8)] = \alpha((1, 2, 3, 4), [(5, 6, 7, 8)]) : L(N A) \quad (10.3.4b)$$

$$(\text{gen})(xs) = \text{gen}((1, 2, 3, 4), (\text{gen})[(5, 6, 7, 8)])$$

both halves are in $\text{gen} \triangleq F(1, \text{moves trans } N(\text{union})) \text{ zip } N(\text{cp}(F(A, -)) E(\alpha))$: the tail folds to $(\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})$, one path per row and each of them one square long, and gen on it is $\text{gen} \triangleq F(1, \text{moves trans } N(\text{union})) \text{ zip } N(\text{cp}(F(A, -)) E(\alpha))$'s walk.

§10.3.5. $Q : F(N A, N(L A)) \rightarrow N(L A)$, an $F(N A, -)$ -algebra: $Q = F(1, \text{moves trans } N(\text{est}(R))) \text{ zip } N(\alpha)$

(10.3.5a)

$$Q : F(N A, N(L A)) \rightarrow N(L A)$$

the fold's algebra: a new column, one path per row

$u = ((1, 2, 3, 4), ([5], [6], [7], [8])) : F(N A, N(L A))$
 $F(1, \text{moves trans } N(\text{est}(R)))$ 1 keeps the column, the paths move
 $\text{moves}([5], [6], [7], [8])$
 $= ([6], [7], [8], [5]),$
 $([5], [6], [7], [8]),$
 $([8], [5], [6], [7])$: $E(N(L A))$ down, unmoved, up
 $\text{trans}(\text{that})$
 $= (\{[6], [5], [8]\}, \{[7], [6], [5]\},$
 $\{[8], [7], [6]\}, \{[5], [8], [7]\})$: $N(E(L A))$ row k gets rows k-1, k, k+1
 $N(\text{est}(R))(\text{that})$
 $= ([5], [5], [6], [5])$: $N(L A)$ the cheapest into each row – chosen BEFORE the new square
 $\text{zip}(\text{that})$
 $= ((1, [5]), (2, [5]), (3, [6]), (4, [5]))$: $N(F(A, L A))$ each row: its square, and the one predecessor that survived
 $N(\alpha)(\text{that})$
 $= ([1, 5], [2, 5], [3, 6], [4, 5])$: $N(L A)$ $\alpha(1, [5]) = [1, 5]$

$$\text{lax}(\text{moves}) \wedge \text{lax}(\text{trans}) \wedge \text{lax}(\text{zip}) \wedge F(1, \text{moves trans } N(\text{est}(R))) = [\text{l}, \text{pm } r] \wedge \text{zip} = \quad (10.3.5b)$$

$$[N(\text{inl}), \text{zip}' N(\text{inr})] \wedge \text{inl } \alpha = \text{wrap} \wedge \text{inr } \alpha = \text{cons} \Rightarrow Q = [N(\text{wrap}), \text{pm } \text{zip}' N(\text{cons})]$$

at the last column Q starts one path per row, and at each earlier one it puts each square in front of the cheapest of the three kept paths it can step to — the algebra $\text{lax}(\text{moves}) \wedge \text{lax}(\text{trans}) \wedge \text{lax}(\text{zip}) \wedge$

$$\text{lax}(\text{setify}) \wedge R R \sqsubseteq R \wedge F(1, N(\text{est}(R))) Q \sqsubseteq \text{gen } N(\text{est}(R))$$

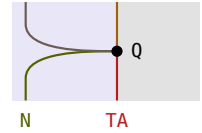
$$\Rightarrow (Q) \text{ setify } \text{est}(R) \sqsubseteq \text{paths } \text{est}(R) \text{ 's last step folds}$$

circuit — the fork is $F(NA, N(LA)) = NA + NA \times N(LA)$

Hinze–Marsden

$$F(N A) (N (T A)) \text{---} \boxed{Q} \text{---} N(T A)$$

$$F(N A, N (T A))$$

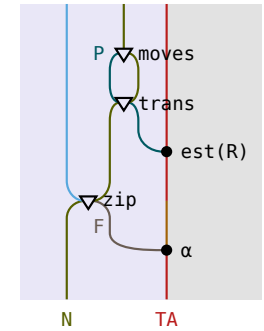


$$FN(T A) \text{---} \boxed{F(1, \text{moves trans } N(\text{est}(R)))} \text{---} FN(T A) \text{---} \boxed{\text{zip}} \text{---} N(F A (T A)) \text{---} \boxed{N(\alpha)} \text{---} N(T A)$$

= the fusion condition $\text{lax}(\text{moves}) \wedge \text{lax}(\text{trans}) \wedge \text{lax}(\text{zip}) \wedge R R \sqsubseteq R \wedge F(1, \text{est}(R)) \alpha \sqsubseteq \text{cp}(F(A, -)) E(\alpha) \text{est}(R)$

$\Rightarrow F(1, N(\text{est}(R))) Q \sqsubseteq \text{gen } N(\text{est}(R))$ read as a definition,

$$F(NA, -) N TA$$



$$F(N A) (N (T A)) \text{---} \left\{ \begin{array}{l} \boxed{\text{l}} \text{---} N(p_0) \text{---} \boxed{N(\text{wrap})} \\ \boxed{\text{r}} \text{---} q \text{---} \boxed{\text{pm}} \text{---} q \text{---} \boxed{\text{zip}'} \text{---} N(q_0) \text{---} \boxed{N(\text{cons})} \end{array} \right\} \text{---} N(T A)$$

$$\text{zip} = 1 + \text{zip}', \alpha = [\text{wrap}, \text{cons}]$$

$$\text{lax(moves)} \wedge \text{lax(trans)} \wedge \text{lax(zip)} \wedge \text{lax(setify)} \wedge R \ R \sqsubseteq R \wedge F(1, N(\text{est}(R))) \ Q \sqsubseteq \text{gen} \quad (10.3.5c)$$

$$N(\text{est}(R)) \Rightarrow \langle Q \rangle \text{ setify } \text{est}(R) \sqsubseteq \text{paths } \text{est}(R)$$

shortest paths on a cylinder: the dynamic programming approach to exhaustive search allows a path of least cost to be found in $O(n \times m)$ time; Q is $\text{lax(moves)} \wedge \text{lax(trans)} \wedge \text{lax(zip)} \wedge$

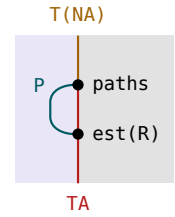
$$F(1, \text{moves trans } N(\text{est}(R))) = [l, pm \ r] \wedge \text{zip} = [N(\text{inl}), \text{zip}' \ N(\text{inr})] \wedge \text{inl } \alpha = \text{wrap} \wedge \text{inr } \alpha = \text{cons}$$

$$\Rightarrow Q = [N(\text{wrap}), pm \ \text{zip}' \ N(\text{cons})] \text{ 's algebra.}$$

circuit

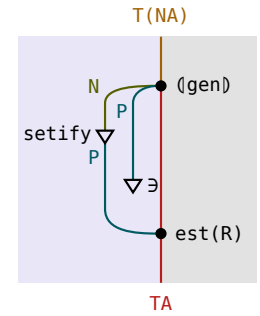
Hinze–Marsden

$$T(N \ A) \text{---} \boxed{\text{paths}} \text{---} P(T \ A) \text{---} \boxed{\text{est}(R)} \text{---} T \ A$$



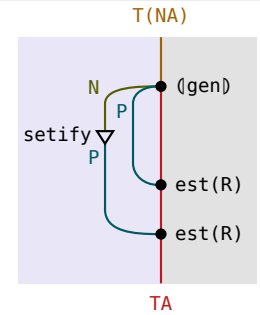
$$= T(N \ A) \text{---} \boxed{[F(N \ A) \ (N \ (P \ (T \ A)))]} \text{---} \boxed{\text{gen}} \text{---} NP(T \ A) \text{---} \boxed{\text{setify}} \text{---} PP(T \ A) \text{---} \boxed{\text{union}} \text{---} P(T \ A) \text{---} \boxed{\text{est}(R)} \text{---} T \ A$$

$\text{paths} \triangleq \langle \text{gen} \rangle \text{ setify union at paths}$



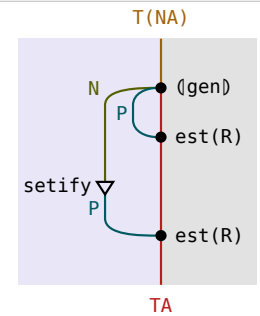
$$\sqsubseteq T(N \ A) \text{---} \boxed{[F(N \ A) \ (N \ (P \ (T \ A)))]} \text{---} \boxed{\text{gen}} \text{---} NP(T \ A) \text{---} \boxed{\text{setify}} \text{---} PP(T \ A) \text{---} \boxed{P(\text{est}(R))} \text{---} P(T \ A) \text{---} \boxed{\text{est}(R)} \text{---} T \ A$$

$P(\text{est}(R)) \ \text{est}(R) \sqsubseteq \text{union } \text{est}(R)$ — a minimum in each set, then a minimum of those; R transitive



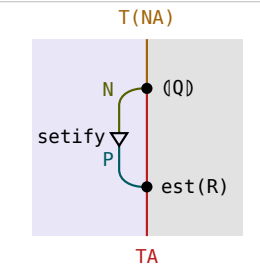
$$\sqsubseteq T(N \ A) \text{---} \boxed{[F(N \ A) \ (N \ (P \ (T \ A)))]} \text{---} \boxed{\text{gen}} \text{---} NP(T \ A) \text{---} \boxed{N(\text{est}(R))} \text{---} N(T \ A) \text{---} \boxed{\text{setify}} \text{---} P(T \ A) \text{---} \boxed{\text{est}(R)} \text{---} T \ A$$

$\text{setify lax natural}$



$$\sqsubseteq T(N \ A) \text{---} \boxed{[F(N \ A) \ (N \ (T \ A))]} \text{---} \boxed{Q} \text{---} N(T \ A) \text{---} \boxed{\text{setify}} \text{---} P(T \ A) \text{---} \boxed{\text{est}(R)} \text{---} T \ A$$

fusion at $\text{lax(moves)} \wedge \text{lax(trans)} \wedge \text{lax(zip)} \wedge R \ R \sqsubseteq R \wedge F(1, \text{est}(R)) \ \alpha \sqsubseteq \text{cp}(F(A, -))$
 $E(\alpha) \ \text{est}(R)$
 $\Rightarrow F(1, N(\text{est}(R))) \ Q \sqsubseteq \text{gen } N(\text{est}(R))$



$$\text{lax}(\text{moves}) \wedge \text{lax}(\text{trans}) \wedge \text{lax}(\text{zip}) \wedge R \subseteq R \wedge F(1, \text{est}(R)) \alpha \subseteq \text{cp}(F(A, -)) E(\alpha) \text{est}(R) \Rightarrow (10.3.5d)$$

$$F(1, N(\text{est}(R))) Q \subseteq \text{gen } N(\text{est}(R))$$

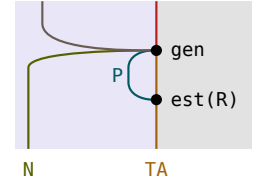
fusion: the condition for fusion in $\text{lax}(\text{moves}) \wedge \text{lax}(\text{trans}) \wedge \text{lax}(\text{zip}) \wedge \text{lax}(\text{setify}) \wedge R \subseteq R \wedge$
 $F(1, N(\text{est}(R))) Q \subseteq \text{gen } N(\text{est}(R))$

$$\Rightarrow (Q) \text{ setify } \text{est}(R) \subseteq \text{paths } \text{est}(R) \text{ 's last step, used to derive a definition of } Q.$$

circuit

$$F(N(A) (N(P(T A)))) \text{---} \boxed{\text{gen}} \text{---} NP(T A) \text{---} \boxed{N(\text{est}(R))} \text{---} N(T A)$$

Hinze–Marsden

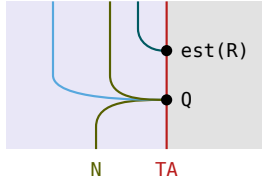
$$F(N(A, N(P(T A))))$$


N

TA

$$\boxed{\text{FNP}(T A)} \text{---} \boxed{F(1, N(\text{est}(R)))} \text{---} FN(T A) \text{---} \boxed{Q} \text{---} N(T A)$$

(7.13), then zip, trans, moves lax natural

$$F(NA, -) \quad N \quad P \quad TA$$


N

TA

(7.13) is $F(1, \text{est}(R)) \alpha \subseteq \text{cp } P(\alpha) \text{est}(R)$, Theorem 7.1 $\text{Map}(f) \wedge 1 \subseteq R$

$\Rightarrow f^\circ F(R) f \subseteq R \Leftrightarrow F(\text{est}(R)) f \subseteq \frac{F(\exists) f}{\exists} \text{est}(R)$ at the map α with $\frac{F(1, \exists) \alpha}{\exists} = \text{cp } P(\alpha)$: extending every path in a set and then taking a minimum is beaten by extending one minimum. It is the crux here, not the greedy theorem.